

THE JavaScript MASTERBOOK

ONE BOOK • ALL CONCEPTS
INTERVIEW READY

UPAMANYU DEKA

Introduction

JavaScript interviews are unpredictable — but your preparation doesn't have to be.

This book is designed as a complete, structured, and practical resource for anyone aiming to master JavaScript for real-world technical interviews. With **180+ carefully curated questions**, it covers the full spectrum of JavaScript concepts: from fundamentals like scope and closures to advanced topics such as event loops, Web APIs, memory optimization, async patterns, and modern ES features.

But this is not just another list of definitions.

Every question is answered in a **deep, explanatory, interview-style format**. You'll find clarity for beginners, depth for experienced developers, and insights that help you understand *why* JavaScript behaves the way it does — not just *what* it does.

Inside, you'll discover:

In-depth explanations written to teach, not confuse

Examples that show exactly how a concept works

Real-life analogies that help difficult ideas stick

Practice questions (both theory and coding) to sharpen your understanding

A structure that mirrors how top companies evaluate candidates

Whether you're preparing for your **first JavaScript interview** or aiming to **crack a senior-level role**, this book gives you the complete foundation and clarity you need. Read it sequentially or jump between topics — each question stands strong on its own, yet contributes to a cohesive understanding of the language.

Think of this book as your personal preparation partner:

Comprehensive. Practical. Interview-focused.

Let's begin your journey toward mastering JavaScript with confidence.

Table of Contents-1

- 001. Difference between var, let, and const
- 002. What is hoisting in JavaScript?
- 003. Difference between == and ===
- 004. What is scope (global, function, block)?
- 005. What are closures?
- 006. What is the event loop?
- 007. Explain call stack and execution context
- 008. What are data types in JavaScript?
- 009. Difference between primitive and reference types
- 010. What is type coercion?
- 011. Explain truthy and falsy values
- 012. What is NaN and how do you check for it?
- 013. Explain 'this' keyword in JavaScript
- 014. Difference between implicit, explicit, default, and new binding
- 015. What is lexical scope?
- 016. Explain function declarations vs function expressions
- 017. What is an IIFE (Immediately Invoked Function Expression)?
- 018. Explain pure functions and side effects
- 019. What is immutability and why does it matter?
- 020. Difference between undefined, null, and NaN
- 021. What is strict mode?
- 022. What is the Temporal Dead Zone (TDZ)?
- 023. What are arrow functions and how are they different from normal functions?
- 024. Explain default parameters, rest operator, and spread operator
- 025. What are template literals?
- 026. Difference between for, for-in, for-of, and forEach
- 027. What are objects and how are they stored in memory?
- 028. Explain pass-by-value vs pass-by-reference
- 029. What is object destructuring and array destructuring?
- 030. Explain Object.freeze, Object.seal, and Object.preventExtensions
- 031. What is prototypal inheritance?
- 032. How does the prototype chain work?
- 033. What are constructor functions?
- 034. Explain class syntax in ES6 and how it's sugar over prototypes

Table of Contents-2

- 035. What is Object.create used for?
- 036. Explain call, apply, bind — and differences between them
- 037. What are higher-order functions?
- 038. What is callback hell and how to avoid it?
- 039. Explain promises and how they work
- 040. What is async/await and how is it different from promises?
- 041. What are microtasks and macrotasks?
- 042. Explain setTimeout, setInterval, and clearTimeout
- 043. What is debouncing and throttling?
- 044. What is event bubbling and capturing?
- 045. How does event delegation work?
- 046. Difference between document, window, and this in different contexts
- 047. Explain DOM vs BOM
- 048. What are Web APIs?
- 049. What is localStorage, sessionStorage, and cookies?
- 050. What is CORS and how does it work?
- 051. Difference between synchronous and asynchronous code
- 052. What is the Fetch API and how is it different from XMLHttpRequest?
- 053. What is JSON.parse and JSON.stringify and what are their pitfalls?
- 054. Explain module patterns in JS — ESM vs CommonJS
- 055. What is tree shaking and dead code elimination?
- 056. What is a polyfill?
- 057. Explain memoization
- 058. What are generators and iterators?
- 059. Explain currying and partial application
- 060. What is the Intl API and how is it used for localization?
- 061. Explain the repaint and reflow process in browsers
- 062. What is garbage collection and how does mark-and-sweep work?
- 063. Explain shadowing and variable masking
- 064. What is event propagation and stopPropagation?
- 065. What is Symbol in JavaScript?
- 066. What is WeakMap and WeakSet?
- 067. What are Map and Set and how do they differ from objects?
- 068. Explain shallow copy vs deep copy
- 069. What is structuredClone?

Table of Contents-3

- 070. What are Web Workers and when should you use them?
- 071. What are Service Workers and PWA concepts?
- 072. Explain Promises.all, Promise.race, and Promise.allSettled
- 073. What is BigInt in JavaScript?
- 074. Explain dynamic imports and code splitting
- 075. What are optional chaining and nullish coalescing operators?
- 076. Explain Proxy and Reflect API
- 077. What is destructuring / aliasing and how is it useful?
- 078. What is module federation in modern JS apps?
- 079. Explain Virtual DOM and reconciliation in React conceptually (JS related)
- 080. What is event-loop starvation?
- 081. Explain call stack overflow and recursion depth limits
- 082. What are tagged template literals?
- 083. What is lazy evaluation in JS?
- 084. How does JavaScript handle memory leaks?
- 085. Explain hoisting with function expressions vs arrow functions
- 086. What is the Temporal API (upcoming JS proposal)?
- 087. What is requestAnimationFrame and when to use it?
- 088. Explain IntersectionObserver and MutationObserver APIs
- 089. What is the difference between innerHTML and textContent?
- 090. What are custom events and how do you dispatch them?
- 091. Explain microtask queue vs nextTick in Node.js
- 092. What is the difference between V8 engine internals and standard JavaScript?
- 093. How does just-in-time (JIT) compilation work in JavaScript engines?
- 094. Explain hidden classes and inline caching in V8
- 095. What are WeakRefs and FinalizationRegistry?
- 096. How does debounce-throttle combo optimize performance?
- 097. What are ArrayBuffer and TypedArray?
- 098. Explain SharedArrayBuffer and Atomics API
- 099. What is structured concurrency (upcoming spec)?
- 100. How does the ECMAScript spec define execution order?
- 101. Explain Function.prototype.toString and environment internal slots
- 102. What is the Realms API and why might it matter for sandboxing?
- 103. How does the module resolution algorithm work in ESM?
- 104. What are decorators and how do they extend class behavior?

Table of Contents-4

- 105. Explain WeakMap-based private fields vs native private fields () in classes
- 106. What are import assertions and why are they used?
- 107. What is structuredClone's difference from deep copy via JSON?
- 108. Explain lazy vs eager evaluation in iterables
- 109. What is monkey-patching and why is it discouraged?
- 110. How do JavaScript engines optimize tail calls (TCO)?
- 111. *What is generator delegation (yield) and how does it work?*
- 112. What are async generators and how are they used with for-await-of?
- 113. Explain top-level await in ES modules
- 114. How does module caching work in ES modules and CommonJS?
- 115. What happens in circular module dependencies in JavaScript?
- 116. Explain bare imports and import maps in browsers
- 117. What are custom error classes and how do you create them?
- 118. How do try-catch-finally blocks behave with async/await?
- 119. What is unhandledrejection and how can it crash your app?
- 120. What are tagged template literals used for in libraries like styled-components?
- 121. Explain the difference between lazy evaluation and eager evaluation in iterables
- 122. How does tail-call optimization (TCO) work, and is it supported in JavaScript engines?
- 123. What are Record and Tuple proposals, and how do they differ from Objects and Arrays?
- 124. What is pattern matching in JavaScript (proposal stage)?
- 125. How does the pipeline operator improve function chaining?
- 126. What is Array.prototype.groupBy and how is it used?
- 127. What is the purpose of Symbol.dispose and the using statement proposal?
- 128. What is Error.cause — new error message cause — and when is it useful?
- 129. What are the phases of the event loop (timers, poll, check, close)?
- 130. What is event-loop starvation and how can you prevent it?
- 131. What is requestIdleCallback and when should you use it?
- 132. How do garbage collection triggers and mark-and-sweep impact performance?
- 133. What causes detached DOM node memory leaks and how to avoid them?
- 134. What is the difference between microtasks, macrotasks, and animation frames?
- 135. How do Performance.now and PerformanceObserver help in profiling?
- 136. What is the difference between queueMicrotask, setTimeout, and requestAnimationFrame?
- 137. How do hidden classes and inline caching affect JS performance internally?
- 138. What is ResizeObserver and how is it different from MutationObserver?
- 139. How does the Clipboard API work for copying and pasting programmatically?

Table of Contents-5

- 140. What is the Notification API and how can you request user permission?
- 141. Explain the Battery Status and Network Information APIs
- 142. What is the Fetch streaming API and how can you consume a streamed response?
- 143. What is the Web Share API and when is it useful?
- 144. How does the Web Crypto API provide secure randomness and hashing?
- 145. What is `crypto.getRandomValues` and why is it safer than `Math.random`?
- 146. Explain WebRTC basics — data channels and peer connections
- 147. What is `AbortController` and how do you use it to cancel fetch requests?
- 148. What is Content Security Policy (CSP) and why is it important?
- 149. What is cross-site scripting (XSS) and how can JavaScript prevent it?
- 150. What is cross-site request forgery (CSRF) and how can JS help mitigate it?
- 151. Explain sandboxed iframes and the same-origin policy in browsers
- 152. What is the Trusted Types API and how does it defend against XSS?
- 153. What is the Cache Storage API and how does it relate to Service Workers?
- 154. How do Progressive Web Apps (PWAs) leverage Service Workers for offline access?
- 155. What is the difference between `deep clone` and `structuredClone` for complex objects?
- 156. How do custom iterators work and how can you build your own iterable object?
- 157. What are `ArrayBuffer` and `TypedArray`, and how are they different from Arrays?
- 158. What are `SharedArrayBuffer` and `Atomics`, and how do they enable thread safety?
- 159. How do `WeakRefs` and `FinalizationRegistry` help manage memory?
- 160. What are transferable objects and how do they improve performance in Workers?
- 161. What is structured concurrency (upcoming spec) and how might it change async patterns?
- 162. Explain monkey patching, why it's discouraged, and alternatives
- 163. What is the Realms API and why might it matter for sandboxing?
- 164. How does the ECMAScript spec define execution order at the spec level?
- 165. What are the pitfalls of floating-point arithmetic ($0.1 + 0.2 \neq 0.3$)?
- 166. How can you achieve precise decimal arithmetic in JavaScript?
- 167. How do `Intl.DateTimeFormat` and `Intl.NumberFormat` support localization?
- 168. What are `pluralRules` and `segmenter` in the Intl API?
- 169. How does Temporal API improve date–time management compared to `Date`?
- 170. What are the limitations of `Math.random` and how to get cryptographically secure randomness?
- 171. What triggers a reflow vs a repaint and how to minimize them?
- 172. How does compositing work in modern browsers?
- 173. What are layout thrashing and forced synchronous layouts?

Table of Contents-6

- 174. What is IntersectionObserver and how can it be used for lazy loading?
- 175. How do custom events improve component communication?
- 176. What is the difference between innerHTML, outerHTML, and textContent?
- 177. What is the difference between CORS preflight and simple requests?
- 178. How does sandbox attribute in iframes affect script execution?
- 179. What are cross-origin resource policies (CORP, COEP, COOP) and why do they matter?
- 180. What is a content-type sniffing attack and how can JS prevent it?
- 181. What is the Observer pattern and how is it implemented in JavaScript?
- 182. What is the Publish–Subscribe pattern and how does it differ from Observer?
- 183. Explain functional composition in JavaScript
- 184. What are Higher-Order Components (HOC) and render-props patterns conceptually?
- 185. What is dependency injection and can it be achieved in JavaScript?
- 186. What are singletons and their drawbacks in JavaScript?
- 187. What is event-driven architecture and how can it be implemented in JavaScript?
- 188. Explain memoization strategies and cache invalidation techniques
- 189. What is reactive programming and how does it differ from imperative programming?
- 190. How does decorator syntax enhance class behavior?
- 191. What are import assertions and how do they ensure module type safety?
- 192. What is module federation and how does it enable micro-frontend architectures?
- 193. What are WeakMap-based private fields and how do they differ from native private fields ()?
- 194. How does lazy vs eager evaluation affect performance in iterables?
- 195. What are WeakKeys and WeakRefs — new memory-safe references?
- 196. How do structuredClone, postMessage, and transferable objects relate?

Difference between var, let, and const

One-line answer: `var` is function-scoped and hoisted to `undefined`; `let`/`const` are block-scoped, hoisted but uninitialized (TDZ). `let` can be reassigned, `const` cannot (reference only).

Why this matters (beginner-friendly)

Choosing the right declaration prevents scope leaks, redeclaration bugs, and subtle hoisting issues—especially in loops and async callbacks.

The concept in depth

- **Scope:** `var` → function scope; `let`/`const` → block scope (`{ }`).
- **Hoisting:** All three are hoisted; only `var` is initialized to `undefined`. `let`/`const` live in the **Temporal Dead Zone** until their declaration line executes.
- **Redeclaration:** `var` allows redeclaration in the same scope; `let`/`const` do not.
- **Reassignment:** `var` and `let` allow reassignment; `const` does not. For objects/arrays, the **binding** is constant but properties/elements may change.

Code examples

```
``js
// Hoisting difference
console.log(a); // undefined (var hoisted and initialized)
var a = 5;

console.log(b); // ReferenceError (TDZ)
let b = 5;

console.log(c); // ReferenceError (TDZ)
const c = 5;
```

``

```
``js
// Block scope vs function scope
for (var i = 0; i < 3; i++) {
  {
    setTimeout(() => console.log("var i:", i), 0);
  }
}
```

```

    }
  }
  // var i: 3,3,3

  for (let j = 0; j < 3; j++) {
    {
      setTimeout(() => console.log("let j:", j), 0);
    }
  }
  // let j: 0,1,2

```

```

```js

```

// const: immutable binding, mutable object
const user = {{ name: "Alice" }};
user.name = "Bob";    // ✓ allowed
// user = {{}}        // ✗ TypeError: Assignment to constant
// variable

```

```

### Common pitfalls & misconceptions

- Thinking `const` makes the **object** immutable (it only freezes the **binding**).
- Using `var` in loops with async callbacks causes late binding bugs.
- Assuming `let`/`const` aren't hoisted—they are, but remain uninitialized (TDZ).

### Internal mechanics (how JS engines/spec handle this)

- Declarations form **bindings** in the current **Lexical Environment**.
- `var` creates a property on the **Variable Environment** and is set to `undefined` at environment creation.
- `let`/`const` bindings exist but remain **uninitialized** until the declaration executes; access before that throws **ReferenceError**.

### When to use / avoid

- Prefer `const` by default; use `let` when you know the value will change.
- Avoid `var` in modern code unless maintaining legacy code.

## Related topics

- Hoisting, TDZ, Scope & Closures, Binding of `this`, Function vs Block scope.

## Practice questions

1. What happens when you access a `let` variable before its declaration?
2. Why can you modify a `const` object's properties but not reassign it?
3. Rewrite the loop below to log 0, 1, 2:

```
```js
```

```
for (var i = 0; i < 3; i++) {  
  {  
    setTimeout(() => console.log(i), 100);  
  }  
}
```

```
```
```

# What is hoisting in JavaScript?

## Hoisting in JavaScript

When JavaScript executes your code, it first performs a setup step where it registers all the variables and functions it can find. This behavior is known as hoisting. Because of hoisting, some variables and functions seem to exist even before the line where they are defined. However, different declarations are hoisted in different ways.

Function declarations are available early and can be used anywhere in their scope. Variables declared using `var` are also hoisted but start with a value of `undefined`. Variables declared with `let`, `const`, or `class` are hoisted too, but they are left uninitialized. If you try to access them before their declaration line, JavaScript throws a `ReferenceError`. This situation is known as the Temporal Dead Zone (TDZ).

Think of JavaScript running your file in two passes inside an execution context. In the first pass, called the creation phase, JavaScript scans your code and sets up memory for all variables and functions. It does not execute any statements yet; it only prepares the environment. In the second pass, called the execution phase, JavaScript runs the code line by line and assigns actual values to the variables or executes functions.

During the creation phase, JavaScript performs the following steps.

### 1. Function declarations

```
```js
```

```
function greet() {  
  console.log("hi");  
}
```

```
```
```

The entire function is stored in memory during setup, which means you can call `greet` anywhere in the same scope, even before its line in the code.

### 2. var variables

```
```js
```

```
var a = 10;
```

```
```
```



The name `a` is created and initialized with `undefined` during the setup phase. The actual assignment (`= 10`) happens later in the execution phase.

### 3. `let`, `const`, and `class`

```
```\njs\nlet x = 1;\nconst y = 2;\nclass Person {}\n```\n
```

These names are created but left uninitialized. Any attempt to read them before their declaration line results in a `ReferenceError`. This waiting period is known as the Temporal Dead Zone.

When the execution phase begins, JavaScript runs your code from top to bottom. When it reaches `var a = 10`, it assigns 10 to the already existing `a`. When it reaches `let x = 1` or `const y = 2`, it initializes them for the first time, exiting the TDZ. Function declarations were already ready to use, so they can be called before their declaration lines.

The TDZ exists to prevent you from using variables before they are properly defined. Without it, many confusing bugs would occur where variables show `undefined` values unexpectedly.

Here are some examples that show how hoisting works in different scenarios.

Example 1: `var` vs `let` and `const`

```
```\njs\nconsole.log(a); // undefined (hoisted name + default value)\nvar a = 5;\n\nconsole.log(b); // ReferenceError (TDZ)\nlet b = 5;\n\nconsole.log(c); // ReferenceError (TDZ)\nconst c = 5;\n```\n
```

#### Example 2: Function declaration vs function expression vs arrow function

```

``js
sayHi(); // works (function declaration is hoisted fully)
function sayHi() {
 console.log("Hi!");
}

try {
 sayHello();
} catch (e) {
 console.log(e.name);
} // TypeError or ReferenceError
var sayHello = function () {
 console.log("Hello!");
};

try {
 wave();
} catch (e) {
 console.log(e.name);
} // ReferenceError (TDZ)
let wave = () => console.log("Wave!");
``

```

With var, the variable sayHello exists during setup and has the value undefined until its assignment line. Function expressions and arrow functions behave like variable assignments and are not ready until the line is executed. With let or const, the variable exists but is uninitialized until that line is reached.

Example 3: class behaves like let and const (TDZ)

```

``js
try {
 new Person();
} catch (e) {
 console.log(e.name);
} // ReferenceError (TDZ)
class Person {}
``

```

To visualize how hoisting works, imagine time flowing from top to bottom in your code.

...

CREATION PHASE (Setup, before running your lines)

- function greet -> ready (you can call it)
- var a -> exists with value undefined
- let b -> exists but uninitialized (TDZ)
- const c -> exists but uninitialized (TDZ)

EXECUTION PHASE (Run your lines)

- 1) console.log(a) -> prints undefined
- 2) var a = 10 -> assigns 10 to a
- 3) console.log(b) -> ReferenceError (still in TDZ)
- 4) let b = 20 -> initializes b to 20 (TDZ ends)
- 5) greet() -> works (declaration was ready)
- 6) const c = 30 -> initializes c to 30 (TDZ ends)

...

There are several common misconceptions about hoisting. Some people think let and const are not hoisted. In reality, they are hoisted, but they remain uninitialized until their declaration line. Others think hoisting moves code to the top. That is not true; the JavaScript engine only creates memory bindings for variables and functions during setup. Another misconception is that function expressions are hoisted like function declarations. Only function declarations are hoisted with their body ready. Function expressions or arrow functions depend on whether the variable is declared using var, let, or const. Finally, var is sometimes used in loops with asynchronous code, which often causes bugs because all iterations share the same variable. It is better to use let in such cases.

There are also some subtle edge cases you may encounter.

Redeclaring with var

```
``js
var x = 1;
var x = 2; // allowed
console.log(x); // 2
``
```

var allows redeclaration in the same scope, which can lead to unexpected results.

TDZ with default parameters referencing later bindings

```
``js
let y = 1;
function f(a = y) {
 return a;
}
console.log(f()); // 1
```

```
function g(a = z) {
 return a;
}
let z = 2;
``
```

In this case, you cannot read `z` in the default parameter of `g` until `z` has been initialized.

Modules and hoisting behave slightly differently. ES modules always run in strict mode and have their own rules. Imports are hoisted and must appear at the top level of the file. They are ready before the module code runs. If you use top-level `await`, it pauses the module's execution until the awaited promise is resolved.

In practice, it is best to use `const` by default and `let` only when you need to change a variable's value. Avoid using `var` in modern JavaScript because it can lead to confusing behavior. Use function declarations when you need to call functions from anywhere in the scope, and use function expressions or arrow functions when you want them to be created at a specific time in execution.

Here is a quick reference summary.

| Declaration                    | Hoisted? | Value during setup  | Early access     |
|--------------------------------|----------|---------------------|------------------|
| -----                          | -----    | -----               | -----            |
| <code>function foo() {}</code> | Yes      | callable            | Works            |
| <code>var x</code>             | Yes      | undefined           | Prints undefined |
| <code>let x</code>             | Yes      | uninitialized (TDZ) | ReferenceError   |
| <code>const x</code>           | Yes      | uninitialized (TDZ) | ReferenceError   |
| <code>class A {}</code>        | Yes      | uninitialized (TDZ) | ReferenceError   |

Finally, here are some practice questions to test your understanding.

1. Why does `console.log(a)` show undefined but `console.log(b)` throws a `ReferenceError` before their declarations if `a` is declared with `var` and `b` is declared with `let`?
2. Predict the output of the following code.

```
```\njs\nsay();\nvar say = function () {\n  console.log("hi");\n};\n```\n
```

3. Fix the loop below so that it prints 0, 1, 2.

```
```\njs\nfor (var i = 0; i < 3; i++) {\n  setTimeout(() => console.log(i), 0);\n}\n```\n
```

You can fix it using `let`:

```
```\njs\nfor (let i = 0; i < 3; i++) {\n  setTimeout(() => console.log(i), 0);\n}\n```\n
```

Or by capturing `i`:

```
```\njs\nfor (var i = 0; i < 3; i++) {\n  ((iCopy) => setTimeout(() => console.log(iCopy), 0))(i);\n}\n```\n
```

## Difference between == and ===

Difference between == and === in JavaScript

In JavaScript, both the double equals (==) and triple equals (===) are comparison operators used to check if two values are equal, but they work differently. The difference lies in whether they perform type conversion before comparing.

The double equals operator (==) checks for equality after converting both values to a common type. This process is called type coercion. JavaScript tries to make both sides the same type before comparing, which can sometimes produce unexpected results. The triple equals operator (===) checks for equality without converting types. It compares both value and data type exactly as they are.

Because of this, == is called the loose equality operator, and === is called the strict equality operator.

When you use ==, JavaScript may convert strings, numbers, booleans, or even null and undefined to make the comparison possible. This can make your code unpredictable if you are not aware of how coercion works.

For example:

```
``js
console.log(5 == "5"); // true, because "5" is converted to a number
console.log(0 == false); // true, because false is converted to 0
console.log(null == undefined); // true, special rule in JS
console.log("0" == false); // true, because both sides are converted to 0
``
```

All these return true even though the data types are different. This happens because the == operator tries to make the values the same type before comparing.

Now, let's look at the === operator.

```
``js
console.log(5 === "5"); // false, number vs string
console.log(0 === false); // false, number vs boolean
console.log(null === undefined); // false, different types
console.log("0" === false); // false, string vs boolean
``
```

With `===`, no type conversion happens. JavaScript checks both the type and the actual value. If either differs, the result is false.

To understand this better, you can think of `===` as a stricter form of comparison. It is like asking, "Are these two values exactly the same, including their type?" On the other hand, `==` is like asking, "Can these two values be considered the same after some conversion?"

When comparing objects or arrays, both operators behave the same way. They only check if both sides refer to the exact same object in memory.

```
``js
let a = [1, 2];
let b = [1, 2];
console.log(a == b); // false
console.log(a === b); // false

let c = a;
console.log(a === c); // true, because both point to the same object
``
```

### Common misconceptions

1. Some developers believe `==` and `===` are interchangeable. They are not. Using `==` can cause unexpected true or false results because of type coercion.
2. `null` and `undefined` are equal with `==` but not with `===`. This can lead to subtle bugs if you are checking for missing values.
3. Comparing objects with `==` or `===` never compares their contents, only their references.
4. Using `==` for numeric comparisons with strings or booleans can create unpredictable results, especially when data comes from user input or APIs.

### Practical examples

```
``js
console.log(1 == true); // true, true becomes 1
console.log(1 === true); // false, number vs boolean

console.log("10" == 10); // true, "10" becomes 10
console.log("10" === 10); // false, string vs number
``
```

```
console.log("" == false); // true, both convert to 0
console.log("" === false); // false, string vs boolean
...

```

As you can see, `==` tries to help by converting types, but this often causes confusion. That is why most developers use `===` by default.

### Best practices

Always use `===` unless you specifically want type coercion. The strict equality operator makes your code more predictable and easier to debug. If you really need to compare values with different types, convert them manually using functions like `Number()`, `String()`, or `Boolean()` before comparing.

```
```js
console.log(Number("5") === 5); // true, both are numbers now
...

```

Quick reference summary

Expression	Result	Explanation
5 == "5"	true	"5" is converted to number 5
5 === "5"	false	different types
0 == false	true	false converts to 0
0 === false	false	number vs boolean
null == undefined	true	special rule in JS
null === undefined	false	different types
NaN == NaN	false	NaN is never equal to anything
NaN === NaN	false	same reason

Practice questions

1. What is the key difference between `==` and `===` in JavaScript?
2. Predict the output of:

```
```js
console.log("" == 0);
console.log("" === 0);

```



'''

3. Why does `null == undefined` return true but `null === undefined` return false?
4. How would you safely compare a string number like "10" with an actual number without unexpected results?

# What is scope (global, function, block)?

What is scope in JavaScript (global, function, block)

Scope is the area of a program where a variable or function name is visible and can be accessed. Understanding scope helps you predict where a value can be read or changed and prevents accidental name collisions. In JavaScript there are three primary kinds of scope you will use every day: global scope, function scope, and block scope. There is also module scope in ES modules, which behaves like file level scope, but this note focuses on the everyday three.

Global scope means a name is available everywhere in your program after it is defined. In a browser, global variables become properties of the window object. In Node.js, the global object is different, and variables declared with `let` or `const` at the top level of a module are not added to the global object. Global scope is convenient but dangerous because any part of the program can read or change that value. Prefer limiting scope where possible.

Function scope is created each time a function is called. Variables declared with `var` inside a function are visible anywhere inside that function but not outside it. Because `var` is function scoped, using it inside `if` or `for` blocks does not limit its visibility to those blocks; it remains available everywhere in the function.

Block scope is created by a pair of braces. The most common blocks are those from `if`, `for`, `while`, `try`, and just plain braces. Variables declared with `let` or `const` are block scoped. They exist only inside that block and are not accessible outside. This prevents accidental leaks and makes code easier to reason about. Classes declared with `class` also follow block scoping rules. Variables in block scope have a temporal dead zone before the declaration line, which means you cannot access them before their declaration executes.

Shadowing happens when an inner scope declares a name that already exists in an outer scope. Inside the inner scope, the new declaration hides the outer one. Shadowing can be useful for clarity when parameter names repeat, but overusing it can reduce readability.

Lexical scope means that JavaScript decides which variables are visible based on where functions and blocks are written in the code, not based on where they are called at runtime. A function can access variables from the scope where it was defined. This idea is the foundation for closures.

Examples

```
``js
// global scope example
let siteName = "Docs";
function show() {
```

```

 console.log(siteName); // can read global
 }
 show();
 ...

``js
// function scope with var
function demo() {
 var x = 1;
 if (true) {
 var x = 2; // same function-scoped variable
 }
 console.log(x); // 2
}
demo();
...

```

```

``js
// block scope with let and const
function demo2() {
 let a = 1;
 if (true) {
 let a = 2; // different, block-scoped variable
 const b = 3;
 console.log(a, b); // 2 3
 }
 console.log(a); // 1
 // console.log(b); // ReferenceError
}
demo2();
...

```

```

``js
// shadowing and lexical scope
const value = "outer";
function make() {
 const value = "inner";
 return function () {

```

```
 console.log(value); // reads "inner" because of lexical scope
 };
}
make();
...
```

### Common misconceptions

1. Variables declared with `var` in a block are block scoped. In reality, `var` is function scoped and leaks out of the block.
2. Global variables declared with `let` or `const` become properties on `window`. In browsers only `var` at the top level does that; `let` and `const` at top level of a script create global bindings but not `window` properties, and in modules they are module scoped.
3. A variable defined inside a function can be accessed after the function returns. This is only possible through closures, not by direct access.

### Practice questions

1. Explain the difference between function scope and block scope using a short code example.
2. Why is global scope considered risky, and how can you reduce reliance on it in a large application.
3. What is shadowing, and when could it reduce readability.

# What are closures?

## What are closures

A closure is one of the most important - and often misunderstood - ideas in JavaScript.

In simple words, a closure is a function that carries a memory of the environment in which it was created. It remembers the variables that were around it at the time it was defined, even after that outer code has finished executing.

To understand why this happens, you first need to recall that JavaScript uses lexical scope - meaning variable visibility is determined by where a function is written in the code, not by where it is called later.

Because of lexical scope, an inner function automatically "knows" about the variables defined in its outer function.

### 1. The backpack analogy - how to imagine a closure

Imagine every function in JavaScript carries an invisible backpack with it.

When the function is created, JavaScript puts into this backpack all the variables that were in scope at that time - the things the function can "see" from where it was written.

When the outer function finishes running, most of its local variables normally disappear from memory. But if an inner function still references them, those variables stay alive - kept safely inside the backpack.

Whenever the inner function is called later (even long after the outer function is gone), it can still open that backpack and find the variables it remembers.

This is a closure in action: the combination of the function and the preserved environment around it.

### 2. A simple example

```
```\njs\nfunction makeCounter() {\n  let count = 0; // this variable lives in makeCounter's scope\n\n  return function () {\n    count++; // inner function can still access count\n    return count;\n  };\n}
```

```
const counter = makeCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
...
```

What happens here:

makeCounter finishes running after the first call.

Normally, its local variable count would disappear.

But the inner function still "remembers" count through its closure - it keeps it alive inside its backpack.

Every time you call counter(), it finds count in that backpack, updates it, and returns the new value.

Even though makeCounter no longer exists in memory as a running function, the variable count remains because the closure is still holding onto it.

3. Closures keep references, not copies

It's very important to understand that closures don't store copies of variables - they store references. That means if a variable's value changes after the closure is created, the closure will see the updated value.

```
``js
function outer() {
  let message = "Hello";
  return function inner() {
    console.log(message);
  };
}
```

```
const greet = outer();
message = "Hi"; // changing outer variable (if accessible)
greet(); // prints "Hello" only if outer scope variable remains unchanged, otherwise references
update dynamically
...
```

Closures are "live links" to variables, not frozen snapshots.

That's why they're so powerful - they can reflect changes over time.

4. Closures allow data privacy

Before ES6 introduced classes with private fields, closures were the main way to create private data in JavaScript - variables that can't be accessed from outside but are still remembered internally.

```
``js
function createAccount() {
  let balance = 0;

  return {
    deposit(amount) {
      balance += amount;
      console.log("Deposited:", amount);
    },
    getBalance() {
      return balance;
    },
  };
}

const account = createAccount();
account.deposit(100);
console.log(account.getBalance()); // 100
console.log(account.balance); // undefined - cannot access directly
...

```

Here, balance acts like a private variable.

It lives in the closure of the functions returned by createAccount, and can't be read or modified except through those inner functions.

5. Closures in asynchronous code

Closures often appear naturally in asynchronous operations like setTimeout, Promise, or event listeners.

The inner function runs later, but still remembers the variables from when it was defined.

```

```js
function greet(name) {
 setTimeout(function () {
 console.log("Hello, " + name);
 }, 1000);
}

greet("Alice");
```

```

Even though greet finishes before one second passes, the callback inside setTimeout still knows what name was - because it carries a closure around it.

6. Real-world uses of closures

Closures aren't just theoretical - they power many common programming patterns:

Data encapsulation: hiding internal state, like private variables.

Function factories: creating multiple customized versions of a function.

Memoization: remembering results for faster future calculations.

Event handlers and callbacks: retaining access to state across time.

Modules: grouping related code with private internal data.

Example - function factory:

```

```js
function multiplier(factor) {
 return function (n) {
 return n * factor;
 };
}

const double = multiplier(2);
const triple = multiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15
```

```


Each inner function remembers its own factor - carried in its own backpack from when it was created.

7. Memory and performance considerations

Closures keep their referenced variables alive in memory as long as they are reachable.

If you store many closures or forget to release them when no longer needed, those variables won't be garbage-collected.

That's not a bug - it's just how JavaScript ensures your closures keep working.

In most normal cases, the runtime cleans up automatically once closures are no longer used.

8. Summary

A closure is the combination of:

a function, and the lexical environment where that function was created.

Or in simpler words:

A closure is a function plus its backpack of remembered variables.

Closures make JavaScript functions powerful and flexible, letting you preserve state, hide data, and control behavior across time - all thanks to lexical scoping and how JavaScript keeps those "backpacks" alive even after the original context is gone.

Examples

```
``js
// private state counter
function makeCounter() {
  let count = 0;
  return function () {
    count += 1;
    return count;
  };
}
const c1 = makeCounter();
console.log(c1()); // 1
console.log(c1()); // 2
``
```

```

``js
// function factory
function greeter(greeting) {
  return function (name) {
    return greeting + ", " + name;
  };
}
const hello = greeter("Hello");
console.log(hello("Sam")); // Hello, Sam
``

```

```

``js
// async with closure
function delayedLog(msg) {
  for (let i = 1; i <= 3; i++) {
    setTimeout(() => console.log(msg, i), i * 100);
  }
}
delayedLog("Step"); // Step 1, Step 2, Step 3
``

```

Common misconceptions

1. Closures copy values. They actually keep references to variables, so you observe updated values, not frozen snapshots.
2. Closures always cause memory leaks. They only keep what is referenced. If nothing references the inner function, the closure can be collected.
3. Using `var` in loops with callbacks works the same as `let`. With `var`, each iteration shares the same variable, which surprises many developers.

Practice questions

1. Implement a once utility so a function runs at most one time and returns the same result on later calls.
2. Write a memoize function that caches results by argument for a pure function.
3. Explain why `var` often behaves unexpectedly in loops with asynchronous callbacks and show a fix.

What is the event loop?

What is the event loop

The event loop is the coordination system that allows JavaScript to appear concurrent, even though it runs on a single thread. It manages what code runs now, what runs next, and when the browser can repaint the screen. Every JavaScript environment, like a browser or Node.js, includes an event loop, though their specific queues and priorities may differ slightly.

When JavaScript first starts running a script, it executes all synchronous code line by line on the call stack. This is the main thread of execution. When asynchronous operations like timers, fetch calls, or user interactions occur, they register callbacks to be executed later, once their operation finishes. Those callbacks are stored in different types of queues.

The event loop constantly checks two key conditions:

Is the call stack empty?

Are there any tasks waiting in the queues?

If the stack is empty, the event loop picks the next appropriate task and pushes it onto the stack to execute.

There are two main kinds of tasks the event loop manages — macrotasks and microtasks — and understanding their order is crucial.

Macrotasks (often just called tasks) include things like `setTimeout`, `setInterval`, `setImmediate` (in Node.js), I/O callbacks, and events such as clicks or network responses. Microtasks include `promise.then()` callbacks, `async/await` continuations, and functions queued using `queueMicrotask`. Microtasks always run before the event loop moves to the next macrotask.

Here's the exact sequence:

The event loop takes one macrotask from the macrotask queue (for example, the initial script or a `setTimeout` callback) and executes it fully, top to bottom.

When that macrotask completes, the event loop checks the microtask queue.

All microtasks in the queue are executed, one after another, until the microtask queue is completely empty.

Once the microtasks finish, the browser gets a chance to perform rendering or painting if needed.

Then, the loop picks the next macrotask and repeats the same steps.

Because microtasks are processed right after each macrotask and before rendering, promise `.then()` callbacks can run sooner than you might expect. For example, if you schedule both a promise and a `setTimeout` with zero delay, the promise handler will always run first because it is a microtask, and microtasks are drained before the next macrotask starts.

Consider this example:

```
``js
console.log("Start");

setTimeout(() => console.log("Timeout callback"), 0);

Promise.resolve().then(() => console.log("Promise microtask"));

console.log("End");
``
```

The output order is:

Start

End

Promise microtask

Timeout callback

This happens because the main script itself is a macrotask. After it finishes, the event loop looks at the microtask queue. The promise `.then()` is there, so it runs before the next macrotask (the `setTimeout` callback).

If you add more microtasks while executing microtasks, those newly added ones are also run before the event loop returns to macrotasks. This is why microtasks can "chain" indefinitely if they keep scheduling more microtasks.

Another important detail is rendering. The browser does not repaint the screen in the middle of microtasks. Rendering occurs only after the microtask queue is empty and before starting the next macrotask. That's why promise-heavy operations can block the visual update even though they seem asynchronous.

In Node.js, the terminology differs slightly, but the logic is similar. Node has phases like timers, pending callbacks, I/O polling, and a microtask queue that behaves like the browser's.

To summarize the correct order clearly:

Run one macrotask (e.g., a piece of code, setTimeout callback, I/O event).

When that macrotask finishes, run all queued microtasks.

If more microtasks appear during this step, keep running them until none remain.

Let the browser render updates.

Move to the next macrotask.

Repeat indefinitely.

This precise sequence ensures that JavaScript remains non-blocking, responsive, and predictable despite running in a single thread.

Examples

```
```js
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// Order: A, D, C, B
```
```

```
```js
// microtasks drain before the next macrotask
setTimeout(() => console.log("timeout"), 0);
Promise.resolve().then(() => console.log("then-1"));
Promise.resolve().then(() => console.log("then-2"));
// then-1, then-2, timeout
```
```

```
``js
// queueMicrotask behaves like a promise microtask
queueMicrotask(() => console.log("microtask"));
console.log("sync");
// sync, microtask
``
```

Common misconceptions

1. `setTimeout` with zero delay runs immediately. It schedules a task that runs only after the current call stack is empty and microtasks have run.
2. Promises are faster by themselves. It is not about speed but about queue priority. Promise callbacks run in the microtask queue which is processed before the next task.
3. The event loop belongs to JavaScript the language. The event loop is provided by the host environment (browsers, Node.js) which integrates timers, I/O, and rendering.

Practice questions

1. Predict the output order of logs when both `setTimeout` and `Promise.then` are used together.
2. Explain why a long running while loop blocks click handlers from running.
3. Show how to yield back to the event loop to keep the UI responsive during heavy computation.

Explain call stack and execution context

Explain call stack and execution context

The call stack is a structure that tracks what function is currently running and which function to return to when it finishes. Each time a function is called, the runtime creates a new frame and pushes it onto the stack. When the function returns, its frame is popped. If the stack grows too deep through unbounded recursion, a stack overflow error occurs.

An execution context is the environment in which a piece of JavaScript runs. It includes the scope chain, the bindings for variables and functions, the value of `this`, and references to outer environments. There is a global execution context created when the script starts. Each function call creates a new function execution context. During function creation, JavaScript also records the lexical environment that will be used later when the function runs; this makes closures possible.

Creation and execution happen in two phases for each context. In the creation phase, the engine allocates memory for declarations and sets up the scope and this binding. In the execution phase, the code runs line by line, reading and writing variables and calling other functions. When a function calls another function, a new context is created and pushed onto the call stack above the current one.

Examples

```
``js
function a() {
  console.log("in a");
  b();
  console.log("back to a");
}
function b() {
  console.log("in b");
}
a();
// Stack behavior: enter a, enter b, exit b, back to a, exit a
``
```

```
``js
// two-phase model inside a function
function sum(x, y) {
  // creation phase sets up x, y, and the environment
  return x + y; // execution phase reads values and returns
}
console.log(sum(2, 3));
```



```
...  
  
```js  
// overflow example (do not run in production)
function recur() {
 return recur();
}
// recur(); // RangeError: Maximum call stack size exceeded
...`
```

### Common misconceptions

1. The call stack shows asynchronous callbacks waiting. Only running frames are on the stack. Asynchronous callbacks wait in queues until picked up.
2. The value of `this` is the same in every function. It depends on how the function is called, not just where it is defined.
3. Execution context and scope are the same thing. Scope is part of the execution context, which includes additional details like `this` and the outer environment.

### Practice questions

1. Describe what happens on the call stack when a function calls another function that then throws an error.
2. Explain how execution context creation and execution phases relate to hoisting.
3. Why does unbounded recursion cause a stack overflow, and how can you avoid it.

# What are data types in JavaScript

## What are data types in JavaScript

JavaScript has a small set of built in types. These types define how values behave, how they compare to each other, and what operations are valid. There are primitive types and reference types. Primitive types are immutable and compared by value. Reference types are objects and compared by reference.

The primitive types are number, string, boolean, null, undefined, symbol, and bigint. Number represents both integers and floating point values including special values like NaN and Infinity. String is a sequence of characters. Boolean represents true or false. Null represents an intentional empty value. Undefined means a variable has been declared but not assigned a value. Symbol creates unique identifiers useful for object keys that should not collide. BigInt represents integers of arbitrary size beyond the safe range of number.

Objects are collections of key value pairs and include plain objects, arrays, functions, dates, and many other built in structures. Functions are callable objects. Arrays are ordered collections with a length property and numeric indices. Most values you create with curly braces or constructors are objects and are passed by reference.

Type inspection can be done with typeof, Array.isArray, and other utilities. typeof works reliably for primitives except that typeof null historically returns object for legacy reasons. Arrays report typeof object, so use Array.isArray to detect them. For class instances, you can use instanceof to check prototype relationships.

## Examples

```
``js
console.log(typeof 42); // "number"
console.log(typeof "hi"); // "string"
console.log(typeof true); // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof Symbol("s")); // "symbol"
console.log(typeof 10n); // "bigint"
console.log(typeof null); // "object" (legacy quirk)
console.log(typeof {}); // "object"
console.log(Array.isArray([])); // true
...

```

```
``js
// objects by reference
const a = { x: 1 };
const b = a;

```

```
b.x = 2;
console.log(a.x); // 2
...

``js
// numbers and string conversions
console.log(Number("10")); // 10
console.log(String(99)); // "99"
...

```

### Common misconceptions

1. Null and undefined mean the same thing. Null is an intentional empty value; undefined means not assigned yet.
2. Arrays are a separate typeof result. Arrays are objects; use Array.isArray to detect them.
3. BigInt and number can be mixed freely. You cannot mix them directly in arithmetic without explicit conversion.

### Practice questions

1. List all primitive types and describe a use case for symbol and bigint.
2. Why does typeof null return object, and how do you reliably test for null.
3. When would you prefer an array over an object and why.

## Difference between primitive and reference types

### Difference between primitive and reference types

Primitive types are number, string, boolean, null, undefined, symbol, and bigint. These values are immutable and compared by value. When you assign a primitive to a new variable or pass it to a function, a copy of the value is made. Changing the new variable does not affect the original.

Reference types are objects, including arrays and functions. These values are stored by reference. When you assign an object to another variable, both variables refer to the same underlying object. Changing a property through one reference is visible through the other. Equality comparisons for objects check whether two references point to the same object, not whether their contents are equal.

Stack and heap are common mental models. Engines are free to implement however they like, but the model is useful: primitives are often small fixed size values and can be copied easily, while objects may live in managed memory with garbage collection and are accessed by references.

### Examples

```
```js
// primitives copy by value
let a = 5;
let b = a;
b = 7;
console.log(a, b); // 5 7
```
```

```
```js
// objects copy by reference
const p = { n: 1 };
const q = p;
q.n = 2;
console.log(p.n); // 2
```
```

```
```js
// equality
console.log({} === {}); // false, different objects
const r = {};
const s = r;
console.log(r === s); // true, same reference
```
```

```
``js
// shallow copy vs deep copy
const user = { name: "A", meta: { views: 1 } };
const shallow = { ...user }; // shallow copy
shallow.meta.views = 5;
console.log(user.meta.views); // 5
``
```

### Common misconceptions

1. Objects are passed by reference. JavaScript passes arguments by value; the value for objects is a reference. Reassigning the parameter does not change the caller's variable, but mutating the object does.
2. Spreading or `Object.assign` always creates a deep copy. They only copy one level by default.
3. Two objects with the same properties are equal with `===`. They are equal only if they are the same reference.

### Practice questions

1. Show how to deep clone a nested object without mutating the original using `structuredClone` or a library.
2. Explain why a function cannot reassign an object parameter to replace the caller's variable but can mutate its properties.
3. Give an example where a shallow copy causes an unexpected mutation in the original object.

# What is type coercion

## What is type coercion

Type coercion is when JavaScript converts a value from one type to another so that an operation can proceed. Coercion happens in two main ways: implicit and explicit. Implicit coercion occurs when operators or comparisons convert values automatically. Explicit coercion is when you convert values directly using Number, String, Boolean, or other APIs.

The language defines conversion rules for many operations. For arithmetic with the plus operator, if either operand is a string and the other is not an object with a special behavior, JavaScript converts the other operand to a string and concatenates. For subtraction, multiplication, and division, JavaScript converts operands to numbers. For comparisons with double equals, the runtime attempts to convert both sides to a common type, which can lead to results that surprise people who are not aware of the rules.

Objects convert to primitives by trying `valueOf` and `toString` in a specific order depending on the operation. Symbols do not coerce to strings implicitly to avoid accidental leaks. Bigints do not mix with numbers without explicit conversion.

## Examples

```
``js
// string concatenation vs numeric addition
console.log("10" + 5); // "15"
console.log(10 + "5"); // "15"
console.log(10 - "5"); // 5
console.log("10" * "2"); // 20
...

```

```
``js
// explicit coercion
console.log(Number("12")); // 12
console.log(String(7)); // "7"
console.log(Boolean("")); // false
console.log(Boolean("hi")); // true
...

```

```
``js
// equality coercion
console.log(0 == false); // true
console.log("" == 0); // true
console.log(null == undefined); // true

```

```

console.log("0" == false); // true
...

```js
// object to primitive
const price = {
  value: 1000,
  valueOf() { return this.value; }
};
console.log(price + 50); // 1050 (valueOf used)
...

```

Common misconceptions

1. Coercion is always bad. Coercion is a tool; confusion comes from not knowing the rules. Use strict equality to avoid surprises where needed.
2. The plus operator always adds numbers. If either side is a string, plus concatenates.
3. Boolean conversion treats any non empty string as true but the string "0" is also true. Only the empty string is false.

Practice questions

1. Show how different operators trigger coercion with examples for +, -, ==, and Boolean conversion.
2. Explain how an object with a custom valueOf can influence arithmetic results.
3. Why does 0 == false evaluate to true but 0 === false is false, and how would you make such comparisons safer.

Explain truthy and falsy values

Explain truthy and falsy values

In JavaScript, every value becomes either true or false when used in a boolean context. Truthy means a value is treated as true; falsy means it is treated as false. This conversion happens with if, while, for conditions, the ternary operator, logical operators like &&, ||, and !, and in APIs that expect a boolean.

There are exactly seven falsy values: false, 0, -0, 0n (BigInt zero), "" (empty string), null, undefined, and NaN. All other values are truthy, including objects, arrays, functions, non-zero numbers, non-empty strings, and symbols. Even unusual values like "0" (a string containing zero), "false" (a non-empty string), [] (empty array), and {} (empty object) are truthy.

JavaScript does not permanently convert the value; it only interprets it as true or false for that operation using an internal ToBoolean step. This is part of type coercion.

Examples

```
```js
if ("hello") console.log("truthy"); // non-empty string is truthy
if (0) console.log("never runs"); // 0 is falsy
if ([]) console.log("runs"); // empty array is truthy
if ({}) console.log("runs"); // empty object is truthy
if (null) console.log("no"); // null is falsy
```
```

```
```js
console.log(Boolean("")); // false
console.log(Boolean(" ")); // true (string with a space)
console.log(Boolean(123)); // true
console.log(Boolean(0)); // false
console.log(Boolean([])); // true
console.log(Boolean({})); // true
```
```

Common misconceptions

1. Empty arrays and empty objects are falsy. All objects are truthy, even empty ones.
2. The string "false" is falsy. Any non-empty string is truthy.
3. NaN behaves like 0 in truthiness. NaN is falsy.

Practice questions

1. List all falsy values in JavaScript and explain why "0" is truthy but 0 is falsy.
2. Explain why [] && {} returns {} but [] || {} returns [].
3. Predict the behavior of if ("0") and if (0) and explain the difference.

What is NaN and how do you check for it

What is NaN and how do you check for it

NaN stands for Not-a-Number. It is a special numeric value that represents the result of an invalid or undefined numeric operation. NaN has type of "number" because it lives in the number type domain, but it signals that a meaningful numeric value could not be produced.

You get NaN from operations like $0 / 0$, parsing non-numeric strings as numbers, square roots of negative numbers (in real arithmetic), or any arithmetic expression that already contains NaN. Once NaN is produced, it contaminates further arithmetic: any operation involving NaN typically yields NaN again.

Examples

```
```js
console.log(typeof NaN); // "number"
console.log(0 / 0); // NaN
console.log(Math.sqrt(-1)); // NaN
console.log(parseInt("abc")); // NaN
console.log(Number("12x")); // NaN
```
```

A unique property of NaN is that it is not equal to anything, including itself. Therefore `NaN === NaN` is false. This design preserves the idea that different invalid results should not compare equal.

To check for NaN, prefer `Number.isNaN(value)`. It returns true only when the value is actually the special NaN value. The older global `isNaN(value)` first coerces the argument to a number, so it can report true for non-numeric strings, which is often not what you want.

```
```js
console.log(NaN === NaN); // false
console.log(Number.isNaN(NaN)); // true
console.log(Number.isNaN("hello")); // false
console.log(isNaN("hello")); // true (coerces to number → NaN)
```
```

Common misconceptions

1. NaN means "not a numeric type". It is a number type value meaning "invalid numeric result".

2. `isNaN` and `Number.isNaN` are interchangeable. `isNaN` coerces; `Number.isNaN` is strict and safer.
3. You can detect NaN using equality or inequality operators. NaN never equals anything, not even itself.

Practice questions

1. Name three operations that can produce NaN and explain why.
2. Why does `NaN !== NaN` evaluate to true, and how do you correctly test for NaN?
3. Write a function `isNumeric(val)` that returns true only for finite numbers (hint: use `typeof`, `Number.isFinite`).

Explain 'this' keyword in JavaScript

Explain this keyword in JavaScript

In JavaScript, the `this` keyword is one of the most confusing topics for beginners because it behaves differently from how people expect in other languages. But once you understand what decides the value of `this` (the call site, not the definition site), it becomes consistent and logical.

Think of `this` as a reference to the current "owner" of the function call - the object that the function is being executed "on."

But JavaScript does not decide `this` when you write the function - it decides it each time the function is called, depending on how the function is called.

1. Global context

When you use `this` in the global scope (outside of any function), the value depends on the environment and mode.

In a browser, outside of strict mode, `this` refers to the global object - that's `window`.

In `Node.js`, it refers to an empty object in modules, not the global object.

In strict mode, `this` is undefined in global functions.

```
``js
console.log(this === window); // true (in browsers, non-strict mode)
``
```

If you add `"use strict";` at the top, then inside a normal function that isn't attached to an object:

```
``js
"use strict";
function show() {
  console.log(this);
}
show(); // undefined
``
```

So the first rule is:

In normal (non-strict) mode, `this` defaults to the global object.

In strict mode, it becomes undefined.

2. Inside methods (object functions)

If a function is called as a property of an object, `this` points to that object.

```
``js
```

```
const user = {
  name: "Alice",
  greet() {
    console.log("Hi, I'm " + this.name);
  },
};
user.greet(); // "Hi, I'm Alice"
```

```

Here, this refers to user because the call was user.greet().

The function doesn't care where it was defined — it only cares about how it was called.

If you separate the function from the object, the connection is lost:

```
const greetFn = user.greet;
greetFn(); // undefined or global object, depending on strict mode
Now this is no longer user, because it's just a normal function call.
```

### 3. Arrow functions and lexical this

Arrow functions are special — they do not have their own this.

Instead, they capture the this from the surrounding scope where they were defined.

That means whatever this was outside the arrow function will also be the value inside it.

This makes arrow functions perfect for callbacks and event handlers where you want to preserve the outer context.

```
```js
const team = {
  title: "Developers",
  listMembers() {
    setTimeout(() => {
      console.log("Team: " + this.title); // uses team's this
    }, 100);
  },
};
team.listMembers(); // "Team: Developers"
```
```

If we used a normal function instead of an arrow here, this inside the `setTimeout` callback would not refer to `team`.

#### 4. `call`, `apply`, and `bind` — setting this manually

JavaScript lets you manually decide what this should be when you call a function.

`call` executes the function immediately, with `this` set to the first argument.

`apply` does the same, but arguments are passed as an array.

`bind` creates a new function with a permanently fixed `this`.

```
``js
function sayHi() {
 console.log(this.name);
}

const user = { name: "John" };
const admin = { name: "Admin" };

sayHi.call(user); // "John"
sayHi.apply(admin); // "Admin"

const boundFn = sayHi.bind(user);
boundFn(); // "John" (always uses user)
...

```

This is useful when you need to control context explicitly, especially when passing functions as callbacks.

#### 5. Constructor calls (new binding)

When you call a function with the `new` keyword, JavaScript automatically:

Creates a new empty object.

Sets `this` inside the function to point to that object.

Runs the function body.

Returns the new object (unless the function returns a different object).

```
``js

```

```
function Person(name) {
 this.name = name;
}
const p = new Person("Alice");
console.log(p.name); // "Alice"
...

```

Here, this inside Person refers to the new instance being created.

If you forget new, this won't refer to the new object — it will follow the default rule instead, often causing bugs.

## 6. Event handlers

In browser event handlers (like onclick), this automatically refers to the element that received the event — unless you're using arrow functions, which do not have their own this.

```
...js
document.querySelector("button").onclick = function () {
 console.log(this); // the button element
};

document.querySelector("button").onclick = () => {
 console.log(this); // probably window, not the button
};
...

```

## 7. Binding priority

If multiple rules could apply, JavaScript uses this priority order:

new binding (constructor)

Explicit binding (call, apply, bind)

Implicit binding (object method call)

Default binding (global or undefined)

Example:

```
...js

```

```
function show() {
 console.log(this.value);
}
const obj = { value: "obj", show };

const bound = show.bind({ value: "bound" });
const instance = new bound(); // uses new binding, not bound one
...

```

Here, even though we bound the function, new takes precedence and creates a new this.

## 8. How to think about this

Don't memorize every case. Instead, ask:

"How is the function being called?"

That single question reveals what this will be.

If it's called with new, this is the new object.

If it's called with obj.method(), this is obj.

If it's called with call or apply, this is whatever you pass in.

Otherwise (plain function), it's undefined in strict mode or global in non-strict.

If it's an arrow function, this is inherited from its surrounding scope.

## Examples

```
```js
function show() {
  console.log(this);
}
show(); // undefined in strict mode; window in sloppy mode
...

```

```
```js
const user = {
 name: "Sam",
 greet() {
 console.log(this.name);
 },
}

```

```

};
user.greet(); // "Sam"

const fn = user.greet;
fn(); // undefined or window depending on mode
...

```js
const obj = {
  name: "A",
  printLater: function () {
    setTimeout(() => console.log(this.name), 50); // arrow keeps outer this
  },
};
obj.printLater(); // "A"
...

```js
function hi() {
 console.log(this.msg);
}
hi.call({ msg: "Hello" }); // "Hello"
...

```

### Common misconceptions

1. this points to the function itself. It points to the call-time receiver object, not the function.
2. Arrow functions take this from the caller. They capture this from where they are defined.
3. Binding once changes this forever. bind returns a new function with a fixed this; the original is unchanged.

### Practice questions

1. Explain the difference in this between arrow functions and regular functions with an example.
2. How would you preserve a method's this when passing it as a callback to an event listener?
3. What does new do to this inside a constructor function?





# Difference between implicit, explicit, default, and new binding

## Difference between implicit, explicit, default, and new binding

JavaScript resolves "this" using four main rules. Default binding applies when a function is called without a receiver object. In non-strict mode, this becomes the global object; in strict mode, this is undefined. Implicit binding applies when you call a function as a property of an object; the object to the left of the dot becomes this. Explicit binding uses call, apply, or bind to set this to a specific object regardless of how the function is invoked. New binding applies when you use new; it creates a fresh object, sets it as this inside the constructor, and returns it unless the constructor returns an object explicitly.

When multiple rules could apply, the priority is: new binding first, then explicit binding, then implicit binding, then default binding.

### Examples

```
``js
function show() {
 console.log(this.name);
}

// default
show(); // undefined (strict) or global name (non-strict)

// implicit
const user = { name: "Sam", show };
user.show(); // "Sam"

// explicit
show.call({ name: "Alex" }); // "Alex"

// new
function Person(name) {
 this.name = name;
}
const p = new Person("Ravi");
console.log(p.name); // "Ravi"
```

...

### Common misconceptions

1. Arrow functions follow these rules. Arrow functions ignore all four and capture this lexically.
2. bind mutates the original function. It returns a new function with fixed this.
3. call or apply can override new. If new is used, new binding wins.

### Practice questions

1. Describe each binding rule and give a one-line example.
2. What happens if you call a bound function with new?
3. In which order are the rules considered when more than one might apply?

# What is lexical scope

## What is lexical scope

Lexical scope means variable visibility is determined by where code is written, not by where it is called. Functions and blocks create scopes. Inner scopes can access names from their outer scopes, but not the reverse. This structure is fixed at parse time and provides a predictable chain for name lookup at runtime.

When code runs, the engine looks for a variable starting in the current scope and then walks outward through enclosing scopes until it finds a match or reaches the global scope. This model enables closures, because a function keeps access to the scope where it was defined even after that outer function returns.

## Examples

```
``js
const outer = "outside";

function a() {
 const inner = "inside";
 console.log(outer); // accesses outer
}
a();
// console.log(inner); // ReferenceError
``
```

```
``js
function makeAdder(x) {
 return function (y) {
 return x + y; // x comes from lexical scope
 };
}
const add5 = makeAdder(5);
console.log(add5(2)); // 7
``
```

## Common misconceptions

1. Lexical scope changes based on the caller. It is determined at definition time.
2. Only functions create scope. Blocks with `let/const` also create scope.
3. Closures create new scopes out of thin air. They capture existing lexical environments.

#### Practice questions

1. Explain lexical scope versus dynamic scope.
2. Why does a function still access outer variables after the outer function returns?
3. Show how lexical scope enables private state.

# Explain function declarations vs function expressions

## Explain function declarations vs expressions

Function declarations and function expressions define functions but differ in when they become available and how they are used. A function declaration appears as a statement beginning with the function keyword. It is hoisted, which means the entire function is available before its line in the code. A function expression defines a function inside an expression, for example by assigning it to a variable. Expressions are not hoisted as callable functions; only the variable's binding is created early (undefined for var, temporal dead zone for let/const), and the function value is set at runtime when that line executes.

Function expressions may be anonymous or named. Named expressions can aid debugging and allow self-reference for recursion.

### Examples

```
``js
// declaration
greet();
function greet() {
 console.log("Hi");
}
...

```

```
``js
// expression
const sayHi = function () {
 console.log("Hello");
};
sayHi();
...

```

```
``js
// named expression for recursion
const factorial = function fact(n) {
 return n <= 1 ? 1 : n * fact(n - 1);
};
console.log(factorial(5)); // 120

```

...

### Common misconceptions

1. All functions are hoisted the same way. Only declarations are fully hoisted.
2. A named function expression leaks its name to the outer scope. The name is only visible inside the function body.
3. Declarations inside blocks behave uniformly across modes. Block scoping rules and strict mode can affect visibility.

### Practice questions

1. What happens if you call a function expression before it is defined?
2. When would you prefer a function expression over a declaration?
3. Why might you give a function expression a name even when assigning it to a variable?

# What is an IIFE (Immediately Invoked Function Expression)?

What is IIFE (Immediately Invoked Function Expression)

An IIFE is a function expression that is executed immediately after it is created. It is written by wrapping a function in parentheses to force it to be an expression, then adding another pair of parentheses to call it. The pattern creates a private scope for variables and avoids leaking names into the global scope. Before block scope with `let` and `const` was available, IIFEs were a primary way to create isolated scopes.

IIFEs are useful for one-time setup, initializing modules, and keeping temporary variables private. Arrow functions can be used as IIFEs too.

## Examples

```
``js
(function () {
 const message = "Runs now";
 console.log(message);
})();
``
```

```
``js
(function (name) {
 console.log("Hello " + name);
})("Sam");
``
```

```
``js
const result = (() => {
 const x = 2,
 y = 3;
 return x * y;
})();
console.log(result); // 6
``
```

## Common misconceptions



1. IIFEs are obsolete after ES6. They still help with one-off initialization and encapsulation.
2. Only function keyword works. Arrow IIFEs work as well.
3. IIFEs create globals. They prevent globals by scoping variables locally.

#### Practice questions

1. Write an IIFE that returns today's ISO date string.
2. Why were IIFEs common before `let` and `const`?
3. How does an IIFE differ from calling a named function declared elsewhere?

## 018. Explain pure functions and side effects

Explain pure functions and side effects

A pure function always produces the same output for the same input and does not cause any observable changes outside itself. It does not read or write global state, mutate its parameters, perform I/O, or rely on time or randomness unless those are passed in as inputs. Pure functions are predictable, easy to test, and simple to reason about.

A side effect in programming refers to any action a function performs that affects something outside itself, or depends on something that can change outside its control. It's called a "side" effect because it happens alongside the main purpose of the function — instead of just returning a value, the function is also changing the world around it.

Let's break that down clearly.

### 1. The idea of "outside world or shared state"

Every function has its own scope — the variables and values defined inside it.

If a function reads or modifies something beyond its scope, that's considered interacting with the outside world or shared state.

Examples of shared state:

Global variables

Data stored in files, databases, or APIs

The browser DOM (elements on a web page)

External services (network requests)

Console logs (which affect the program's visible output)

Any variable or object that was created outside the function and then modified inside it

So, whenever a function touches these — by reading, writing, or depending on them — it is performing a side effect.

### 2. Why side effects happen

Most real-world programs exist to cause side effects.

If you write to a database, send an email, or update the screen — you're changing the world outside the function. That's useful and necessary.

However, side effects make code less predictable because the same function call might produce different results depending on what's happening elsewhere in the system. For example:

A function that reads the current time or a random number will give different outputs each call.

A function that modifies a shared variable affects other parts of the program in ways that may not be obvious.

This means that side effects create coupling — parts of the program become dependent on each other's hidden behaviors.

### 3. Why minimizing side effects matters

When a function has side effects, it becomes harder to test, debug, or reason about:

You can't easily predict the result without knowing the full program state.

You can't safely reuse it in different contexts.

Bugs may appear when multiple functions modify the same data in unexpected orders.

That's why in good program design, side effects are not eliminated (since they're needed for interaction) but isolated — kept at the boundaries of the system.

This approach makes your core logic pure (depending only on inputs and producing outputs) and pushes unavoidable side effects to specific, controlled areas (like one module handling I/O or DOM updates).

### 4. Example to think about

Imagine a web application that calculates total price and then updates the page:

The calculation part (`subtotal + tax`) is pure — it always gives the same result for the same inputs.

The DOM update (`document.querySelector("#total").textContent = total`) is a side effect — it changes something visible outside the function.

If you separate these concerns — one function that purely computes, and another that updates the DOM — you can easily test the computation logic without worrying about browser behavior.

Later, if the display code changes (say, switching to a different UI framework), your pure logic remains safe and reusable.

### 5. Side effects are not evil — they just need boundaries

Every useful program must eventually perform side effects, because that's how it communicates with the world (displaying output, saving data, sending requests).

The key is controlling them:

Keep pure logic functions independent and predictable.

Let a small part of the code handle side effects explicitly (for example, one function responsible for logging or rendering).

This approach keeps your codebase more modular, easier to test, and less prone to bugs that come from unpredictable changes in shared state.

Examples

```
```js
// pure
function add(a, b) {
  return a + b;
}
```
```

```
```js
// impure: reads and writes external state
let total = 0;
function addToTotal(x) {
  total += x;
  return total;
}
```
```

```
```js
// impure: mutates input
function pushItem(arr, v) {
  arr.push(v);
  return arr;
}
```
```

```
```js
// pure alternative: returns a new array
function append(arr, v) {
  return arr.concat(v);
}
```
```

### Common misconceptions

1. Pure functions cannot use variables. They can use parameters and constants; they just cannot rely on or mutate external changing state.
2. Logging is harmless. Logging is a side effect.

3. A function that happens to return the same value today is pure. Purity requires guarantees for all time with identical inputs and no external changes.

#### Practice questions

1. Give three benefits of keeping your core logic pure.
2. Convert an impure function that pushes into an array into a pure version.
3. List five examples of side effects in typical web applications.

# What is immutability and why does it matter?

What is immutability and why does it matter

Immutability means that once a value is created, it is not changed. Instead of modifying data in place, you create new values that reflect the change. JavaScript primitives (string, number, boolean, null, undefined, symbol, bigint) are immutable. Objects and arrays are mutable by default, but you can adopt immutable patterns to avoid accidental shared mutations.

Immutability reduces hidden coupling, makes reasoning and debugging easier, and helps avoid bugs where two parts of a program unintentionally affect each other through shared references. It also enables simple change detection strategies and time-travel debugging in state management libraries.

To work immutably with objects and arrays, prefer methods that return new structures such as array map, filter, slice, spread syntax for objects and arrays, and Object.assign. Use structuredClone for deep copies when needed. Object.freeze can prevent modification of an object's properties, but it is shallow and does not freeze nested objects.

Examples

```
```js
// primitive example
let a = "hi";
let b = a;
b += "!";
console.log(a, b); // "hi", "hi!"
```
```

```
```js
// immutable update for object
const user = { name: "Sam", meta: { visits: 1 } };
const updated = {
  ...user,
  meta: { ...user.meta, visits: user.meta.visits + 1 },
};
console.log(user.meta.visits, updated.meta.visits); // 1, 2
```
```

```
```js
// immutable array operations
```

```
const arr = [1, 2, 3];
const doubled = arr.map((x) => x * 2);
const appended = [...arr, 4];
console.log(arr, doubled, appended);
...
```

Common misconceptions

1. const makes an object immutable. const prevents reassignment of the binding, not mutation of the object's contents.
2. Immutability is always slower. For typical app-level data, the clarity and safety outweigh minor copying costs; libraries optimize structural sharing.
3. JSON.stringify or Object.freeze makes deep immutability. Object.freeze is shallow; deep immutability needs recursive freezing or libraries.

Practice questions

1. Why does immutability make state management and debugging easier?
2. Show how to increment a deeply nested counter without mutating the original object.
3. What tools does JavaScript provide to help you work immutably?

Difference between undefined, null, and NaN

Difference between undefined, null, and NaN

In JavaScript, all three - undefined, null, and NaN - represent "absence" in some form. But they describe different kinds of absence, and mixing them up often causes confusion.

Think of them like this:

undefined -> "The variable exists, but no one has given it a value yet."

null -> "I deliberately set this to nothing."

NaN -> "I tried to get a number, but the result is nonsense."

Let's explore these one by one.

1. undefined - value not assigned

undefined is the default state of things that exist but haven't been assigned any value.

You don't have to explicitly write undefined; JavaScript gives it automatically in several cases:

Declared but not initialized variable

Missing function return value

Accessing a non-existent object property

Function parameter not passed during a call

Examples:

```
``js
```

```
let x;
```

```
console.log(x); // undefined (declared but no value)
```

```
function doSomething() {}
```

```
console.log(doSomething()); // undefined (no return statement)
```

```
const user = { name: "Alice" };
```

```
console.log(user.age); // undefined (property doesn't exist)
```

```
function greet(name) {
```

```
  console.log("Hello " + name);
```

```
}
```

```
greet(); // name is undefined
```

```
``
```

So undefined is JavaScript's way of saying:

"This thing is real, but it doesn't currently hold a value."

undefined often happens unintentionally - when something is missing or not yet set.

2. null - intentional absence

null represents a value that's intentionally empty.

Developers assign null themselves to mean, "This should have no value."

```
```js
```

Example:

```
let selectedUser = null; // means: no user selected yet
```
```

Unlike undefined, which JavaScript assigns automatically, null is assigned by you when you want to signal "nothing here on purpose."

Some real use cases:

Resetting a variable:

```
user = null; // clear previous user
```

Placeholder for future object values:

```
let result = null;
```

```
if (dataFound) result = process(data);
```

Indicating "not applicable" or "no result."

Even though null means "nothing," it's still a valid JavaScript value and must be assigned explicitly.

The typeof null oddity

```
typeof null; // "object"
```

This is a bug in JavaScript's design that dates back to the earliest versions.

It was never fixed for backward compatibility.

But it doesn't mean that null is an object - it's just an old quirk.

To correctly check for null, always use:

```
value === null;
```

3. NaN - invalid number result

NaN stands for Not-a-Number, but it actually is a number type - just a special one that means "this number operation failed."

You usually get NaN when:

You try to convert a non-numeric value to a number and it doesn't make sense.

You perform a mathematical operation that has no valid result.

```
```js
```

Examples:

```
Number("abc"); // NaN
parseInt("hello"); // NaN
0 / 0; // NaN
Math.sqrt(-1); // NaN
```

NaN is contagious - once a calculation involves NaN, the whole result becomes NaN.

```
NaN + 5; // NaN
NaN * 2; // NaN
...

```

The tricky part is that:

```
NaN === NaN; // false
```

This happens because JavaScript treats NaN as a special "unreliable" value that never equals anything - even itself.

To check for it, use:

```
Number.isNaN(value); // modern reliable way
```

Concept	Description	Type	Created by	Equality Example	How to check
<b>undefined</b>	variable exists but uninitialized, missing property, or missing return	"undefined"	JS assigns automatically	<code>'x' === undefined</code>	<code>typeof x === "undefined"</code>
<b>null</b>	intentional "no value" assigned by developer	"object" <i>(Legacy bug)</i>	developer sets manually	<code>'x' === null</code>	direct comparison
<b>NaN</b>	invalid numeric computation or coercion	"number"	numeric operation fails	<code>'NaN' === NaN</code> -> false	<code>Number.isNaN(x)</code>

## 5. Practical mental model

Use undefined for uninitialized or system-generated missing values.

You rarely need to assign it yourself.

Use null when you deliberately clear or empty a variable.

It's your way of saying: "I know this variable exists, but it should have no value right now."

Treat NaN as a special numeric error - it only shows up when math or conversions go wrong.

## 6. A simple story to remember

Imagine three boxes on a desk:

One box is empty because nobody ever put anything in it -> undefined

Another box has a note that says "intentionally left empty" -> null

A third box has nonsense written on it that doesn't make sense as a number -> NaN

All three are "empty" in some way, but each tells a different story about why they're empty.

## Examples

```
``js
let x;
console.log(x); // undefined
const obj = {};
console.log(obj.missing); // undefined

let y = null;
console.log(y === null); // true

console.log(typeof null); // "object" (legacy quirk)

console.log(Number("abc")); // NaN
console.log(NaN === NaN); // false
console.log(Number.isNaN(NaN)); // true
...

```

## Common misconceptions

1. undefined and null are interchangeable. undefined implies "not assigned yet"; null is an explicit, intentional empty value.
2. typeof null === "null". It returns "object" due to a long-standing quirk.
3. NaN compares equal to NaN. NaN never equals anything; use Number.isNaN to test it.

## Practice questions

1. When would you deliberately use null instead of leaving a variable undefined?
2. How do you distinguish between a missing property and a property explicitly set to null?
3. Why does NaN === NaN return false and how do you correctly test for NaN?

# What is strict mode

## What is strict mode

Strict mode is an optional mode in JavaScript that makes the language behave in a safer, more predictable way by turning silent errors into visible ones and by disallowing some problematic features. You enable it by placing "use strict"; at the top of a script file or at the beginning of a function body. In ES modules, strict mode is enabled by default.

Strict mode helps catch mistakes early. Assigning to an undeclared variable throws a ReferenceError instead of creating an accidental global. Certain syntax that often leads to bugs is disallowed. The this value in plain functions becomes undefined instead of implicitly pointing to the global object, which prevents hidden global access. Duplicate parameter names are banned, octal escape sequences are disallowed, attempts to delete plain variable bindings throw, and writes to non-writable properties throw in strict mode rather than failing silently in some engines.

## Examples

```
``js
"use strict";
x = 10; // ReferenceError: x is not defined
``
```

```
``js
function demo() {
 "use strict";
 y = 5; // ReferenceError
}
``
```

```
``js
function show() {
 console.log(this);
}
show(); // non-strict: global object; strict: undefined
``
```

## Common misconceptions

1. Strict mode always makes code faster. It mainly improves safety; performance is engine-dependent.
2. Strict mode is only for modules. Modules are strict by default, but scripts and functions can opt in with "use strict".

3. Strict mode breaks working code for no reason. It surfaces real mistakes like accidental globals and unsafe patterns.

#### Practice questions

1. List three runtime differences you get in strict mode versus non-strict.
2. How does strict mode change the default this in plain functions.
3. Why do writes to read-only properties throw in strict mode.

## What is the Temporal Dead Zone (TDZ)?

### What is the Temporal Dead Zone (TDZ)

The Temporal Dead Zone (TDZ) is a short period during your program's execution when a variable has been declared in memory but is not yet ready to use. It exists only for variables declared using `let`, `const`, or `class`.

When JavaScript starts running a block of code - like inside a function or `{ }` braces - it sets aside memory space for all variables it finds inside that block. This happens before the actual lines of code start executing. However, for `let`, `const`, and `class`, JavaScript does not assign them any initial value at this stage. The variables are marked as "uninitialized." They are known to exist, but you cannot access them yet.

The time between when the block begins and when JavaScript reaches the actual line where you declare the variable is called the Temporal Dead Zone. The word "temporal" refers to time, and "dead zone" means you cannot use that variable during that time. If you try to access it before the declaration line runs, JavaScript immediately throws a `ReferenceError` instead of silently giving `undefined`.

Once the interpreter reaches the variable's declaration line and runs it, the variable becomes fully initialized. From that moment onward, you can safely read or write its value.

The TDZ exists to prevent confusing bugs. In older JavaScript (before ES6), variables declared with `var` were automatically set to `undefined` when the program started running. This led to situations where developers accidentally used variables before they were truly ready, without realizing it. With the TDZ, JavaScript makes it clear: you cannot use a variable before it's actually declared.

This rule makes code more predictable. It ensures that every variable is used only after it has been properly defined and initialized, which avoids subtle errors and improves readability.

So, the TDZ is not an error itself - it's a protective mechanism that enforces good timing and discipline in how you use your variables.

## Examples

```
``js
{
 // console.log(x); // ReferenceError (TDZ)
 let x = 10; // TDZ ends here
 console.log(x); // 10
}
``
```

```
``js
{
 // console.log(y); // ReferenceError
 const y = 3;
}
``
```

Default parameters can also hit TDZ if they reference variables declared later.

```
``js
let a = 1;
function f(b = a) {
 return b;
} // ok
function g(b = c) {
 return b;
} // ReferenceError when called
let c = 2;
``
```

## Common misconceptions

1. let and const are not hoisted. They are hoisted but left uninitialized until the declaration runs; the gap is the TDZ.
2. TDZ is a special exception type. It is behavior that results in ReferenceError.
3. Only blocks have TDZ. Functions and class bodies also create TDZ for their let/const/class bindings.

### Practice questions

1. Explain why TDZ improves code safety with a short example.
2. Show a default-parameter case that triggers TDZ and explain why.
3. How does TDZ relate to the creation and execution phases of an execution context.

# What are arrow functions and how are they different from normal functions?

What are arrow functions and their differences from normal functions

Arrow functions are a compact way to write function expressions introduced in ES6. They change syntax (shorter) and semantics (important differences from "normal" functions created with the function keyword). The biggest semantic change is lexical this: an arrow function does not create its own this; instead, it captures the this from the surrounding (enclosing) scope at the moment the arrow is created.

## WHAT "LEXICAL THIS" REALLY MEANS

In a normal function, this is decided by how the function is called (method call, call/apply/bind, new, etc.). In an arrow function, this is decided by where the function was written. The arrow looks outward to the nearest non-arrow function (or top-level/module scope) and uses that this.

- Consequence: call, apply, and bind cannot change an arrow function's this-they still pass arguments, but the this stays whatever was captured.
- Practical upside: perfect for callbacks where you want to keep using the outer object's this (e.g., inside setTimeout, array methods, or promise handlers).
- Practical gotcha: arrows are a poor choice for object methods if you want this to be the receiver object; use a normal method syntax instead.

## OTHER LEXICALLY CAPTURED META-BINDINGS

Arrow functions also do not have their own arguments, super, or new.target. They close over the nearest outer ones (if any).

- No arguments: if you need a parameter list, use rest parameters (...args), which give you a real array.
- In classes, super used inside an arrow function refers to the super of the containing method-useful, but subtle.
- new.target is likewise taken from the outer scope (and arrows themselves cannot be constructors anyway).

## NOT CONSTRUCTIBLE; NO PROTOTYPE

You cannot use new with an arrow function. They're not constructors and have no prototype property. If you need instances via new, use a normal function or a class.

## SYNTAX OPTIONS (AND LITTLE PITFALLS)

- Parameters



- Zero parameters: `() => 42`
  - One parameter (no default/destructuring): `x => x * 2` (parentheses optional)
  - Multiple/default/destructured: `(x, y = 1) => x + y`, `{id} => id`
- Bodies
- Expression body (concise): `x => x * 2` implicitly returns the expression result.
  - Block body: `(x) => { const y = x * 2; return y; }` requires return to send a value back.
- Returning object literals
- Use parentheses: `() => ({ a: 1 })` (without them the braces are parsed as a block).
- Automatic semicolon insertion
- With concise bodies there's no return keyword; with block bodies you must return explicitly.
- Names
- Arrow functions are syntactically anonymous, but engines infer a name from the assignment target: `const add = (a,b)=>a+b`; `add.name` is often "add".
- ### ASYNC ARROWS
- You can write async arrow functions: `const fetchIt = async url => { const r = await fetch(url); return r.json(); }`
- They still capture this lexically; `await` works the same as in async function.
- ### WHEN ARROWS SHINE
- Callbacks that need the outer this
  - In methods: `setTimeout(() => this.doSomething(), 0)` keeps the instance this without `.bind(this)`.
  - Functional style array methods
  - `items.map(x => x.id)` is concise and clear.
  - Short utilities

- One-liners (predicates, transforms) become very readable.

## WHEN TO AVOID ARROWS

- Prototype methods or object methods that rely on dynamic this
  - If you write `obj = { total: 0, add: () => { this.total++ } }`, this won't be `obj`; it'll be whatever was outer when the arrow was defined. Use `add() { this.total++ }` instead.
- Event listeners that expect this to be the element (legacy/non-addEventListener patterns)
  - Arrows will not give you the element as this. Prefer a normal function if you need this bound to the element.
- Constructors / generator functions / places where you need arguments, `new.target`, or your own this
  - Arrows simply cannot do those.

## ABOUT PERFORMANCE AND MEMORY

- Performance is generally comparable; choose arrows for semantics/readability, not speed.
- In classes, defining arrow properties like `handler = () => { ... }` binds this per instance (great for callbacks) but creates a separate function per instance (slightly more memory) vs sharing a method on the prototype. Use this pattern when convenient, but be aware of the trade-off.

## INTEROP WITH BIND/CALL/APPLY

- `fn.call(obj, x)` will pass `x`, but the arrow's `this` won't become `obj`. If you need to rebind this, use a normal function.
- You can still use `call`/`apply` for arguments only with an arrow; this won't change.

## STRICT MODE AND PARAMETER RULES

- Arrows behave like strict functions for disallowed patterns (e.g., no duplicate parameter names when defaults/rest are present).
- They inherit strictness from their environment (modules are strict by default).

## COMMON MISCONCEPTIONS (CLEAR UP FAST)

- "Arrow functions are just shorter syntax."

Shorter, yes-but also different semantics for this, arguments, super, new.target, and constructibility.

- "I can use arrow functions anywhere I can use a function."

Not as constructors; not when you need your own dynamic this; not as generators.

- "I can fix an arrow's this with .bind."

No-you can't change this of an arrow after creation. .bind returns a new function, but the arrow's this is already lexically locked.

#### MENTAL CHECKLIST WHEN CHOOSING ARROW VS NORMAL FUNCTION

1. Do I need my own this (dynamic receiver, event element, prototype method)? -> Use normal function.
2. Do I want to preserve outer this in a callback or tiny helper? -> Use arrow.
3. Do I need arguments, new, prototype, or yield? -> Use normal function.
4. Am I inside a class and want an instance-bound handler for convenience? -> Arrow property can be great.

#### Examples

```
```js
const double = (x) => x * 2; // implicit return
const add = (a, b) => {
  return a + b;
}; // block body
```
```

```
```js
const obj = {
  n: 0,
  incLater() {
    setTimeout(() => {
      this.n++;
    }, 0); // arrow keeps outer this
  },
};
```

```
...
```

```
```js
```

```
const sum = (...nums) => nums.reduce((a, b) => a + b, 0);
```

```
...
```

### Common misconceptions

1. Arrow functions are just shorter syntax. They also change this and arguments behavior.
2. You can use new with an arrow function. Arrow functions are not constructors.
3. Arrow functions are always better. Prefer normal functions when you need your own this, arguments, or when defining methods on prototypes.

### Practice questions

1. Explain lexical this with a small example using setTimeout.
2. When should you avoid arrow functions and use a normal function instead.
3. How do you handle variable arguments in an arrow function.

# Explain default parameters, rest operator, and spread operator

Explain default parameters, rest, and spread operators

## Understanding Default Parameters, Rest, and Spread

These three features were added in modern JavaScript (ES6) to make working with function arguments and arrays easier and more natural. They all look similar because they use three dots (...), but they serve different purposes depending on where they appear.

### 1. Default Parameters - "Give me a fallback value"

When you write a function, sometimes the caller doesn't pass a value for one of the parameters.

Before default parameters existed, that would make the value undefined, and you had to handle it manually.

Default parameters let you specify a value that will automatically be used if no argument is provided, or if undefined is passed.

Think of it as giving your function a safety net.

Example idea:

If you have a function that greets a user, and no name is given, you can tell JavaScript to use "Guest" by default.

So instead of checking manually if the value exists, the function always works - it greets either the given name or the default one.

This makes functions more reliable and self-explanatory.

### 2. Rest Parameters - "Collect all the leftovers"

The rest parameter lets a function accept any number of arguments.

Normally, functions can only work with a fixed number of parameters (for example, one or two).

But sometimes you don't know how many values the caller will pass - maybe one, maybe ten.

By adding ... before the parameter name, JavaScript automatically gathers all the "extra" arguments into a real array.

That's why it's called "rest" - it collects the rest of the arguments that were not assigned to other parameters.

In plain English:

If you say, "give me the rest of them," that's what this feature does - it takes everything that remains and packs it together so you can use it easily inside the function.

This is very useful when building flexible functions - for example, when summing a list of numbers, processing all given inputs, or combining variable-length data.

### 3. Spread Operator - "Unpack things out again"

The spread operator also uses three dots (...), but instead of collecting values, it does the opposite - it unpacks them.

If you have an array (a list of values) and want to pass each value separately, the spread operator lets you do that easily.

You can think of it as pouring out the contents of a container.

For example:

If you have a list of numbers and want to pass them into a function that expects individual numbers, the spread operator breaks that array into separate pieces so the function can see them individually.

It's like saying:

"Here, take each of these items, not the box they're in."

The same idea works for objects: the spread operator can copy all key-value pairs from one object into another, which makes merging or cloning objects very simple.

The relationship between them

Even though rest and spread both use three dots, they do opposite things:

- Rest collects multiple values into one bundle (when used in a function definition).
- Spread unpacks one bundle into multiple values (when used in a function call or an array/object literal).

You can remember it like this:

Rest gathers, Spread scatters.

Why these features matter

Before these were added to JavaScript, you had to write extra code to:

- Handle missing arguments manually.
- Loop over the mysterious arguments object (which wasn't even a real array).

- Merge arrays or objects with clumsy methods.

Now, with default, rest, and spread, your code becomes cleaner, more expressive, and easier to understand.

They help you write functions that are more flexible and less error-prone, especially when working with variable data.

Default parameters

```
```js
function greet(name = "there") {
  return "Hi " + name;
}
greet(); // "Hi there"
greet("Sam"); // "Hi Sam"
```
```

Rest parameters

```
```js
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
```
```

Spread with arrays and iterables

```
```js
const a = [1, 2];
const b = [3, 4];
const combined = [...a, ...b]; // [1,2,3,4]
console.log([... "hi"]); // ['h','i']
```
```

Spread with objects (shallow copy/merge)

```
```js
const user = { name: "A", meta: { v: 1 } };
```
```

```
const copy = { ...user, role: "admin" }; // shallow copy
...
```

### Common misconceptions

1. Default expressions are evaluated at function definition time. They run at call time only if the argument is undefined.
2. Rest gives an arguments-like object. Rest gives a true array; arguments is array-like and not in arrow functions.
3. Object spread does a deep clone. It copies only one level; nested objects remain shared.

### Practice questions

1. Combine default parameters with destructuring in a function signature.
2. Refactor a function using arguments into one using rest.
3. Explain why spreading an object with nested objects is not a deep copy.



# What are template literals?

## What are template literals

Template literals are strings written with backticks that support embedded expressions, multi-line text, and tagged processing. They simplify string building compared to concatenation and preserve line breaks as written.

Embedded expressions use `${ ... }` to inject values. The expression can be any JavaScript expression. Tagged templates call a function with the literal's parts before construction, enabling custom escaping, `i18n`, or formatting logic.

## Examples

```
``js
const name = "Sam",
 score = 42;
console.log(`Hello ${name}, score: ${score}`);
``
```

```
``js
const multi = `Line 1
Line 2`;
``
```

```
``js
function join(strings, ...vals) {
 return strings.map((s, i) => s + (vals[i] ?? "")).join("");
}
const user = "<admin>";
console.log(join`Hello ${user}`);
``
```

## Common misconceptions

1. Template literals auto-sanitize output. They do not; security depends on your tag function or escaping.
2. Only variables can be inside `${}`. Any expression can.

3. Backticks are slower. They compile to efficient string operations; performance differences are negligible in normal code.

#### Practice questions

1. Rewrite a concatenated string using a template literal.
2. Create a tag that uppercases all interpolated values.
3. Explain how multi-line handling differs from normal quotes.

# Difference between for, for-in, for-of, and forEach

Difference between for, for-in, for-of, and forEach

What each one is for

## 1. for (classic index loop)

- Best when you need full control over the index and step (e.g., count by 2s, go backwards, stop early based on a condition).
- Works with anything that has a numeric length and index access (like arrays and strings).
- You can use break and continue.
- Works fine with await inside an async function (each iteration can await).

Example

```
``js
const arr = [10, 20, 30];
for (let i = 0; i < arr.length; i++) {
 console.log(i, arr[i]);
}
``
```

## 2. for...in (keys of an object)

- Iterates over enumerable property keys (names) of an object.
- Includes inherited keys unless you filter with hasOwnProperty.
- Not recommended for arrays (order can be surprising; it visits non-index properties too).
- You can use break and continue.
- Works fine with await inside an async function (each iteration can await).

Example with an object

```

```js
const user = { name: "A", age: 30 };
for (const key in user) {
  if (Object.prototype.hasOwnProperty.call(user, key)) {
    console.log(key, user[key]);
  }
}
```

```

### 3. for...of (values from an iterable)

- Iterates the values produced by any iterable: arrays, strings, Maps, Sets, typed arrays, generator results, etc.
- Ideal for arrays when you want the values directly (no index math).
- You can use break and continue.
- Works well with await inside an async function (each loop can await before moving to the next item).
- For Maps/Sets, you get entries or values in insertion order.

#### Examples

```

```js
// Array values
for (const value of [10, 20, 30]) {
  console.log(value);
}

// String characters
for (const ch of "hi") {
  console.log(ch);
}
```

```

```
// Map entries (each is [key, value])
const m = new Map([
 ["x", 1],
 ["y", 2],
]);
for (const [k, v] of m) {
 console.log(k, v);
}
...

```

#### 4. forEach (array method that runs a callback per item)

- Calls your function for each element of the array.
- You cannot use break or continue to stop early; it always runs to the end.
- Returning from inside the callback only returns from the callback, not the outer function.
- About async/await: if you put await inside the forEach callback, the loop does not "pause" between items. All callbacks are scheduled and your awaits run inside them, but the outer code keeps going. If you need to process items one-by-one in order, prefer for...of or for.

#### Example

```
```js
[10, 20, 30].forEach((value, index) => {
  console.log(index, value);
});
...

```

"Await-aware" explained simply

- "Await-aware" just means: if you use await inside the loop, the loop actually waits for the asynchronous work to finish before moving to the next item.

- classic for and for...of: yes, they wait (when used inside an async function).
- forEach: no, it does not wait between items; it fires the callbacks and moves on.
- for...in: behaves like for (you can await inside and it will wait in an async function).

Example of processing items one by one with pauses

```
```js
// Good: processes in sequence, waiting each time
async function processSequentially(items) {
 for (const item of items) {
 await doAsyncWork(item); // waits before moving on
 }
}
```
```

If you tried the same with forEach, the outer flow wouldn't wait for each `doAsyncWork` to finish before starting the next one.

When to use which

- Use for when you need index control, custom step sizes, or want to break/continue at specific counts.
- Use for...in for enumerating object keys (and guard with hasOwnProperty). Avoid it for arrays.
- Use for...of for clean iteration over values (arrays, strings, Maps, Sets, etc.). It's the go-to loop for arrays when you don't need the index.
- Use forEach for quick, no-break, no-await list processing where order/pausing doesn't matter and readability is your priority.

Access patterns and ordering

- Arrays: for and for...of preserve natural order; forEach also follows array order; for...in can produce unexpected key order and include non-index keys—avoid it for arrays.

- Objects: plain objects are not iterable, so `for...of` will not work directly; use `for...in` (with `hasOwnProperty`) or `Object.keys/Object.values/Object.entries` with `for...of`.
- Maps/Sets: `for...of` gives items in insertion order (with Map entries as `[key, value]`).
- Strings: `for...of` yields characters (including correct handling for many Unicode cases).

Common pitfalls (and fixes)

1. "for...in on arrays is fine, right?"

It can include non-index keys and odd ordering. Prefer `for`, `for...of`, or `forEach` for arrays.

2. "I can break out of `forEach` when I'm done."

You can't break or continue from `forEach`. If you need to stop early, use `for` or `for...of`.

3. "I used `await` in `forEach` but it still ran everything at once."

`forEach` doesn't pause between items. If you need to wait per item, switch to `for...of` (inside an `async` function).

4. "for...of works on objects too."

Not by default; plain objects aren't iterable. Use `for...in` (with `hasOwnProperty`) or:

```
``js
for (const [k, v] of Object.entries(obj)) {
  /* ... */
}
```

5. "I need the index with `for...of`."

Use `entries`:

```
``js
for (const [index, value] of arr.entries()) {
  /* ... */
}
```

```
}  
...  

```

A few handy patterns

Object entries with for...of

```
```js  
const obj = { a: 1, b: 2 };
for (const [k, v] of Object.entries(obj)) {
 console.log(k, v);
}
...

```

Array with index using entries

```
```js  
const arr = ["x", "y", "z"];  
for (const [i, v] of arr.entries()) {  
  console.log(i, v);  
}  
...  

```

Sequential async processing

```
```js  
async function run(items) {
 for (const item of items) {
 await doAsyncWork(item);
 }
}

```



...

Early exit

```
``js
for (const value of arr) {
 if (value === target) break;
}
...
```

Common misconceptions

1. `for...in` and `for...of` are interchangeable. They are not: `for...in` gives property names (keys); `for...of` gives iterable values.
2. `forEach` is the same as a loop. It cannot `break/continue`, and it doesn't pause with `await`.
3. Plain objects work with `for...of`. They don't; use `Object.keys/values/entries` or `for...in` (with a `hasOwnProperty` check).
4. You can always replace `for` with `forEach`. If you need index math, early exit, or `await` per item, prefer `for` or `for...of`.

Practice questions

1. You have to process a list of tasks one by one, waiting for each network call to finish before starting the next. Which loop do you choose and why?
2. Show how to iterate key/value pairs of a plain object in insertion-like order without using `for...in`.
3. Explain why `'break'` does not work inside `forEach` and provide an alternative that supports early exit.

# What are objects and how are they stored in memory?

What are objects and how are they stored in memory

An object is a container of key-value pairs. Keys are strings or symbols; values can be anything (numbers, strings, arrays, functions, other objects). Objects are dynamic: you can add, change, or remove properties at runtime.

How objects live in memory (the mental model)

Primitives (number, string, boolean, bigint, symbol, null, undefined) are stored directly as values. Objects live on the heap; variables hold a reference (a pointer) to them. If two variables point to the same object, changing it through one is visible through the other. When an object becomes unreachable from any live reference (including from other reachable objects), the garbage collector reclaims it. Cycles don't prevent collection as long as the whole cycle becomes unreachable.

Ways to create objects

1. Object literal (most common)

```
```js
const user = { name: "Ava", age: 30 };
```
```

Shorthand properties and methods:

```
```js
const name = "Ava";
const age = 30;
const user = {
  name, // same as name: name
  age,
  greet() {
    // method shorthand
  }
};
```
```

```

 return `Hi, I'm ${this.name}`;
 },
};
...

```

Computed property names:

```

```js
const key = "score";
const obj = { [key]: 42 }; // { score: 42 }
...

```

2. Constructor function (older pattern)

```

```js
function Person(name) {
 this.name = name;
}

Person.prototype.sayHi = function () {
 return "Hi " + this.name;
};

const p = new Person("Leo");
...

```

Properties set inside the constructor are per-instance; methods placed on the prototype are shared by all instances (memory-efficient).

## 3. class (modern, friendlier syntax around prototypes)

```

```js
class Person {

```

```

constructor(name) {
  this.name = name;
}
sayHi() {
  return `Hi ${this.name}`;
}
static species() {
  // class (static) method
  return "Homo sapiens";
}
}
const p = new Person("Mia");
...

```

Under the hood, classes still use prototypes. Instance methods are on `Person.prototype`. Static methods are on `Person` itself.

4. Object.create (build with a specific prototype)

```

```js
const base = { kind: "base" };
const child = Object.create(base); // prototype = base
child.x = 1;
...

```

Great for building objects with a chosen prototype without invoking constructors.

#### 5. From existing entries

```

```js
const entries = [

```

```

["a", 1],
["b", 2],
];
const obj = Object.fromEntries(entries); // { a: 1, b: 2 }
...

```

Own vs inherited properties and the prototype chain

Each object has an internal link to a prototype (another object or null). When you access `obj.prop`, the engine:

1. Looks for an "own" property on `obj`.
2. If missing, looks on `obj`'s prototype.
3. Continues up the chain until it finds it or reaches `null`.

No copying happens; lookup is dynamic. This is why methods placed on `Constructor.prototype` are shared across instances.

Defining, reading, enumerating properties

Property names can be strings or symbols. String keys that look like integers (e.g., "0", "1") may get special ordering during enumeration, but for most cases assume insertion order for own string keys, with symbol keys not included in typical enumeration.

Useful patterns for iteration:

```

``js
const user = { name: "Ava", age: 30 };

// keys only
for (const k of Object.keys(user)) {
  console.log(k, user[k]);
}

```

```
}
```

```
// values only
```

```
for (const v of Object.values(user)) {
```

```
  console.log(v);
```

```
}
```

```
// entries (key/value pairs)
```

```
for (const [k, v] of Object.entries(user)) {
```

```
  console.log(k, v);
```

```
}
```

```
...
```

for...in walks enumerable keys including inherited ones; guard with hasOwnProperty when needed.

Property attributes (descriptors)

Every property has attributes: value, writable, enumerable, configurable (for data properties) or get/set (for accessor properties).

```
```js
```

```
const user = {};
```

```
Object.defineProperty(user, "id", {
```

```
 value: 123,
```

```
 writable: false, // cannot change value
```

```
 enumerable: false, // won't show in Object.keys
```

```
 configurable: false, // cannot delete or reconfigure
```

```
});
```

```
...
```

Accessors (computed values, side effects):

```

```js
const meter = {
  _value: 0,
  get value() {
    return this._value;
  },
  set value(v) {
    if (v >= 0) this._value = v;
  },
};
```

```

Symbols as keys (non-colliding, non-string)

```

```js
const ID = Symbol("id");
const o = { [ID]: 99 };
```

```

Symbol keys are not found by `Object.keys`/`for...in``. Use `Object.getOwnPropertySymbols(o)`` or `Reflect.ownKeys(o)`` to see them.

Common built-in object utilities (the greatest hits)

Creation and prototypes

- `Object.create(proto, descriptors?)`` Create with a given prototype.
- `Object.getPrototypeOf(obj)`` / `Object.setPrototypeOf(obj, proto)`` Read/change prototype (changing later can be slow; prefer setting at creation).
- `obj.__proto__`` Legacy getter/setter for prototype (avoid in production).

## Introspection

- `Object.keys(obj)` enumerable own string keys.
- `Object.values(obj)` enumerable own string values.
- `Object.entries(obj)` pairs of `[key, value]`.
- `Object.getOwnPropertyNames(obj)` own string keys including non-enumerable.
- `Object.getOwnPropertySymbols(obj)` own symbol keys.
- `Reflect.ownKeys(obj)` all own keys (strings + symbols).
- `Object.getOwnPropertyDescriptor(obj, key)` read attributes.

## Copying / merging (shallow)

- `Object.assign(target, ...sources)` copies own enumerable string + symbol properties (shallow).
- Spread syntax `{ ...obj }` also shallowly copies enumerable own properties.
- Deep copy: `structuredClone(obj)` (modern) deep-clones many structured values; falls back to libraries for older environments or special cases.

## Equality and identity

- `Object.is(a, b)` like `===` but treats `NaN` equal to `NaN` and distinguishes `+0` vs `-0`.

## Mutability control (top-level only)

- `Object.preventExtensions(obj)` block adding new props.
- `Object.seal(obj)` prevent add/delete; keep writing if writable.
- `Object.freeze(obj)` prevent add/delete/reconfigure and writing.
- Use recursively for deep effects or write a `deepFreeze` helper.

## Converting to data

- `JSON.stringify(obj)` to JSON string (ignores functions and symbols, throws on cycles).



- `JSON.parse(str)` back to an object.

### Boxing and primitives

Objects can have `toString`, `valueOf`, or `Symbol.toPrimitive` to control how they convert to strings or numbers.

```
``js
const money = {
 amount: 1000,
 [Symbol.toPrimitive](hint) {
 return hint === "number" ? this.amount : `$$${this.amount}`;
 },
};
String(money); // "$1000"
+money; // 1000
``
```

### Methods on the object vs prototype methods

Instance methods exist on each object, prototype methods are shared. In classes:

```
``js
class Counter {
 count = 0; // public field per instance (each gets its own)
 inc() {
 this.count++;
 } // shared function on prototype
}
``
```

Arrow methods as fields (`handler = () => {}`) capture `this` per instance (great for callbacks), but consume more memory than a single shared prototype method.

## Objects vs Map/Set

Use a plain object when keys are known strings/symbols and you want prototype features, JSON, and simple literals. Use `Map` when keys can be anything (including objects), when you need guaranteed insertion order + efficient size, and methods like `map.get`, `map.set`, `map.has`. Use `Set` for unique value collections. `WeakMap/WeakSet` hold weak references to object keys/values and don't prevent garbage collection (useful for caches without leaks).

## Cloning and immutability

Shallow copies copy only one level; nested objects remain shared.

```
```js
const user = { name: "Ava", meta: { visits: 1 } };
const copy = { ...user }; // shallow
copy.meta.visits++; // also changes user.meta.visits
...

```

Deep copy options:

- `structuredClone(user)` (modern, handles many types).
- Libraries or custom recursive clone for legacy/edge cases.

If you want immutable patterns, avoid mutating original objects; create new copies with changed fields:

```
```js
const updated = {
 ...user,
 meta: { ...user.meta, visits: user.meta.visits + 1 },
};
...

```

## Performance notes (pragmatic)

Engines optimize objects using hidden classes/shapes; consistent property creation order and shape (define all fields in the constructor, avoid adding/removing fields later) can help performance. `Object.setPrototypeOf` and changing shapes at runtime can deoptimize; prefer stable layouts.

## Memory, reachability, and leaks

Objects stay alive as long as something reachable references them. Accidental retention (e.g., a global cache never cleared, long-lived event listeners, closures holding large data) can cause leaks. Use:

- Remove event listeners when no longer needed.
- Null out references when appropriate.
- Prefer WeakMap/WeakSet for caches keyed by objects.

## Putting it all together: typical patterns

### Factory function (no `new`)

```
```js
function createUser(name) {
  return {
    name,
    greet() {
      return `Hi ${this.name}`;
    },
  };
}
```
```

## Class

```
```js
class User {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return `Hi ${this.name}`;
  }
}
```
```

## Prototype + Object.create

```
```js
const proto = {
  greet() {
    return `Hi ${this.name}`;
  },
};
const u = Object.create(proto);
u.name = "Ava";
```
```

## Defining non-enumerable or read-only properties

```
```js
const obj = {};
Object.defineProperty(obj, "id", {
  value: 1,
```

```
writable: false,  
enumerable: false,  
});  
...
```

Common misconceptions

1. Assigning an object to another variable copies it. It copies the reference; both variables point to the same object.
2. Prototype properties are copied into the object. They are not; they're looked up dynamically through the chain.
3. `Object.freeze` makes everything inside immutable. It's shallow; nested objects are still mutable unless you freeze them too.
4. `for...of` works on plain objects. Plain objects are not iterable; use `Object.keys/values/entries` or `for...in` with `hasOwnProperty`.
5. JSON is a reliable clone for all objects. It drops functions/symbols and fails on cycles or special types; prefer `structuredClone` when available.

Practice questions

1. Show four different ways to create an object that has a `greet()` method, and explain which method is shared vs per-instance in each case.
2. Explain what happens in memory when you execute `const a = { x: 1 }; const b = a; b.x = 2;`
3. How would you create a non-enumerable, read-only property `id` on an object? How would you verify its descriptor?
4. What's the difference between `{ ...obj }`, `Object.assign({}, obj)`, and `structuredClone(obj)`?
5. When would you choose a `Map` over a plain object? Give two concrete reasons.

Explain pass-by-value vs pass-by-reference

Explain pass-by-value vs pass-by-reference

When you pass something to a function in JavaScript, the language always passes it **by value** — but *what that value represents* depends on whether the thing you're passing is a **primitive** or an **object**.

To understand the difference, let's first see what each category really is.

1. Two categories of data types

- **Primitives:** numbers, strings, booleans, null, undefined, BigInt, and symbols.

These hold *actual values* directly (like `42` or `"hello"`).

- **Objects:** arrays, functions, objects, maps, sets, etc.

These do not store the value itself in the variable.

Instead, the variable stores a **reference** — a kind of "address" or pointer to where the real data lives in memory (on the heap).

So, in JavaScript:

- Primitives → simple direct values.

- Objects → references to memory locations that hold the data.

2. Pass-by-value (primitives)

When you pass a **primitive** to a function, JavaScript copies its actual value into the parameter.

That means changes inside the function don't affect the original variable, because the function is working on its own copy.

Think of it like photocopying a document — the function gets the copy, not the original.

If it writes on that paper, your original stays untouched.

So when we say "pass-by-value," it literally means:

> "The function received a new copy of the value."

3. Pass-by-reference (objects — conceptually)

Now, when you pass an ****object****, what gets copied is ****not the whole object****, but the ****reference**** (the address pointing to it).

That means both the original variable and the function parameter now point to the ****same object**** in memory.

If the function changes something *_inside_* that object, both will see that change — because they're both looking at the same underlying data.

But there's a subtle twist:

Even though this behavior *_looks like_* pass-by-reference, JavaScript is technically still ****passing by value**** — the value just happens to be a *_reference_*.

In other words, the function gets a *_copy of the reference_*, not the object itself.

But because the reference points to the same memory location, changing the contents inside that object affects both.

However, if you make the parameter point to a new object inside the function, that new reference doesn't affect the original one.

They'll then point to different places.

4. The mental model — "pointer vs value"

You can think of it like this:

- A primitive variable holds a ****value**** directly.
- An object variable holds an ****address tag**** pointing to where the object lives.

When you call a function:

- For a primitive: the value (like `10` or `"hi"`) is copied.
- For an object: the reference (the tag pointing to the heap) is copied.

Now both the original and the parameter hold tags that point to the same place.

This explains why modifying the object's inside properties affects the outside variable too — both are looking at the same location.

5. Practical implications

1. ****Immutable vs mutable:****

- Primitives are immutable (you can replace them, but not modify them directly).
- Objects are mutable (their contents can change).

2. ****Function design:****

- When you pass a primitive, you can be sure the original won't be affected.
- When you pass an object, if you don't want to mutate the original, you should clone or copy it first.

3. ****Common beginner confusion:****

- "I thought JavaScript passes by reference because my array changed!"
 - No — the reference itself was passed *_by value_*, but both refer to the same object.
- "If I reassign the parameter inside the function, shouldn't it change outside too?"
 - No — that's a new reference; the outer one still points to the old object.

6. How memory fits into this

In memory:

- Primitives live directly on the stack.
- Objects live on the heap.
- Variables hold stack entries that either contain a direct value (for primitives) or a pointer to a heap object (for objects).

When a function runs:

- New local variables and parameters are placed on its stack frame.
- When the function exits, those local stack variables are discarded.
- But the heap objects they pointed to may continue to exist if something outside the function still references them.

That's why objects persist beyond function calls — they're not copied around; only their references are.

7. The key takeaway

- JavaScript always passes arguments ****by value****.
- For primitives, that value is the actual data.
- For objects, that value is a *_reference_* to where the data lives.
- So modifications to object properties affect the same object, but reassignment inside the function doesn't affect the original variable.

Or in simpler terms:

> You copy the value of the box —

> but for objects, that value is the ****key to a shared box****, not the box itself.

Examples

```
``js
function inc(n) {
  n = n + 1;
}
let a = 5;
inc(a);
console.log(a); // 5
...

```

```
``js
function setName(obj) {
  obj.name = "Alex";
}
const user = { name: "Sam" };
setName(user);
console.log(user.name); // "Alex"
...

```

```
``js
function replace(obj) {
  obj = { name: "New" };
}
replace(user);
console.log(user.name); // still "Alex"
...

```

Common misconceptions

1. JavaScript is pass-by-reference. The reference is passed by value.
2. Reassigning a parameter changes the caller's variable. Only mutations to the object's contents are observed by the caller.
3. Passing objects is unsafe. Avoid mutation or copy when needed; passing references itself is fine.

Practice questions

1. Why does reassigning a parameter not affect the caller?
2. Show how to avoid accidental mutation when passing objects into functions.
3. Implement a deep clone and explain when you would use it.

What is object destructuring and array destructuring?

What is object destructuring and array destructuring

Destructuring lets you extract values from arrays and properties from objects into variables using a compact pattern. Arrays destructure by position; objects destructure by property name. You can rename, set defaults, skip elements, and destructure nested structures.

Examples

```
```js
```

```
const arr = [10, 20, 30];

const [a, b] = arr; // a=10, b=20

const [, , c] = arr; // c=30

const [x = 1, y = 2] = []; // defaults
```

```
```
```

```
```js
```

```
const user = { name: "Sam", age: 30 };

const { name, age } = user;

const { name: fullName } = user; // rename

const { role = "guest" } = user; // default
```

```
```
```

```
```js
```

```
const data = { meta: { count: 5 }, items: [1, 2] };

const { meta: { count }, items: [first] } = data;
```

```
```
```

```
```js
```

```
function draw({ x = 0, y = 0, color = "black" } = {}) {}
```

```
```
```

Common misconceptions

1. Destructuring mutates the source. It only reads.
2. Object destructuring depends on order. It matches by key name.
3. Missing properties cause errors. Provide defaults to handle undefined.

Practice questions

1. Extract the second and fourth items of an array into a and b.
2. Destructure user.profile.name with a default of "Anonymous".
3. Write a function that destructures options with defaults in its parameter list.

Explain Object.freeze, Object.seal, and Object.preventExtensions

Explain Object.freeze, Object.seal, and Object.preventExtensions

Object.preventExtensions stops adding new properties to an object. Existing properties can still be changed or deleted depending on their descriptors. Object.isExtensible checks whether new properties can be added.

Object.seal prevents adding or deleting properties and marks all existing properties as non-configurable. Values of writable properties can still be changed. Object.isSealed checks the sealed state.

Object.freeze prevents adding, deleting, or reconfiguring properties and makes all existing properties non-writable and non-configurable. Values cannot be changed. Object.isFrozen checks the frozen state. All three methods are shallow: nested objects are unaffected unless also processed.

Examples

```
```js
const a = { x: 1 };
Object.preventExtensions(a);
a.y = 2; // ignored or throws in strict mode
delete a.x; // allowed if configurable
...

```

```
```js
const b = { x: 1 };
Object.seal(b);
delete b.x;   // false
b.x = 2;      // ok if writable
...

```

```
```js

```

```
const c = { x: 1 };
Object.freeze(c);
c.x = 2; // ignored or throws in strict mode
...
```

### Common misconceptions

1. freeze is deep immutability. It only freezes the top level.
2. seal prevents value changes. It prevents deletion and reconfiguration but not writing (if writable).
3. preventExtensions is equivalent to seal. It only blocks adding new properties; deletion and configuration still depend on descriptors.

### Practice questions

1. Summarize the differences among preventExtensions, seal, and freeze.
2. Write a deepFreeze utility and explain when you would use it.
3. What happens when you assign to a frozen property in strict vs non-strict mode?

# What is prototypal inheritance?

## What is prototypal inheritance

Prototypal inheritance is the way JavaScript lets objects share behavior without copying it. Every object can have an internal link to another object called its prototype. When you read a property from an object and the engine doesn't find an "own" property, it automatically walks this prototype link to look for the property on the prototype, then that prototype's prototype, and so on until it reaches null. This chain is consulted at read time for every lookup; nothing is copied automatically. Because of this, a single shared method defined on one prototype can serve many instances at once, saving memory and keeping behavior consistent. If you later change the shared method on the prototype, all instances immediately observe the change on their next property read (unless they have their own property with the same name, which "shadows" the prototype's property).

In day-to-day code, you usually encounter prototypal inheritance through constructor functions and classes. A constructor's prototype object holds the shared methods; instances created with `new` link to that prototype. With class, the instance methods you write are placed on the class's prototype under the hood, and instances created with `new Class()` link to it. You can also build prototype chains directly with `Object.create(parent)`, which makes an object whose prototype is parent without involving constructors. This flexibility allows both classical, class-like hierarchies and ad-hoc delegation where objects share only what they need.

Understanding prototypal inheritance clarifies several behaviors: method dispatch uses the receiver object as this even if the method was found higher in the chain; assignment creates or updates own properties by default (it does not climb the chain to overwrite a prototype property); and property enumeration can include or exclude inherited properties depending on the mechanism you use. The model is dynamic and late-bound, so you should avoid mutating prototypes at runtime in widely-shared libraries to reduce surprises; prefer establishing shapes once and treating prototypes as stable interfaces.

## Examples

```
```js
// Sharing a method via a prototype (constructor style)

function Person(name) {
  this.name = name;
}

Person.prototype.sayHi = function () {
  return "Hi " + this.name;
}
```



```

};

const a = new Person("Ava");
const b = new Person("Ben");
a.sayHi(); // "Hi Ava"
b.sayHi(); // "Hi Ben"
console.log(a.sayHi === b.sayHi); // true (shared function)
...

```

```

``js
// Direct delegation with Object.create
const mover = {
  move() {
    return this.label + " moves";
  },
};
const robot = Object.create(mover);
robot.label = "R2";
robot.move(); // "R2 moves"
...

```

```

``js
// Shadowing a prototype property
const base = { kind: "base" };
const child = Object.create(base);
child.kind = "child"; // own property shadows base.kind
...

```

Common misconceptions

1. Prototypal inheritance copies methods into children. Nothing is copied; reads consult the chain on demand.

2. Methods run with this bound to the prototype. this is the receiver object (the thing before the dot), not the prototype where the method lives.
3. Assigning obj.x updates the prototype's x. Assignment creates/updates an own property unless an accessor or descriptor intercepts it.
4. Classes remove prototypes. class is syntax over prototypes; the mechanism is unchanged.

Practice questions

1. Theory: In your own words, describe how a property read travels along the prototype chain and how shadowing works.
2. Coding: Implement a Shape constructor with an area method on Shape.prototype. Make Rectangle inherit from Shape's prototype and override area on Rectangle.prototype. Show both in action.
3. Coding: Create a base object with Object.create and prove that adding a method to the base later becomes visible on already-created children.

How does the prototype chain work?

How does the prototype chain work

The prototype chain is the ordered sequence of objects the engine searches to resolve a property read. When you evaluate `obj.prop`, the engine checks for an own property `prop` on `obj`. If it does not exist, it follows `obj`'s internal `[[Prototype]]` link to another object (often the constructor's prototype) and repeats the check. This continues until the property is found or the end of the chain (`null`) is reached. If nothing defines `prop`, the result is `undefined`. Because lookup is dynamic, changing any object along the chain affects future reads; because own properties take precedence, adding an own property with the same name "shadows" the one found higher in the chain.

Method calls use the same lookup. When you call `obj.method()`, the engine finds `method` using the chain, but during the call it binds this to `obj` (the receiver). That allows a single shared function living on a prototype to behave as if it belonged to each instance. Writes behave differently: a plain assignment `obj.prop = v` creates or updates an own property on `obj`; it does not climb the chain to modify a prototype property. Accessors (`get/set`) and property descriptors can change this behavior by intercepting reads/writes. Enumeration tools differ too: `for...in` traverses enumerable keys including inherited ones; `Object.keys` shows only enumerable own string keys; `Object.getOwnPropertyNames` includes non-enumerable own string keys; `Reflect.ownKeys` includes all own keys (strings and symbols).

The global chain for typical objects ends at `Object.prototype`, which provides methods like `toString` and `hasOwnProperty`. Arrays and functions have their own prototype objects that themselves link to `Object.prototype`. Null-prototype objects (`Object.create(null)`) deliberately have no inherited properties, which is useful for "dictionary" maps without prototype interference. Understanding the chain helps you reason about performance (engines optimize common shapes), avoids accidental leaks of shared mutable state on prototypes, and clarifies why late changes to a prototype surface everywhere.

Examples

```
```js
// Three-level chain: A <- B <- C
const A = {
 tag: "A",
 greet() {
 return "from A";
 },
}
```

```

};

const B = Object.create(A);
const C = Object.create(B);
C.greet(); // "from A"
C.greet = function () {
 return "from C";
};
C.greet(); // "from C" (shadowing)
...

```js
// Assignment creates own property; prototype remains unchanged
const base = { x: 1 };
const child = Object.create(base);
child.x = 2;
console.log(child.x, base.x); // 2, 1
...

```js
// Prototype accessors can intercept writes
const P = {
 _v: 0,
 get v() {
 return this._v;
 },
 set v(n) {
 this._v = n < 0 ? 0 : n;
 },
};
const o = Object.create(P);
o.v = -5; // setter on prototype runs with this = o

```

```
o.v; // 0
```

```
...
```

### Common misconceptions

1. If a property exists on the prototype, assignment updates that one. Default assignment creates/updates an own property instead.
2. Methods run with this equal to the prototype. this is the receiver.
3. Changing a prototype only affects future instances. Reads always consult the current prototype, so existing instances observe changes immediately (unless shadowed).
4. for...in shows only own properties. It walks inherited enumerable keys too.

### Practice questions

1. Theory: Explain why `child.prop = value` does not update `parent.prop` and how accessors can alter this behavior.
2. Coding: Build `A -> B -> C` where `A` supplies a method. Prove that shadowing on `C` hides `A`'s method, then delete from `C` and show the lookup falls back to `A` again.
3. Coding: Use a prototype setter to validate assignments on child instances; show that the setter's `this` refers to the instance.

# What are constructor functions?

## What are constructor functions

Constructor functions are the pre-class way to initialize similar objects that share behavior. A constructor is just a normal function intended to be invoked with `new`. The `new` operator performs four coordinated steps: (1) it allocates a fresh empty object, (2) it sets that object's internal `[[Prototype]]` to the constructor's prototype property, (3) it binds this inside the constructor to that fresh object so you can assign per-instance fields, and (4) it returns the new object automatically unless you explicitly return a non-primitive value. By placing methods on `ConstructorName.prototype` instead of inside the constructor, all instances share one function, which is memory-efficient and keeps behavior consistent.

Constructor functions compose into hierarchies by linking prototypes. To emulate "subclassing," you create the child's prototype from the parent's prototype (`Object.create`) and call the parent constructor within the child constructor to initialize shared fields. You also repair the child's `prototype.constructor` property if you care about reflection. While ES6 class offers cleaner syntax, understanding constructor functions explains what class does under the hood and clarifies why instances see prototype methods, how this is bound during construction, and why returning an object from a constructor changes the returned value regardless of prototype setup.

## Examples

```
```js
function Person(name) {
  this.name = name;
}

Person.prototype.sayHi = function () {
  return "Hi " + this.name;
};

const a = new Person("Ava"),
      b = new Person("Ben");

a.sayHi();
b.sayHi();
```
```

```

```js
function Parent(x) {
  this.x = x;
}

function Child(x, y) {
  Parent.call(this, x); // initialize Parent fields
  this.y = y;
}

Child.prototype = Object.create(Parent.prototype);
Child.prototype.constructor = Child;
...

```js
// Returning a non-primitive overrides the default return
function Odd() {
 this.tag = "Odd";
 return { note: "I ignore the prototype link" };
}

const o = new Odd();
// o has no link to Odd.prototype
...

```

### Common misconceptions

1. new copies methods into instances. It links instances to the prototype; methods remain on the prototype.
2. Returning an object from a constructor is harmless. It replaces the instance entirely, possibly breaking the intended prototype link.
3. The constructor property is always meaningful. After replacing a prototype, restore constructor manually if you rely on it.

### Practice questions

1. Theory: List the four steps performed by new and explain why placing methods on the prototype saves memory.
2. Coding: Implement a Point(x, y) with a distance method on Point.prototype. Create two points and verify the method is shared.
3. Coding: Implement Parent/Child inheritance using Object.create + Parent.call, and verify Child instances access Parent's prototype methods.



# Explain class syntax in ES6 and how it's sugar over prototypes

Explain class syntax in ES6 and how it's sugar over prototypes

Classes in JavaScript are a more readable way to write what the language has always done with functions and prototypes. They do not introduce a new inheritance system like in Java or C++; they just make the old prototype-based pattern easier to write and understand. When you define a class, JavaScript actually creates a special kind of function under the hood - the constructor. Every method you write inside the class body is automatically added to that class's prototype, not copied into each object. So when you create an object from a class using the new keyword, that object gets linked to the class's prototype, just like it would if you used a constructor function. That's why calling a method on one instance doesn't duplicate the code - all instances share the same method from the prototype.

When you extend one class from another using extends, JavaScript connects the new class's prototype to the parent class's prototype. This means if a method isn't found on the child, it looks upward in the chain to find it on the parent. Inside the child's constructor, you can call super() to run the parent's constructor, ensuring the base setup happens before adding the child's specific logic. You can also call super.someMethod() to reuse parent methods from inside overrides.

Classes also provide a few new conveniences that make object-oriented programming easier. For example, you can define fields (variables) directly inside the class body instead of only inside the constructor. Fields declared with the # prefix are private, meaning they can only be accessed inside that class. This gives real data privacy - something older JavaScript patterns couldn't enforce. There are also static methods and fields, which belong to the class itself instead of instances. Static methods are used for utility operations or factory functions related to the class, while static fields can store constants or configuration shared across all instances.

Despite these modern features, classes in JavaScript are still built on the same prototype mechanism. Each class creates a constructor function and a prototype object that stores shared methods. Instances link to that prototype, and method calls use that link to find and execute the right code. Nothing is copied; the language simply hides the prototype wiring behind a cleaner syntax. That's why classes are often said to be "syntactic sugar" - they sweeten the old prototype system but don't change it. Understanding this helps you realize that features like inheritance, method overriding, and shared behavior in JavaScript are all just prototype relationships under the hood.

Examples

```
```js
```

```

class Shape {
  area() {
    return 0;
  }
}

class Rectangle extends Shape {
  constructor(w, h) {
    super();
    this.w = w;
    this.h = h;
  }
  area() {
    return this.w * this.h;
  } // override
  static kind() {
    return "rect";
  }
}

const r = new Rectangle(3, 4);
r.area(); // 12
Rectangle.kind(); // "rect"
...

```

```

```js
class Counter {
 #count = 0; // private
 inc() {
 this.#count++;
 }
 value() {
 return this.#count;
 }
}

```

```

 }
}
const c = new Counter();
c.inc();
c.value(); // 1
...

```js
// Field initializers
class Task {
    status = "new"; // per-instance field
    run() {
        this.status = "done";
    }
}
...

```

Common misconceptions

1. class changes inheritance to classical copying. It still uses prototypes and delegation.
2. Methods are per-instance with class. Methods live on the prototype; fields are per-instance.
3. Private fields are just naming tricks. They are enforced; accessing `#private` outside the class is a syntax error.

Practice questions

1. Theory: Explain where instance methods and static methods are stored at runtime and how extends links prototypes.
2. Coding: Implement a Base class with a greet method, extend it with Admin that overrides greet and calls `super.greet()`.
3. Coding: Create a class with a `#private` counter and public `inc/value` methods; prove that external code cannot access `#counter`.

What is Object.create used for?

What is Object.create used for

`Object.create(proto, descriptors?)` constructs a new object whose internal `[[Prototype]]` is `proto` and whose own properties can be defined via an optional `descriptors` map. This is the most direct way to express "make an object that delegates to this other object" without invoking constructors. Because the prototype is set at creation time, you avoid the performance and complexity costs of altering the prototype later. Passing `null` creates a "dictionary" object with no inherited properties (no `toString`, no `hasOwnProperty`), which is ideal when you want a clean key space for arbitrary keys.

The `descriptors` parameter lets you define properties with precise control over writability, enumerability, configurability, and getters/setters in one call. Using `Object.create` makes delegation explicit and encourages composition: you can assemble objects that share only what they need by choosing suitable prototypes. It is also convenient for building test doubles or simple hierarchies, where you can add methods to the parent after children exist and they will see the new methods on subsequent reads.

Examples

```
```js
const base = {
 greet() {
 return "hi";
 },
};

const child = Object.create(base, {
 id: { value: 1, writable: true, enumerable: true, configurable: true },
});

child.greet(); // "hi"
```
```

```
```js
// Null-prototype "dictionary"
const dict = Object.create(null);
```

```
dict["__proto__"] = "ok"; // safe: not an inherited key
Object.hasOwn(dict, "__proto__"); // true
...

```

```
``js
// Add to the base later
const a = Object.create(base);
a.name = "A";
base.who = function () {
 return this.name;
};
a.who(); // "A"
...

```

### Common misconceptions

1. Object.create clones the prototype. It links; nothing is copied.
2. You need constructors to set up inheritance. Object.create directly sets the prototype relationship.
3. Null-prototype objects behave like normal objects. They lack Object.prototype utilities; use Object.hasOwn and Reflect helpers.

### Practice questions

1. Theory: When would you prefer Object.create over class or constructors?
2. Coding: Create a null-prototype dictionary and demonstrate safe key storage and lookup without collisions.
3. Coding: Build a base -> child chain with Object.create, add a method to the base afterwards, and show the child can call it.

# Explain call, apply, bind — and differences between them

Explain call, apply, bind and differences between them

In JavaScript, functions are special objects that can be called in many different ways. By default, when you call a regular function, the value of `this` inside it depends on *how* you call it — not where it was written. Sometimes you need to control what `this` points to, or you might need to pass arguments flexibly from another source. That's where **`call`**, **`apply`**, and **`bind`** come in. These three methods belong to every function in JavaScript, and they let you explicitly decide what `this` should refer to when the function runs.

The **`call`** method runs a function immediately, but allows you to specify what `this` should be and pass arguments one by one. For example, `sayHello.call(user, "Hi")` calls `sayHello` right away with `this` bound to `user`. It's just like calling `user.sayHello("Hi")`, but gives you manual control over the context. The **`apply`** method works almost the same way — it also invokes the function immediately — but instead of taking arguments individually, it expects them as a single array (or array-like structure). For instance, `sayHello.apply(user, ["Hi"])` does the same as `sayHello.call(user, "Hi")`. The main difference is only in how you pass the arguments. This was particularly useful before the spread operator (`...`) was introduced, since you could easily forward arrays of arguments to a function.

The **`bind`** method is slightly different. It doesn't call the function right away. Instead, it creates and returns a *new function* that remembers the `this` value you provided and optionally some of the arguments you passed. When you later call that returned function, it will automatically use the bound `this` and pre-filled arguments. For example, if you write `const greetJohn = greet.bind(user, "John")`, you now have a new function that always greets using that same `user` context and name, no matter how or where you call it. This is extremely useful when you need to pass functions as callbacks — like event listeners or `setTimeout` handlers — and want them to remember which object they belong to.

Here's a simple way to visualize it:

```
``js
function say(greeting) {
 console.log(greeting + ", I am " + this.name);
}

const person = { name: "Alice" };
```

```
// call - runs immediately
say.call(person, "Hello"); // "Hello, I am Alice"

// apply - runs immediately, arguments as array
say.apply(person, ["Hi"]); // "Hi, I am Alice"

// bind - doesn't run yet
const boundSay = say.bind(person, "Hey");
boundSay(); // "Hey, I am Alice"
...

```

Another common use case is **borrowing methods**. For example, `Array.prototype.slice.call(arguments)` converts the special `arguments` object (which is array-like but not a real array) into a true array. You can do this because `call` allows you to use a method from one object (`Array.prototype`) on another object (`arguments`) by setting what `this` should be. Similarly, you can use `Math.max.apply(null, numbers)` to find the maximum value in an array, since `Math.max` expects individual numbers, not an array.

It's also worth noting how **arrow functions** differ here. Arrow functions do not have their own `this` — they inherit it from the surrounding scope where they were defined. Because of that, using `call`, `apply`, or `bind` on arrow functions has no effect on `this`; they simply ignore those changes. That's one of the main differences between arrow functions and regular ones when it comes to controlling context.

Finally, there's a special rule when combining **`bind`** and **`new`**. If you create a bound function and later use it as a constructor (with the `new` keyword), the newly created object becomes `this`, overriding the bound value. This ensures that bound functions still behave correctly when used to create objects. So, binding only fixes `this` for normal function calls, not for object construction.

In short:

- **`call`** -> runs the function immediately, takes `this` and individual arguments.
- **`apply`** -> runs immediately, takes `this` and arguments as an array.
- **`bind`** -> returns a new function that remembers its `this` and optional pre-filled arguments for later.

These tools make JavaScript more flexible by letting you borrow functions, stabilize the meaning of `this`, and reuse logic in different contexts without rewriting it.

## Examples

```
```js
function show(prefix) {
  return prefix + this.name;
}

const user = { name: "Ava" };
show.call(user, ">> "); // ">> Ava"
show.apply(user, ["** "]); // "** Ava"
const bound = show.bind(user, ":: ");
bound(); // ":: Ava"
...

```

```
```js
// Borrowing array methods
function firstArg() {
 return arguments[0];
}

Array.prototype.slice.call(arguments); // turn arguments into a real array
...

```

```
```js
// Partial application with bind
function add(a, b, c) {
  return a + b + c;
}

const add5 = add.bind(null, 5);
add5(10, 20); // 35

```


...

Common misconceptions

1. bind mutates the original function. It returns a new function; the original remains unchanged.
2. You can rebind an arrow's this with call/apply/bind. Arrow functions' this is fixed lexically.
3. bind prevents new from changing this. Constructing a bound function with new binds this to the newly created instance.

Practice questions

1. Theory: In one paragraph, contrast call, apply, and bind and describe when each is most convenient.
2. Coding: Write a logger that prints this.tag; demonstrate changing receivers with call/apply and permanently with bind.
3. Coding: Create a multiply(a, b) function and a double(x) by partially applying multiply with bind.

What are higher-order functions?

What are higher-order functions

A higher-order function (HOF) is a function that takes one or more functions as inputs, returns a function as output, or both. This idea lets you separate "what to do" (the callback) from "how to do it" (the control flow). In practice, `Array.prototype.map`, `filter`, and `reduce` are classic HOFs: you provide small functions that describe how to transform, select, or combine items, and the HOF handles iteration, indexing, and collection building. HOFs are central to functional programming patterns, enabling composition, reuse, and testable logic where side effects are limited and easy to locate.

Beyond arrays, HOFs underpin event systems (registering handlers), async flows (then handlers, executor functions), and middleware/decorators (wrapping behavior to add cross-cutting concerns like logging, caching, and retry). Two related techniques often used with HOFs are partial application (pre-filling some arguments) and currying (turning a multi-argument function into a chain of one-argument functions). Used thoughtfully, HOFs reduce boilerplate and make data pipelines clear; used excessively, they can obscure simple logic behind many tiny layers. The balance is to encapsulate common patterns while keeping core steps explicit.

Examples

```
```js
// map, filter, reduce
const nums = [1, 2, 3, 4];
const squares = nums.map((n) => n * n);
const evens = nums.filter((n) => n % 2 === 0);
const sum = nums.reduce((acc, n) => acc + n, 0);
...
```
```

```
```js
// once: call a function only the first time
function once(fn) {
 let called = false,
 value;
 return function (...args) {
```

```

 if (!called) {
 called = true;
 value = fn.apply(this, args);
 }
 return value;
 };
}
...

```js
// curry for two-argument function
function curry2(fn) {
  return (a) => (b) => fn(a, b);
}
const add = (x, y) => x + y;
const add2 = curry2(add);
add2(3)(4); // 7
...

```

Common misconceptions

1. HOFs are only for arrays. They appear anywhere you pass or return functions (events, promises, middleware).
2. Currying and partial application are identical. Currying makes N unary functions; partial application fixes some arguments in place.
3. HOFs are always slower. In real apps, clarity and maintainability outweigh micro-overheads; measure before optimizing.

Practice questions

1. Theory: Explain why separating iteration mechanics (map/filter/reduce) from per-item logic improves testability.

2. Coding: Implement a `compose(...fns)` utility that composes functions right-to-left; test with simple math functions.
3. Coding: Implement a `memoize(fn)` that caches results based on arguments for pure functions.

What is callback hell and how to avoid it?

What is callback hell and how to avoid it

Callback hell is the tangled control flow that emerges when you nest many asynchronous callbacks inside one another. Each step depends on the previous step's result, so you place the next callback inside the prior one's success handler, creating a pyramid shape. This structure makes code hard to read, error paths easy to forget, and sequencing brittle (e.g., double invocations or missed errors). The problem is not callbacks themselves; it's unstructured composition that interleaves business logic with control flow and error handling.

The escape is to use composable abstractions. Promises represent future values and let you chain steps with `then` and `catch` in a flat structure; errors propagate by default, so you don't have to thread error callbacks manually. `async/await` builds on promises and lets you write sequential async steps top-to-bottom with `try/catch` for errors while still returning promises. Additional techniques include extracting named functions instead of inline anonymous ones, using `Promise.all` to run independent tasks in parallel, wrapping callback-style APIs with Promise constructors (`promisify`), and centralizing error handling so you have one place to log and recover.

Examples

```
```js
// From nested callbacks to promise chaining
fetch(url1)
 .then((r1) => r1.json())
 .then((d1) => fetch(url2 + d1.id))
 .then((r2) => r2.json())
 .then((d2) => console.log(d2))
 .catch((err) => console.error(err));
```
```

```
```js
// The same flow with async/await
async function run() {
 try {
```

```

const r1 = await fetch(url1);
const d1 = await r1.json();
const r2 = await fetch(url2 + d1.id);
const d2 = await r2.json();
console.log(d2);
} catch (e) {
 console.error(e);
}
}
...

```js
// Promisify a callback API
function delay(ms) {
  return new Promise((res) => setTimeout(res, ms));
}
...

```

Common misconceptions

1. async/await eliminates promises. It is syntax on top of promises; they remain fundamental.
2. Promises are always sequential. Promise.all, allSettled, and any provide parallel coordination.
3. Callbacks are obsolete. They are still used in many APIs; promisify or wrap them for composition.

Practice questions

1. Theory: Describe three concrete problems with deeply nested callbacks and how promises mitigate each.
2. Coding: Convert a "pyramid" of setTimeout calls into a promise chain with a delay(ms) helper.
3. Coding: Rewrite a two-step dependent async flow using async/await with proper try/catch and a finally clean-up.

Explain promises and how they work

Explain promises and how they work

In JavaScript, a Promise is a special object that represents the result of an operation that hasn't finished yet but will finish in the future — either successfully or with an error. It acts as a placeholder for a value that will be available later. Think of it as a box that starts empty but is guaranteed to eventually contain either a result or a reason why it failed. This idea lets JavaScript handle long-running tasks (like fetching data from a server) without freezing the browser or blocking other code from running.

A promise has three states:

1. Pending — it's still working on the task and hasn't produced a result yet.
2. Fulfilled (resolved) — the task completed successfully and produced a value.
3. Rejected — the task failed and produced an error or reason.

Once a promise moves from pending to either fulfilled or rejected, it stays settled — it never changes again. You can attach handlers to a promise to know when that happens. The `then()` method lets you specify what should happen when the promise fulfills, and `catch()` specifies what to do if it rejects. The `finally()` method runs no matter what, useful for cleanup (like hiding a loading spinner).

Under the hood, JavaScript schedules these `then` and `catch` callbacks to run as microtasks, meaning they execute right after the current synchronous code finishes but before other queued tasks like timers. This guarantees consistent ordering: your promise handlers always run after the code that created them, even if the promise resolves instantly.

Here's a simple example:

```
```js
const promise = new Promise((resolve, reject) => {
 setTimeout(() => resolve("Data loaded"), 1000);
});

promise
```

```

.then((result) => console.log(result)) // runs after 1s: "Data loaded"
.catch((error) => console.error(error))
.finally(() => console.log("Done"));
...

```

When this code runs, the `Promise` constructor starts an asynchronous operation (simulated by `setTimeout`). After 1 second, it calls `resolve`, marking the promise as fulfilled. JavaScript then calls the function you provided to `then`, passing in the result. Even though the work started earlier, the callback runs later, once the main thread is free.

The key strength of promises is chaining. The `then` method itself returns a new promise, allowing you to connect multiple asynchronous steps linearly. If a `then` handler returns a plain value, the next promise in the chain automatically resolves with that value. If it returns another promise, JavaScript waits for that promise to finish before moving on. If a handler throws an error, the next promise becomes rejected automatically. This consistent rule is what flattens complex asynchronous flows into simple, readable sequences.

Example:

```

```js
fetch("/user")
  .then((response) => response.json())
  .then((user) => fetch(`/posts?user=${user.id}`))
  .then((response) => response.json())
  .then((posts) => console.log(posts))
  .catch((err) => console.error("Something went wrong:", err));
...

```

Each `then` waits for the previous promise to settle before running. Errors in any step skip the rest and jump straight to `catch`, simplifying error handling that would otherwise require multiple nested callbacks.

Promises also provide combinator methods for working with multiple async tasks:

- `Promise.all([a, b, c])` runs all at once and fulfills when *_all_* succeed (or rejects on the first failure).
- `Promise.allSettled([a, b, c])` waits for all to finish, whether successful or not, returning their outcomes.
- `Promise.race([a, b, c])` settles as soon as *_any_* one promise settles (either fulfillment or rejection).
- `Promise.any([a, b, c])` fulfills on the first success and rejects only if *_all_* fail.

For instance:

```
```js
const api1 = fetch("/data1");
const api2 = fetch("/data2");

Promise.all([api1, api2])
 .then(([r1, r2]) => Promise.all([r1.json(), r2.json()]))
 .then(([d1, d2]) => console.log("Both done", d1, d2))
 .catch((err) => console.error("At least one failed", err));
```
```

Here, both API requests start together. `Promise.all` waits until both complete, then continues. This is efficient because it doesn't run them sequentially — they happen in parallel.

Now, let's understand all this through a real-world analogy.

Imagine ordering food at a restaurant. You place your order (start an async task), and the waiter gives you a token number — that's your promise. The token doesn't contain the food yet; it just represents a guarantee that your meal will eventually be ready or the restaurant will tell you it can't be served (error). While waiting, you're free to chat or browse your phone — your program isn't blocked. Later, when your meal is ready, the kitchen signals fulfillment, and your waiter delivers the result. If they run out of ingredients, the promise is rejected and you're informed of the error. The token (promise) itself never changes; it only moves from "pending" to "fulfilled" or "rejected." You can even attach multiple handlers — maybe one person is waiting for the food (then) and another for the bill (finally).

This analogy helps illustrate that promises aren't about doing tasks faster — they're about managing time and coordination. They let your code keep running other tasks while waiting for asynchronous operations, without getting tangled in callback pyramids.

In summary:

- A promise starts pending and settles once — either fulfilled or rejected.
- `then`, `catch`, and `finally` let you handle those outcomes cleanly.
- Promise chaining turns asynchronous flows into readable sequences.
- Combinators coordinate multiple async operations in parallel.
- Promises make asynchronous code predictable, composable, and far easier to reason about.

Examples

```
```js
// Basic chaining
fetch("/data.json")
 .then((r) => r.json())
 .then((data) => data.items)
 .catch((err) => console.error("failed", err))
 .finally(() => console.log("done"));
```

```js
// Combinators
const a = fetch("/a");
const b = fetch("/b");
Promise.all([a, b])
 .then(([ra, rb]) => Promise.all([ra.json(), rb.json()]))
 .then(([ja, jb]) => {
 /* use both */
 })
 .catch(console.error);
```
```

```
``js
// Creating a promise
function delay(ms) {
  return new Promise((res) => setTimeout(res, ms));
}
delay(200).then(() => console.log("after 200ms"));
``
```

Common misconceptions

1. A promise can resolve twice. Settle happens once; further resolve/reject calls are ignored.
2. then runs immediately. Handlers run in the microtask queue after current synchronous work.
3. Promise.all ignores rejections. It rejects fast on the first rejection; use allSettled to observe all outcomes.

Practice questions

1. Theory: Explain how returning a value, returning a promise, and throwing inside then affect the next link in the chain.
2. Coding: Implement a timeout wrapper that rejects if a promise doesn't settle within ms; test it with fetch.
3. Coding: Fetch three URLs in parallel, parse all JSON results, and handle partial failures gracefully using allSettled.

What is async/await and how is it different from promises?

What is async/await and how is it different from promises

async/await is syntax that builds on promises to make asynchronous code read like synchronous code. Declaring a function async means it always returns a promise. Inside an async function, await pauses that function until the awaited promise settles: if it fulfills, await yields its value; if it rejects, await throws that reason. This pause is cooperative—JavaScript's single thread is not blocked; the runtime schedules the rest of the async function as a microtask continuation. Errors are handled with ordinary try/catch around awaits, and finally works as expected, which makes control flow clearer than long then chains for sequential steps.

Despite the different look, the semantics are still promise-based. Awaiting a non-promise converts it to an already-fulfilled promise; returning a value from an async function resolves the returned promise with that value; throwing creates a rejected promise. For parallel work, start multiple promises first and then await them together (e.g., Promise.all). If you write await inside a plain for loop with a slow async operation, you serialize the work one-by-one; that's correct when order matters but wasteful when tasks are independent. The right pattern is to kick off all tasks, then await their combined completion. Understanding these patterns yields readable, efficient async code that behaves predictably under error conditions.

Examples

```
```js
// Sequential with try/catch
async function load() {
 try {
 const r1 = await fetch("/a");
 const a = await r1.json();
 const r2 = await fetch("/b?id=" + a.id);
 const b = await r2.json();
 return { a, b };
 } catch (e) {
 console.error(e);
 throw e;
 }
}
```

```

 } finally {
 console.log("done");
 }
}
...

```js
// Parallel start, then await together
async function loadBoth() {
    const pa = fetch("/a");
    const pb = fetch("/b");
    const [ra, rb] = await Promise.all([pa, pb]);
    const [a, b] = await Promise.all([ra.json(), rb.json()]);
    return { a, b };
}
...

```js
// Convert a .then chain to async/await
async function getItem() {
 const r = await fetch("/item");
 return r.json();
}
...

```

### Common misconceptions

1. await blocks the thread. It suspends the async function only; the event loop continues running other work.
2. async functions return plain values. They return promises; use await or then to get the result.
3. await is a replacement for Promise.all. It is not; use Promise.all for true parallelism when tasks are independent.

## Practice questions

1. Theory: Describe how `await` interacts with the microtask queue and why `try/catch` maps cleanly to promise rejection handling.
2. Coding: Convert a 3-step `then` chain into an `async` function with proper error handling and a `finally` block.
3. Coding: Given an array of URLs, implement (a) sequential fetch with `await` in a loop and (b) parallel fetch with `Promise.all`—compare behavior and performance.

# What are microtasks and macrotasks?

Explain microtasks and macrotasks

Modern JavaScript runs on a **single-threaded** event loop. It executes code, handles events and renders the UI by processing a queue of tasks one after the other. To keep the browser responsive, the runtime splits work into **macrotasks** (also called "tasks") and **microtasks**.

Understanding the difference helps you predict when your code will run and how it interacts with the rest of the page.

## ### What is a macrotask?

Macrotasks represent the "big" units of work handled by the browser. Each macrotask comes from a different source: an entire script block, a callback from an event (click, scroll, etc.), timers like `setTimeout()` and `setInterval()`, or I/O such as network responses. The event loop picks one macrotask from the queue, runs it to completion (including any synchronous code), then moves on to the next one. Because the browser only processes one macrotask at a time, long-running tasks can block rendering and make the page feel sluggish.

## ### What is a microtask?

Microtasks are tiny jobs that must run *after* the current macrotask finishes but *before* the browser does anything else. They're created by promises (`.then`, `.catch` and `.finally` callbacks) and the `queueMicrotask()` API. When a macrotask ends, the event loop empties the microtask queue — executing each microtask in order — **before** it renders the page or processes the next macrotask. This guarantees that promise handlers always run after the code that scheduled them, but before other asynchronous events.

## ### How the event loop orders them

The simplified event loop behaves like this:

1. Run a macrotask (e.g., an event handler or `setTimeout` callback).
2. When that macrotask finishes, execute **all** pending microtasks. If a microtask queues more microtasks, they run before the loop continues.
3. Render updates and handle UI painting.
4. Repeat with the next macrotask.

Because microtasks run **\*\*before\*\*** rendering and before the next macrotask, they have higher priority. This ensures that promise chains update data consistently and avoids race conditions. However, a microtask that continually re-queues itself can starve rendering and freeze the UI.

### ### Example: promises vs timers

Consider the following code:

```
``js
console.log("start");
setTimeout(() => console.log("timer"), 0);
Promise.resolve().then(() => console.log("promise"));
console.log("end");
``
```

The output is:

```
``
start
end
promise
timer
``
```

`setTimeout` schedules a **\*\*macrotask\*\*** with a minimum delay of 0 ms. Even though the delay is zero, it won't run until after the current macrotask and all microtasks finish.

`Promise.resolve().then(...)` schedules a **\*\*microtask\*\***, so its callback runs immediately after the current macrotask completes, and before the `setTimeout` callback.. As a result, "promise" appears before "timer".

### ### Real-world analogy



Imagine a restaurant kitchen. The head chef (the event loop) works on one **order** at a time (a macrotask). Between orders he must finish some **quick chores** (microtasks) like wiping the counter or plating dishes from finished orders. Even if a new order arrives (`setTimeout`), the chef won't start it until he has completed both the current order and all the small chores. Microtasks keep the kitchen tidy and ensure the next order starts with a clean slate.

### ### Summary

- **Macrotasks** include running scripts, event callbacks, timers (`setTimeout`/`setInterval``), and I/O. The event loop processes them one by one.
- **Microtasks** come from promises and `queueMicrotask()`. They run after the current macrotask, but before the browser renders and before the next macrotask.
- The microtask queue is emptied completely before moving on, even if microtasks add more microtasks.
- Using microtasks lets you schedule code to run as soon as possible without blocking user events; however, an infinite microtask loop can freeze the page.

### ### Common misconceptions

1. **Promises run immediately.** The executor function inside a `new Promise(...)` runs synchronously, but the `.then()` and `.catch()` callbacks run as microtasks after the current macrotask finishes.
2. **`setTimeout(fn, 0)` is synchronous.** Even with a delay of 0ms, `setTimeout` schedules a macrotask that runs after microtasks and the current macrotask.
3. **Microtasks can be ignored.** Because microtasks run before rendering, forgetting to empty the microtask queue (for example, by chaining lots of promises) can delay UI updates and cause performance issues.

### ### Practice questions

1. **Theory:** Describe the order of logs in the following snippet and explain why:

```
``js
console.log(1);

Promise.resolve().then(() => console.log(2));

queueMicrotask(() => console.log(3));
```

```
setTimeout(() => console.log(4), 0);
console.log(5);
...
```

2. **Coding:** Implement a function `nextTick(fn)` that schedules a callback as a microtask when available or falls back to `setTimeout(fn, 0)`.

3. **Coding:** Create a timer that uses `setTimeout` recursively instead of `setInterval`. Why might this be preferable when the task takes longer than the interval?

# Explain setTimeout, setInterval, and clearTimeout

Explain `setTimeout`, `setInterval` and cancellation functions

JavaScript timers let you schedule code to run later or repeatedly. They're essential for tasks like animations, polling a server or debouncing user input. The language provides four related functions: `setTimeout()`, `setInterval()`, `clearTimeout()` and `clearInterval()`. Understanding how they work and when to use each will help you write responsive programs.

## ### `setTimeout(fn, delay[, ...args])`

`setTimeout()` sets a one-off timer. It takes a callback function, a delay in milliseconds and optional arguments to pass to the callback. After at least `delay` milliseconds have elapsed and the current call stack is empty, the callback runs. The call returns a numeric **timer ID** which you can pass to `clearTimeout()` to cancel the timer. If `delay` is omitted or coerced to `0`, the callback is scheduled as soon as possible but still runs after all pending microtasks and the current macrotask.

```
``js
const id = setTimeout(() => {
 console.log("Hello after 1s");
}, 1000);

// Cancel the timeout before it runs:
clearTimeout(id);
``
```

Notes:

- JavaScript runs tasks one at a time using something called the "event loop." When you use `setTimeout()` to schedule a function, you're asking the browser to run that function after a certain number of milliseconds. However, this waiting time isn't an exact guarantee. If other code is already running or if the browser is busy handling things like user events or rendering the page, your timer's callback will wait its turn and may fire later than you requested.

Browsers also set a lower limit on how frequently timers can fire when they're nested deeply. After you schedule a few timers inside one another (beyond about five levels of nesting), most browsers

won't let them run faster than roughly every 4 milliseconds. This "clamping" prevents runaway loops from bogging down the tab. In practice, it means that when you repeatedly schedule one `setTimeout()` from inside another, the callbacks might not run back-to-back; the browser will slow them down slightly to keep the page responsive.

- The callback runs asynchronously and does not block subsequent code execution.

### `setInterval(fn, delay[, ...args])`

`setInterval()` repeatedly calls a function with a fixed delay between calls. It returns an **interval ID**. The callback continues to run until you call `clearInterval(id)`. Interval IDs share the same pool as timeout IDs; you can technically clear an interval with `clearTimeout()`, but it's better practice to match functions.

```
```js
let count = 0;
const intervalId = setInterval(() => {
  console.log("Tick", ++count);
  if (count === 5) {
    clearInterval(intervalId); // stop after 5 ticks
  }
}, 1000);
```
```

Important considerations:

When you use `setInterval()` to run a function repeatedly, the browser tries to call that function every X milliseconds regardless of how long the function itself takes to finish. If your function takes longer to run than the interval you've set, the browser can end up stacking multiple calls on top of each other, it might start the next call before the previous one has finished. This can slow your page or create unexpected overlaps. A common workaround is to use `setTimeout()` inside your function to schedule the next run only after it has completed. That way, each run waits for the previous one to finish before starting the timer again, so nothing overlaps.

Browsers also have built-in limits to prevent timers from running too rapidly. If you have a chain of timers (or intervals) nested more than a few levels deep, most browsers will automatically slow them

down so they don't fire more often than about once every four milliseconds. This "throttling" helps keep the page responsive by avoiding a flood of very fast, back-to-back timer events.

### ### Cancelling timers: `clearTimeout()` and `clearInterval()`

`clearTimeout()` and `clearInterval()` stop scheduled timers. Both functions accept the numeric ID returned by `setTimeout` or `setInterval` and cancel any pending execution. If the ID is invalid, nothing happens. Timer IDs are stored on the global `window` object, so clearing an interval with `clearTimeout()` technically works, but for readability you should use the matching clear function.

```
``js
// Schedule and then cancel a one-off timer
const timeoutId = setTimeout(doSomething, 5000);
clearTimeout(timeoutId);

// Schedule and then cancel a repeating timer
const intervalId = setInterval(doSomethingElse, 1000);
clearInterval(intervalId);
...

```

### ### Real-world analogy

Think of `setTimeout` as setting a single **alarm clock**: you set it for the future, go about your day, and when it rings you perform the task. `setInterval` is like a **repeating alarm** that keeps ringing every morning until you switch it off. The IDs returned are like the alarm handles; calling `clearTimeout` or `clearInterval` is pressing the off button.

### ### Summary

- **`setTimeout`** schedules a callback to run once after a delay. It returns an ID used to cancel the timer.
- **`setInterval`** schedules a callback to run repeatedly with a fixed delay between executions and returns an ID. Use `clearInterval` to stop it.

- For better control over periodic tasks (especially when the task itself takes time), prefer a recursive `setTimeout` over `setInterval`.
- `clearTimeout` and `clearInterval` cancel timers. Passing an invalid ID does nothing.
- Timers execute asynchronously; even a delay of 0 ms doesn't make the callback synchronous.

### ### Common misconceptions

1. **"`setTimeout(fn, 0)` runs immediately."** A zero delay still defers execution until after the current call stack and microtasks are finished.
2. **"`setInterval` is always precise."** Intervals don't compensate for execution time; if the callback takes longer than the interval, calls can overlap or drift. Use recursive timeouts when accuracy matters.
3. **"You can only cancel a timeout with `clearTimeout`."** While timeouts and intervals share an internal ID pool, mixing clear functions works but is bad practice.

### ### Practice questions

1. **Theory:** Why might repeated calls with `setInterval` drift over time? How does recursive `setTimeout` alleviate this?
2. **Coding:** Write a function `delay(ms)` that returns a promise and resolves after `ms` milliseconds using `setTimeout`.
3. **Coding:** Implement a countdown from 10 to 0 that prints a number every second and then prints "go!" using a timer. Include a button to cancel the countdown with `clearInterval`.

# What is debouncing and throttling?

Explain debouncing and throttling

Web pages often listen to frequent events such as `resize`, `scroll` and `keyup` also multiple user initiated events like clicks. Without control, handlers for these events can fire dozens of times per second, causing performance issues or unnecessary network requests. **Debouncing** and **throttling** are two patterns for rate-limiting function calls. They serve similar goals but behave differently.

## ### Debouncing

Debouncing delays a function call until a certain amount of time has passed since the last invocation. When an event triggers repeatedly, the debounce wrapper resets its timer each time; only when no new events occur during the wait period does it finally call the function. This technique reduces the number of calls by ensuring the function executes **once** after a burst of activity 【52406376895244+L118-L128】. Common use-cases include auto-saving form data or making API requests after a user stops typing.

Example debounce implementation:

```
```js
function debounce(fn, delay) {
  let timerId;
  return (...args) => {
    clearTimeout(timerId);
    timerId = setTimeout(() => fn.apply(this, args), delay);
  };
}

const searchInput = document.getElementById("search");
searchInput.addEventListener(
  "input",
  debounce(() => {
    // This runs only after the user stops typing for 300 ms
  })
);
```
```

```

 performSearch(searchInput.value);
 }, 300)
);
...

```

**\*\*Advantages:\*\*** Debouncing reduces resource consumption by preventing unnecessary calls.

**\*\*Disadvantages:\*\*** It introduces a delay before the action runs; the function won't execute until the user stops triggering events.

### ### Throttling

Throttling ensures that a function runs at most once per specified interval. When events occur more frequently than the limit, extra calls are ignored or delayed. Unlike debouncing, throttling does not wait for the activity to stop — it guarantees regular execution (e.g., every 100 ms) regardless of how many events fire. Throttling is useful for events like `scroll` or `mousemove` where you need periodic updates (such as updating a progress bar or showing scroll position).

A basic throttle implementation can use a flag and timestamps:

```

```js
function throttle(fn, limit) {
  let lastCall = 0;
  return (...args) => {
    const now = Date.now();
    if (now - lastCall >= limit) {
      lastCall = now;
      fn.apply(this, args);
    }
  };
}

```

```

window.addEventListener(
  "scroll",

```



```

throttle() => {
  console.log("Scroll position:", window.scrollY);
}, 100)
);
...

```

This version executes the callback immediately and then prevents further calls until the time window has passed. A more advanced implementation might schedule the final call after the event burst ends.

****Advantages:**** Throttling enforces a consistent rate of execution. It can keep UI updates smooth under heavy event load. ****Disadvantages:**** It may skip intermediate events; if the interval is too long, the handler might miss important changes.

Choosing between them

Both techniques limit how often a function runs:

- Use ****debounce**** when you want to wait for a "pause" in activity — for example, delaying an API call until the user stops typing.
- Use ****throttle**** when you want periodic updates regardless of continuous activity — for example, updating the scroll position every 100 ms during a scroll.

Real-world analogies

- ****Debouncing**** is like waiting until a friend finishes speaking before responding. If they keep talking, you hold off; you only reply once they pause.
- ****Throttling**** is like setting a timer to check your phone every minute. Even if you get dozens of notifications in between, you only look when the minute timer goes off.

Summary

Debouncing waits for a quiet period before running a function. Each time the button is clicked, a timer is started (or restarted). If no additional clicks happen during that timer window (say 3

seconds), the API call executes. If another click occurs before the delay elapses, the timer resets and the countdown starts over. This ensures that the API is called only once after the user has stopped clicking.

Throttling allows the action to happen immediately, but then suppresses additional triggers for a set period. On the first click, the API call runs right away. For the next few seconds (again, say 3 seconds), any further clicks are ignored. Only after that interval has passed will a new click result in another API call. This guarantees that the function is called at most once per interval, regardless of how often the user clicks.

Debounce: Imagine clicking a button that triggers an API call. When the function is debounced, a delay is introduced between the moment the user interacts and the moment the call actually fires. For example, if you set a delay of three seconds, the API call will execute only after three seconds have passed without any further clicks. If the user clicks again before those three seconds are up, the delay resets, and the function waits for another full three seconds from the most recent click before firing.

Throttle: With throttling, the first click triggers the API call immediately, just as it normally would. However, subsequent clicks are ignored until a set period has elapsed. Continuing the three-second example, after the initial call runs, no additional calls will occur until three seconds have passed. Only then will another click trigger a new API call.

- Debouncing reduces call count but adds delay. Throttling provides regular updates but may drop events.

- Both patterns are typically implemented using `setTimeout` and timestamps.

Common misconceptions

1. *****Debounce and throttle are the same.***** They both rate-limit calls, but debounce waits for inactivity while throttle guarantees periodic execution.
2. *****Debounce functions always improve UX.***** Over-debouncing can make an interface feel unresponsive, because the action waits until the user stops for long enough.
3. *****Throttle always catches every event.***** Throttling discards events between intervals; if the interval is too long, important changes may be skipped.

Practice questions

1. **Coding:** Write your own `debounce` function that accepts a callback and a delay, and demonstrate it with a `keyup` handler that sends an API request only when the user stops typing.
2. **Coding:** Implement a `throttle` function that ensures a callback fires once every 200 ms, and test it on a `scroll` event.
3. **Theory:** Describe a scenario where debouncing would be inappropriate but throttling would work well, and vice versa.

What is event bubbling and capturing?

Explain event bubbling and capturing

When you click, type or otherwise interact with elements on a page, the browser fires an "event" that moves through the document tree. How and where you respond to that event depends on understanding the two propagation phases:

Capturing (also called trickling) - The event is dispatched from the top of the DOM hierarchy (window or document) and moves downward through each ancestor element until it reaches the actual target. Listeners registered with `addEventListener()` using `{capture: true}` fire in this phase, starting from the outermost ancestor and ending with the target. Capturing is disabled by default because most code relies on bubbling, but it's available when you need to intercept an event before it hits its destination.

Target phase - Once the event reaches the element that originally triggered it (for example, the button you clicked), any event listeners attached directly to that element run, regardless of whether they were registered for capturing or bubbling.

Bubbling - After the target has processed the event, it bubbles back up the DOM tree. The event moves from the target's parent up to the root, invoking listeners along the way. This is the default behavior for most event types. When you call `addEventListener()` without options, you're registering a listener that will fire during this bubbling phase.

Understanding these phases allows you to decide where to attach your handlers:

Normal use case (bubbling): Attach a listener on a parent element to handle events from many children (a technique known as event delegation). For example, a click on a list item bubbles up to the `<ul>`, and a click handler on the `<ul>` can inspect `event.target` to determine which `<li>` was clicked.

Capturing use case: Attach a listener with `{capture: true}` when you need to act before the event reaches the target. This can be useful for intercepting keyboard shortcuts or gestures at the document level or for implementing custom UI controls that need to block events from hitting certain elements.

Several other points are important for both beginners and experienced developers:

You can stop an event from continuing along its propagation path by calling `event.stopPropagation()`. To prevent other handlers on the same element from running as well, call `event.stopImmediatePropagation()`.

The properties `event.target` and `event.currentTarget` differ: `event.target` is the element where the event originated, while `event.currentTarget` is the element whose listener is currently executing.

Not all events bubble (e.g. `focus` and `blur` do not), and some behave inconsistently across older browsers, so always test your code in the environments you support.

By controlling where in the capture/bubble phases your handlers run and understanding how the event travels, you can build more efficient and responsive UIs, whether you're just starting out or writing complex component systems.

Bubbling in action

Consider nested elements:

```
```html
<div id="outer">
 <button id="inner">Click me</button>
</div>
```
```

If you add click handlers on both elements:

```
```js
document
 .getElementById("outer")
 .addEventListener("click", () => console.log("outer"));

document
 .getElementById("inner")
 .addEventListener("click", () => console.log("inner"));
```
```

...

Clicking the button logs:

...

inner

outer

...

The click originates at `#inner`, triggers its handler, then bubbles up to `#outer` and runs the outer handler.

Capturing

Capturing is the mirror image of bubbling. The event starts at the top of the tree and flows down to the target, invoking handlers registered for the capture phase. To attach a listener during capture, provide `{ capture: true }`:

```
```js
```

```
document
```

```
 .getElementById("outer")
```

```
 .addEventListener("click", () => console.log("outer capture"), {
```

```
 capture: true,
```

```
 });
```

```
document
```

```
 .getElementById("inner")
```

```
 .addEventListener("click", () => console.log("inner"));
```

```
```
```

Now clicking the button logs:

...

outer capture

inner

...

The capturing handler runs before the target and bubbling handlers because the event travels down the tree first.

Controlling propagation

Inside an event handler you can stop the event from continuing along the propagation path:

- `event.stopPropagation()` halts further propagation through both capturing and bubbling phases.
- `event.stopImmediatePropagation()` additionally prevents other handlers on the same element from running.

Stopping propagation is useful when you don't want parent elements to receive the event (for example, clicking inside a modal dialog should not close the underlying page).

Real-world analogy

Imagine dropping a stone into a pond. The initial splash is the **target phase**. Ripples that travel outward to the shore resemble **bubbling**: the disturbance spreads from the point of impact to the edges. Now imagine someone at the shore sending a vibration back toward the point where the stone hit — that's **capturing**: energy travels from the outside in.

Summary

- DOM events have three phases: capturing (from root down), target, and bubbling (from target up).
- Most event listeners fire during bubbling by default. Pass `{ capture: true }` to listen during the capturing phase.
- `stopPropagation()` and `stopImmediatePropagation()` let you prevent an event from continuing along its path.
- Understanding propagation helps with advanced patterns like **event delegation** and preventing unwanted side effects.

Common misconceptions

1. **Events only bubble.** Many events bubble by default, but capturing exists and is enabled by passing `{ capture: true }`.
2. **`event.target` equals `event.currentTarget`.`** `event.target`` is the element where the event originated; `event.currentTarget`` is the element whose listener is currently executing. They differ when handling bubbling events on ancestors.
3. **Bubbling can't be stopped.** Calling `event.stopPropagation()`` halts the event's travel up (and down) the DOM tree.

Practice questions

1. **Theory:** In what order do the following handlers fire when capturing and bubbling are both used? Explain why:

```
```html
<div id="parent">
 <button id="child">Click</button>
</div>

<script>
 parent.addEventListener("click", () => console.log("parent capture"), {
 capture: true,
 });
 parent.addEventListener("click", () => console.log("parent bubble"));
 child.addEventListener("click", () => console.log("child"));
</script>
...```
```

2. **Coding:** Build a modal dialog component. Clicking outside the dialog should close it; clicking inside should stop propagation so the backdrop doesn't close.



3. **Coding:** Create a custom event and dispatch it on a child element. Add listeners at different phases and log `event.target` and `event.currentTarget` to see how they differ.

## How does event delegation work?

Explain how event delegation works

Attaching event listeners to every element in a list can be inefficient, especially when the list is long or dynamic. **Event delegation** solves this problem by taking advantage of event bubbling: instead of listening on each child element, you attach a single listener on a common ancestor. When an event bubbles up, you check which child triggered it and handle it appropriately. This approach reduces the number of handlers and works for elements added later.

### ### Why delegation works

When a user interacts with an element, the event travels up through its ancestors during the bubbling phase. Because of this, a handler on a parent element can "see" events from its children. In a delegated handler you inspect `event.target` (the original element that fired the event) and decide whether to respond. You often use `element.closest()` to ensure the target matches the selector you care about.

### ### Basic example: highlighting table cells

Suppose you have a table with many cells:

```
``html
<table id="data-table">
 <tr>
 <td>Cell A</td>
 <td>Cell B</td>
 </tr>
 <tr>
 <td>Cell C</td>
 <td>Cell D</td>
 </tr>
 <!-- More rows... -->
</table>
```

```
...
```

Instead of adding a click listener on each `td`, you can delegate:

```
```js
const table = document.getElementById("data-table");

table.addEventListener("click", (event) => {
  // Find the nearest td; ignore clicks outside cells
  const cell = event.target.closest("td");
  if (!cell || !table.contains(cell)) return;

  // Remove existing highlight
  table
    .querySelectorAll(".selected")
    .forEach((td) => td.classList.remove("selected"));
  cell.classList.add("selected");
});
```
```

Because the listener is on the `

### ### Delegation with data attributes

Event delegation also makes it easy to build component APIs. For example, a menu might contain buttons with `data-action` attributes indicating what to do:

```
```html
<ul id="menu">
  <li><button data-action="save">Save</button></li>

```

```

<li><button data-action="load">Load</button></li>
<li><button data-action="delete">Delete</button></li>
</ul>
...

```js
document.getElementById("menu").addEventListener("click", (event) => {
 const button = event.target.closest("button");
 if (!button) return;
 const action = button.dataset.action;
 switch (action) {
 case "save":
 saveFile();
 break;
 case "load":
 loadFile();
 break;
 case "delete":
 deleteFile();
 break;
 }
});
...

```

Adding new actions later requires only new HTML. The single listener on `

` handles all current and future buttons.

### ### Benefits of event delegation

- **\*\*Performance:\*\*** Fewer listeners reduce memory usage and avoid attaching thousands of handlers to similar elements.

- **Dynamic content:** Elements created after the page loads are still handled, because the parent listener continues to receive events.
- **Simpler management:** One handler centralizes logic; you don't need to add or remove listeners when elements appear or disappear.

### Real-world analogy

Imagine a party where guests bring their own cups. Instead of stationing a waiter at every table to collect empty cups, you place a single bin near the exit. As guests leave (events bubble up), they drop their cups into the bin (the delegated handler). There's no need to monitor each seat.

### Summary

- **Event delegation** uses event bubbling to handle events from multiple children with one parent listener.
- Inside the delegated handler, inspect `event.target` or use `closest()` to identify the relevant child.
- Delegation is great for lists, tables, menus and any dynamic content that may change after initial rendering.

### Common misconceptions

1. **"Delegation only works for click events."** Delegation works for any event that bubbles (e.g. `input`, `mouseover`).
2. **"You must use capturing to delegate."** Delegation relies on bubbling; capture phase isn't needed.
3. **"Delegation is slower."** On the contrary, delegating reduces overhead by attaching fewer listeners.

### Practice questions

1. **Coding:** Create a list with "Delete" buttons next to each item. Use event delegation on the list `<ul>` to handle clicks and remove the corresponding `<li>`.
2. **Coding:** Build a tab component where clicking a tab activates its panel. Use delegation so new tabs added later still work.

3. **Theory:** Explain why delegation fails on events that do not bubble (e.g., `focus`). How can you handle those events on many elements?

# Difference between document, window, and this in different contexts

Difference between `document`, `window` and `this` in different contexts

JavaScript in the browser exposes several objects representing different parts of the environment. Two of the most important are the **`window`** and **`document`** objects, and the keyword **`this`** behaves differently depending on how a function is called. Mixing them up can lead to subtle bugs. Let's clarify their roles.

## ### `document`: the page itself

The `document` object represents the web page loaded in the browser. It is part of the DOM (Document Object Model) and provides methods and properties to access and manipulate HTML elements. You can select elements by ID, class, tag name or CSS selector and modify them at runtime. For example, `document.getElementById('title')` returns an element with the given ID, and `document.querySelectorAll('.card')` returns a list of elements with the class `card`. Internally, `document` is accessible via `window.document`, but you rarely prefix it. The DOM defines a logical structure for documents, allowing us to create, manipulate or delete elements and attributes.

## ### `window`: the browser or tab

The `window` object represents the browser window or frame itself. It sits at the top of the Browser Object Model (BOM) and exposes features like screen size, history and location. Properties such as `window.innerWidth`, `window.location`, `window.history` and methods like `alert()`, `setTimeout()`, `open()` or `close()` belong to the `window` object. In a browser, all global variables and functions become properties of `window`. For instance, declaring `var x = 5;` makes `window.x === 5` true. This is why you can often omit `window.` when calling `alert()` or `setTimeout()`. Note that the BOM isn't standardized, so some properties may vary between browsers.

## ### The value of `this`

`this` is a special keyword whose value is determined by how a function is invoked. It does **not** point to the function itself, but to the object on which the function was called.

- **Global context:** Outside any function, `this` refers to the global object (`window` in browsers) in non-strict mode. In strict mode, `this` in the global scope is `undefined`.

- **\*\*Simple function call:\*\*** When a function is called without an explicit receiver, `this` is the global object in non-strict mode, or `undefined` in strict mode.
- **\*\*Object method:\*\*** When a function is invoked as a method of an object (`obj.method()`), `this` is bound to that object.
- **\*\*DOM event handler:\*\*** In an event handler added via `element.addEventListener`, `this` is set to the element on which the event fired. In inline event handlers (e.g., `onclick="..."`), `this` also refers to the element.
- **\*\*Arrow functions:\*\*** Arrow functions do not have their own `this`; instead they capture the `this` value from the surrounding lexical scope. This makes them unsuitable for event handlers if you rely on `this` pointing to the element.

### ### Examples

```

```js
// Global context
console.log(this === window); // true in non-strict mode

function show() {
  console.log(this);
}

show(); // logs 'window' (or 'undefined' in strict mode)

const person = {
  name: "Ada",
  greet() {
    console.log(this.name);
  },
};

person.greet(); // 'Ada'; 'this' refers to the object

document.getElementById("btn").addEventListener("click", function () {
  console.log(this === document.getElementById("btn")); // true; 'this' is the element
});

```



```
document.getElementById("btn").addEventListener("click", () => {  
  console.log(this === window); // true; arrow functions inherit 'this' from the outer scope  
});  
...
```

Real-world analogy

Think of `document` as the **blueprint** for a house and its contents. It lists every room and piece of furniture and allows you to remodel the house on the fly. The `window` is the **actual house** — the physical container that holds the blueprint and provides features like doors (navigation) and windows (screen properties). The keyword `this` is like a pronoun whose meaning depends on who is speaking — it refers to the current "actor" at the moment the code runs.

Summary

- `document` represents the loaded web page and provides methods to access and modify its elements.
- `window` represents the browser window and exposes global functions, timer APIs and browser-specific features.
- `this` is bound at call time and varies depending on whether the function is called globally, as a method, as a constructor, or as an event handler.
- Arrow functions capture `this` from their lexical scope instead of creating a new binding.

Common misconceptions

1. **"document and window are the same."** While `document` is a property of `window`, it specifically refers to the DOM of the page, whereas `window` includes the DOM plus browser APIs like `alert()` and `location`.
2. **"this always refers to the object where a function is defined."** `this` depends on how a function is called, not where it's defined.
3. **"Arrow functions make this predictable everywhere."** Arrow functions inherit `this` from their outer scope; using them in event listeners or object methods may not give you the element or object you expect.

Practice questions

1. **Coding:** Write a function that logs `this` and call it: (a) globally; (b) as a method of an object; (c) as an event handler. Observe how `this` changes.
2. **Coding:** Create a custom object with a method that references `this`. Bind that method to another object using `.call()` or `.apply()` and observe the output.
3. **Theory:** Why is it generally unnecessary to write `window.alert()` instead of `alert()` in browser code? What happens to `this` if you enable strict mode?

Explain DOM vs BOM

Explain DOM vs BOM

JavaScript runs in the context of a web browser, which provides two related but distinct models: the **Document Object Model (DOM)** and the **Browser Object Model (BOM)**. Both expose objects and methods to your scripts, but they serve different purposes.

DOM (Document Object Model)

The DOM is a **standardized programming interface** that represents an HTML or XML document as a tree of nodes. When an HTML document is loaded, the browser parses it and creates a DOM tree. JavaScript can traverse this tree, create new elements, remove existing ones, and update attributes and styles. Functions like `getElementById()`, `querySelector()`, `appendChild()` and properties like `document.body` or `document.title` allow you to manipulate the structure and content of the page. As GeeksforGeeks notes, the DOM defines the logical structure of documents and provides methods to access and modify tags, IDs, classes and attributes.

BOM (Browser Object Model)

The BOM refers to the **collection of objects provided by the browser** that let JavaScript interact with the browser itself rather than the document. Unlike the DOM, there is no formal standard for the BOM, so implementations vary slightly among browsers. The root of the BOM is the `window` object. It exposes properties like `navigator` (information about the browser), `location` (current URL), `history` (the user's navigation history), `screen` (screen size and color depth), and `document`. The BOM also includes methods for controlling windows such as `open()`, `close()`, `moveTo()` and `resizeTo()`. For example:

```
```js
// Using the BOM

console.log(window.location.href); // current URL

console.log(window.navigator.userAgent); // browser user agent string

const newWin = window.open(
 "https://example.com",
 "_blank",
 "width=400,height=300"
);
```

```
// ... later

newWin.close();

...

```

### ### Key differences

Aspect	DOM	BOM
Purpose	Represents and manipulates the content of a web page	Represents the browser window and its features
Standardization	Fully standardized by the W3C	Not formally standardized; implementation varies
Root object	<code>document</code>	<code>window</code>
Examples of API	<code>getElementById()</code> , <code>querySelector()</code> , <code>createElement()</code>	<code>window.open()</code> , <code>window.close()</code> , <code>navigator</code> , <code>screen</code>
Access to page data	Directly manipulates elements and attributes	Does not directly access page content
Lifespan	Exists only when a document is loaded	Exists as long as the browser window/tab exists

### ### Real-world analogy

Think of a website as a **book** in a library. The DOM is the table of contents and the pages of the book; it represents the structure of the content and lets you read or edit chapters. The BOM is the **building** that houses the library — it includes the doors, windows and elevators. It lets you open a new room (tab), check the building's address (URL) or find out the size of the reading rooms (screen).

### ### Summary

- The **DOM** is a standardized API for representing and interacting with documents. It lets you create, access and modify HTML elements and their attributes.
- The **BOM** is a browser-specific collection of objects that let you interact with the environment: windows, frames, navigation history and screen information.
- They complement each other: `document` (DOM) is a property of `window` (BOM), but the DOM focuses on the page content while the BOM focuses on the container.

### ### Common misconceptions

1. **"The BOM is part of the DOM."** The BOM includes `window` and related objects which exist outside the document. The DOM deals only with the document's content and structure.
2. **"The BOM follows a standard API."** There is no formal specification for the BOM; browser vendors implement it differently, so not all methods behave identically across browsers.

3. **\*\*"You can manipulate the page through the BOM alone."\*\*** While the BOM provides access to high-level browser features, it does not provide methods to access or change specific HTML elements.

### ### Practice questions

1. **\*\*Coding:\*\*** Use the DOM to create a new `

` element, set its text content, append it to the body and then use the BOM to open a new tab displaying its inner text.
2. **\*\*Theory:\*\*** List three properties or methods available on `window` but not on `document`. What are their purposes?
3. **\*\*Theory:\*\*** Why can't you rely on the BOM for consistent behavior across browsers? What precautions should you take when using methods like `window.open()`?

# What are Web APIs?

Explain Web APIs

At its core, JavaScript is a programming language that manipulates numbers, strings, objects and arrays. But when you run JavaScript in a web browser you get access to a rich set of **Web APIs** — additional objects and functions built into the browser that let you do things like manipulate the DOM, fetch data over the network, store data locally and interact with device hardware. These APIs are not part of the JavaScript language itself; they are provided by the host environment.

## ### What are APIs?

An **Application Programming Interface (API)** is a set of constructs made available in a programming environment to perform complex tasks more easily. APIs abstract away underlying implementation details and expose a convenient interface. For example, instead of writing low-level code to process audio, you can call the Web Audio API which wraps that complexity.

## ### Browser APIs vs third-party APIs

Web APIs fall into two broad categories:

- **Browser (built-in) APIs** are integrated into the browser. They let you access data from the browser and the computer (e.g., geolocation, camera, file system) and perform tasks like drawing graphics or storing data offline. The Web Audio API, for instance, provides constructs to manipulate audio in the browser.
- **Third-party APIs** are delivered by external services such as Google Maps or Facebook and allow you to integrate their functionality into your app. Unlike browser APIs, you need to load these via external scripts or modules.

## ### Relationship between JavaScript, APIs and other tools

MDN explains that client-side programming typically involves several layers: **JavaScript** (the core language), **browser APIs** on top of it, **third-party APIs**, and optionally libraries or frameworks. JavaScript by itself can't, for example, fetch a resource over the network; it needs the Fetch API. Libraries like React and frameworks like Angular build on these APIs to provide higher-level abstractions.

### ### Common categories of browser APIs

Modern browsers provide a huge number of APIs. Here are some of the most common categories:

Here is the table reformatted for consistent alignment:

Category	Examples and use cases
<b>**Document manipulation**</b>	The DOM API lets you create, remove and update HTML and CSS; it underlies all dynamic web pages.
<b>**Network requests**</b>	The Fetch API and older <code>XMLHttpRequest</code> allow you to request data from servers without reloading the page, enabling AJAX and dynamic content.
<b>**Graphics**</b>	The Canvas API and WebGL API let you draw 2D and 3D graphics, manipulate images and build games or visualizations.
<b>**Audio/video**</b>	APIs like <code>HTMLMediaElement</code> , the Web Audio API and WebRTC enable audio playback, capture from microphones and cameras, and real-time communication.
<b>**Device access**</b>	The Geolocation API retrieves the user's location, while other APIs allow access to device sensors and hardware.
<b>**Client-side storage**</b>	The Web Storage API ( <code>localStorage</code> / <code>sessionStorage</code> ) and IndexedDB provide ways to store data on the client without sending it to the server.

### ### Example: Fetching data with the Fetch API

```
```js
// Fetch JSON data from a server
fetch("https://api.example.com/data")
  .then((response) => response.json())
  .then((data) => {
    console.log("Received data:", data);
  })
  .catch((err) => console.error("Request failed:", err));
```
```

The `fetch()` function is part of the Fetch API. It returns a promise that resolves to a `Response` object; calling `response.json()` parses the JSON body. Behind the scenes, the browser handles networking and security. Without this API you'd need to rely on older, less consistent interfaces.

### ### Real-world analogy

Web APIs are like **\*\*power outlets\*\*** in a house. The JavaScript language itself is the wiring and switches; the outlets (APIs) let you plug in powerful appliances like vacuum cleaners (audio/video), ovens (file access) or televisions (graphics) without worrying about how electricity is generated. Different rooms (categories) offer different outlets tailored to specific devices.

### ### Summary

- Web APIs are sets of functionality provided by the browser or third parties that extend what JavaScript can do.
- **Browser APIs** expose features like DOM manipulation, network requests, graphics, media, device access and storage.
- **Third-party APIs** offer services from external providers such as maps, payments or social features.
- APIs sit on top of JavaScript; libraries and frameworks build on top of APIs to create higher-level abstractions.

### ### Common misconceptions

1. **"Web APIs are part of JavaScript."** APIs are provided by the browser or external services; they are not defined by the ECMAScript language specification.
2. **"All APIs require a network."** Many browser APIs (DOM, Canvas, Web Audio) operate entirely locally and don't involve network requests.
3. **"Using a library replaces Web APIs."** Libraries and frameworks build on Web APIs. Even when using React or jQuery, under the hood they still call DOM methods or Fetch.

### ### Practice questions

1. **Coding:** Use the Geolocation API to get the user's current latitude and longitude and display it on the page. Handle errors if the user denies permission.
2. **Coding:** Build a simple drawing app using the Canvas API that lets the user draw lines with the mouse.
3. **Theory:** Explain the difference between a browser API like `localStorage` and a third-party API like the Google Maps API. What extra steps are needed to use the latter?



# What is localStorage, sessionStorage, and cookies?

Explain `localStorage`, `sessionStorage` and cookies

Web applications often need to remember information between page loads. Browsers provide several mechanisms to store data on the client side: **cookies**, **sessionStorage** and **localStorage**. Each has different characteristics regarding scope, lifetime, size and how data travels.

## ### Cookies

Cookies are small pieces of data (name/value pairs) that a server sends to the browser. The browser stores cookies and sends them back to the same server with subsequent requests. Cookies allow web applications to remember state across HTTP requests, which are otherwise stateless. A typical use case is session management: after a user signs in, the server sets a cookie containing a session ID; on later requests the browser includes that cookie so the server knows the user is authenticated. Cookies are also used for personalization and tracking.

Key properties of cookies:

- **Size and number limits:** Browsers restrict the number of cookies per domain and limit each cookie to around 4KB.
- **Automatic transmission:** Cookies are sent with every HTTP request to their associated domain, which can impact performance on slow connections.
- **Expiration:** Cookies can be set with an `Expires` or `Max-Age` attribute to persist for a given time. Without those attributes they are **session cookies** and are deleted when the browser session ends.
- **Access:** In JavaScript you can read and write cookies using `document.cookie`, but they are not as straightforward to manage compared to Web Storage.

## ### Web Storage API: `sessionStorage` and `localStorage`

The Web Storage API provides a simpler key/value storage mechanism than cookies. Data stored via Web Storage never travels to the server; it stays entirely on the client. Two separate storage areas exist per origin:

- **`sessionStorage`** is scoped to the **browser tab** and **origin**. Each tab (and its iframes) gets its own session storage. Closing the tab clears the data. Data is not shared across tabs or browser windows. You access it via `window.sessionStorage`.
- **`localStorage`** is scoped to the **origin** only. All pages from the same origin share the same storage, and the data persists even when the browser is closed and reopened. You access it via `window.localStorage`.

Both return a `Storage` object with methods:

```
``js
localStorage.setItem("name", "Ada"); // store
const value = localStorage.getItem("name"); // retrieve
localStorage.removeItem("name"); // delete one item
localStorage.clear(); // delete all items

// sessionStorage works similarly
sessionStorage.setItem("counter", "1");
...

```

PLEASE NOTE IMPORTANT:

Choosing among cookies, `sessionStorage` and `localStorage` depends on what you need to store and who needs to read it.

### **When to use cookies**

- **Server-side needs**: Cookies are automatically included in HTTP requests to their associated domain. This makes them suitable for storing session identifiers, authentication tokens or user preferences that the server needs to see on each request.
- **Cross-page or cross-tab persistence**: Cookies persist across tabs and sessions if you set an expiration date, so they can maintain login state or language preferences.
- **Small data**: Because cookies are limited to a few kilobytes each and contribute to the size of every request, they should only hold small pieces of information.

Although cookies are often set by the server via the ``Set-Cookie`` header, client-side scripts can read and write them using ``document.cookie``. Keep in mind that cookies marked as ``HttpOnly`` by the server cannot be accessed from JavaScript, and sensitive cookies should always be set with the ``Secure`` and ``SameSite`` attributes to mitigate security risks.

### **\*\*When to use ``sessionStorage``\*\***

- **\*\*Per-tab or per-window data\*\***: If you need to store temporary state that is specific to a single browser tab—for example, progress through a multi-step form or data that should reset when the user closes the tab—``sessionStorage`` is ideal. Each tab gets its own storage area, and the data is cleared when that tab is closed.
- **\*\*Client-only data\*\***: Use ``sessionStorage`` for data that the server doesn't need, since it never leaves the browser.

### **\*\*When to use ``localStorage``\*\***

- **\*\*Persistent client-side data\*\***: ``localStorage`` retains data across browser sessions. It's well suited for things like theme preferences, "remember me" flags, or other settings that should survive a page refresh or browser restart.
- **\*\*Larger storage needs\*\***: Browsers typically allow several megabytes of storage via ``localStorage``, so it can hold more data than cookies. However, it's still best to avoid storing highly sensitive information, as any script running on the page can read it.

### **\*\*Summary of selection criteria\*\***

1. **\*\*Does the server need to read it?\*\*** Use cookies for data that must accompany every request (e.g., session IDs). Use Web Storage for data the server never needs.
2. **\*\*How long should it last?\*\*** Use ``sessionStorage`` for temporary, per-tab data; ``localStorage`` for data that persists until explicitly cleared; cookies for short-term or long-term server-visible data depending on their expiration.
3. **\*\*How big is the data?\*\*** Cookies are limited to a few kilobytes and should stay small. ``sessionStorage`` and ``localStorage`` can hold significantly more.
4. **\*\*Security considerations\*\*** Cookies can be protected with ``HttpOnly`` and ``Secure`` flags and sent over HTTPS, making them suitable for credentials. Data in Web Storage is accessible to any script running in that origin, so don't store secrets there.

In practice, it's common to see both client-side and server-side code setting and reading cookies. For example, a front-end app might set a cookie to track a non-essential preference or to trigger analytics, while the server uses its own cookies for authentication.

Important characteristics:

- **Persistent vs temporary:** `localStorage` persists across sessions; `sessionStorage` lasts until the tab or window is closed.
- **Per-origin isolation:** Storage is partitioned by origin; pages from different domains cannot read each other's storage.
- **Synchronous operations:** Reading and writing to Web Storage are synchronous; large or frequent writes can block the main thread.
- **Capacity:** Browsers typically allow several megabytes of storage, far more than the few kilobytes allowed for cookies.

### ### Comparing cookies, `sessionStorage` and `localStorage`

| Feature             | Cookies                                                            | <code>sessionStorage</code>                  | <code>localStorage</code>                         |
|---------------------|--------------------------------------------------------------------|----------------------------------------------|---------------------------------------------------|
| Lifetime            | Expires/Max-Age controlled; session cookies deleted on session end | Cleared when the tab is closed               | Persists until explicitly cleared                 |
| Scope               | Sent with each request to the same domain                          | Available only within the current tab        | Shared across all tabs/windows of the same origin |
| Storage size        | ~4 KB per cookie; limited number                                   | Several MB                                   | Several MB                                        |
| Data sent to server | Yes, automatically                                                 | No                                           | No                                                |
| Use cases           | Sessions, authentication, personalization, tracking                | Per-tab temporary state (e.g., wizard steps) | Persistent client-side data, preferences, caches  |

### ### Real-world analogy

Imagine a hotel. A **cookie** is like a hotel key card that you must present every time you enter your room; the hotel (server) issues and recognizes the card to know which room you should access. **`sessionStorage`** is like a personal note pad you carry during your stay; it exists only as long as you're in the hotel and doesn't leave the building. **`localStorage`** is a storage locker you rent in town; it remains yours even when you leave the hotel and come back later.

### ### Summary

- **Cookies** are small name/value pairs sent to and from the server. They enable sessions, personalization and tracking but are limited in size and number.
- **`sessionStorage`** stores data per tab and origin. It's cleared when the tab closes and isn't shared across tabs.
- **`localStorage`** stores data per origin and persists across browser sessions until explicitly cleared.

- Unlike cookies, data in Web Storage (both session and local) is not sent to the server and can hold much more data.

### ### Common misconceptions

1. **\*\*\*`localStorage` is secure storage.\*\*\*** Data in Web Storage is accessible to any script on the page. Do not store sensitive information (like passwords) there.
2. **\*\*\*`sessionStorage` persists across tabs.\*\*\*** Each tab has its own session storage; closing the tab deletes its data.
3. **\*\*\*Cookies can hold large amounts of data.\*\*\*** Each cookie is limited to a few kilobytes and browsers limit the number of cookies per domain.

### ### Practice questions

1. **\*\*Coding:\*\*** Store a user's preferred theme ("dark" or "light") in `localStorage` and apply it when the page loads.
2. **\*\*Coding:\*\*** Create a page counter using `sessionStorage` that increments every time the user reloads the tab but resets when the tab is closed.
3. **\*\*Theory:\*\*** Describe scenarios where cookies are necessary instead of Web Storage. What security attributes (e.g., `HttpOnly`, `Secure`) should be set on cookies used for authentication?

# What is CORS and how does it work?

Explain CORS and how it works

Modern web applications often need to request resources from different domains — for example, a single-page app hosted on `example.com` may fetch data from an API at `api.example.net`. **Cross-Origin Resource Sharing (CORS)** is a mechanism that allows (or disallows) such cross-origin requests in a secure way. To appreciate why CORS exists, you first need to understand the **same-origin policy**.

## ### Same-origin policy

Browsers enforce a security model called the **same-origin policy**: a script running on a web page can only read data from the same protocol, domain and port that served it. This prevents malicious pages from accessing sensitive information on another site via `fetch()` or `XMLHttpRequest`. For example, a page loaded from `https://bank.com` cannot make a request to `https://mail.com` and read the response.

## ### What is CORS?

Cross-Origin Resource Sharing (CORS) is an HTTP-header based protocol that relaxes the same-origin policy for approved requests. It allows a server to specify which origins are permitted to read its resources. When a page makes a cross-origin request, the browser adds special CORS headers and may send a **preflight** request to check whether the server will accept the actual request.

## ### How CORS works

1. **Simple requests:** If the request uses a safe HTTP method (`GET`, `HEAD` or sometimes `POST` with simple headers) and does not include custom headers, the browser automatically adds an `Origin` header specifying the requesting domain. The server's response must include `Access-Control-Allow-Origin` with either the requesting origin or `*` to permit the read.
2. **Preflight requests:** For requests that could modify server data (e.g. `PUT`, `DELETE`, or `POST` with non-simple headers) the browser first sends an `OPTIONS` request to the server containing `Access-Control-Request-Method` and `Access-Control-Request-Headers`. The server responds with `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods` and `Access-Control-Allow-Headers` to indicate whether the actual request is allowed. If approved, the browser proceeds with the actual request; otherwise it aborts.
3. **Credentials:** By default, cross-origin requests do **not** include cookies or HTTP authentication. To send credentials, the client must set `fetch(url, { credentials: 'include' })` and the

server must respond with `Access-Control-Allow-Credentials: true` along with a specific `Access-Control-Allow-Origin` (not `\*`), otherwise the browser will reject the response.

4. **\*\*Errors:\*\*** When a CORS request fails, JavaScript code cannot see the details. The browser simply reports a generic error to prevent information leaks.

### ### Example

```
```js
// From https://domain-a.com
fetch("https://api.domain-b.com/data.json")
  .then((resp) => resp.json())
  .then((data) => console.log("Data:", data))
  .catch((err) => console.error("CORS error:", err));
```
```

If `api.domain-b.com` includes `Access-Control-Allow-Origin: https://domain-a.com` in its response, the browser allows the script to read the data. If the header is missing or the origin is not allowed, the request still reaches the server, but the browser blocks the response and triggers a CORS error.

### ### Real-world analogy

Think of visiting a secure building. Normally, only employees (same origin) can enter any office. CORS is like a guest policy: an employee (server) can put a list on the door (CORS headers) of which visitors from other companies (origins) are allowed to enter and what rooms (methods/headers) they can access. Before allowing a visitor into restricted rooms, security might call ahead (the preflight request) to confirm it's okay.

### ### Summary

- The **\*\*same-origin policy\*\*** restricts scripts to resources from the same scheme, domain and port.
- **\*\*CORS\*\*** relaxes this policy by letting servers indicate which origins may access their resources via HTTP headers.
- **\*\*Simple requests\*\*** require only an `Origin` header and a matching `Access-Control-Allow-Origin` response; **\*\*preflight requests\*\*** use the `OPTIONS` method to negotiate allowed methods and headers for non-simple requests.

- Browsers handle CORS enforcement; failure results in a generic error visible in the console but not to JavaScript.

### ### Common misconceptions

1. **CORS is a client-side fix.** CORS is enforced by browsers and configured on servers. Client-side code cannot override CORS restrictions; the server must send the appropriate headers.
2. **CORS allows any cross-origin request.** Only the origins explicitly allowed by the server are permitted. Omitting the `Access-Control-Allow-Origin` header will still cause the browser to block access.
3. **CORS is a security vulnerability.** CORS is a security feature that *prevents* unauthorized cross-origin reads. When misconfigured (e.g. using `Access-Control-Allow-Origin: \*` with credentials), it can open vulnerabilities, but properly configured CORS improves security.

### ### Practice questions

1. **Theory:** Describe the difference between the same-origin policy and CORS. What problem does CORS solve?
2. **Coding:** Write a simple Express or Node server that responds to `GET /api/data` with JSON and sets `Access-Control-Allow-Origin: \*`. Test fetching this endpoint from a different domain.
3. **Theory:** Why are preflight requests necessary? What headers does the browser send during a preflight, and how should the server respond?



# Difference between synchronous and asynchronous code

## Difference between synchronous and asynchronous code

JavaScript traditionally executes code **\*\*synchronously\*\***: it runs one statement after another in the order you wrote them, and each statement must finish before the next begins. MDN describes synchronous programs as those where the browser "steps through the program one line at a time ... waiting for the line to finish its work before going on to the next". This is straightforward to reason about, but it has a drawback—long-running functions block the single JavaScript thread. For example, an inefficient prime-number generator can freeze the user interface for seconds because nothing else can happen until it returns.

**\*\*Asynchronous programming\*\*** solves this problem by allowing long-running operations (such as network requests or file access) to start and then return immediately. The program remains responsive, and when the task finishes it provides the result via a callback, promise or event. MDN notes that asynchronous programming lets your program "start a potentially long-running task and still be able to be responsive to other events". Events like HTTP requests, camera access or file pickers are handled asynchronously, so your code isn't blocked while waiting for a response.

### ### How synchronous and asynchronous code differ

- **\*\*Execution order\*\*** - In synchronous code, statements execute sequentially; the call stack must be empty before the browser can do anything else. In asynchronous code, long-running tasks start and then yield control back to the browser. When the task completes, its callback or promise handler runs later via the event loop. This makes the application appear to "do two things at once" even though JavaScript is single-threaded.

- **\*\*Blocking vs. non-blocking\*\*** - A synchronous function blocks the main thread: user interactions and rendering wait until it finishes. The MDN asynchronous requests guide warns that synchronous requests "block the execution of code" and cause the UI to freeze. Asynchronous functions, on the other hand, don't block; the browser continues handling user input while awaiting the result.

- **\*\*Handling results\*\*** - Synchronous calls return their result immediately. Asynchronous calls return a promise or accept a callback that will be invoked when the result is ready. Using promises (`.then()`/`catch()`) or `async`/`await` makes asynchronous flows easier to read than nested callbacks.

### ### Example: long synchronous vs. asynchronous tasks

```
``js
```

```
// Synchronous example: blocks the UI
```

```

function generateLargePrimes(count) {
 const primes = [];
 let num = 2;
 function isPrime(n) {
 for (let i = 2; i <= Math.sqrt(n); i++) {
 if (n % i === 0) return false;
 }
 return true;
 }
 while (primes.length < count) {
 if (isPrime(num)) primes.push(num);
 num++;
 }
 return primes;
}

// Calling this will freeze the UI until it finishes
const primes = generateLargePrimes(100000);

```

```

// Asynchronous example using setTimeout
function generatePrimesAsync(count) {
 return new Promise((resolve) => {
 setTimeout(() => {
 resolve(generateLargePrimes(count));
 }, 0); // schedule on the event loop
 });
}

generatePrimesAsync(100000).then((primes) => {
 console.log("Generated primes asynchronously");
});

```

```
console.log("UI stays responsive while primes are computed");
```

```
...
```

In the synchronous version the browser must finish generating primes before doing anything else. In the asynchronous version we wrap the computation in a `Promise` that resolves after a timeout. The promise allows the event loop to process other events (like user input) before running the heavy computation in a later tick.

### ### Real-world analogy

Imagine standing in line at a coffee shop. A **synchronous** process would require you to wait at the counter until your drink is made before the next customer can order; the barista can serve only one person at a time. An **asynchronous** process is like taking a numbered ticket: you place your order, receive a token and then sit down. While you chat with friends, the barista prepares multiple orders. When your number is called, you pick up your drink. You were free to do other things while waiting, and the barista could work on many orders without people blocking the counter.

### ### Common misconceptions

1. **Asynchronous code runs on multiple threads.** In browsers, JavaScript still runs on a single thread; asynchronous functions simply defer execution until the call stack is free. Web APIs or worker threads perform the heavy work in the background, but your callbacks run on the main thread.
2. **Asynchronous code is always faster.** It doesn't make a task execute sooner; it just prevents the UI from freezing. A network request still takes the same time to complete; asynchronous handling lets your program respond to other events during that time.
3. **`async` functions run in parallel.** Declaring a function `async` means it returns a promise and allows you to use `await`. It does not make the function concurrent by itself.

### ### Practice questions

1. **Theory:** Explain why long-running synchronous functions cause the browser to become unresponsive, and how asynchronous functions avoid that problem.
2. **Coding:** Convert a synchronous function that fetches data from an API using `XMLHttpRequest` into an asynchronous version using the Fetch API and promises. Ensure that the UI remains responsive while the data is loading.

3. **Coding:** Write an `async` function that performs three network requests in parallel using `Promise.all()` and returns the combined results. How would you handle errors if one of the requests fails?

# What is the Fetch API and how is it different from XMLHttpRequest?

What is the Fetch API and how is it different from XMLHttpRequest?

Modern web applications need to communicate with servers without reloading the page. Two main browser APIs provide this capability: **XMLHttpRequest** (XHR) and the newer **Fetch API**. Both allow you to make HTTP requests from JavaScript, but they differ significantly in syntax, features and design.

## Fetch API overview

`fetch()` is a modern, promise-based interface for making HTTP requests in JavaScript. It replaces the older `XMLHttpRequest` (XHR) with a cleaner syntax and integrates well with features like CORS and service workers. A basic `fetch()` call looks like this:

```
```js
fetch("https://api.example.com/data")
  .then((response) => {
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    return response.json(); // parse JSON body
  })
  .then((data) => console.log(data))
  .catch((err) => console.error(err));
```
```

`fetch()` takes two arguments:

1. **Resource** - a URL or a `Request` object representing the resource to fetch.
2. **Options object** - an optional `RequestInit` object where you configure the request.

It returns a **\*\*Promise\*\*** that resolves to a `Response` object once the server responds with headers. If a network error occurs (e.g., DNS failure), the promise rejects. However, HTTP error status codes (4xx/5xx) do **\*\*not\*\*** cause rejection; you must check `response.ok` yourself.

---

### ### Configuring requests via the options object

The optional `init` object passed to `fetch()` lets you specify details such as the HTTP method, headers, body, credentials, caching, and more. Key properties include:

| Property                 | Purpose / Examples                                                                                                                                                                                            |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>method</code>      | HTTP method (e.g., <code>'GET'</code> , <code>'POST'</code> , <code>'PUT'</code> , <code>'DELETE'</code> ). Default is <code>'GET'</code> .                                                                   |
| <code>headers</code>     | An object or <code>Headers</code> instance specifying request headers.                                                                                                                                        |
| <code>body</code>        | Data to send with methods like <code>POST</code> or <code>PUT</code> . Can be a string, <code>Blob</code> , <code>FormData</code> , <code>URLSearchParams</code> , <code>ArrayBuffer</code> , or typed array. |
| <code>mode</code>        | <code>'cors'</code> , <code>'no-cors'</code> , or <code>'same-origin'</code> - controls CORS behavior.                                                                                                        |
| <code>credentials</code> | <code>'omit'</code> , <code>'same-origin'</code> , or <code>'include'</code> - controls whether cookies and HTTP auth data are sent.                                                                          |
| <code>cache</code>       | Caching strategy: <code>'default'</code> , <code>'no-store'</code> , <code>'reload'</code> , <code>'force-cache'</code> , or <code>'only-if-cached'</code> .                                                  |
| <code>redirect</code>    | How redirects are handled: <code>'follow'</code> , <code>'error'</code> , or <code>'manual'</code> .                                                                                                          |
| <code>referrer</code>    | Specifies the <code>Referer</code> header ( <code>'about:client'</code> , <code>'no-referrer'</code> , a URL).                                                                                                |
| <code>keepalive</code>   | If <code>true</code> , allows the request to outlive page unloads (useful for analytics pings).                                                                                                               |
| <code>signal</code>      | An <code>AbortSignal</code> from an <code>AbortController</code> to cancel the request.                                                                                                                       |

---

### ### Making a POST request and sending data

To send data to the server, change the `method` and supply a suitable `body`. When sending JSON, set the `Content-Type` header and `body` to a stringified object:

```
```js
async function createUser(user) {
  const response = await fetch("https://api.example.com/users", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      // you can add other headers like Authorization here
    },
  },
```

```

    body: JSON.stringify(user),
  });

  if (!response.ok) {
    throw new Error("Failed to create user: " + response.status);
  }

  return response.json();
}

// Usage:
createUser({ name: "Alice", age: 30 })
  .then((data) => console.log("User created:", data))
  .catch((err) => console.error(err));
...

```

Alternatively, for form submissions you can send `FormData`:

```

```js
const formData = new FormData();
formData.append("title", "Hello");
formData.append("image", fileInput.files[0]);

fetch("/upload", {
 method: "POST",
 body: formData,
});
...

```

Here, you should **not** set a `Content-Type` header; the browser will set the correct multipart boundary.

---

### ### Working with responses

A `Response` object provides properties and methods to inspect and consume the reply:

- `response.ok`: boolean indicating status in the range 200-299.
- `response.status`: numeric status code.
- `response.headers`: a `Headers` object to read response headers.
- `response.text()`: returns a promise that resolves to the body as a string.
- `response.json()`: parses JSON and resolves to the JS object.
- `response.blob()`: resolves to a `Blob`, suitable for binary data (e.g., images).
- `response.arrayBuffer()`: resolves to an `ArrayBuffer`.
- `response.body`: a `ReadableStream` you can read incrementally.

Example of downloading a file as a blob and converting it into an object URL:

```
```js
fetch("https://example.com/logo.png")
  .then((res) => res.blob())
  .then((blob) => {
    const url = URL.createObjectURL(blob);
    const img = document.createElement("img");
    img.src = url;
    document.body.appendChild(img);
  });
```
```

---



### ### Error handling

Network errors (no response, DNS failure) reject the fetch promise. HTTP errors do not, so always check `response.ok` or `response.status` and throw accordingly. Errors thrown in `.then()` handlers or within `async` functions propagate to the nearest `.catch()` or `try...catch` block.

You can cancel requests using `AbortController`:

```
```js
const controller = new AbortController();
const { signal } = controller;

fetch('/long-request', { signal })
  .then(res => /* handle response */)
  .catch(err => {
    if (err.name === 'AbortError') {
      console.log('Request was cancelled');
    }
  });

// Cancel after 2 seconds
setTimeout(() => controller.abort(), 2000);
...

---
```

Using custom `Request` and `Headers` objects

You can prebuild requests and headers:

```
```js
const headers = new Headers({
```

```
"Content-Type": "application/json",
Authorization: "Bearer token",
});
```

```
const request = new Request("/data", {
 method: "POST",
 headers,
 body: JSON.stringify({ foo: "bar" }),
 credentials: "include", // send cookies
});
```

```
fetch(request).then(/* ... */);
...
```

---

### ### Caching and service workers

The `cache` option influences how the browser interacts with its HTTP cache. Using `no-store` ensures a fresh request; `force-cache` retrieves from cache even if expired. Service workers can intercept and respond to fetch events, enabling offline support and advanced caching. You can also use the [Cache Storage API](<https://developer.mozilla.org/en-US/docs/Web/API/Cache>) to programmatically cache responses.

---

### ### Summary

- **Promises & async/await:** Fetch uses promises, making asynchronous code more readable than XHR callbacks.
- **Flexible options:** You can set HTTP method, headers, body content, CORS mode, credentials, cache behavior, redirects and abort signals via the options object.

- **Response handling:** Check `response.ok`/`response.status`, then use `.json()`, `.text()`, `.blob()`, `.arrayBuffer()`, or the streaming `response.body` to consume the body.
- **Cancellation:** Use `AbortController` to cancel long-running requests.
- **Integration:** Fetch works in browsers and in modern Node.js, and plays nicely with service workers and CORS.

With this knowledge you should be able to perform most network interactions—GET, POST, file uploads, streaming downloads, and more—without needing to refer to external resources.

### XMLHttpRequest overview

`XMLHttpRequest` is the original API for AJAX (Asynchronous JavaScript and XML). It allows you to send HTTP requests, track their progress and handle responses via event listeners. XHR can operate in both synchronous and asynchronous modes, but synchronous requests block the main thread and are deprecated because they freeze the UI. A typical XHR usage involves creating a new `XMLHttpRequest`, calling `.open()`, attaching `onload`/`onerror` handlers and then calling `.send()`.

### Key differences between Fetch and XHR

- **Syntax and promises** - `fetch()` returns a promise that resolves with a `Response` object. You can chain `.then()` handlers or use `await` to process the response. XHR uses event callbacks (`load`, `error`, `progress`) and does not return a promise. Promise-based code tends to be cleaner and avoids "callback hell".
- **Error handling** - With `fetch`, network errors reject the promise, but HTTP errors (status codes 4xx/5xx) do **not**—you must check the `response.ok` property. XHR surfaces HTTP errors in its `status` property; you manually check `xhr.status` inside the `onload` handler.
- **Headers and bodies** - The Fetch API accepts an options object where you can set method, headers, body and other settings. It supports Request/Response streams and easily handles JSON or binary data. XHR also supports setting headers via `.setRequestHeader()`, but its API is less flexible and does not handle streams.
- **Cancellation** - To abort a fetch request you create an `AbortController` and pass its signal to `fetch()`. With XHR you call `.abort()` directly on the XHR instance. Fetch does not yet support progress events (though they are proposed), while XHR emits `progress` events useful for displaying upload/download progress.
- **Caching and service workers** - Fetch integrates with service workers and allows controlling caching via the `cache` option. XHR has no built-in cache control; caching must be handled manually or via browser heuristics.

- **\*\*Environment support\*\*** - XHR is built into browsers and has long been supported. It is not natively available in older Node.js versions. Fetch is part of modern JavaScript; it works in browsers, recent Node.js and Deno. Many existing Node.js libraries still use XHR for historical reasons, and some features like progress events are still exclusive to XHR.

### ### Example: XHR vs. fetch

```
``js
// XMLHttpRequest example
function loadUser_XHR(id) {
 return new Promise((resolve, reject) => {
 const xhr = new XMLHttpRequest();
 xhr.open("GET", `https://jsonplaceholder.typicode.com/users/${id}`, true);
 xhr.responseType = "json";
 xhr.onload = () => {
 if (xhr.status === 200) resolve(xhr.response);
 else reject(new Error("Request failed: " + xhr.status));
 };
 xhr.onerror = () => reject(new Error("Network error"));
 xhr.send();
 });
}

loadUser_XHR(1).then((user) => console.log("XHR user", user));

// Fetch example
async function loadUser_Fetch(id) {
 const response = await fetch(
 `https://jsonplaceholder.typicode.com/users/${id}`
);
 if (!response.ok) throw new Error("HTTP error: " + response.status);
 return response.json();
}
```

```
}
```

```
loadUser_Fetch(1)

 .then((user) => console.log("Fetch user", user))

 .catch((err) => console.error(err));

'''
```

In the fetch version, the intent is clear: we request the resource, check `response.ok`, parse JSON and return it. Error handling can be centralized with `try/catch`. The XHR version requires more boilerplate to set up handlers and parse the response.

### ### Real-world analogy

You can think of XHR as ordering food by calling a restaurant and waiting on hold while the operator writes down your order. You must stay on the line and listen for status updates, and if you hang up you have to start over. Fetch is like placing an order through a modern delivery app: you send your order (a promise) and receive a notification when it's ready. You can cancel the order with a tap (`AbortController`) and track its status in the app. Both achieve the same goal, but the user experience is smoother with the newer tool.

### ### Practice questions

1. **Theory:** Describe two advantages of the Fetch API over `XMLHttpRequest` and one feature that XHR still offers that Fetch does not.
2. **Coding:** Write a function that retrieves a list of posts from an API using `fetch()`, parses the JSON and handles HTTP errors gracefully. Then rewrite the same functionality using `XMLHttpRequest` and compare the readability.
3. **Exploration:** Research how to cancel fetch requests using `AbortController` and implement a button that aborts a pending request. Why is aborting important in single-page applications?

# What is JSON.parse and JSON.stringify and what are their pitfalls

What is `JSON.parse` and `JSON.stringify`, and what are their pitfalls?

Below is a beginner-friendly explanation of `JSON.parse()` and `JSON.stringify()`, with examples and common pitfalls. The goal is to demystify how JSON works in JavaScript so you can safely convert between JSON text and JavaScript values.

---

## ## What is JSON?

**\*\*JSON (JavaScript Object Notation)\*\*** is just text: a string that uses a strict format to represent data structures such as objects and arrays. Because it's just text, you can send it over the network or save it to disk. JavaScript can then parse this text back into real objects.

A simple JSON string looks like this:

```
```json
{ "name": "Alice", "age": 30, "hobbies": ["reading", "hiking"] }
```
```

Notice that keys and string values are **\*\*always wrapped in double quotes\*\***, and trailing commas are not allowed.

---

## ## `JSON.parse()` - turning JSON text into values

The `JSON.parse(text, reviver?)` function takes a JSON string (`text`) and converts it into a JavaScript value. It also accepts an optional **\*\*reviver function\*\*** to transform values during parsing.

### ### Basic usage

```

```js
const text = '{"name":"Alice","age":30}';
const obj = JSON.parse(text);
console.log(obj.name); // "Alice"
console.log(obj.age); // 30
```

```

### ### Using a reviver

A reviver lets you customize how values are constructed. It receives each key and value, from the deepest properties up to the root, and you return the final value to use. For example, to convert ISO date strings into `Date` objects:

```

```js
const json = '{"event":"meeting","time":"2025-11-07T12:00:00Z"}';
const event = JSON.parse(json, (key, value) => {
  // if a string matches ISO date format, convert to a Date
  return typeof value === "string" && /^\d{4}-\d{2}-\d{2}T/.test(value)
    ? new Date(value)
    : value;
});
console.log(event.time instanceof Date); // true
```

```

### ### Pitfalls and caveats

- **Numeric precision:** JavaScript stores numbers as 64-bit floats. Very large integers (beyond  $2^{53}$ ) can lose precision when parsed. MDN warns that numbers "may lose precision in the process". If you need to safely transport big integers, serialize them as strings and convert them back (e.g., to `BigInt`) after parsing.

- **Type revival:** JSON supports only objects, arrays, strings, numbers, booleans and `null`. Complex types (`Date`, `Map`, custom classes) are parsed as plain objects. Use the `reviver` to reconstruct them.
- **Invalid JSON throws:** `JSON.parse()` will throw a `SyntaxError` if the input string isn't valid JSON—single quotes, comments or trailing commas aren't allowed. Always wrap parsing in `try/catch` when dealing with user input.
- **Security:** Never parse untrusted JSON that includes executable code. JSON is data only; it doesn't allow functions, so any function-like content indicates something is wrong.

---

### ## `JSON.stringify()` - turning values into JSON text

`JSON.stringify(value, replacer?, space?)` converts a JavaScript value to a JSON string. It can take an optional **replacer** (to filter or transform properties) and **space** (to pretty-print the output).

#### ### Basic usage

```
```js
const obj = { name: "Bob", age: 25 };
const text = JSON.stringify(obj);
console.log(text); // {"name":"Bob","age":25}
```
```

#### ### Replacer

A `replacer` lets you control which properties are included or how they are transformed. It can be an array of keys to include, or a function that processes each key/value.

```
```js
const person = { name: "Carol", password: "secret", age: 28 };

// Only include selected keys
```



```

const publicData = JSON.stringify(person, ["name", "age"]);
console.log(publicData); // {"name":"Carol","age":28}

// Use a function to filter/transform values
const sanitized = JSON.stringify(person, (key, value) => {
  if (key === "password") return undefined; // omit passwords
  return value;
});
console.log(sanitized); // {"name":"Carol","age":28}
...

```

Pretty printing

The `space` argument adds indentation for readability:

```

```js
const obj = { a: 1, b: { c: 2, d: 3 } };
const pretty = JSON.stringify(obj, null, 2);
console.log(pretty);

/*
{
 "a": 1,
 "b": {
 "c": 2,
 "d": 3
 }
}
*/
...

```

Use `null` for the replacer if you don't need to filter anything.

### ### Pitfalls and caveats

- **Unsupported values:** JSON does not support `undefined`, functions or symbols. When encountered as property values, they are **omitted**; in arrays they become `null`. For example:

```
```js
JSON.stringify({ a: undefined, b: 2 }); // '{"b":2}'
JSON.stringify([1, undefined, 3]); // '[1,null,3]'
JSON.stringify({
  say() {
    return "hi";
  },
}); // '{}'
```

- **Special numbers:** `Infinity`, `-Infinity` and `NaN` are not valid JSON values; they are converted to `null`.

- **Non-enumerable / symbol-keyed properties:** Only an object's own enumerable string-keyed properties are serialized. Symbol-keyed and non-enumerable properties are ignored.

- **Custom serialization:** If an object has a `toJSON()` method, `JSON.stringify()` will call that method to get a serializable representation. This lets you define how your objects are stringified.

- **Circular references:** If you attempt to stringify an object that references itself (directly or indirectly), `JSON.stringify()` will throw a `TypeError`. Use a custom replacer to handle circular structures or use a library like `flatted`.

Example: handling BigInt and dates

```
```js
const obj = {
```

```

big: BigInt("9007199254740993"), // > Number.MAX_SAFE_INTEGER
date: new Date(),
};

```

```

const json = JSON.stringify(obj, (key, value) => {
 // convert BigInt to string
 if (typeof value === "bigint") return value.toString();
 // convert Date to ISO string
 if (value instanceof Date) return value.toISOString();
 return value;
});

```

```

console.log(json);
// {"big":"9007199254740993","date":"2025-11-07T13:00:00.000Z"}

```

```

const parsed = JSON.parse(json, (key, value) => {
 if (key === "big") return BigInt(value);
 if (key === "date") return new Date(value);
 return value;
});
...

```

---

## ## Summary

Function	Purpose	Gotchas to remember
-----	-----	-----
-----	-----	-----

| `JSON.parse()` | Converts a JSON string to a JavaScript value | Large numbers lose precision; only supports basic types; invalid JSON throws; use `reviver` to handle dates and BigInts |

| `JSON.stringify()` | Converts a JavaScript value to a JSON string | Ignores `undefined`, functions and symbols; `Infinity`/`NaN` become `null`; circular references throw; use `replacer`/`toJSON()` for custom serialization |

By understanding the basic usage and pitfalls of `JSON.parse()` and `JSON.stringify()`, you can confidently serialize and deserialize data without surprises—even as a beginner.

### ### Example: pitfalls in practice

```
``js
const obj = {
 name: "Alice",
 big: 9007199254740993n, // larger than Number.MAX_SAFE_INTEGER
 greet: () => "hi",
 nested: {},
};
obj.nested.self = obj; // circular reference

try {
 console.log(JSON.stringify(obj));
} catch (err) {
 console.error("Error during stringify:", err.message);
}

// Safely stringify by converting unsupported values
const safe = JSON.stringify(obj, (key, value) => {
 if (typeof value === "bigint") return value.toString();
 if (typeof value === "function" || value === undefined) return undefined;
 return value;
});
```

```

console.log(safe);

// Parsing with reviver
const parsed = JSON.parse(safe, (key, value) => {
 if (key === "big") return BigInt(value);
 return value;
});
console.log(parsed.big === 9007199254740993n); // true
...

```

The first attempt to stringify `obj` throws because of the circular reference. The custom replacer removes the function and `undefined` values and converts the bigint to a string, allowing the serialization to succeed. The reviver then restores the bigint when parsing.

### ### Real-world analogy

Think of JSON serialization as packing items into a standardized shipping box. Only certain item types (strings, numbers, booleans, `null`, arrays and plain objects) fit in the box. If you try to pack a live plant (function), a piece of paper with no label (`undefined`) or something infinitely large (`Infinity`), the packer will either throw the item away or mark its slot as empty. If an item contains a loop of rope that attaches back to itself (circular reference), the packer doesn't know how to untangle it and refuses to pack the box at all. You must convert unusual items into a supported form before shipping and mark them carefully so that the receiver can reconstruct them.

### ### Practice questions

1. **Theory:** Why does `JSON.stringify({x: undefined, y: NaN})` produce `{ "y": null }`? What happens if you try to stringify a function?
2. **Coding:** Write a custom replacer and reviver to serialize and deserialize a `Map` object. How can you preserve the key/value pairs?
3. **Troubleshooting:** Describe how you would detect and handle circular references when stringifying a deep object graph. Compare using a replacer function versus using the built-in `structuredClone()` method.

# Explain module patterns in JS — ESM vs CommonJS

## Explain module patterns in JS: ESM vs CommonJS

JavaScript uses modules to organize code into reusable pieces. Historically, Node.js adopted CommonJS (CJS), while browsers and the ECMAScript standard now support ES modules (ESM). Each pattern has its own syntax and characteristics.

### ### CommonJS (CJS)

CJS is the original module system for Node.js. Modules are loaded **\*\*synchronously\*\*** using `require()`, and they export values via `module.exports` or `exports`. Because `require()` is just a function, you can call it dynamically based on conditions. CJS modules run immediately when required and their exports are cached in `require.cache`. According to the Node documentation, CJS modules have access to variables like `__filename` and `__dirname` and can modify `module.exports` to expose functionality.

Example:

```
```.js
// math.js (CommonJS)
function add(a, b) {
  return a + b;
}
module.exports = { add };

// index.js
const math = require("./math");
console.log(math.add(2, 3));
...

```

ECMAScript modules (ESM)

ES modules are the official standard for JavaScript modules. They use the `import` and `export` keywords and are loaded **asynchronously**. Because import statements are **static** (must be at the top level), bundlers can analyze dependencies ahead of time and eliminate unused exports (tree shaking). ESM does not expose CommonJS-specific features such as `require`, `module.exports`, `__filename` or `__dirname`. LogRocket notes that ES modules are more readable and are the standard moving forward, whereas CommonJS is primarily used for backward compatibility.

Example:

```
``js
// math.mjs

export function add(a, b) {
  return a + b;
}

// index.mjs

import { add } from './math.mjs';
console.log(add(2, 3));
``
```

Key differences

- **Loading strategy** - CJS loads modules synchronously; code executes as soon as `require()` is called. This works well in Node because modules are on disk, but it blocks execution during network fetches in the browser. ESM loads modules asynchronously, returning a promise when using dynamic `import()`. Static imports are hoisted and executed before other code.
- **Syntax and static analysis** - CJS uses `require()` and `module.exports`; ESM uses `import`/`export`. Because ESM uses static declarations, bundlers can perform tree shaking to remove unused exports. CJS allows imports anywhere, even conditionally, which makes static analysis harder.
- **Interop** - Mixing CJS and ESM can be tricky. ESM default export maps to `module.exports.default` in CJS. Node requires the file extension `.mjs` or a `"type": "module"` field in `package.json` to enable ESM.
- **Environment support** - CJS is built into Node and works in any version. ESM is supported in modern browsers and in Node 12+ behind certain flags; older tools may still rely on CJS.

- **Tree shaking** - Bundlers like webpack can eliminate unused code in ESM because the structure is static. This is harder with CJS because functions can be required dynamically.

Choosing between CJS and ESM

For new projects targeting modern environments, ES modules are recommended because they are the standard, allow static analysis and tree shaking, and work in both browser and Node with minimal configuration. However, you may need CommonJS when using legacy Node modules or building libraries that must support old tooling. Many projects provide dual builds (e.g., `index.cjs`` and `index.mjs``) to support both systems.

Practice questions

1. **Theory:** Describe two advantages of ES modules over CommonJS and one reason you might still use CommonJS.
2. **Coding:** Convert a CommonJS module that exports multiple functions into an ES module using named exports, and adjust its import statements.
3. **Exploration:** Research how to dynamically load modules using `import()` in ES modules and how that compares to conditional `require()` calls. When might dynamic loading be useful?

What is tree shaking and dead code elimination

What is tree shaking and dead code elimination

Bundlers and compilers try to remove code that is never used. There are two related concepts:

****dead code elimination (DCE)**** and ****tree shaking****. DCE is a compiler optimization that discards unreachable code; tree shaking is a technique applied specifically to JavaScript modules to include only the parts of a dependency graph you actually use.

Dead code elimination

Dead code elimination removes unreachable or unused code after an entire file or bundle has been generated. Traditional optimizers mark functions and variables that are never referenced and delete them. However, because JavaScript allows dynamic behavior (e.g., using ``eval`` or property lookup by string), a bundler cannot always detect unused code. DCE is limited to obvious cases.

Tree shaking

Tree shaking is a form of dead code elimination tailored for ES modules. Web.dev compares it to pruning a dependency tree: bundlers analyze static ``import`` statements and only include the exported members you actually import. Instead of starting with all code and removing dead pieces, tree shaking builds your bundle from the top down, adding only "live" code that is referenced. It works best when using ES modules because ``import`/`export`` syntax is static; CommonJS ``require`` is dynamic and hampers static analysis. Named imports help bundlers know exactly which exports you need.

Example:

```
``js
// math.js
export function add(a, b) {
  return a + b;
}
export function subtract(a, b) {
  return a - b;
}
```

```
// app.js
import { add } from "./math.js";
console.log(add(2, 3));
...
```

When bundling `app.js`, a tree-shaking bundler includes only the `add` function and omits `subtract`, reducing bundle size. If you import the entire module (`import * as math`), the bundler may include both functions because it cannot know which properties will be used.

Tips for effective tree shaking

- Use ES module syntax (`import`/`export`) rather than CommonJS. Avoid dynamic `require`.
- Prefer named imports over namespace or default imports so bundlers know exactly what to include.
- Avoid side effects (code that runs when the module is imported); mark pure modules with `"sideEffects": false` in your `package.json` so bundlers can safely remove unused files.
- Understand that tree shaking does not remove code executed for side effects; functions like polyfills or global initialization will always be included.

Practice questions

1. **Theory:** Explain how tree shaking differs from traditional dead code elimination and why static `import` statements are important.
2. **Coding:** Refactor a module that exports multiple functions so that unused functions can be tree-shaken away when bundling. Test with a bundler like webpack or Rollup.
3. **Exploration:** Research how the `"sideEffects"` flag in `package.json` affects tree shaking. Try enabling/disabling it in a simple project and observe the bundle size.

What is a polyfill?

What is a polyfill

Web standards evolve, but not all browsers support new features immediately. A ****polyfill**** is a piece of code that implements a feature on browsers that do not natively support it. MDN defines a polyfill as a JavaScript implementation that provides modern functionality to older browsers. Developers write polyfills to mimic APIs like ``Object.assign``, ``Array.prototype.includes`` or CSS features so that applications work across browsers.

How polyfills work

Polyfills typically check whether a feature exists and, if not, define it. For example, to add ``Array.prototype.includes`` support in older browsers:

```
``js
if (!Array.prototype.includes) {
  Array.prototype.includes = function (search, start = 0) {
    for (let i = start; i < this.length; i++) {
      if (
        this[i] === search ||
        (Number.isNaN(this[i]) && Number.isNaN(search))
      ) {
        return true;
      }
    }
    return false;
  };
}
```

When run in modern browsers, the ``if`` condition prevents overriding the native implementation. When run in older browsers, it adds the method so that code using ``includes`` will work.

Polyfills vs transpilers and shims

- **Transpilers** (e.g., Babel) convert modern syntax to older syntax (e.g., arrow functions to traditional functions). They cannot add new APIs. Polyfills complement transpilers by providing missing methods.
- **Shims** are similar to polyfills; some authors use the terms interchangeably, but "shim" often refers to code that wraps existing APIs to provide a consistent interface.
- **Polyfills** mimic features with as close to spec-compliant behavior as possible but may have performance limitations compared to native implementations.

When to use polyfills

Use polyfills when you need to support older browsers that lack specific features. Many libraries and frameworks include polyfills automatically based on browser targets (e.g., core-js). Be careful not to include unnecessary polyfills because they increase bundle size. Also, never polyfill features that change global behavior (like `Promise`) if you cannot guarantee spec compliance; some environments may already provide partial implementations.

Practice questions

1. **Theory:** Define a polyfill and explain how it differs from a transpiler.
2. **Coding:** Write a polyfill for the `String.prototype.startsWith()` method and test it on an array of strings.
3. **Exploration:** Investigate how modern build tools like Babel and core-js work together to provide polyfills based on targeted browsers. How can you reduce the number of polyfills included in your bundle?

Explain memoization

Explain memoization

In computing, ****memoization**** is an optimization technique that caches the results of expensive function calls and returns the cached result when the same inputs occur again. Memoization stores results in a cache to reduce processing time and memory, particularly when calls are expensive.

Why use memoization

Many algorithms repeatedly compute the same subproblems (e.g., in recursion). Without memoization, each call recomputes results, wasting time. Memoization saves results in a lookup table so that repeated calls with the same arguments are instantaneous. This is especially useful for functions with deterministic outputs (pure functions) where the same input always yields the same output.

Implementing memoization in JavaScript

A common pattern uses closures to create a cache:

```
``js
function memoize(fn) {
  const cache = {};
  return function (...args) {
    const key = JSON.stringify(args);
    if (key in cache) {
      return cache[key]; // return cached result
    }
    const result = fn.apply(this, args);
    cache[key] = result;
    return result;
  };
}
```

```
// Example: memoized Fibonacci

function fib(n) {
  if (n <= 1) return n;
  return fib(n - 1) + fib(n - 2);
}

const memoizedFib = memoize(fib);

console.log(memoizedFib(35)); // computed
console.log(memoizedFib(35)); // cached
...

```

The `memoize` helper returns a new function that checks the cache before invoking the original function. The key is derived from the arguments so each input set gets its own cached result.

Considerations

- Memoization is most effective for pure functions with no side effects. Functions that depend on external state or cause side effects should not be memoized.
- Caching consumes memory; use strategies like size limits (LRU caches) or TTL (time-to-live) to avoid excessive memory usage.
- Functions with complex arguments may need custom key generators instead of simple JSON stringification.

Practice questions

1. **Theory:** Explain why memoization improves performance in recursive algorithms like Fibonacci.
2. **Coding:** Implement a memoized version of a function that computes factorials. Compare its performance to a naive recursive version for large inputs.
3. **Exploration:** Research how libraries like Lodash's `_.memoize` implement memoization and how you might customize the cache behavior.

What are generators and iterators

What are generators and iterators

Think of an **iterable** as something you can loop over, one item at a time. Arrays, strings, and Maps are all iterables in JavaScript. What makes them iterable is that they implement a special protocol: when you start iterating (for example, with `for...of`), JavaScript calls a method named `Symbol.iterator` on the object to get an **iterator**.

An **iterator** is just an object with a `next()` method. Each call to `next()` returns an object with two properties:

- `value`: the next item in the sequence.
- `done`: a boolean that tells whether the sequence is finished.

Here's a simple iterator for counting from 1 to 5:

```
``js
const counter = {
  current: 1,
  last: 5,
  [Symbol.iterator]() {
    return {
      next: () => {
        if (this.current <= this.last) {
          return { value: this.current++, done: false };
        }
        return { done: true };
      },
    };
  },
};
```

```
// You can iterate with for...of:
for (const num of counter) {
  console.log(num); // logs 1, 2, 3, 4, 5
}
...
```

When you call `for...of` on `counter`, JavaScript calls `counter[Symbol.iterator]()` to get the iterator. It then repeatedly calls `next()`, pulling values until `done` becomes `true`. Because each value is computed only when requested, iterators can represent very large or even infinite sequences without storing everything in memory.

Generator functions: a shortcut for writing iterators

Writing custom iterator objects by hand can be verbose. **Generator functions**—declared with `function*`—make this easier. When you call a generator function, it doesn't run immediately; instead, it returns a **generator object**. This object is both iterable and an iterator. Inside the generator, you use `yield` to produce values one at a time. Execution "pauses" at each `yield` and resumes when `next()` is called again.

Let's rewrite the counter using a generator:

```
```js
function* countUpTo(max) {
 for (let i = 1; i <= max; i++) {
 yield i; // pause here and return the value
 }
}

const numbers = countUpTo(5);
for (const n of numbers) {
 console.log(n); // 1, 2, 3, 4, 5
}
...
```
```


Because `yield` pauses the function, generators naturally maintain state between iterations. They also let you:

- **Send values back in**: When calling `next(value)`, the `value` becomes the result of the previous `yield` expression. This can be used to modify the generator's internal logic.
- **Finish early**: Calling `return(value)` on a generator ends it early and returns `value` as the final result.
- **Throw errors into the generator**: Calling `throw(error)` will cause the corresponding `yield` expression to throw, letting you handle errors inside the generator.

Here's a more interesting example: an infinite Fibonacci sequence generator:

```
```js
function* fibonacci() {
 let a = 0,
 b = 1;
 while (true) {
 yield a;
 [a, b] = [b, a + b];
 }
}

const fib = fibonacci();
console.log(fib.next().value); // 0
console.log(fib.next().value); // 1
console.log(fib.next().value); // 1
console.log(fib.next().value); // 2
// ...and so on, potentially forever
...
```
```

Since the generator never finishes by itself, you must decide when to stop iterating—either by breaking a loop after a few values or by calling `return()`.

Summary for beginners

- An **iterable** is anything you can loop over with `for...of`. It must implement `Symbol.iterator` and return an iterator.
- An **iterator** is an object with a `next()` method that returns `{ value, done }`. You control how values are produced.
- **Generator functions** (`function*`) are a convenient way to create iterators. Each `yield` produces a value and pauses the function until the next value is requested.
- Generators simplify stateful or infinite sequences and let you write cleaner asynchronous code (when combined with `async` generators).

By understanding these concepts, you can build custom sequences on demand and handle complex iteration patterns more cleanly.

Use cases

- Producing sequences lazily, like infinite series or streams of events.
- Implementing asynchronous flow control using async generators (`async function*`) and `for await ... of`.
- Flattening nested structures by yielding values recursively.

Practice questions

1. **Theory:** Describe the difference between an iterable and an iterator. What methods must an iterator implement?
2. **Coding:** Write a generator function that yields values from a nested array (e.g., `[1, [2, 3], 4]`) in a single sequence.
3. **Exploration:** Research async generators and `for await...of`. How do they simplify working with streams or asynchronous data sources?

Explain currying and partial application

Explain currying and partial application

Think of a multi-argument function like a machine with several input slots. **Currying** turns that machine into a series of single-slot machines chained together. Each call fills one slot and returns a new function that expects the next input. The original function only runs when all slots have been filled.

In concrete terms, if you start with a function `f(a, b, c)`, a curried version would look like `f(a)(b)(c)`. The function doesn't execute until you've provided all three arguments. That might seem strange at first, but it allows you to create partially specialized versions of the function by "locking in" some of the inputs.

Here's the earlier example, annotated:

```
```js
// A normal 3-argument function
function sum(a, b, c) {
 return a + b + c;
}

// A curried version
function currySum(a) {
 return function (b) {
 return function (c) {
 return a + b + c;
 };
 };
}

// Fill the arguments one by one
const add1 = currySum(1); // returns a function waiting for b and c
const add1and2 = add1(2); // returns a function waiting for c
```

```
console.log(add1and2(3)); // prints 6 (1+2+3)
```

```
...
```

A helper function can automate currying so you don't have to write nested functions by hand, but the idea is the same: you gradually supply arguments until the function has enough to run.

```

```

### ### Partial application

**Partial application** is a looser technique: it lets you pre-fill **some** arguments of a function to create a new function. Unlike curried functions, the partially applied function still expects all remaining arguments at once. In the example you provided:

```
```js
```

```
function multiply(a, b, c) {
```

```
  return a * b * c;
```

```
}
```

```
function partial(fn, ...fixedArgs) {
```

```
  return function (...restArgs) {
```

```
    return fn(...fixedArgs, ...restArgs);
```

```
  };
```

```
}
```

```
const doubleAndTriple = partial(multiply, 2, 3); // fixes a = 2, b = 3
```

```
console.log(doubleAndTriple(4)); // 24 (2 * 3 * 4)
```

```
```
```

Here, `doubleAndTriple` is a new function where two of the original three arguments are pre-set, and you only need to supply the last one. Partial application can also be done with built-in methods like `Function.prototype.bind()`, which fixes the `this` context and leading arguments.

---

### ### When are these useful?

- **Creating specialized functions**: Suppose you have a logging function `log(level, namespace, message)`. You can curry or partially apply it to make a `debugLog = log.bind(null, 'debug', 'MyApp')` so you only need to supply the message.
- **Function composition**: In functional programming, you often build complex behavior by composing small functions. Curried functions make composition easier because they always return unary functions.
- **Reusable configuration**: If a function accepts many configuration options, currying lets you create pre-configured versions for different situations (e.g., `makeApiCall(baseUrl)(endpoint)(params)`).
- **Readable callbacks**: Sometimes event handlers or array methods require a callback with a single parameter. Currying can wrap a multi-argument function into the expected signature while keeping the original logic intact.

For beginners, it helps to see currying and partial application as ways to reuse and adapt functions without rewriting them. Currying breaks a function into a chain of one-argument steps, while partial application "locks in" some arguments and returns a function that expects the rest. Both are powerful techniques for writing flexible, declarative code once you get comfortable with passing functions around.

### ### Differences between currying and partial application

- **Arity** - Currying always returns unary functions; partial application may return functions that expect multiple arguments.
- **Execution** - Curried functions wait until called with all arguments; partial application returns a new function that calls the original with a mix of fixed and new arguments.
- **Implementation** - Currying typically uses nested functions and may provide a flexible wrapper that collects arguments until the original arity is met. Partial application uses `bind()` or a wrapper to pass preset arguments.

### ### Use cases

- Creating specialized functions such as logging functions with preset date or level.
- Building composable functions in functional programming.

- Simplifying event handlers by pre-filling context.

### ### Practice questions

1. **Theory:** Explain how currying transforms a two-argument function into a series of unary functions. How does partial application differ?
2. **Coding:** Write a curry helper that converts any three-argument function into a curried version. Test it with a function that concatenates three strings.
3. **Exploration:** Consider the `Function.prototype.bind()` method. How can it be used for partial application? What are the limitations compared to writing your own partial helper?

# What is the Intl API and how is it used for localization?

What is the Intl API and how is it used for localization

Internationalizing applications involves presenting dates, numbers, lists and messages in formats that users expect based on their locale. The **Intl** object is a namespace providing the ECMAScript Internationalization API. It exposes constructors such as `Intl.DateTimeFormat`, `Intl.NumberFormat`, `Intl.ListFormat` and others. According to MDN, the Intl API offers language-sensitive string comparison, number formatting and date/time formatting and provides standard ways to display data in a user's locale.

## ### Locale identifiers

Locales are strings like `"en-US"` (English, United States) or `"fr-FR"` (French, France). They may include language, country and optional numbering system or calendar (e.g., `"de-DE-u-ca-gregory"`). The Intl API uses the best available locale fallback when a specific locale is not fully supported.

## ### Formatting dates and times

`Intl.DateTimeFormat` formats dates and times according to locale. You can pass options to specify styles (e.g., `"long"` or `"short"`):

```
```js
const date = new Date("2025-11-06T08:30:00Z");
const fr = new Intl.DateTimeFormat("fr-FR", {
  dateStyle: "long",
  timeStyle: "short",
});
console.log(fr.format(date)); // "6 novembre 2025 à 09:30" (example)
```
```

## ### Formatting numbers and currencies

`Intl.NumberFormat` formats numbers, currencies and percentages. Options allow specifying minimum decimals, currency display and grouping:

```

```js
const price = 1234.5;
const usd = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
});
const de = new Intl.NumberFormat("de-DE", {
  style: "currency",
  currency: "EUR",
});
console.log(usd.format(price)); // "$1,234.50"
console.log(de.format(price)); // "1.234,50 €"
```

```

MDN notes that `Intl.NumberFormat` is used to create language-sensitive number formatting.

### ### Other Intl features

- **RelativeTimeFormat** formats phrases like "in 3 minutes" or "5 days ago".
- **ListFormat** formats lists using conjunctions appropriate to the locale (e.g., "A, B and C" vs "A, B y C").
- **Collator** compares and sorts strings according to language-specific rules, useful for ordering names or dictionary entries.
- **PluralRules**, `DateTimeFormat`, `NumberFormat` and `UnitFormat` help handle pluralization, numeric units and measurement systems.

### ### Practical use of Intl

Using Intl helps avoid writing custom formatting logic and improves consistency across browsers. It is not a translation library; it formats data, not text. For full localization you still need to translate messages separately. Many frameworks integrate Intl, and Node.js includes Intl by default.



### ### Practice questions

1. **Theory:** What is the purpose of the Intl API? List three constructors provided by Intl and their roles.
2. **Coding:** Format the number `12345.6789` as currency in Japanese yen and as a percentage in German locale using `Intl.NumberFormat`.
3. **Exploration:** Research how `Intl.RelativeTimeFormat` can be used to display relative dates (e.g., "yesterday", "in 2 weeks"). Implement a helper that takes a number of days and returns a human-readable relative time string for different locales.

# Explain the repaint and reflow process in browsers

## # Explain the repaint and reflow process in browsers

### ## Introduction

When you build a web page, the browser has to calculate how every element should appear and then draw those pixels to the screen. **Reflow** and **repaint** are two distinct steps in this rendering pipeline. Understanding what triggers reflows versus repaints helps you write more efficient code and avoid janky interfaces. In simple terms, reflow changes the **layout** of elements (positions and sizes), while repaint changes only the **visual appearance** (colours, backgrounds, shadows) without moving anything.

### ## Detailed explanation

#### ### What is a reflow?

Reflow (also called **layout**) occurs when the browser recalculates the geometry and position of elements. Any change that affects an element's layout—such as adding or removing DOM nodes, changing the `display` or `position` CSS property, toggling a class that changes `margin` or `font-size`, or resizing the browser window—forces the browser to walk through the DOM tree, measure elements and determine their new positions.

Reflows can be expensive because one element's layout often depends on its parents and children. Adjusting a single element can ripple up and down the DOM tree, causing the browser to recalculate many nodes. While modern engines optimize this process, large reflows can still block interactivity and cause "jank."

#### ### What is a repaint?

Repaint (also called **render** or **redraw**) happens after the layout is calculated. It updates the visual styles of elements without affecting their geometry. Changing a colour, background image or visibility (`visibility: hidden`) triggers a repaint. Repaints are generally cheaper than reflows because the browser does not need to compute positions; it simply needs to fill new pixels. However, they still use resources and can be noticeable if triggered frequently.

### ### Triggers and performance tips

Common triggers for reflow include:

- Adding, removing or moving DOM elements.
- Changing display types (`display: none` ↔ `block`), fonts or sizes.
- Resizing the window or an element.
- Calculating sizes with properties like `offsetHeight` or `getComputedStyle`.

Triggers for repaint include:

- Changing colour, background, borders or shadows.
- Adjusting `visibility` or `outline`.

To minimize performance hits:

1. **Batch DOM changes**: group multiple style or DOM changes together rather than interleaving reads and writes. This reduces repeated reflows.
2. **Use CSS classes** instead of repeatedly modifying inline styles. One class change triggers a single reflow.
3. **Avoid layout thrashing**: reading layout properties like `offsetTop` or `clientHeight` immediately after writing styles forces the

browser to reflow synchronously. Cache values when possible.

4. **\*\*Reduce deep nesting\*\***: complex DOM hierarchies require more work during reflow, so flatten your markup where practical.

## ## Real-world analogy

Imagine your browser is a moving company. A **\*\*reflow\*\*** is like rearranging furniture in a room—if you move a sofa, you might need to shuffle other pieces around to make everything fit again. A **\*\*repaint\*\*** is like repainting the walls or changing the curtains—nothing has moved, but the appearance has changed. Painting the walls is quicker than moving furniture, but doing either too often will tire the crew.

## ## Example

Here's a simple demonstration of how different changes affect the rendering process:

```
```html
<style>
.box {
  width: 100px;
  height: 100px;
  background: skyblue;
  margin: 10px;
}
.moved {
  margin-top: 100px;
} /* triggers reflow */
.recolored {
  background: salmon;
}
```

```

    } /* triggers repaint */
</style>

<div id="box" class="box"></div>
<button id="move">Move</button>
<button id="color">Change colour</button>

<script>
    const box = document.getElementById("box");
    document.getElementById("move").onclick = () => box.classList.toggle("moved");
    document.getElementById("color").onclick = () =>
        box.classList.toggle("recolored");
</script>
'''

```

Clicking "Move" adds or removes a class that changes the box's margin, causing a reflow because the layout changes. Clicking "Change colour" only changes the background colour, causing a repaint.

Common misconceptions

- **Repaints are free.** While cheaper than reflows, repaints still require the browser to redraw pixels and can hurt performance if triggered rapidly.
- **Only DOM changes trigger reflow.** Reading layout properties can also force a reflow if the browser must flush pending changes to answer your query.
- **Reflows always block the UI.** Modern browsers perform some reflows asynchronously or off the main thread, but large reflows can still cause noticeable delays.

Practice questions

1. What is the difference between repaint and reflow?
2. Name two actions that trigger a reflow and two that trigger only repaint.
3. Why is it beneficial to batch multiple DOM changes together?
4. How might reading `offsetHeight` immediately after changing a style affect performance?

What is garbage collection and how does mark-and-sweep work?

What is garbage collection and how does mark-and-sweep work?

Introduction

In languages like C or C++, developers must manually allocate and free memory. JavaScript is more forgiving: it automatically reclaims memory that's no longer needed through **garbage collection (GC)**. GC makes it easier to write code without worrying about leaks, but understanding how it works helps you write more efficient and leak-free programs.

Reachability and memory management

JavaScript engines consider objects **reachable** if they can be accessed from the code. Roots of reachability include global variables, variables on the current call stack and variables captured in closures. As long as there is a chain of references from a root to an object, that object stays in memory. Once there is no way to reach an object, it becomes eligible for garbage collection.

The mark-and-sweep algorithm

The classic algorithm used by JS engines is **mark-and-sweep**:

1. **Mark phase**: Starting from root objects, the GC traverses references and marks each object it encounters as live or reachable. It follows all property references, array elements, closures, etc., marking anything connected to a root.
2. **Sweep phase**: After marking, the GC scans through memory and frees any

object that was not marked. Unmarked objects are unreachable and their memory is returned to the system.

This algorithm avoids freeing objects that are still in use. Modern implementations add optimizations like ****generational collection**** and ****incremental collection****:

- *_Generational GC_* divides objects into "new" and "old" generations. Most objects die young, so the collector scans the young generation frequently and the old generation less often. This reduces overall work because short-lived objects are collected quickly.
- *_Incremental and idle-time GC_* break the mark-and-sweep work into smaller chunks that run during idle moments, preventing long pauses that would freeze the main thread.

Memory leaks and patterns

Despite automatic GC, you can still create memory leaks:

- ****Lingering references****: Storing objects in long-lived containers (global arrays, Maps, caches) prevents them from being collected. Clear entries when you no longer need them.
- ****Detached DOM nodes****: Removing an element from the DOM does not free it if your code still references it. Always nullify references to elements you remove.
- ****Closures capturing unnecessary variables****: Capturing large objects in closures may keep them alive longer than necessary. Avoid capturing heavy data if you only need a small part.

Real-world analogy

Imagine your working desk as computer memory. You keep important documents within reach (roots). Occasionally you clean your desk: you go through each document, marking which ones you still need. Anything unmarked is thrown away. To save time, you might check recent documents more often (generational GC) and tidy up small areas during breaks (incremental GC).

Example: monitoring memory usage

While you cannot manually force garbage collection in most environments, you can write code that simulates leaks:

```
```js
// Simulate a leak by storing lots of data in a global array
const cache = [];

function allocate() {
 // allocate ~1 MB string
 const data = new Array(1024 * 1024).join("x");
 cache.push(data);
 console.log("Allocated", cache.length, "MB");
}

setInterval(allocate, 1000);

// To fix the leak, clear the cache periodically
setInterval(() => {
 cache.splice(0, cache.length);
 console.log("Cache cleared");
}, 10000);
```
```

This code continuously allocates memory; clearing the cache allows the GC to reclaim it. Tools like Chrome DevTools' Memory panel help identify leaks.

Common misconceptions

- *****Garbage collection is deterministic.***** GC runs at unspecified times when the engine decides memory needs to be reclaimed. You cannot rely on exact timing.
- *****Objects are collected immediately after they become unreachable.***** There may be delays; incremental GC might wait until the next idle period.
- *****GC frees everything.***** If you maintain references to objects (in caches, closures or global variables) they remain reachable and won't be collected.

Practice questions

1. What does it mean for an object to be "reachable" in JavaScript?
2. Describe the two phases of the mark-and-sweep algorithm.
3. How does generational garbage collection improve performance?
4. Give an example of a memory leak in JavaScript and how to fix it.

Explain shadowing and variable masking

Explain shadowing and variable masking

Introduction

JavaScript variables live in `_scopes_`-regions of code where a name is defined.

When a variable is defined in an inner scope with the same name as a variable in an outer scope, the inner variable ****shadows**** the outer one. Within the inner scope, references to that name refer to the inner variable, effectively masking the outer variable from access. Shadowing is a normal part of lexical scoping, but accidental shadowing can lead to bugs.

Understanding scope and shadowing

Consider this example:

```
``js
let greeting = "Hello";

function sayHi() {
  let greeting = "Hi"; // shadows the outer variable
  console.log(greeting); // prints 'Hi'
}

sayHi();
console.log(greeting); // prints 'Hello'
``
```

Inside ``sayHi``, the ``greeting`` declared with ``let`` hides the ``greeting`` in the outer scope; the outer variable is still there but cannot be accessed until the inner scope ends. Once ``sayHi`` finishes, the outer ``greeting`` is used again.

Illegal shadowing

Not all shadowing is legal. Mixing `var` and `let`/`const` for the same identifier in overlapping scopes can cause errors. Because `var` is function-scoped, declaring a `var` variable inside a block (`if`, `for`) leaks it to the entire function. If an outer scope already has a `let` variable with the same name, trying to declare a `var` inside will throw a `SyntaxError`. To avoid illegal shadowing, use `let` and `const` consistently and avoid reusing names.

Best practices

- Use clear, descriptive variable names to reduce the chance of collisions.
- Limit the scope of variables—declare them where they are needed.
- Consider using linting tools (like ESLint) to warn about accidental shadowing.
- Avoid using `var` in modern code; prefer `let` and `const` to get block-scoped variables.

Real-world analogy

Imagine you work at an office with two break rooms (scopes). Both rooms have a coffee machine labeled "Coffee." When you're in the inner break room, pressing the "Coffee" button gives you the coffee from that room—not the coffee from the outer break room. Once you leave the inner room, the outer machine is available again. Similarly, inner variables take precedence over outer ones while inside that scope.

Example: pitfalls of shadowing

```

``js
const count = 10;

function updateCount() {
  // This inner count shadows the outer one. Did we mean to overwrite it?
  let count = count + 1; // ReferenceError: Cannot access 'count' before initialization
}

updateCount();
``

```

In `updateCount`, `let count` declares a new `count` in the function scope, which masks the outer `count`. However, JavaScript cannot initialize `count` with its own value (`count + 1`), because at that point the inner `count` is still uninitialized—leading to a `ReferenceError`. The fix is to use a different variable name or remove the `let` keyword.

Common misconceptions

- **Shadowing always causes errors.** Shadowing is often intentional (e.g., looping variables). It only becomes problematic when you unintentionally reuse a name and operate on the wrong variable.
- **Variables declared with `var` are block-scoped.** `var` is function-scoped, so it can leak outside of block constructs and shadow variables unexpectedly.

Practice questions

1. What is variable shadowing and how does it relate to lexical scoping?
2. Why can declaring a `var` variable inside a block cause a `SyntaxError` if there is an outer `let` variable with the same name?

3. How can you avoid accidental shadowing in your code?
4. What happens in the code example above, and how would you fix it?

What is event propagation and stopPropagation?

What is event propagation and `stopPropagation`?

Introduction

Web pages are interactive thanks to **events**—clicks, key presses, scrolls, etc. When an event happens on an element, it doesn't just stay there; it propagates through the DOM. Understanding this propagation helps you decide where to attach event listeners and how to control event flow.

Phases of event propagation

When an event is dispatched, it travels through three phases:

1. **Capturing (trickling)** - The event moves down from the root (`window` or `document`) through ancestors to the target element. By default, listeners do not run during this phase unless `capture: true` is specified when registering the listener.
2. **Target** - The event reaches the target element and runs handlers attached directly to it.
3. **Bubbling** - After the target is processed, the event bubbles back up through ancestors, invoking handlers on each. This is the default phase for most event listeners.

`event.target` vs. `event.currentTarget`

Inside an event handler, `event.target` refers to the element where the event originated, while `event.currentTarget` refers to the element whose listener is currently executing. When using event delegation (attaching a handler to a parent element to handle events from its children), check `event.target` to determine which child was clicked.

Stopping propagation

Sometimes you need to prevent an event from reaching other listeners. Two methods are available:

- **`event.stopPropagation()`** - Prevents the event from bubbling further up the DOM. Other handlers on the current element will still run.
- **`event.stopImmediatePropagation()`** - Stops bubbling and prevents any remaining handlers on the current element from running.

Stopping propagation can be useful when you want to ensure a handler runs exclusively or to prevent default behavior on parent elements. However, overusing it can make your code harder to reason about. Prefer letting events bubble and using event delegation when possible.

Capturing listeners

To listen during the capturing phase, pass `{ capture: true }` as the third argument to `addEventListener()`. This is useful when you want a parent to intercept an event before it reaches the target. For example:

```
```js
document.addEventListener(
 "click",
 (e) => {
 console.log("capturing at document");
 },
 { capture: true }
);
```



```
document.body.addEventListener("click", () => {
 console.log("bubbling at body");
});

// Click anywhere on the page to see the order: capturing runs first
...

```

### ## Real-world analogy

Imagine shouting a message in a multi-story building. If you stand on the ground floor and call out someone's name, the sound travels up and down the stairs. People on lower floors hear it first (capturing), then the person you want hears it (target), then people on higher floors hear echoes as the sound bubbles back up. If someone interrupts and stops the message (calls `stopPropagation``), it doesn't go further.

### ## Common misconceptions

- **\*\*\*`stopPropagation()` stops default actions.\*\*\*** It only affects propagation; to prevent default actions (like following a link), call `event.preventDefault()``.
- **\*\*\*Listeners always run during bubbling.\*\*\*** You can register listeners during capturing by passing `{ capture: true }`.
- **\*\*\*`event.target` is always the element with the listener.\*\*\*** It refers to the element where the event originated; `event.currentTarget`` may be different if you attached the listener to an ancestor.

### ## Practice questions

1. What are the three phases of event propagation?
2. When would you use `event.stopPropagation()`` versus

``event.stopImmediatePropagation()``?

3. How do ``event.target`` and ``event.currentTarget`` differ?

4. How do you attach a handler to run during the capturing phase?

# What is Symbol in JavaScript?

## # What is a `Symbol` in JavaScript?

### ## Introduction

Symbols are a primitive data type introduced in ES6. They were added to provide unique, non-string keys for object properties. Each symbol value is guaranteed to be unique—even if two symbols are created with the same description—and cannot be automatically converted to a string.

Because of these qualities, symbols are ideal for defining "hidden" or collision-free properties on objects.

### ## Key characteristics

- **Uniqueness** - Every call to `Symbol()` returns a distinct value. Even symbols created with the same description are not equal.

```
```\js
const s1 = Symbol("id");
const s2 = Symbol("id");
console.log(s1 === s2); // false
...`
```

- **Immutability** - A symbol's value cannot be changed.
- **Non-enumerability** - Properties keyed by symbols do not appear in `for...in` loops or `Object.keys()` results. Use `Object.getOwnPropertySymbols(obj)` to retrieve them.
- **Global registry** - `Symbol.for(key)` checks a runtime-wide registry. If a symbol with the given key exists, it returns it; otherwise, it creates one. This allows sharing symbols across modules.
- **Well-known symbols** - JavaScript defines built-in symbols that change how objects behave. Examples include `Symbol.iterator` (makes an object iterable), `Symbol.toStringTag` (defines the default string tag of an

object) and `Symbol.hasInstance`` (customizes `instanceof``).

Creating and using symbols

Basic usage

```
``js
const secret = Symbol("secretId");
const user = {
  name: "Alice",
  [secret]: 12345, // Symbol as property key
};

console.log(user.name); // "Alice"
console.log(user[secret]); // 12345
...

```

The ``secret`` property is not visible with typical enumeration methods. It can only be accessed by using the symbol variable itself.

Global registry

```
``js
const uid1 = Symbol.for("uid");
const uid2 = Symbol.for("uid");
console.log(uid1 === uid2); // true - retrieved the same symbol
console.log(Symbol.keyFor(uid1)); // 'uid'
...

```

Using `Symbol.for()`` is useful for sharing a symbol across different parts of your code base; it stores the symbol in the global symbol registry.

Real-world analogy

Think of symbols like secret identifiers. Imagine you're labeling files in office drawers. You can write any label (string) on a folder, but two people might choose the same label and accidentally put their files together. Symbols are like using a unique, secret sticker that only you know. Even if others describe the sticker the same way, their stickers will be different and won't collide with yours.

Common misconceptions

- *****Symbols are private variables.***** They can't be accessed accidentally through normal property iteration, but any code that holds a reference to the symbol can access the property.
- *****Symbols replace strings for all keys.***** Symbols are useful for unique, hidden keys or when customizing built-in behavior. For normal property keys, strings are perfectly fine.
- *****Symbols are convertible to strings.***** Symbols do not implicitly convert to strings; trying to concatenate one throws a ``TypeError``. Use ``String(sym)`` or ``sym.description`` for debugging.

Practice questions

1. What makes each ``Symbol()`` unique?
2. How do symbol-keyed properties differ from string-keyed properties in enumeration?
3. What is the purpose of ``Symbol.for()``?
4. Name two well-known symbols and describe their use.

What is WeakMap and WeakSet?

What is `WeakMap` and `WeakSet`?

Introduction

ES6 introduced **WeakMap** and **WeakSet**—specialized collections that hold objects and allow them to be garbage-collected if there are no other references.

Unlike regular `Map` and `Set`, they provide *weak references* to their contents, which helps avoid memory leaks when associating data with objects.

WeakMap

A **WeakMap** is a collection of key/value pairs where keys must be objects or non-registered symbols. The value can be any type. The important characteristic is that a key's presence in a WeakMap does **not** prevent the object from being garbage-collected. When the key object is collected, its entry in the WeakMap disappears automatically.

Because keys may disappear unpredictably, WeakMaps do **not** support iteration methods (`forEach`, `keys`, `values`, etc.) or a `size` property—if they did, iterating over keys would reveal when garbage collection happens, making behavior non-deterministic.

Example usage

```
``js
const cache = new WeakMap();

function getData(obj) {
  if (cache.has(obj)) return cache.get(obj);
  const data = heavyComputation(obj);
```

```

    cache.set(obj, data);
    return data;
}

let key = {};
getData(key); // stores data in cache
key = null; // drop the only reference to the key
// at some point later, the key and its associated data will be collected
...

```

WeakMap is useful for storing metadata or caching results associated with objects without preventing them from being freed.

WeakSet

A **WeakSet** is a collection of objects or non-registered symbols. Each value may appear only once, and like WeakMap keys, values are held weakly. If an object stored in a WeakSet has no other references, it can be garbage-collected, and its entry vanishes. WeakSets also lack iteration methods and a `size` property for the same reason: values can disappear at any time.

Example usage

```

```js
const visited = new WeakSet();

function process(node) {
 if (visited.has(node)) {
 return; // avoid processing the same node twice
 }
}

```

```

visited.add(node);

// ... process node ...
}

// When node is removed elsewhere and no longer referenced,
// it will automatically disappear from visited
'''

```

## ## Real-world analogy

Imagine renting lockers (objects) at a gym. You keep a notebook to track which locker holds which customer's clothes. If a customer leaves and clears out their locker (no other references), the locker and its entry in your notebook are freed automatically. You don't maintain a master list of all lockers in use because people leave at different times. WeakMap and WeakSet behave similarly: they store data associated with objects, but entries vanish when those objects are no longer needed.

## ## Common misconceptions

- **WeakMap keeps objects alive.** The key's reference is weak; it does not prevent garbage collection.
- **You can iterate over WeakMap/WeakSet.** They deliberately omit iteration methods to hide garbage collection behavior.
- **WeakMap can have string keys.** Only objects and non-registered symbols are allowed as keys.

## ## Practice questions

1. Why do WeakMaps and WeakSets not support iteration?
2. What kinds of keys/values can be stored in a WeakMap and WeakSet?



3. Describe a scenario where a WeakMap is preferable to a regular Map.
4. What happens when the only reference to an object stored in a WeakSet is removed?

# What are Map and Set and how do they differ from objects?

## # What are `Map` and `Set`, and how do they differ from objects?

### ## Introduction

ES6 introduced two new collection types—`Map` and `Set`—that complement traditional objects and arrays. They offer more flexible key and value handling, deterministic iteration order and convenient methods. Understanding how they differ from plain objects helps you choose the right data structure.

### ## `Map`

A `Map` is a collection of key/value pairs. The key can be **any type**—object, number, string, boolean, symbol, even `NaN`—and the value can also be any type. Maps preserve insertion order and provide built-in methods to manipulate and inspect entries. In contrast, plain objects only accept strings or symbols as keys (other types are coerced to strings).

### ### Key features of `Map`

- **Arbitrary key types**: keys retain their type and are compared using the SameValueZero algorithm, meaning `NaN` is considered equal to itself.
- **Insertion order**: when iterating, entries are returned in the order they were inserted.
- **Size property**: `map.size` returns the number of entries.
- **Convenient methods**: `set(key, value)`, `get(key)`, `has(key)`, `delete(key)`, `clear()`, and iteration methods like `map.keys()`, `map.values()`, `map.entries()`.
- **Object keys**: maps allow using objects as keys without converting them to

strings.

### ### Example

```
``js
const m = new Map();
m.set("a", 1);
m.set(42, "answer");
const objKey = { id: 1 };
m.set(objKey, "object value");

console.log(m.get("a")); // 1
console.log(m.get(42)); // 'answer'
console.log(m.get(objKey)); // 'object value'
console.log(m.size); // 3

for (const [key, value] of m) {
 console.log(key, value);
}
...
``
```

### ## `Set`

A `Set` is a collection of **unique values**. Like `Map`, it preserves insertion order and provides methods to manipulate its contents. The main idea is that a value can appear only once; repeated calls to `set.add(value)` have no effect.

### ### Key features of `Set`

- **Uniqueness**: values are stored once; duplicates are ignored.

- **Any value type**: numbers, strings, objects, etc., can be added.
- **Methods**: `add(value)`, `has(value)`, `delete(value)`, `clear()`, `size`, and iteration via `for...of`, `set.keys()`, `set.values()`, and `set.entries()` (entries return `[value, value]` for compatibility).
- **Efficient lookups**: checking membership with `set.has(value)` is typically  $O(1)$ , while checking `array.includes(value)` is  $O(n)$ .

### ### Example

```

```js
const s = new Set();

s.add("apple");

s.add("banana");

s.add("apple"); // duplicate ignored

console.log(s.size); // 2

console.log(s.has("banana")); // true

for (const item of s) {
  console.log(item); // 'apple', then 'banana'
}
```

```

### ## Differences from plain objects

| Aspect                                     | Map / Set                                                                                                       | Plain objects                                                                         |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Key types</b>                           | Map accepts any type; Set stores values of any type                                                             | Keys must be strings or symbols; other types are coerced to strings                   |
| <b>Ordering</b>                            | Preserves insertion order for iteration                                                                         | Mostly insertion order in modern engines but historically not guaranteed              |
| <b>Size retrieval</b>                      | <code>map.size</code> / <code>set.size</code> returns the number of entries                                     | You must compute the count manually, e.g., <code>Object.keys(obj).length</code>       |
| <b>Iteration methods</b>                   | Built-in iterators: <code>keys()</code> , <code>values()</code> , <code>entries()</code> , <code>forEach</code> | Use <code>Object.keys</code> , <code>for...in</code> , or <code>Object.entries</code> |
| <b>Performance for additions/deletions</b> | Optimized for dynamic updates                                                                                   | Less efficient for large collections with frequent additions or deletions             |

### ## When to use each

- Use **Map** when you need keys of any type, want predictable iteration order,

or need a large dictionary with frequent insertions/deletions.

- Use **`Set`** when you need a collection of unique values, such as tracking unique visitors or removing duplicates from an array.
- Use **plain objects** for simple key/value pairs where keys are known and string/symbol keys are sufficient—objects have less overhead and simpler syntax.

### ## Practice questions

1. What key types can a `Map` accept compared to a plain object?
2. How does a `Set` ensure that each value is stored only once?
3. Name two advantages of using a `Map` over an object.
4. When might a plain object be more appropriate than a `Map` or `Set`?

# Explain shallow copy vs deep copy

## # Explain shallow copy vs deep copy

### ## Introduction

When you copy objects or arrays in JavaScript, you can do so **\*\*shallowly\*\*** or **\*\*deeply\*\***. Understanding the difference is critical when working with complex data structures. A **\*\*shallow copy\*\*** duplicates only the top-level properties; nested objects or arrays are shared between the source and the copy. A **\*\*deep copy\*\*** duplicates every level of the structure so that the copy is entirely independent.

### ## Shallow copy

A shallow copy produces a new object whose properties point to the same values as the original. If those values are primitives (numbers, strings, booleans), the copy has its own copy of the primitive. If they are objects or arrays, both objects reference the same nested object. Mutating nested data affects both the original and the copy.

### ### Example

```
```js
const original = {
  name: "Alice",
  address: { city: "Miami" },
  scores: [10, 20],
};

const shallow = { ...original }; // spread syntax creates a shallow copy
```

```
shallow.name = "Bob"; // affects only the copy
shallow.address.city = "Tampa"; // affects both shallow and original
shallow.scores.push(30); // affects both arrays
```

```
console.log(original.address.city); // 'Tampa'
console.log(original.scores); // [10, 20, 30]
...

```

All standard built-in copy operations—spread syntax (`{...obj}`), `Array.prototype.concat()`, `Array.prototype.slice()`, `Object.assign()`—create ****shallow copies****.

When to use shallow copies

Shallow copies are efficient for flat objects (no nested references) or when you intentionally want the copy to share nested structures with the original. For instance, copying a configuration object that contains immutable nested objects is safe.

Deep copy

A deep copy duplicates everything recursively, so that the new object shares no references with the original. Modifying the copy does not affect the original and vice versa. Deep copies are necessary when working with mutable nested data that should be independent.

Ways to deep copy

1. ****Recursive copying****: write a function that iterates through properties and recursively copies objects and arrays.
2. ****JSON serialization****: for JSON-friendly data, you can use

``JSON.parse(JSON.stringify(obj))``. This method fails for functions, ``Date``, ``Map``, ``Set``, ``undefined``, and cyclic structures.

3. **`structuredClone()`**: a built-in method that deep clones objects and supports many types and cyclic references; see the next topic for more details.

Example using `structuredClone()`

```
``js
const original = { date: new Date(), list: [1, 2, 3] };
const deep = structuredClone(original);

deep.list.push(4);
console.log(original.list); // [1, 2, 3] - unchanged

// Changing the date in the original doesn't affect the copy
original.date.setFullYear(2030);
console.log(deep.date.getFullYear()); // original change has no effect
``
```

Real-world analogy

Think of photocopying a document. A **`shallow copy`** is like making a copy where all attachments (post-it notes) remain stuck on the original. Both the original and copy share the same attachments, so moving or modifying an attachment affects both. A **`deep copy`** is like making a copy and reprinting all attachments separately; the new document has its own independent notes.

Common misconceptions

- **`"Spread syntax always makes a deep copy."`** It only copies the first level; nested objects remain shared.

- *****Deep copies are always better.***** They are more expensive to create and may not be needed for flat structures. Use shallow copies when you don't need full independence.
- *****`JSON.parse(JSON.stringify())` can clone anything.***** It cannot clone functions, `Date`, `Map`, `Set`, undefined values, or cyclic references.

Practice questions

1. What is the key difference between a shallow and deep copy?
2. Name two methods to perform a deep copy. What are their limitations?
3. Why might you choose a shallow copy over a deep copy in certain situations?
4. What happens when you modify a nested object in a shallow copy?

What is structuredClone?

What is `structuredClone`?

Introduction

Deep copying complex objects in JavaScript can be tricky. Many popular techniques (spread syntax, `Object.assign`, `JSON.parse(JSON.stringify())`) fail for functions, dates, maps, sets, typed arrays, or cyclic structures. The `structuredClone()` method provides a built-in way to **deep clone** most JavaScript values safely.

What does `structuredClone()` do?

`structuredClone()` creates a deep copy of a given value using the **structured clone algorithm**. It can clone primitives, plain objects, arrays, typed arrays, Maps, Sets, Dates, RegExps and more. The resulting clone is completely independent; changes to the clone do not affect the original.

Syntax

```
``js
const clone = structuredClone(value, options?);
``
```

- **value**: any structured-cloneable type—primitives, objects, arrays, typed arrays, Maps, Sets, Dates, etc.
- **options** (optional): an object with a `transfer` property. You can transfer **transferable objects** (ArrayBuffer, MessagePort) to the clone instead of copying them. When an object is transferred, it is detached from the original and attached to the new object.

Return value and exceptions

The function returns a deep copy of the input.

If any part of the input contains unserializable data (like DOM nodes, functions or WeakMaps), `structuredClone()` throws a `DataCloneError`.

Examples

Cloning basic objects and arrays

```
```js
const original = { a: 1, b: { c: 2 }, d: [3, 4] };
const copy = structuredClone(original);

copy.b.c = 42;
copy.d.push(5);
console.log(original.b.c); // 2 - original unchanged
console.log(original.d); // [3, 4] - original unchanged
...
```
```

Cloning and transferring an ArrayBuffer

```
```js
const buffer = new ArrayBuffer(8);
const clone = structuredClone(buffer, { transfer: [buffer] });

// The original buffer is now detached and unusable
console.log(buffer.byteLength); // 0
console.log(clone.byteLength); // 8
...
```
```

Real-world analogy

Imagine duplicating a file on your computer. A typical copy duplicates the file's contents, while leaving the original intact. `structuredClone()` is like using a special copying tool that not only handles simple documents but also copies entire folders, compressed files and even broken links—everything is duplicated accurately. If you choose to transfer a large folder instead of copying it, the original folder disappears and only the new one remains.

Common misconceptions

- *****`structuredClone()` is the same as JSON serialization.***** JSON serialization cannot clone functions, dates, maps, sets or typed arrays, and it fails on cyclic objects. `structuredClone()` handles many of these cases and preserves special types.
- *****It can clone any JavaScript value.***** Some types (DOM nodes, functions, WeakMap/WeakSet) are not structured-cloneable and will throw `DataCloneError`.

Practice questions

1. What does `structuredClone()` do that `JSON.parse(JSON.stringify())` cannot?
2. What types can be transferred rather than cloned using the `transfer` option?
3. What happens to the original object when you transfer a transferable resource?
4. Name two values that cannot be cloned with `structuredClone()`.

What are Web Workers and when should you use them?

What are web workers and when should you use them?

Introduction

JavaScript in the browser normally runs on a ****single main thread**** that handles user input, renders the page and executes your code. If your code performs a heavy computation, the browser cannot respond to user input until that computation finishes, leading to a frozen interface. ****Web workers**** provide a simple way to run scripts in background threads so long-running tasks don't block the UI.

What is a web worker?

A web worker is created by calling `new Worker('worker.js')`, which spawns a new thread running the specified script. Workers have their own global context (similar to a separate window), cannot directly access the DOM and communicate with the main thread via message passing. Workers run scripts in background threads and can perform tasks without interfering with the user interface. Once created, a worker can send messages to the main thread and receive messages back using `postMessage()` and the `onmessage` event handler.

Basic structure

****Main thread (page)****

```
``js
// main.js
const worker = new Worker("worker.js");
```

```

worker.onmessage = (e) => {
  console.log("Received from worker:", e.data);
};

worker.postMessage({ type: "start", value: 40 });

// later, terminate the worker
// worker.terminate();
...

```

****Worker script****

```

````js
// worker.js
self.onmessage = (e) => {
 const { type, value } = e.data;
 if (type === "start") {
 const result = fib(value);
 self.postMessage(result);
 }
};

function fib(n) {
 return n <= 1 ? n : fib(n - 1) + fib(n - 2);
}
...

```

Let's walk through the interaction step by step to see what each line does and when each handler is invoked.

### ### 1. Creating the worker

```
```js
const worker = new Worker("worker.js");
```
```

On the main thread (the page), this line spawns a new background thread and instructs it to run the code contained in `worker.js`. A worker has its own global scope—separate from the main thread—and cannot access the DOM or use most of the `window` object. The `Worker` constructor returns a `Worker` instance that the main thread can use to communicate with this background thread.

### ### 2. Setting up a message handler on the main thread

```
```js
worker.onmessage = (e) => {
  console.log("Received from worker:", e.data);
};
```
```

The `onmessage` property of the `Worker` instance is an event handler for the `"message"` event. It is called whenever the worker thread sends a message back to the main thread. The handler receives an event object (`e`), whose `data` property contains whatever data the worker posted. At this point, nothing is called yet; you're just registering a callback so you can handle responses in the future.

### ### 3. Posting a message to the worker

```
```js
worker.postMessage({ type: "start", value: 40 });
```
```

Here, the main thread sends a message to the worker. It uses `postMessage()`, which serializes the given data and delivers it to the worker thread. Because workers communicate via message passing, data is copied rather than shared. In this example, the main thread sends an object with a `type` of `"start"` and a `value` of `40`, instructing the worker to begin a computation.

### ### 4. Handling the message in the worker

In `worker.js`, the worker registers its own `onmessage` handler:

```
```js
self.onmessage = (e) => {
  const { type, value } = e.data;
  if (type === "start") {
    const result = fib(value);
    self.postMessage(result);
  }
};
```
```

The `self` keyword inside the worker refers to the worker's global scope. When the main thread calls `postMessage()`, the worker's `onmessage` handler fires, receiving the data in `e.data`. The worker checks the message type; if it is `"start"`, it calls the `fib()` function to compute the 40th Fibonacci number. After finishing, it calls `self.postMessage(result)` to send the result back to the main thread.

### ### 5. Completing the round trip

Once the worker posts the result, the browser delivers it to the main thread and triggers the handler you assigned earlier:

```
```js
worker.onmessage = (e) => {
  console.log("Received from worker:", e.data);
};
```
```

At this point, `e.data` contains the Fibonacci number computed in the worker. The callback logs the value. Any subsequent messages from the worker will also trigger this handler.



### ### 6. Cleaning up

If you no longer need the worker, you can terminate it:

```
```js
// worker.terminate();
```
```

Calling `terminate()` immediately stops the worker's thread and frees its resources.

Putting it all together: the main thread creates a worker, sets up a response handler (`onmessage`), and sends a message using `postMessage()`. The worker receives that message in its own `onmessage` handler, performs the computation, and sends the result back using `postMessage()`. The main thread then receives the result via its `onmessage` callback. Because the heavy computation runs in a separate thread, the user interface remains responsive throughout.

### ## Types of workers

- **Dedicated workers**: The most common type. A dedicated worker is tied to a single script. Only the thread that created it can communicate with it.
- **Shared workers**: A shared worker can be accessed from multiple scripts running in different windows or tabs, provided they are from the same origin.
- **Service workers**: A special worker that intercepts network requests, enabling offline caching and background sync. Service workers are not directly used for computational tasks, but they share some worker characteristics.

### ## When to use web workers

Use web workers when you need to perform CPU-intensive or blocking operations that would otherwise freeze the UI. Examples include:

- **Data processing**: Sorting large arrays, parsing big JSON files, doing cryptographic operations or image manipulation.
- **Network requests**: Although fetch runs asynchronously, combining requests with heavy processing (like decompressing large files) benefits from a worker.
- **Real-time calculations**: Physics simulations or game logic.

Avoid using a worker for simple tasks or frequent updates that would incur unnecessary message passing overhead. Also remember that workers cannot directly access DOM elements or most of the `window` object.

## ## Real-world analogy

Think of a web worker as a personal assistant. While you (the main thread) interact with users and handle immediate tasks, your assistant can work on a complex report in another room. You occasionally exchange notes (messages), but you don't look over each other's shoulders. If you want the report to stop, you tell your assistant to stop working (terminate the worker).

## ## Common misconceptions

- **"Web workers can access the DOM."** They cannot. Only the main thread can manipulate the DOM.
- **"Workers share memory with the main thread."** They communicate by copying or transferring data via messages; data is not shared.
- **"Workers always improve performance."** Spawning a worker has overhead. For small tasks, it's cheaper to run them on the main thread.

## ## Practice questions

1. What problem do web workers solve?

2. How do the main thread and a worker communicate?
3. Why can't a worker access the DOM?
4. When might using a worker be unnecessary or counterproductive?

# What are Service Workers and PWA concepts?

## # What Are Service Workers and PWA Concepts?

Progressive Web Apps (PWAs) bridge the gap between traditional web pages and native applications. They use modern browser APIs to deliver reliable, installable, offline-capable experiences. At the heart of a PWA is the **\*\*service worker\*\***, a script that runs separately from the main page and controls how the app interacts with the network.

### ## What is a service worker?

A service worker is a background script registered by your application. Once installed and activated, it sits between your app and the network, intercepting network requests and deciding how to respond. Because it runs independently of any web page, it can respond to events even when your site is not open. Service workers can:

- **\*\*Cache assets and data\*\*** so your app can work offline or with poor connectivity.
- **\*\*Serve cached responses\*\*** immediately while fetching updated data in the background.
- **\*\*Manage background tasks\*\*** like push notifications or background sync.
- **\*\*Act as a proxy\*\*** to modify or log requests and responses.

Service workers have no direct access to the DOM. They communicate with pages via the ``postMessage`` API and must be served over HTTPS for security.

### ### Lifecycle

Service workers follow a predictable lifecycle:

1. **\*\*Registration\*\*** - Your page calls ``navigator.serviceWorker.register('/sw.js')`` to start installing a worker. Registration happens on page load and must succeed before the service worker can control pages.
2. **\*\*Installation\*\*** - The service worker downloads and runs the installation code. You typically pre-cache core assets in the ``install`` event so they are available offline.
3. **\*\*Activation\*\*** - After installation, the service worker activates. It clears old caches and takes control of pages within its scope. New versions wait to activate until all pages using the old version are closed, so updates don't disrupt the current users.

4. **\*\*Idle/Fetch\*\*** - Once active, the worker listens for events such as `fetch`, `push` and `sync`. It decides how to respond to network requests—either from the cache, network or a combination.

Only one service worker can control a given scope (directory path) at a time. When you update your worker script, the browser downloads the new version, installs it in the background and waits to activate until the old version has no more clients.

### ### Registering and using a service worker

Here's a minimal example that installs a service worker and caches an asset:

```
```.js
// main.js - register the service worker
if ("serviceWorker" in navigator) {
  window.addEventListener("load", () => {
    navigator.serviceWorker.register("/sw.js").catch((err) => {
      console.error("Service worker registration failed:", err);
    });
  });
}

// sw.js - service worker
const CACHE_NAME = "pwa-cache-v1";
const ASSETS = ["/", "/styles.css", "/index.html", "/logo.png"];

self.addEventListener("install", (event) => {
  // Pre-cache core assets
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => cache.addAll(ASSETS))
  );
});
```

```

self.addEventListener("activate", (event) => {
  // Remove old caches
  event.waitUntil(
    caches
      .keys()
      .then((keys) =>
        Promise.all(
          keys
            .filter((key) => key !== CACHE_NAME)
            .map((key) => caches.delete(key))
        )
      )
  );
});

self.addEventListener("fetch", (event) => {
  // Respond with cache first, then network
  event.respondWith(
    caches.match(event.request).then((cached) => {
      return (
        cached ||
        fetch(event.request).then((response) => {
          // Update the cache for next time
          return caches.open(CACHE_NAME).then((cache) => {
            cache.put(event.request, response.clone());
            return response;
          });
        })
      );
    })
  );
});

```

```
});
```

```
...
```

In the example, the service worker caches static assets during installation and serves them from the cache when offline. It also updates the cache whenever a fresh version is downloaded.

PWA concepts beyond service workers

1. **Web App Manifest** - A JSON file (`manifest.webmanifest`) describing your app's name, icons, theme colors and how it should appear when installed on a user's home screen. It allows users to add your PWA to their device.
2. **HTTPS** - PWAs require secure contexts. Browsers block service worker registration on insecure origins to prevent man-in-the-middle attacks.
3. **Responsive design** - PWAs should adapt to different screen sizes and device capabilities.
4. **Installability** - When a PWA meets criteria (served over HTTPS, has a manifest, registered service worker and user engagement), the browser prompts users to install the app.
5. **Offline and connectivity independence** - Caching and local data storage enable the app to function even when the network is unavailable.

Real-world analogy

Imagine a restaurant that uses a **waiter** to handle orders. Instead of every customer shouting orders to the kitchen (the network), the waiter (the service worker) intercepts orders, writes them down, and checks if some meals are already prepared (cached). If the meal is ready, the waiter serves it immediately. Otherwise, the waiter sends the order to the kitchen and serves it when it's done. If the restaurant closes (the user goes offline), the waiter can still serve meals that were prepared earlier.

Common pitfalls and best practices

- **Scope placement**: Place your service worker file at the top of the directory you want it to control. A worker registered at `/sw.js` controls the entire site, while `/blog/sw.js` only controls files under `/blog`.
- **Updates not activating**: New versions wait until all pages using the old worker are closed. Use `self.skipWaiting()` in the `install` event and `clients.claim()` in the `activate` event to take control immediately, but beware that this can refresh pages unexpectedly.

- ****Caching too much****: Cache only assets that are necessary for offline use. Uncontrolled caching can fill storage or serve stale content.
- ****Offline fallbacks****: Provide fallback pages or messages when resources cannot be fetched.

Practice questions

****Conceptual questions****

1. What is a service worker and how does it differ from a regular web worker?
2. Describe the lifecycle of a service worker. Why doesn't a new service worker take control immediately after installation?
3. How does a Progressive Web App benefit from a service worker? List at least three capabilities.
4. Why must service workers be served over HTTPS?
5. Explain how caching strategies (cache-first, network-first, stale-while-revalidate) work. When would you use each?

****Coding exercises****

1. Write a script to register a service worker and log whether registration succeeded or failed.
2. Modify the example service worker so that it serves a fallback HTML page (`offline.html`) when the requested page is not available offline and the network is down.
3. Implement a caching strategy where the service worker always fetches resources from the network first and falls back to the cache if the network request fails.
4. Add code to your service worker to display a push notification when receiving a `push` event. How would you handle the user clicking on the notification?

Explain Promises.all, Promise.race, and Promise.allSettled

Explaining `Promise.all()`, `Promise.race()`, and `Promise.allSettled()`

Modern JavaScript uses promises to represent asynchronous operations such as network requests, timers or file reads. When you have **multiple promises** that should run in parallel, the language provides helper methods to orchestrate them. The three most common helpers—`Promise.all()`, `Promise.race()`, and `Promise.allSettled()`—behave differently when resolving or rejecting a group of promises. Understanding these differences helps you choose the right tool for your scenario.

`Promise.all()`

`Promise.all()` accepts an iterable (usually an array) of promises and returns a **new promise**. This returned promise fulfills when **every** input promise fulfills. If **any** input promise rejects, the returned promise rejects immediately with that reason. The order of results corresponds to the order of the input promises, not the order in which they resolved.

Use `Promise.all()` when you need all results to continue—for example, fetching user details, preferences and settings before rendering a dashboard. It runs all promises concurrently and aggregates their results.

```
```js
// Fetch three resources concurrently
const userPromise = fetch("/api/user");
const prefsPromise = fetch("/api/preferences");
const settingsPromise = fetch("/api/settings");

Promise.all([userPromise, prefsPromise, settingsPromise])
 .then(async ([userRes, prefsRes, settingsRes]) => {
 // Parse JSON responses
 const [user, prefs, settings] = await Promise.all([
 userRes.json(),
 prefsRes.json(),
 settingsRes.json(),
]);
 });
```
```

```

    });
    // Now we have all the data and can render
    renderDashboard(user, prefs, settings);
  })
  .catch((err) => {
    // If any request failed, handle it here
    console.error("At least one request failed:", err);
  });
  ...

```

Key points

- The returned promise resolves to an ****array of results**** in the same order as the input promises.
- If any promise rejects, the entire operation fails immediately. This is useful when you cannot proceed without all results.
- The original promises continue running even if one rejects—you cannot cancel them with `Promise.all()` alone.

`Promise.race()`

`Promise.race()` takes an iterable of promises and returns a promise that settles (fulfills or rejects) as soon as ****the first input promise settles****. The returned promise adopts the value or reason of the first settled promise.

Use `Promise.race()` when you want to proceed with whichever promise finishes first, regardless of success or failure. Common patterns include implementing timeouts or selecting the fastest source.

```

``js
// Timeout helper: rejects if a promise takes too long
function withTimeout(promise, ms) {
  return Promise.race([
    promise,
    new Promise((_, reject) =>

```

```

    setTimeout(() => reject(new Error("Operation timed out")), ms)
  ),
]);
}

```

```

withTimeout(fetch("/api/data"), 3000)
  .then((res) => res.json())
  .then((data) => console.log("Data loaded within 3s", data))
  .catch((err) => console.error(err.message));
...

```

Key points

- Whichever promise settles first (resolve or reject) decides the result.
- Other promises keep running in the background; use cancellation mechanisms like ``AbortController`` to abort network requests if needed.
- Suitable for implementing ****fallbacks****, such as requesting data from multiple mirrors and using the first response.

`Promise.allSettled()`

``Promise.allSettled()`` returns a promise that fulfills ****after all input promises have settled****, regardless of whether they fulfilled or rejected. The result is an array of objects describing the outcome of each promise. Each object has a ``status`` property (``fulfilled`` or ``rejected``) and either a ``value`` or a ``reason`` property.

Use ``Promise.allSettled()`` when you want to wait for ****all promises to finish**** but don't want one failure to short-circuit the rest. For example, you might want to display partial results while noting which requests failed.

```

```js
const urls = ["/api/user", "/api/preferences", "/api/broken"];
const fetchPromises = urls.map((url) => fetch(url));

```

```

Promise.allSettled(fetchPromises).then((results) => {
 results.forEach((result, index) => {
 if (result.status === "fulfilled") {
 console.log(`Request ${index} succeeded`);
 } else {
 console.warn(`Request ${index} failed:`, result.reason);
 }
 });
});
...

```

### ### Key points

- Always resolves, never rejects. You handle successes and failures separately by inspecting each result.
- The order of the results matches the order of the input promises.
- Useful for parallel operations where failures are acceptable or expected (e.g., loading optional resources).

### ## Choosing the right helper

| Scenario                            | Use                                 |
|-------------------------------------|-------------------------------------|
| -----                               | -----                               |
| Need all results or fail fast       | <code>`Promise.all()`</code>        |
| Use the earliest result             | <code>`Promise.race()`</code>       |
| Wait for all, regardless of outcome | <code>`Promise.allSettled()`</code> |

### ### Real-world analogy

Imagine ordering parts from several suppliers:

- **`Promise.all()`** is like waiting for all your suppliers to deliver before you can start assembling. If any supplier fails to deliver, your project stalls.
- **`Promise.race()`** is like taking the first quote that arrives. You proceed with whichever supplier responds first.
- **`Promise.allSettled()`** is like checking in at the end of the day to see which suppliers delivered and which didn't. You then decide what to do with the partial orders.

## ## Practice questions

### **Conceptual questions**

1. Explain the difference between `Promise.all()` and `Promise.allSettled()`. When would you choose one over the other?
2. In `Promise.race()`, what happens if the first promise rejects? How can you handle this case gracefully?
3. Why does `Promise.all()` reject as soon as any promise rejects? How could you modify your code to collect all errors instead of failing fast?
4. Describe a real use case for each of the three methods discussed.

### **Coding exercises**

1. Write a function `loadAll(urls)` that takes an array of URLs, fetches them concurrently and returns an array of response bodies using `Promise.all()`. It should reject if any fetch fails.
2. Implement a helper `firstResolved(promises)` that returns the value of the first fulfilled promise and ignores any rejections. Use `Promise.race()` along with additional logic to skip rejected promises.
3. Write a function that fetches multiple resources and logs which succeeded and which failed using `Promise.allSettled()`. Then modify it to retry failed requests once.

# What is BigInt in JavaScript?

## # What Is `BigInt` in JavaScript?

JavaScript's `number` type is a 64-bit double-precision floating-point value. This format can exactly represent integers up to  $2^{53} - 1$  (9,007,199,254,740,991). Beyond this range, integer arithmetic loses precision, resulting in rounding errors. To solve this problem, ECMAScript introduced the **BigInt** type.

## ## Introducing BigInt

`BigInt` is a built-in primitive for representing **whole numbers of arbitrary size**. Unlike regular numbers, BigInt values can grow as large as memory allows without losing precision. You create a BigInt by either appending an `n` to an integer literal or by calling the `BigInt()` constructor.

```
``js
const huge1 = 9007199254740993n; // note the n suffix
const huge2 = BigInt("123456789012345678901234567890");

console.log(huge1 + 2n); // 9007199254740995n
console.log(huge2 * 10n); // 1234567890123456789012345678900n
``
```

A BigInt literal cannot contain a decimal point or exponent; BigInt values always represent integer quantities. Internally, BigInts use an arbitrary-precision representation separate from the IEEE-754 format used by `number`.

## ### Operations on BigInts

Most arithmetic operations (`+`, `-`, `\*`, `/`, `%`, `\*\*`) work with BigInts, but **you cannot mix BigInt and `number` in a single operation**. Doing so throws a `TypeError` to prevent implicit coercion and precision loss. Always convert between types explicitly:

```
``js
const n = 42;
```

```

const big = 10n;

// Invalid: TypeError
// console.log(n + big);

// Convert number to BigInt
const result1 = BigInt(n) + big; // 52n

// Convert BigInt to number (may lose precision if big is huge)
const result2 = Number(big) + n; // 52
...

```

Division with ``/`` returns a truncated result (any fractional part is discarded), because `BigInt` represents only whole numbers. Bitwise operators except ``>>>`` (unsigned right shift) work as well.

### ### Limitations and caveats

- **No mixing with regular numbers**: `BigInts` and numbers don't implicitly convert. Always cast explicitly.
- **Math library is unsupported**: ``Math`` methods (``Math.sqrt``, ``Math.sin``, etc.) do not accept `BigInts`. Use third-party libraries for advanced operations.
- **JSON serialization**: ``JSON.stringify()`` throws when encountering a `BigInt` because JSON doesn't have a `BigInt` type. Convert `BigInts` to strings before serialization.
- **Inconsistent API support**: Some browser APIs accept only numbers. Check whether `BigInt` is supported before using it.

### ### When to use BigInt

`BigInt` is useful when working with:

- **Cryptography** and **large hashes** that require precise integer math.
- **Financial and scientific calculations** where integer precision is critical.

- **Counters** or **IDs** that can exceed the range of 64-bit numbers (e.g., blockchain block numbers).

However, BigInts cannot represent decimal fractions. For currency calculations involving cents, consider using a decimal library or storing amounts as integers of the smallest unit (e.g., cents) with BigInt.

### Real-world analogy

Think of `number` as a **typical calculator**—it has a limited number of digits it can display. Once you exceed those digits, it starts rounding. BigInt is like a **scientific calculator** with expandable memory. It may be slower, but it allows you to keep adding digits without losing any.

### Practice questions

#### Conceptual questions

1. What problem does BigInt solve that the regular `number` type cannot? Give an example where a `number` loses precision.
2. How do you create a BigInt literal? Why can't BigInt values have decimals?
3. Why does JavaScript throw a `TypeError` when you try to add a `number` to a BigInt? How can you perform such an addition correctly?
4. List at least three use cases where BigInt is a better choice than `number`.
5. What happens when you divide one BigInt by another? Explain why fractional results are handled the way they are.

#### Coding exercises

1. Implement a function `factorialBig(n)` that returns the factorial of a non-negative integer `n` using BigInt. For example, `factorialBig(20)` should return `2432902008176640000n`.
2. Write a function that sums a list of numbers and BigInt values. It should return a BigInt and handle type conversions appropriately.
3. Create a function `compareBig(a, b)` that accepts two numbers or BigInts (or a mix) and returns `-1`, `0` or `1` depending on whether `a` is less than, equal to, or greater than `b`.



4. Modify `JSON.stringify()` to serialize objects containing `BigInts` by converting them to strings. Write a helper `stringifyWithBigInt()` that replaces `BigInts` with their string representation before serialization.

# Explain dynamic imports and code splitting

## # Dynamic Imports and Code Splitting in JavaScript

As applications grow, bundling all of your code into one large file slows down initial page loads. To improve performance, modern tools let you **\*\*split code into smaller chunks\*\*** that load only when needed. JavaScript's dynamic `import()` function plays a central role in this strategy.

### ## Static vs. dynamic imports

Traditionally, modules are loaded using static `import` statements:

```
``js
import { add } from "./math.js";
...

```

Static imports must appear at the top level of your file and are resolved during compilation. All dependencies get bundled into the initial script. In contrast, **\*\*dynamic imports\*\*** are functions that return a promise and can be called at runtime:

```
``js
// Load the math module only when needed
async function onCalculate() {
 const math = await import("./math.js");
 console.log(math.add(2, 3));
}

button.addEventListener("click", onCalculate);
...

```

With dynamic imports, you can load modules on demand, such as when a user clicks a button or navigates to a new route. This reduces the upfront cost of downloading code that might never be used.

## ## Code splitting

**Code splitting** is the practice of dividing your application code into separate bundles that can be loaded independently. When using bundlers like webpack, Rollup or Parcel, dynamic `import()` calls signal to the bundler that a new chunk should be created. Each chunk contains only the code necessary for that part of the application.

Benefits of code splitting include:

- **Faster initial load** - Users download only the core functionality needed to render the first screen.
- **Lazy loading** - Additional features (e.g., an admin panel or charting library) load only when the user triggers them.
- **Parallel downloads** - Browsers can download multiple chunks concurrently.

### ### Naming and controlling chunks

Many bundlers let you assign custom names to chunks for easier debugging and caching. In webpack, you can specify a `webpackChunkName` comment:

```
```js
import(
  /* webpackChunkName: "chart" */
  "./components/Chart.js"
).then((module) => {
  const Chart = module.default;
  new Chart();
});
```
```

The bundler generates a file like `chart.js` that is loaded only when the import is executed.

## ## Real-world example: Route-based splitting

In single-page applications (SPAs), it's common to split code by route. Each route's component and its dependencies are put into a separate chunk. When the user navigates to a route, the framework dynamically imports the component and displays it. For example, in React with `React.lazy`:

```
``jsx

import React, { Suspense } from "react";

const AdminPage = React.lazy(() => import("./pages/AdminPage"));

function App() {
 return (
 <Router>
 <Route path="/" element={<Home />} />
 <Route
 path="/admin"
 element={
 <Suspense fallback={<Spinner />}>
 <AdminPage />
 </Suspense>
 }
 />
 </Router>
);
}

...

```

Here the `AdminPage` component is bundled separately. It is fetched when the user navigates to `/admin` and rendered inside a fallback UI while loading.

## ## Common pitfalls

- **\*\*Dynamic imports return a promise\*\*** - Always handle them asynchronously. If you forget to `await` or use `.then()`, your code may try to access undefined exports.
- **\*\*CORS restrictions\*\*** - When loading modules from other domains, ensure the server sets the appropriate CORS headers. Browsers enforce same-origin policies for module scripts.
- **\*\*Multiple imports\*\*** - Calling dynamic `import()` multiple times for the same module may return cached copies, but bundlers can generate duplicate chunks if configured incorrectly.
- **\*\*Not prefetching\*\*** - For anticipated interactions (like the next page), use `<link rel="prefetch">` or bundler features to prefetch chunks during idle time.

## ## Analogy

Imagine your application as a toolkit. **\*\*Static imports\*\*** pack all tools into a single heavy toolbox. You might carry tools you never use. **\*\*Dynamic imports\*\*** let you keep seldom-used tools on a shelf and grab them only when needed, making the initial load lighter and more efficient.

## ## Practice questions

### **\*\*Conceptual questions\*\***

1. What is the difference between static `import` and dynamic `import()`? Why can dynamic imports be placed inside functions or conditional blocks?
2. How does code splitting improve page performance? Describe scenarios where splitting code by route or component makes sense.
3. Explain how bundlers like webpack use dynamic imports to create separate chunks. What happens when multiple dynamic imports refer to the same module?
4. What are potential downsides of relying heavily on dynamic imports?

### **\*\*Coding exercises\*\***

1. Rewrite a simple module `greet.js` that exports a function. Then write code that uses dynamic `import()` to load and execute `greet()` only after the user clicks a button.
2. Using webpack (or another bundler of your choice), configure a project to split code into separate bundles when certain routes are visited. Verify that the generated chunks are loaded on demand.
3. Write a helper `lazyLoad(path)` that wraps dynamic `import()` and caches the loaded module, ensuring that subsequent calls don't fetch it again.

4. Implement a fallback UI that displays a spinner while a dynamically imported component is loading. Once loaded, render the component.

# What are optional chaining and nullish coalescing operators?

## # Optional Chaining (`?.`) and Nullish Coalescing (`??`) Operators

JavaScript applications often work with deeply nested objects. Accessing a property somewhere down the chain can throw an error if an intermediate property is undefined. To simplify safe access and default values, the language introduced two operators: **\*\*optional chaining\*\*** and **\*\*nullish coalescing\*\***.

### ## Optional chaining (`?.`)

The optional chaining operator (`?.`) allows you to safely access properties, call functions or index arrays on a value that might be `null` or `undefined`. If any part of the chain is `null` or `undefined`, the entire expression short-circuits and returns `undefined` instead of throwing a `TypeError`.

#### ### Accessing nested properties

Without optional chaining:

```
```js
const city = user && user.address && user.address.city;
```
```

With optional chaining:

```
```js
const city = user?.address?.city;
```
```

Both expressions return the same result—`undefined` if `user` or `user.address` is missing—but the latter is cleaner and less error-prone. Optional chaining can be used for:

- **\*\*Property access\*\***: `obj?.prop`

- **\*\*Array/Map access\*\***: `arr?.[index]`
- **\*\*Method calls\*\***: `obj.method?.()`

Each `?.` checks the part before it. If that part is `null` or `undefined`, the entire expression returns `undefined` and stops evaluating. Optional chaining does not catch other falsy values (such as `0` or `''`).

### ### Calling functions and optional methods

When calling a function that might not exist, optional chaining prevents errors:

```
```js
button.onclick = null;

// Later ...

button.onclick?.(event); // Does nothing instead of throwing
```
```

In this case, if `onclick` is `null`, the call is skipped. Optional chaining only short-circuits the immediate member access; side effects on the left side (e.g., increment operators) still happen before the check.

### ### Not a substitute for validation

Optional chaining tells you something is missing but doesn't fill the gap. If a value is required, you must still validate it and handle the missing case appropriately.

### ## Nullish coalescing (`??`)

The nullish coalescing operator (`??`) returns its right-hand operand when the left-hand operand is `null` or `undefined`; otherwise it returns the left-hand operand. It's useful for providing default values only when a value is truly absent, not when it is another falsy value.

```
```js
```



```
const name = user.name ?? "Anonymous";  
...
```

If `user.name` is `null` or `undefined`, `name` becomes `'Anonymous'`. If `user.name` is an empty string or zero, those values are preserved. This differs from the logical OR operator (`||`), which treats any falsy value—`0`, `NaN`, `''`, `false`—as a signal to use the fallback.

Combining optional chaining and nullish coalescing

Together, these operators allow concise, safe access with defaults:

```
```js  
const zip = user?.address?.zip ?? "00000";
...
```

If `user` or `user.address` is `undefined`, or if `zip` itself is `undefined` or `null`, `zip` defaults to `'00000'`.

### ## Analogy

Imagine navigating through a series of doors in a building. Optional chaining is like checking whether each door exists before you walk through it. If a door is missing, you stop instead of walking into an error. Nullish coalescing is like saying "If there is no room here (null or undefined), use this backup room; otherwise, use the room you found."

### ## Practice questions

#### **\*\*Conceptual questions\*\***

1. What problem does the optional chaining operator solve? Give an example where it prevents a run-time error.
2. How does `obj?.prop` differ from `obj.prop` in terms of short-circuiting behaviour? What happens if `obj` is `null` or `undefined`?

3. Compare the nullish coalescing operator (`??`) with the logical OR operator (`||`). What values cause each to use the default?
4. Can you use optional chaining with function calls and array indexing? Provide syntax examples.

### **\*\*Coding exercises\*\***

1. Given a nested object describing a product (`product.specs.dimensions.height`), write a function that safely retrieves the height using optional chaining. If any part is missing, return `0` as the default.
2. Implement a function `getUserName(user)` that returns `user.name` if present; otherwise returns `'Guest'`. Use both optional chaining and nullish coalescing.
3. Write a function that takes an array of user objects and returns the first defined `email` property using optional chaining and nullish coalescing, or returns `'no email'` if none exist.
4. Demonstrate how misuse of `||` instead of `??` could accidentally treat an empty string as missing. Rewrite the example correctly.

# Explain Proxy and Reflect API

## # Proxy and Reflect APIs in JavaScript

JavaScript objects expose certain behaviours—reading a property, assigning a value, calling a function, checking membership—with built-in semantics. The **Proxy** API allows you to customize these fundamental operations by wrapping an object in an intermediary. The **Reflect** API complements Proxy by providing methods that perform the default behaviour of those operations in a uniform way.

### ## Proxy: intercepting object operations

A **proxy** is created with `new Proxy(target, handler)`. The `target` is the object being wrapped, and the `handler` is an object whose properties are functions (called *traps*) that intercept operations on the target. When an operation occurs, the corresponding trap executes; you can run custom logic, block the operation, or forward it to the original target using `Reflect`.

#### ### Basic usage

```
``js
const person = { name: "Alice", age: 30 };

const handler = {
 get(target, prop, receiver) {
 console.log(`Reading property ${prop}`);
 return Reflect.get(target, prop, receiver);
 },
 set(target, prop, value, receiver) {
 if (prop === "age" && value < 0) {
 throw new Error("Age cannot be negative");
 }
 console.log(`Setting ${prop} to ${value}`);
 return Reflect.set(target, prop, value, receiver);
 },
}
```

```
};
```

```
const proxyPerson = new Proxy(person, handler);
```

```
console.log(proxyPerson.name); // logs: Reading property name, then 'Alice'
```

```
proxyPerson.age = 35; // logs: Setting age to 35
```

```
proxyPerson.age = -5; // throws Error
```

```
...
```

In this example, the `get` trap logs property reads and then uses `Reflect.get()` to perform the default behaviour. The `set` trap validates the age property before assigning it.

### ### Common traps

- `get(target, prop, receiver)` - intercepts property reads.
- `set(target, prop, value, receiver)` - intercepts property writes; return `true` if successful.
- `has(target, prop)` - traps the `in` operator.
- `deleteProperty(target, prop)` - traps `delete obj[prop]`.
- `apply(target, thisArg, argumentsList)` - traps function calls on callable targets.
- `construct(target, args, newTarget)` - traps object instantiation when the proxy is used with `new`.

Each trap can modify behaviour or delegate to the original using `Reflect` methods.

### ### Use cases for proxies

- **Validation and sanitization** - Check or normalize values before storing them.
- **Logging and debugging** - Track when and how properties are accessed or modified.
- **Virtualized collections** - Represent large or remote datasets and fetch data lazily when properties are accessed.
- **Reactive frameworks** - Libraries like Vue use proxies to detect changes and trigger UI updates.
- **Default values** - Return fallback values when a property doesn't exist.

## ## Reflect: default behaviour as functions

`Reflect` is a namespace object that provides methods corresponding to fundamental object operations. Each method takes arguments explicit rather than relying on special syntax. Reflect methods always perform the default operation without custom side effects.

Some commonly used methods include:

- `Reflect.get(target, prop, receiver)` - Default property access.
- `Reflect.set(target, prop, value, receiver)` - Default assignment; returns a boolean indicating success.
- `Reflect.has(target, prop)` - Equivalent to `prop in target`.
- `Reflect.deleteProperty(target, prop)` - Equivalent to `delete target[prop]`.
- `Reflect.apply(target, thisArg, argsArray)` - Calls a function with a specified `this` value and argument list.
- `Reflect.construct(target, args, newTarget)` - Creates an instance of a constructor function.

When writing proxy traps, using Reflect ensures your proxy behaves consistently with the language's default semantics.

## ## Analogy

Think of a proxy as a **\*\*security guard\*\*** standing in front of a building. Every time someone tries to go inside (access a property), the guard checks their credentials, logs their entry or perhaps stops them. The building itself is the target object. The Reflect API is like the building's default operation manual—when the guard decides to allow someone in, they follow the manual to open the door and let them proceed normally.

## ## Practice questions

### **\*\*Conceptual questions\*\***

1. What is the purpose of the handler object in a Proxy? List at least four traps and explain when they are invoked.

2. How does a proxy differ from a normal object when performing operations like property access, assignment or function invocation?
3. Why is the Reflect API useful inside proxy traps? What advantages does it provide over directly interacting with the target?
4. Give an example of how proxies can be used to implement data validation or default values.
5. Describe how reactive frameworks use proxies to detect changes in objects.

### **\*\*Coding exercises\*\***

1. Create a proxy for an object that logs any attempt to read or write its properties and prevents the deletion of any property.
2. Write a proxy that enforces that only string keys starting with an underscore (`\_`) can be set. All others should throw an error.
3. Implement a proxy for an array that returns `0` whenever an out-of-bounds index is read. Use Reflect to delegate all other operations.
4. Build a proxy that records the time of every method call on an object and stores it in an array called `calls` on the target.

# What is destructuring / aliasing and how is it useful?

## # Destructuring, Aliasing and Their Usefulness in JavaScript

Modern JavaScript includes syntax for **destructuring**—extracting values from arrays and objects into distinct variables. It helps unpack complex data structures into convenient local variables with concise syntax. **Aliasing** within destructuring lets you rename properties to avoid naming conflicts or to choose clearer variable names.

### ## Array destructuring

Array destructuring assigns variables based on the position of elements:

```
``js
const rgb = [255, 200, 100];
const [red, green, blue] = rgb;
```

```
console.log(red); // 255
console.log(green); // 200
console.log(blue); // 100
``
```

You can skip elements by leaving a blank space (`, `) and provide default values if the array is shorter than expected:

```
``js
const coords = [10];
const [x = 0, y = 0, z = 0] = coords;
// x = 10, y = 0, z = 0
``
```

The rest operator (`,...`) gathers the remaining elements:

```

``js
const numbers = [1, 2, 3, 4, 5];
const [first, ...rest] = numbers;
// first = 1, rest = [2, 3, 4, 5]
...

```

## ## Object destructuring

Object destructuring matches properties by name rather than position:

```

``js
const user = { id: 123, name: "Alice", age: 25 };
const { id, name } = user;
// id = 123, name = 'Alice'
...

```

Properties that don't exist produce `undefined`, but you can assign defaults:

```

``js
const { nickname = "Anon" } = user;
// nickname = 'Anon'
...

```

## ### Aliasing (renaming properties)

Sometimes a property name is not a valid identifier or conflicts with another variable in scope. You can assign it to a new variable with a different name:

```

``js
const person = { firstName: "Bob", "last-name": "Smith" };
const { firstName: first, "last-name": last } = person;

```



```
// first = 'Bob', last = 'Smith'
'''
```

Aliasing is also useful when destructuring within a function parameter:

```
```js  
function printUser({ name: fullName, age }) {  
  console.log(`${fullName} is ${age} years old.`);  
}  
  
printUser({ name: "Carol", age: 31 });  
// prints: Carol is 31 years old.  
'''
```

Here the parameter deconstructs the `name` property into a local variable `fullName` and extracts `age` directly.

Nested and mixed patterns

Destructuring can dig into nested objects and arrays:

```
```js  
const data = {
 user: {
 id: 42,
 preferences: {
 theme: "dark",
 languages: ["en", "es", "fr"],
 },
 },
};
```

```
const {
 user: {
 id: userId,
 preferences: {
 theme,
 languages: [primaryLang, ...otherLangs],
 },
 },
} = data;
```

```
// userId = 42
// theme = 'dark'
// primaryLang = 'en'
// otherLangs = ['es', 'fr']
...
```

## ## Use cases and benefits

- **Cleaner code** - Assign multiple variables in a single statement instead of writing repetitive property accesses.
- **Convenient defaults** - Specify fallback values when data might be incomplete.
- **Readable function signatures** - Destructure parameters to name only the needed properties and ignore the rest.
- **Aliasing** - Rename properties to avoid name conflicts, clarify meaning or match naming conventions.
- **Easier pattern matching** - Combine destructuring with loops or pattern matching to process data structures succinctly.

## ## Analogy

Think of destructuring as unpacking a gift basket. Instead of grabbing the whole basket and pulling out items one by one, you list what you need on the table: apples here, oranges there, and leftover

treats in a pile. Aliasing is like labeling the apples as "fruit" and the oranges as "citrus" to make their purpose clearer.

## ## Practice questions

### **\*\*Conceptual questions\*\***

1. How does array destructuring determine which variable receives which value? What happens if the array has fewer elements than variables?
2. Describe how object destructuring matches properties. What happens when a property is not present in the source object?
3. Why would you rename a destructured property? Give an example where aliasing improves clarity or avoids a conflict.
4. Explain how the rest operator (`...`) works in array and object destructuring.
5. Can you use destructuring in function parameters? How does this improve function readability?

### **\*\*Coding exercises\*\***

1. Given `const point = [3, 4, 5]`, use destructuring to assign `x`, `y` and `z` variables. Provide a default of `0` for any missing coordinate.
2. Write a function `swapFirstTwo(arr)` that takes an array and returns a new array where the first two elements are swapped. Use array destructuring.
3. Create a function `describeBook(book)` that takes an object with properties `{ title, author, year }` and logs a sentence. Use destructuring in the parameter list and rename `year` to `published`.
4. Destructure the following nested object to extract `theme`, the first language, and assign the rest of the languages to a variable called `others`: `{ settings: { theme: 'light', languages: ['en', 'de', 'jp'] } }`.

# What is module federation in modern JS apps?

## # Module Federation in Modern JavaScript Applications

As web applications become larger and teams more distributed, breaking your app into independently deployable pieces—often called **micro-frontends**—helps manage complexity. Sharing code between these pieces can be challenging. **Module Federation**, introduced in webpack 5, allows multiple builds to share code and load modules from each other at runtime.

### ## Core concepts

At its heart, module federation enables one application (the **host**) to consume modules exposed by another application (the **remote**). Both host and remote are separate builds with their own dependency graphs. Instead of bundling shared code into the host, the host dynamically loads modules from the remote when needed.

### ### Host application

The host defines which remote applications it depends on via the `ModuleFederationPlugin` in its webpack configuration:

```
``js
// webpack.config.js in host

const { ModuleFederationPlugin } = require("webpack").container;

module.exports = {
 plugins: [
 new ModuleFederationPlugin({
 name: "host",
 remotes: {
 app2: "app2@http://localhost:3002/remoteEntry.js",
 },
 shared: { react: { singleton: true }, "react-dom": { singleton: true } },
 }),
],
}
```

```
],
};
...
```

Here `app2` is a remote application available at the given URL. The `shared` section ensures both host and remote use the same instance of shared libraries like React.

### ### Remote application

The remote app exposes modules via its own `ModuleFederationPlugin` configuration:

```
```js  
// webpack.config.js in remote  
module.exports = {  
  plugins: [  
    new ModuleFederationPlugin({  
      name: "app2",  
      filename: "remoteEntry.js",  
      exposes: {  
        "./Button": "./src/components/Button.js",  
      },  
      shared: { react: { singleton: true }, "react-dom": { singleton: true } },  
    }),  
  ],  
};  
...
```

This configuration exposes a `Button` component located at `./src/components/Button.js`. The `filename` option (`remoteEntry.js`) is the file the host will load to discover the exposed modules.

Consuming a remote module

In the host application, import the remote module using a special syntax understood by webpack:

```
```.js
// In a React component inside the host
import React, { Suspense } from "react";

const RemoteButton = React.lazy(() => import("app2/Button"));

export default function Home() {
 return (
 <div>
 <h1>Welcome to the host app!</h1>
 <Suspense fallback={<div>Loading button...</div>}>
 <RemoteButton />
 </Suspense>
 </div>
);
}
...`
```

When the `RemoteButton` component is rendered, webpack fetches `remoteEntry.js` from the remote and loads the `Button` module on demand. Because both apps share React as a singleton, there are no version conflicts.

## ## Advantages of module federation

- **Independent deployment** - Remotes can be updated or deployed without redeploying the host. Apps can evolve at their own pace.
- **Code sharing** - Common libraries and components are shared rather than duplicated, reducing bundle sizes and improving consistency.
- **Runtime integration** - Modules are loaded when needed, enabling dynamic features or experiments without full rebuilds.

- **Micro-frontend architecture** - Teams can build and own separate parts of a larger application, coordinating via well-defined interfaces.

## ## Considerations and challenges

- **Version compatibility** - Shared dependencies must be compatible across host and remotes. Singleton configuration helps ensure only one version is used.
- **Complex setup** - Correctly configuring hosts, remotes and shared libraries requires careful planning. Tools like `@module-federation/nextjs-mf` or `@module-federation/vite` simplify integration with popular frameworks.
- **Network latency** - Loading remote modules introduces network requests. Use caching and prefetching to mitigate latency.
- **Security** - Loading code from another domain means you must trust the remote application. Implement proper content security policies and version controls.

## ## Analogy

Imagine your application as a city. Some districts are built by different teams (remote apps). Instead of duplicating common resources like power plants (libraries) in every district, the city builds shared infrastructure. When a district needs a new service (a module), it requests it from the central provider and plugs it in without reconstructing the whole district.

## ## Practice questions

### **Conceptual questions**

1. What problem does module federation solve in large applications? Describe the roles of the host and the remote.
2. In the webpack configuration for the host, what does the `remotes` property specify? What is the purpose of the `shared` property?
3. How does a remote application expose a module? What is the `filename` option used for?
4. Explain the advantages and potential challenges of using module federation.
5. How could module federation support a micro-frontend architecture?

### **Coding exercises**

1. Set up a minimal host and remote using webpack 5. Expose a component from the remote and import it in the host. Ensure that both share React as a singleton.
2. Modify the host so that it loads the remote component lazily using `React.lazy` and displays a fallback while loading.
3. Experiment with sharing a utility library (e.g., `Lodash`) between the host and remote. Observe how changing the version in one app affects the other.
4. Design a simple micro-frontend dashboard where each widget (e.g., weather, news, stock) is built as a separate remote. Use module federation to assemble the widgets in the host at runtime.



# Explain Virtual DOM and reconciliation in React conceptually (JS related)

## # Understanding the Virtual DOM and Reconciliation in React

React revolutionized UI development by abstracting direct DOM manipulation. Its secret weapon is the **Virtual DOM**—an in-memory representation of the real DOM—and an efficient **reconciliation** algorithm that updates the real DOM only when necessary.

### ## Why the Virtual DOM?

Manipulating the DOM is relatively slow. Every change triggers layout and paint operations, which can degrade performance when updates are frequent. React solves this by building a lightweight tree of elements (the virtual DOM) that mirrors the structure of the real DOM. When state or props change, React creates a **new virtual DOM** and compares it to the previous one. This diffing process determines the minimal set of changes needed to update the real DOM.

### ### How the virtual DOM works

1. **Initial render** - React constructs a tree of JavaScript objects representing the DOM structure and renders it to the real DOM.
2. **State/prop updates** - When application data changes, React builds a new virtual DOM tree reflecting those changes.
3. **Diffing (reconciliation)** - React compares the new tree with the previous tree. For each node, it determines whether to update an existing element, replace it, or leave it unchanged.
4. **Real DOM updates** - React batches the changes and applies them efficiently to the actual DOM. Only the nodes that changed are updated; unchanged parts are left alone.

By minimizing direct DOM operations and batching updates, React achieves significant performance gains, especially in complex interfaces.

### ## The reconciliation algorithm

React's reconciliation algorithm uses a set of heuristics to perform the diff efficiently in  $O(n)$  time:

- **Different element types lead to full replacement** - If the previous element is a `<div>` and the new element is a `<span>`, React discards the old subtree and mounts a new one.
- **Same type updates** - If the types match, React updates only the changed attributes (e.g., updating a `className` prop) and leaves the DOM node in place. Component state persists across renders when the component type stays the same.
- **Keys and lists** - When rendering lists of elements, assigning a `key` prop to each element helps React identify which items have changed, been added or removed. Using stable, unique keys prevents expensive reordering and preserves state.

### ### Example of list reconciliation

```
```jsx
function TodoList({ items }) {
  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>{item.text}</li>
      ))}
    </ul>
  );
}

// Using indices as keys can lead to incorrect updates when the list changes order.
```
```

By using `item.id` as the key, React can match each `<li>` across renders and update only the text for modified items.

### ## Benefits of the virtual DOM

- **Performance** - Updates are computed in memory and only applied to the DOM when necessary, reducing costly reflows and repaints.
- **Declarative style** - Developers describe what the UI should look like for a given state. React handles updating the DOM to match that state, freeing you from manual DOM manipulation.

- **Predictable updates** - React batches updates and applies them deterministically, which helps avoid inconsistent UI states.

## ## Analogy

Think of the virtual DOM as an architect's **blueprint** of a building. Before renovating, the architect revises the blueprint (new virtual DOM) and compares it to the current blueprint (previous virtual DOM). They then instruct the builders to modify only the walls or rooms that changed (real DOM updates). Without the blueprint, builders would wander the building, making unnecessary changes and causing chaos.

## ## Common pitfalls

- **Using indices as keys** - When rendering lists, using the array index as a key can cause incorrect component reuse and visual glitches when items are reordered or removed. Always use a stable identifier if possible.
- **Unnecessary wrapper elements** - Extra divs in the JSX tree create additional nodes in the virtual DOM. Use React fragments (`<> ... </>`) to avoid wrapping elements unnecessarily.
- **Large component trees** - Deeply nested structures can still result in many diff operations. Consider splitting components and memoizing where appropriate.

## ## Practice questions

### **Conceptual questions**

1. Explain the purpose of the virtual DOM. How does it improve performance compared to direct DOM manipulation?
2. Describe the steps React takes during reconciliation. What happens when two elements have different types?
3. Why are keys important when rendering lists? What problems arise when keys are not used or are non-unique?
4. How does React batch updates to the DOM? Why is batching beneficial?
5. Give an example where using the array index as a key causes an issue when updating a list.

### **Coding exercises**

1. Write a React component that renders a list of users. Demonstrate how adding and removing users affects the DOM when using proper keys vs. using the index.
2. Build a simple counter component and log each phase: initial render, state update, virtual DOM creation, diff and DOM update (use `useEffect` and console logs to illustrate the lifecycle).
3. Create a component that intentionally reorders items without keys and observe how React updates the DOM. Then fix it by adding unique keys.
4. Implement a small custom hook that memoizes a list component to avoid re-rendering when the list items haven't changed. Explain how memoization interacts with reconciliation.

# What is event-loop starvation?

## # What Is Event Loop Starvation?

JavaScript executes on a single thread. It uses an **event loop** to schedule and run different kinds of tasks: macro-tasks (e.g., `setTimeout`, I/O callbacks) and micro-tasks (e.g., `Promise` callbacks, `MutationObservers`). The event loop repeatedly takes a task from the macro-task queue, executes it until completion, then runs all micro-tasks queued during that execution before moving on.

**Starvation** occurs when some tasks never get a chance to run because the event loop is perpetually busy with other tasks.

## ## How starvation happens

Starvation usually stems from one of two issues:

1. **Long-running synchronous code** - Since JavaScript is single-threaded, synchronous functions block the event loop. If a function performs heavy computation without yielding control back to the loop, it prevents any pending tasks from executing.

2. **Flooding the micro-task queue** - Micro-tasks have higher priority than macro-tasks. After a macro-task finishes, the engine runs all micro-tasks before taking the next macro-task. If your code continually queues micro-tasks (for example, repeatedly calling `Promise.resolve().then(...)` in a loop), the engine may run micro-tasks indefinitely, causing macro-tasks like `setTimeout` callbacks to be delayed or never run at all.

## ### Example: micro-task starvation

```
``js
function scheduleMacroTask() {
 setTimeout(() => {
 console.log("Macrotask executed");
 }, 0);
}

function floodMicrotasks() {
```

```

for (let i = 0; i < 1e5; i++) {
 Promise.resolve().then(() => {
 // Simulate quick microtasks
 });
}

}

scheduleMacrotask();
floodMicrotasks();

// You might expect 'Macrotask executed' to log immediately,
// but the macrotask is delayed until all microtasks finish.
...

```

In this example, a single `setTimeout` callback (a macro-task) is scheduled, but a loop adds 100,000 micro-tasks. Because micro-tasks run before the next macro-task, the event loop spends a long time clearing the micro-task queue. The timeout callback doesn't run until all those micro-tasks complete.

### ### Example: synchronous blocking

```

```js
function longComputation() {
  // CPU-intensive loop that blocks the event loop
  const start = Date.now();
  while (Date.now() - start < 5000) {
    // Simulate work
  }
}

console.log("Start");
setTimeout(() => console.log("Timeout fired"), 0);
longComputation();

```

```
console.log("End");
```

```
// 'Timeout fired' logs only after longComputation completes (~5 seconds later).
```

```
...
```

Because `longComputation()` runs synchronously for 5 seconds, the event loop cannot process the `setTimeout` callback until it finishes. Any user interaction or UI updates also pause during this time, making the page appear frozen.

Avoiding starvation

- **Break up long tasks** - Split heavy computations into smaller chunks and schedule the next chunk with `setTimeout` or `setImmediate` (in Node.js). This yields control back to the event loop, allowing other tasks to run.
- **Use micro-tasks responsibly** - Avoid rapidly queuing micro-tasks in a tight loop. Instead, batch work or schedule some tasks as macro-tasks using `setTimeout` with a delay of 0.
- **Use `requestIdleCallback`** - For non-urgent work (analytics, logging), schedule tasks during idle periods. The browser calls `requestIdleCallback` when it's safe to run low-priority tasks without blocking critical rendering or input.
- **Web workers** - Offload CPU-intensive tasks to a Web Worker, which runs in its own thread and doesn't block the event loop.

Analogy

Imagine a single checkout lane at a grocery store. People in line (tasks) are served one at a time. Micro-tasks are like VIP customers that always cut to the front. If too many VIPs arrive, regular customers may never reach the counter. Likewise, if one customer brings a cart full of items and the cashier never pauses, everyone behind them waits. To keep the line moving, the cashier occasionally pauses long customers to serve others, or opens a new lane (web worker) for large orders.

Practice questions

Conceptual questions

1. Explain the difference between micro-tasks and macro-tasks in the event loop. Why can flooding the micro-task queue lead to starvation of macro-tasks?

2. Provide two examples of long-running synchronous code that could cause event loop starvation. How do they affect the user experience?
3. How does breaking a computation into smaller pieces and scheduling them with `setTimeout` or `requestIdleCallback` help prevent starvation?
4. Describe scenarios where using a Web Worker is preferable to running code on the main thread.

****Coding exercises****

1. Write a function that processes an array of 1 million items without blocking the UI. Break the work into chunks and use `setTimeout` to schedule each chunk.
2. Modify the `floodMicrotasks()` example so that it only queues micro-tasks in batches of 100, allowing `setTimeout` callbacks to run between batches.
3. Implement a progress bar that updates in real time while computing a large Fibonacci number. Use `requestIdleCallback` to schedule the computation and ensure the UI remains responsive.
4. Write a simple Web Worker that performs heavy computation (e.g., prime number generation) and communicates results back to the main thread. Demonstrate that the main UI does not freeze during the computation.

Explain call stack overflow and recursion depth limits

Understanding the Call Stack, Recursion Depth Limits and Stack Overflow in JavaScript

JavaScript executes code in a **single thread** using a call stack, which is essentially a stack data structure used to track function calls. Each time a function is invoked, an **execution context**—containing its parameters, local variables and the location where the function should return—is pushed onto this stack. When the function finishes, its context is popped off and execution returns to the caller. This push-and-pop sequence continues as your program runs.

The call stack and recursion

Recursion is a technique where a function calls itself (directly or indirectly) to solve a problem by breaking it down into smaller pieces. Each recursive call pushes another execution context onto the call stack. As long as there is a **base case**—a condition that stops the recursion—the stack will eventually unwind and return control back down the chain of calls.

If a recursive function either lacks a proper base case or calls itself too deeply, the call stack grows until the JavaScript engine cannot allocate any more stack frames. At that point, a **stack overflow error** occurs (for example, `RangeError: Maximum call stack size exceeded`). Different browsers and environments have different stack size limits; in many engines the maximum safe recursion depth is in the range of a few thousand calls, so relying on very deep recursion can be fragile.

Example: factorial with safe recursion

```
```\njs\n// A safe recursive implementation using a clear base case\nfunction factorial(n) {\n  if (n < 0) throw new Error("Negative values are not allowed");\n  if (n === 0 || n === 1) return 1; // base case\n  return n * factorial(n - 1); // recursive case\n}\n\nconsole.log(factorial(5)); // 120\n...`
```

This `factorial` function stops when `n` reaches 0 or 1, preventing unbounded growth of the call stack. If you omit the base case or inadvertently call the function with a negative value, the recursion never ends and a stack overflow occurs.

### ### Demonstrating stack overflow

```
```js
function endless(n) {
  return endless(n + 1); // no base case - runs until the stack overflows
}

try {
  endless(0);
} catch (e) {
  console.error("Stack overflow:", e.message);
}
```
```

Depending on your environment, this code will throw a `RangeError` after several thousand recursive calls. It shows how unbounded recursion can exhaust the call stack.

### ## Why recursion depth is limited

JavaScript engines implement the call stack with a fixed amount of memory. Each new call pushes an execution context, so a deep recursive algorithm can exhaust this memory quickly. Some languages support **tail call optimization**, a technique that reuses stack frames for certain recursive patterns. JavaScript's specification allows tail call optimization in strict mode, but most engines don't implement it yet, so you shouldn't assume it will save stack space.

For algorithms that require many iterations, consider converting recursive logic to an **iterative approach** using loops or explicit stacks. Iterative solutions often avoid the risk of stack overflow and can be easier to reason about when working with large datasets.

## ## Real-world analogy

Imagine a stack of plates at a buffet: you can take a plate off the top or add one on top, but you never remove or insert plates in the middle. Each function call is like placing a new plate on the stack. If you keep adding plates without ever removing them, eventually the stack becomes unstable and collapses—that's the stack overflow. To prevent it, you must stop placing plates once you reach a certain height (the base case) or remove plates as you go (using iteration).

## ## Common pitfalls and misconceptions

- **\*\*Infinite recursion is not the only cause of stack overflow.\*\*** A finite recursion that simply goes too deep will also overflow the stack. Always design your recursion to stop early when possible.
- **\*\*Tail recursion isn't automatically optimized in JavaScript.\*\*** Even in strict mode, most engines do not implement tail call optimization, so writing a tail-recursive function will not necessarily prevent stack overflow.
- **\*\*Using global variables for recursion counters can hide problems.\*\*** It's better to pass state through function parameters or use local variables so you don't inadvertently depend on external mutable state.
- **\*\*Stack overflow is not just about recursion.\*\*** A long chain of synchronous function calls (even without recursion) can also exceed the stack limit if it nests too deeply.

## ## Practice questions

1. **\*\*Theory:\*\*** Explain the call stack in your own words. What happens to the stack when a function calls another function? How does it differ when a function returns?
2. **\*\*Theory:\*\*** Why does a `RangeError: Maximum call stack size exceeded` occur? Describe two ways to prevent it when using recursion.
3. **\*\*Coding:\*\*** Write a recursive function that sums the elements of a nested array (arrays can contain numbers or other arrays) but uses an explicit stack (an array) internally instead of relying on the call stack. This prevents stack overflow on very deeply nested arrays.
4. **\*\*Coding:\*\*** Convert the following recursive Fibonacci function into an iterative version that uses a loop and avoids recursion:

```
``js
function fibonacci(n) {
 if (n <= 1) return n;
```

```
 return fibonacci(n - 1) + fibonacci(n - 2);
 }
 ...
```

5. **Theory:** What is tail call optimization? Why isn't it currently relied upon in production JavaScript code?

These questions and examples will help you deepen your understanding of how the call stack works, why recursion must be designed carefully, and how to avoid stack overflow errors in real projects.

# What are tagged template literals?

## # Tagged Template Literals in JavaScript

Template literals, enclosed by backticks (`), allow embedded expressions (`\${...}`) and multi-line strings. **\*\*Tagged template literals\*\*** build upon this by passing the literal's parts to a **\*\*tag function\*\***. This tag can interpret the string in any way it likes—formatting, escaping, localization, or even generating custom data structures.

## ## How tagged templates work

When you write a tagged template literal like ``tag`Hello, ${name}!``, JavaScript translates it into a call to ``tag`()`. The tag function receives:

1. **\*\*An array of strings\*\*** containing the literal text segments (everything outside `\${...}` placeholders). This array is frozen and reused for repeated calls with the same literal, so you can cache results for performance.
2. **\*\*Substitution values\*\***—one argument for each `\${...}` expression in order of appearance.

You are free to return any value from the tag function, not just a string. This makes tagged templates very flexible.

## ### Example: simple formatting

```
``js
function highlight(strings, ...values) {
 // `strings` is an array of literal segments
 // `values` holds the results of each expression
 return strings.reduce((result, str, i) => {
 const value =
 values[i] !== undefined ? `${values[i]}</strong>` : "";
 return result + str + value;
 }, "");
}
```

```
const user = "Alice";
const age = 30;
const html = highlight`Name: ${user}, Age: ${age}`;
console.log(html); // Name: Alice, Age: 30
'''
```

Here, the `highlight` function wraps each interpolated value in `<strong>` tags. The tag gets the literal pieces (`"Name: ", ", Age: "`) in the `strings` array and the values (`user`, `age`) in the `values` array. By interleaving them, it produces a formatted string.

### ### Accessing raw strings

By default, escape sequences like `\n` are interpreted in template literals. If you need the raw text, the `strings` array has a `raw` property containing the unprocessed versions. This is useful for writing custom parsers that need to interpret backslashes or special characters exactly as written.

```
```js
function showRaw(strings) {
  console.log(strings.raw[0]);
}

showRaw`Line one\nLine two`; // logs "Line one\nLine two"
'''
```

Practical uses

- **Custom DSLs and domain-specific parsing:** Tagged templates are often used to implement mini-languages within JavaScript. Libraries like GraphQL or styled-components use tags to parse structured strings and generate queries or CSS at build time.
- **Localization and internationalization:** A tag function can look up translation keys for strings and substitute variables based on locale settings.
- **Escaping untrusted input:** You can create a safe HTML tag that escapes user input to prevent cross-site scripting attacks:

```

```js
function escapeHTML(strings, ...values) {
 const escape = (str) =>
 String(str)
 .replace(/&/g, "&")
 .replace(/</g, "<")
 .replace(/>/g, ">")
 .replace(/"/g, """)
 .replace(/'/g, "'");
 return strings.reduce(
 (result, str, i) => result + str + escape(values[i] ?? ""),
 ""
);
}

const userInput = '<script>alert("hi");</script>';
const safeHtml = escapeHTML`User says: ${userInput}`;
console.log(safeHtml);

// Output: User says: <script>alert("hi");</script>
...

```

- **\*\*Currying and performance optimizations:\*\*** Since the strings array is immutable and reused, tag functions can cache results for identical literal patterns, improving performance when parsing complex templates repeatedly.

### ## Common misconceptions

- **\*\*Not only for strings.\*\*** A tag function can return any JavaScript value—an object, array, DOM element, or even a promise. Some libraries use tags to build complex queries or React components.

- **\*\*Template parts are not concatenated automatically.\*\*** The tag must handle the `strings` and `values` arrays explicitly. Forgetting to join them will result in unexpected output.

- **\*\*Tags don't modify the template literal syntax.\*\*** The syntax inside ``${...}`` expressions is normal JavaScript. The tag only sees the evaluated values, not the expressions themselves.

## ## Real-world analogy

Think of a mail merge program that uses templates with placeholders like ``Dear {{name}}``. The program reads the template, pulls out the fixed text ("Dear ") and the variable parts (e.g., the recipient's name), then combines them to produce a personalised letter. A tagged template literal works similarly: the tag function is the mail merge program, and the literal is the template.

## ## Practice questions

1. **\*\*Theory:\*\*** What arguments are passed to a tag function when evaluating a tagged template literal? How is the ``raw`` property used?
2. **\*\*Theory:\*\*** Explain how tagged templates can help prevent cross-site scripting when inserting user-supplied content into HTML.
3. **\*\*Coding:\*\*** Write a tag function called ``formatCurrency`` that takes a template like ``formatCurrency`Total: ${amount}`` and returns a string that formats the ``amount`` as US dollars using ``Intl.NumberFormat``.
4. **\*\*Coding:\*\*** Create a tag function that builds an array of objects from a comma-separated list: ``toPairs`a=1,b=2,c=3`` should return ``[{ key: 'a', value: '1' }, ...]``.
5. **\*\*Theory:\*\*** How does a tagged template differ from simply calling a function with a string argument? What advantages does the tagged template syntax provide?

These questions encourage you to explore both the mechanics and creative uses of tagged template literals.



# What is lazy evaluation in JS?

## # Understanding Lazy Evaluation in JavaScript

**\*\*Lazy evaluation\*\*** (or call-by-need) is an evaluation strategy in which expressions are not computed until their values are actually required. This contrasts with JavaScript's default **\*\*eager evaluation\*\***, where expressions are evaluated as soon as they're encountered. Lazy evaluation can improve performance by avoiding unnecessary work and supports patterns like infinite data structures.

While JavaScript does not implement general lazy evaluation for all expressions, it provides mechanisms that behave lazily in specific contexts. You can also build your own lazy constructs using functions, generators and proxies.

## ## Built-in lazy behaviour in JavaScript

JavaScript already employs lazy evaluation in a few operators and features:

### ### Short-circuiting logical operators

The logical AND (`&&`) and OR (`||`) operators evaluate operands from left to right and stop as soon as the result is determined. For example, in ``a && b``, ``b`` is evaluated only if ``a`` is truthy. Similarly, in ``a || b``, ``b`` is evaluated only if ``a`` is falsy. This allows you to use these operators to conditionally execute code:

```
``js
function doExpensiveWork() {
 console.log("Doing expensive work");
 return true;
}

let flag = false;

// doExpensiveWork() runs because flag is false
flag || doExpensiveWork();

// doExpensiveWork() does not run because flag is truthy
```

```
flag = true;

flag || doExpensiveWork();

...


```

### ### Property getters

ES5 introduced getters, which are functions associated with object properties. A getter is executed lazily—the computation happens only when the property is accessed, not when the object is created:

```
``js
const user = {
 firstName: "Alice",
 lastName: "Smith",
 get fullName() {
 console.log("fullName computed");
 return `${this.firstName} ${this.lastName}`;
 },
};

// fullName is not computed here
console.log("User created");

// fullName is computed only when accessed
console.log(user.fullName);

...


```

### ### Generators and iterators

Generator functions (``function*``) produce values on demand. When you call a generator, it returns an iterator. Each call to ``next()`` resumes execution until the next ``yield`` statement, returning a new value. This allows you to represent large or even infinite sequences without precomputing them.

```
``js


```

```
function* naturalNumbers() {
 let i = 1;
 while (true) {
 yield i++;
 }
}

const numbers = naturalNumbers();
console.log(numbers.next().value); // 1
console.log(numbers.next().value); // 2
// numbers.next() can be called indefinitely
...
```

Generators enable lazily building pipelines of operations with the `for...of` loop or the spread operator. Since values are produced one at a time, memory usage stays low even when iterating over large sequences.

### ## Creating custom lazy evaluations

You can simulate laziness using higher-order functions. A common pattern is to wrap a computation in a function and call it only when needed:

```
```js
function lazy(fn) {
  let evaluated = false;
  let result;
  return () => {
    if (!evaluated) {
      result = fn();
      evaluated = true;
    }
    return result;
  }
}
```

```

    };
}

const lazyValue = lazy(() => {
  console.log("Computing...");
  return Math.random();
});

// Nothing logged yet
const value1 = lazyValue(); // logs 'Computing...'
const value2 = lazyValue(); // returns cached result without logging again
...

```

This pattern, known as **memoization**, caches the result after the first computation. Combined with closures, it defers the expensive work until the value is requested.

Another approach is to use `Proxy` to intercept property access and compute values lazily. However, proxies add complexity and should be used judiciously.

Benefits and trade-offs

- **Performance:** By delaying computations until needed, lazy evaluation can avoid unnecessary work, which is helpful when dealing with large data structures or expensive operations.
- **Infinite data structures:** Generators allow you to model potentially infinite sequences (like the Fibonacci numbers) without running out of memory.
- **Control flow:** Lazy evaluation makes it easy to implement custom control flow constructs, such as conditional evaluation in template engines or domain-specific languages.

However, laziness also has costs:

- **Debugging complexity:** Deferred computations can make it harder to trace when and where values are computed.

- **Memory retention:** If cached values capture large objects, they may remain in memory longer than expected. WeakMaps or manual cache invalidation can help mitigate this.
- **Non-uniform language support:** JavaScript is eager by default. Lazy patterns are implemented manually and require discipline to maintain.

Real-world analogy

Consider ordering a made-to-order sandwich: the ingredients aren't assembled until you place the order. If you never order, the sandwich is never prepared, saving time and ingredients. Lazy evaluation works the same way—computation happens only when you need the result.

Practice questions

1. **Theory:** Explain the difference between eager and lazy evaluation. Provide examples of each in JavaScript.
2. **Theory:** How do the logical operators `&&` and `||` demonstrate lazy behaviour? What would happen if JavaScript evaluated both operands eagerly?
3. **Coding:** Implement a generator function that yields the squares of natural numbers (1, 4, 9, ...) indefinitely. Then write a loop that prints the first 10 squares.
4. **Coding:** Create a function `lazySum(...nums)` that returns a thunk (a parameterless function). When the thunk is called, it calculates and returns the sum of `nums`, logging "Calculating" only the first time it runs.
5. **Theory:** Discuss the pros and cons of using lazy evaluation in a web application. When might laziness introduce problems instead of solving them?

These exercises will help you understand how to implement and reason about laziness in a language that is eager by default.

How does JavaScript handle memory leaks?

How JavaScript Handles Memory and Avoids Memory Leaks

JavaScript is a garbage-collected language. You don't explicitly free memory; instead, the engine automatically reclaims memory that is no longer needed. Understanding how garbage collection works and how leaks can occur will help you write more efficient, leak-free code.

Memory management basics

A JavaScript program allocates memory in several phases:

1. **Allocation:** When variables and objects are created, the engine reserves space in memory.
2. **Use:** The program reads and writes to these objects as needed.
3. **Release (garbage collection):** When objects become unreachable—there is no way for running code to access them—the garbage collector frees the memory so it can be reused.

Modern engines use the **mark-and-sweep** algorithm. Starting from "roots" such as global variables, the call stack, and closure scopes, the collector marks all reachable objects. Anything not marked is considered unreachable and is collected. Cyclic references (objects referencing each other) are not a problem because an unreachable cycle is still unmarked and will be reclaimed.

What is a memory leak?

A **memory leak** occurs when the program retains references to objects that are no longer needed, preventing the garbage collector from reclaiming their memory. Over time, these unused objects accumulate, increasing memory consumption and potentially degrading performance.

Common sources of memory leaks

- **Global variables:** Accidentally creating global variables (by omitting `let`, `const` or `var`) or attaching data to the `window` object keeps them alive for the life of the page. Avoid polluting the global scope.

- **Closures holding onto data:** Functions that capture variables from outer scopes can keep those variables alive even when they're no longer needed. For example, storing a large object in a closure used by an event handler may leak memory if the handler remains attached after the object is irrelevant.
- **Uncleared timers and intervals:** Functions passed to `setInterval()` or `setTimeout()` maintain references to their environments. If you never call `clearInterval()` or `clearTimeout()`, the callback (and everything it references) remains in memory.
- **Detached DOM nodes:** Removing DOM elements from the document tree doesn't automatically free associated memory if you still hold references to them in JavaScript. Keeping old elements in arrays or caches without releasing them prevents garbage collection.
- **Accidental caches:** Libraries or your own code may cache data (e.g., API responses, computed values) but never remove stale entries. Unbounded caches grow over time. Use size limits or time-to-live policies.
- **Closures in loops:** Creating functions inside loops that capture loop variables can unintentionally hold onto large structures if the closures persist longer than necessary.

Techniques to avoid leaks

- **Limit scope and use `const`/`let`:** Declare variables with block scope so they don't leak into the global object. Minimise the lifetime of variables.
- **Remove event listeners:** If you attach an event listener to an element or the window, detach it when it's no longer needed. Many modern frameworks automatically handle this for you, but vanilla JavaScript requires manual cleanup.
- **Clear timers:** Always pair `setInterval()` with `clearInterval()` and `setTimeout()` with `clearTimeout()` when the task is complete or when the component is destroyed.
- **Null out references:** When you're done with an object (especially large arrays or DOM elements), set variables referencing it to `null` or remove them from arrays. This makes it easier for the garbage collector to determine that the object is unreachable.

- **Use weak collections:** `WeakMap` and `WeakSet` hold **weak references** to their keys. If no other references exist, the garbage collector can reclaim the key object and its associated value automatically. Weak collections are ideal for memoizing data keyed by DOM elements or other objects without preventing garbage collection.

- **Monitor memory usage:** Modern browsers provide memory profiling tools (e.g., Chrome DevTools' Memory panel) that help detect leaks by showing detached nodes and snapshots of heap usage over time.

Real-world analogy

Imagine your program is like a house with rooms (scopes) holding objects. When you no longer need an item, you should remove it from all rooms; otherwise, it will clutter the house forever. The garbage collector is like a cleaning service that checks which objects are reachable (in rooms that someone can enter) and removes objects left in inaccessible spaces. If you forget to take objects out of accessible rooms, the cleaning service won't touch them—leading to clutter (memory leaks).

Practice questions

1. **Theory:** Describe the mark-and-sweep algorithm in your own words. Why are cyclic references not inherently problematic in modern JavaScript engines?
2. **Theory:** List three common sources of memory leaks in JavaScript applications and explain how to prevent each.
3. **Coding:** Write a function that attaches an event listener to a button and updates a counter. Modify your function to remove the listener when the button is removed from the DOM, ensuring no leak occurs.
4. **Coding:** Create a cache using `Map` that stores results of an expensive computation. Add a method to clear the cache after 10 entries to avoid unbounded memory growth.
5. **Theory:** Explain the difference between `Map` and `WeakMap` in the context of garbage collection. When would you choose one over the other?

By understanding how the garbage collector works and being mindful about references, you can avoid memory leaks and keep your applications fast and efficient.

Explain hoisting with function expressions vs arrow functions

Hoisting and Function Types: Declarations vs. Expressions vs. Arrow Functions

JavaScript's **hoisting** behaviour can be confusing. It refers to the way variable and function declarations are moved to the top of their containing scope during compilation. Understanding how hoisting applies to different kinds of functions—function declarations, function expressions and arrow functions—will help you avoid unexpected `TypeError` or `ReferenceError` messages.

Function declarations are hoisted

A **function declaration** looks like this:

```
``js
console.log(square(5)); // Works because `square` is hoisted

function square(n) {
  return n * n;
}
``
```

Function declarations are fully hoisted. Both the function's name and its body are moved to the top of the current scope. You can call the function before its declaration appears in the code.

Function expressions are not hoisted in the same way

A **function expression** assigns a function to a variable:

```
``js
console.log(cube); // logs undefined

// console.log(cube(2)); // would throw TypeError: cube is not a function
``
```

```

var cube = function (n) {
    return n * n * n;
};

console.log(cube(2)); // 8

```

Variables declared with `var` are hoisted but initialised with `undefined` until their assignment is reached. Therefore, calling `cube(2)` before assignment fails: the variable exists, but holds `undefined`, which is not callable.

Using `let` or `const` for function expressions tightens the rules further. Variables declared with `let`/`const` are placed in a **temporal dead zone**—a period between entering the scope and the declaration line where they exist but cannot be accessed. Accessing them early throws a `ReferenceError`:

```

// js
// console.log(area(3)); // ReferenceError: Cannot access 'area' before initialization

const area = function (r) {
    return Math.PI * r * r;
};

console.log(area(3)); // Works after initialization

```

Arrow functions behave like function expressions

An **arrow function** is always an expression; there is no such thing as an "arrow function declaration." Arrow functions are assigned to variables, which means they follow the same hoisting rules as other variable assignments. You cannot invoke an arrow function before its definition:

```

// js

```

```
// greet(); // ReferenceError or TypeError depending on declaration
```

```
const greet = (name) => {  
  console.log(`Hello, ${name}!`);  
};
```

```
greet("Alice"); // Works
```

```
...
```

If you use `var` instead of `const`, `greet` is hoisted as `undefined`. Calling it before assignment will throw a `TypeError` because you're trying to call `undefined` as a function. With `let` or `const`, accessing `greet` before the declaration triggers a `ReferenceError` due to the temporal dead zone.

Additional differences beyond hoisting

While hoisting is about when a function becomes available, function expressions and arrow functions also differ in behaviour:

- **`this` binding:** Arrow functions do not have their own `this`. They capture `this` from the surrounding lexical context. Function declarations and expressions get their own `this` depending on how they're called. This difference makes arrow functions unsuitable as methods on objects that rely on dynamic `this`.
- **`arguments` object:** Arrow functions do not have an `arguments` object; you must use rest parameters (`...args`) to access arguments. Regular functions have their own `arguments` object.
- **Constructors:** Arrow functions cannot be used as constructors. Calling them with `new` throws a `TypeError`. Traditional function declarations and expressions can be invoked as constructors if designed for that purpose.

Real-world analogy

Think of hoisting like a stage play. Script lines (declarations) are pinned to the top of the script before the actors begin. The actors can deliver lines (call functions) right away if the lines were pinned (function declarations). However, if a line is written on a cue card handed out later (function expression), the actor doesn't know what to say until the cue card arrives. Arrow functions are always on cue cards; the actor can't read them until they're handed over.

Common pitfalls

- **Assuming all functions are hoisted.** Only declarations are. Expressions and arrow functions need to be defined before use.
- **Mixing `var`, `let` and `const`.** Understand that `var` declarations hoist and initialise to `undefined`, while `let` and `const` create a temporal dead zone.
- **Using arrow functions as methods.** Arrow functions capture `this` lexically; using them as object methods can lead to unexpected `this` values.

Practice questions

1. **Theory:** Explain the difference between hoisting of function declarations and variables. How does the temporal dead zone apply to functions assigned to `let` or `const` variables?
2. **Coding:** Predict the output of the following code and explain why:

```
```js
show();

var show = () => console.log("arrow");

function show() {
 console.log("declaration");
}

show();
```
```

3. **Coding:** Rewrite the following function declaration as an arrow function. How would you call it to avoid hoisting pitfalls?

```
```js
function multiply(a, b) {
 return a * b;
}
```
```

4. **Theory:** Describe the differences in `this` and `arguments` handling between arrow functions and traditional functions. Give examples where choosing one over the other makes a difference.
5. **Coding:** Write a function that uses an arrow function inside a method. Demonstrate how the arrow captures `this` from its enclosing scope.

These exercises will solidify your understanding of hoisting and function types, helping you avoid common runtime errors.

What is the Temporal API (upcoming JS proposal)?

The Temporal API - A Modern Date and Time API for JavaScript

Dates and times are notoriously tricky. JavaScript's built-in `Date` object has long been criticised for confusing behaviour, limited time-zone support and awkward APIs. To address these shortcomings, TC39 (the committee that evolves JavaScript) has proposed a new **Temporal** API. Though still experimental in many environments, Temporal aims to provide a robust, modern way to work with dates, times, durations and time zones.

Why Temporal was created

The original `Date` object was modelled on Java's `java.util.Date`, which was later superseded in Java because of its design flaws. `Date` combines a timestamp and calendar fields into a single mutable object, uses local time by default, lacks full time-zone support, and has inconsistent parsing and arithmetic methods. Temporal's design addresses these issues by:

- **Separating concepts**: Temporal introduces distinct classes for instants (`Temporal.Instant`), time-zone-aware date-times (`Temporal.ZonedDateTime`), plain date-times without a time zone (`Temporal.PlainDateTime`), dates, times, years/months, durations and more. Separating these concepts reduces confusion and makes APIs more explicit.
- **Immutability**: Temporal objects are immutable. Operations like `add()` or `with()` return new objects instead of modifying the original, avoiding side effects.
- **Built-in time-zone and calendar support**: Temporal can represent dates in any IANA time zone and supports different calendar systems. It handles daylight saving transitions and leap seconds correctly.
- **Consistent parsing and arithmetic**: Temporal provides methods like `from()`, `toString()`, `add()` and `subtract()` that behave predictably, allowing easy conversion between types and reliable date arithmetic.

Overview of key Temporal classes

- **Temporal.Now** provides static methods that return current date and time values in different forms. For example, `Temporal.Now.plainDateISO()` gives today's ISO date in the local time zone, and `Temporal.Now.zonedDateTimeISO()` returns a zoned date-time with the device's time zone.
- **Temporal.Instant** represents a single point on the timeline (like a UNIX timestamp) with nanosecond precision. It is time-zone agnostic and can be converted to other Temporal types.

- **`Temporal.ZonedDateTime`** pairs a calendar date-time with a specific time zone. It's useful when you need an exact moment plus a zone—for example, scheduling meetings across time zones or converting times for travel itineraries.

- **`Temporal.PlainDate`**, **`Temporal.PlainTime`** and **`Temporal.PlainDateTime`** represent dates and times without a time zone. They model concepts like "Christmas Day" or "8:00 AM" without reference to a particular offset from UTC, which is essential for recurring events.

- **`Temporal.Duration`** represents the difference between two temporal values in terms of years, months, days, hours, minutes, seconds and smaller units. Duration arithmetic is precise and respects calendar rules.

Other classes handle specific combinations (like `Temporal.PlainYearMonth` or `Temporal.PlainMonthDay`) and there are conversion helpers like `Temporal.Instant.fromEpochMilliseconds()`.

Basic usage examples

```
```js
// Getting the current date in ISO format
const today = Temporal.Now.plainDateISO();
console.log(today.toString()); // e.g., '2025-11-06'

// Creating and manipulating a ZonedDateTime
const meeting = Temporal.ZonedDateTime.from({
 year: 2025,
 month: 12,
 day: 15,
 hour: 9,
 minute: 30,
 timeZone: "America/New_York",
});
const newTime = meeting.add({ hours: 2 });
```

```
console.log(newTime.toString()); // adds two hours without modifying `meeting`
```

```
// Converting between types
```

```
const instant = Temporal.Instant.fromEpochMilliseconds(Date.now());
```

```
const zoned = instant.toZonedDateTimeISO("Europe/Paris");
```

```
console.log(zoned.toString());
```

```
// Working with durations
```

```
const duration = Temporal.Duration.from({ days: 2, hours: 5 });
```

```
const later = today.add(duration);
```

```
console.log(later.toString());
```

```
...
```

These examples illustrate Temporal's clarity: you explicitly specify time zones and units, and methods return new immutable objects.

### ## How Temporal differs from Date

- **Immutability vs. mutability:** `Date` objects are mutable (methods like `setHours()` modify the original). Temporal objects are immutable; methods like `add()` return new values.
- **Time-zone awareness:** `Date` stores a timestamp but defaults to the local time zone when converting to strings. Temporal separates zone-aware and zone-agnostic types, making conversions explicit.
- **Precision:** Temporal offers nanosecond precision and robust arithmetic. `Date` is limited to milliseconds and has quirks like months being zero-indexed.
- **Parsing:** Temporal's `from()` methods accept object literals with named fields, removing ambiguities of date string parsing.
- **Calendar systems:** Temporal supports non-Gregorian calendars, accommodating internationalisation needs.

### ## Availability and status

As of 2025, the Temporal API is still experimental. It is implemented in some environments but not available in all browsers. You can test Temporal in supported engines or via polyfills, but check



compatibility before using it in production. When the API is widely supported, it is likely to become the recommended way to handle date and time in JavaScript.

### ## Real-world analogy

Consider scheduling flights. The original `Date` API is like using a single clock that only shows your local time. When you travel across time zones, you constantly need to recalculate departure and arrival times. The Temporal API is like having a world clock and a calendar: you can specify the departure time, time zone and calendar clearly, perform accurate arithmetic and convert to local time zones without confusion.

### ## Practice questions

1. **Theory:** What problems with the existing `Date` object motivated the creation of the Temporal API? List at least three issues.
2. **Theory:** Describe the differences between `Temporal.Instant`, `Temporal.ZonedDateTime` and `Temporal.PlainDateTime`. When would you use each?
3. **Coding:** Create a `Temporal.ZonedDateTime` for July 1, 2025 at 15:00 in the `Asia/Tokyo` time zone. Convert it to New York time (`America/New_York`) and output the resulting date-time.
4. **Coding:** Write a function that takes two `Temporal.PlainDate` objects and returns the number of days between them using a `Temporal.Duration`.
5. **Theory:** Explain how immutability in Temporal objects helps prevent bugs. Give an example where using a mutable `Date` object could lead to unexpected behaviour.

These questions encourage you to explore the Temporal API and understand how it addresses the shortcomings of the old `Date` API.

# What is requestAnimationFrame and when to use it?

## # `requestAnimationFrame`: Scheduling Efficient Animations in the Browser

Modern web applications often require smooth animations—moving elements, scrolling lists, or game loops. Using `setTimeout()` or `setInterval()` to drive animations can cause janky motion and wasted CPU cycles because they run independently of the browser's rendering schedule. The `requestAnimationFrame()` API provides a better way to schedule visual updates.

### ## What does `requestAnimationFrame()` do?

`requestAnimationFrame(callback)` tells the browser that you want to perform an animation. The browser calls your `callback` just **before** the next repaint. The callback receives a **high-resolution timestamp** indicating when the repaint is scheduled. After executing your callback, the browser paints the updated frame. Because `requestAnimationFrame()` is aligned with the display's refresh rate (often 60 frames per second), it produces smoother animations and reduces unnecessary work.

Key characteristics:

- **Runs before repaint:** The callback is invoked just before the browser paints a new frame, so any DOM changes are drawn immediately. This reduces the risk of layout thrashing and ensures your updates are visible on the next screen refresh.
- **Automatic throttling:** When the page is in a background tab or the browser is minimised, `requestAnimationFrame()` callbacks are paused, conserving CPU and battery. Using `setInterval()` continues to run timers even when the page is hidden.
- **Self-rescheduling:** `requestAnimationFrame()` calls are one-shot. If you want continuous animation, you must call `requestAnimationFrame()` again from within your callback.
- **Provides timing information:** The callback receives a timestamp parameter (similar to `performance.now()`) that you can use to calculate elapsed time and animate at consistent speed.

### ## Basic example: animating a moving box

```
```html
<style>
#box {
  position: relative;
```

```

width: 50px;
height: 50px;
background: coral;
}
</style>
<div id="box"></div>
<script>
const box = document.getElementById("box");
let startTime;
function move(timestamp) {
  if (!startTime) startTime = timestamp;
  const elapsed = timestamp - startTime;
  // move 100 pixels per second
  const distance = Math.min(elapsed * 0.1, 500);
  box.style.transform = `translateX(${distance}px)`;
  if (distance < 500) {
    requestAnimationFrame(move);
  }
}
requestAnimationFrame(move);
</script>
...

```

In this example, a box moves smoothly to the right by 500 px at 100 px per second. Each frame calculates how much time has passed and moves the box accordingly. When the box reaches 500 px, the animation stops by not scheduling another frame.

When should you use `requestAnimationFrame()`?

- **Animations and game loops:** Whenever you are updating visual properties (position, opacity, scale) or drawing to a ``<canvas>``, `requestAnimationFrame()` ensures your updates sync with the display.

- ****Smooth scroll or parallax effects:**** Use it to update scroll positions or CSS transforms during user interactions for the smoothest performance.
- ****Throttling expensive tasks:**** You can wrap layout-intensive code inside ``requestAnimationFrame()`` to ensure it runs at most once per frame, preventing layout thrashing from multiple DOM reads/writes in quick succession.

Canceling scheduled frames

``requestAnimationFrame()`` returns an integer ID. If you need to stop a scheduled callback (for example, when a component unmounts), call ``cancelAnimationFrame(id)``. Cancelling prevents the callback from running if it hasn't been executed yet.

```
```js
const id = requestAnimationFrame(myCallback);

// later ...

cancelAnimationFrame(id);
```
```

Real-world analogy

Imagine painting a flipbook: rather than drawing arbitrarily at random times, you wait for the moment right before turning to the next page to draw the next frame. This coordination ensures each frame appears exactly when the viewer flips the page. ``requestAnimationFrame()`` provides similar coordination with the browser's rendering loop.

Common misconceptions

- ****It doesn't guarantee 60 fps on its own.**** ``requestAnimationFrame()`` only schedules callbacks before repaints. If your callback performs expensive work, it can still cause janky frames. Keep your callback work lightweight or offload heavy tasks to Web Workers.
- ****It doesn't run in Node.js environments.**** ``requestAnimationFrame()`` is part of the browser APIs, though environments like Electron and Deno may provide polyfills.
- ****You still need to call it repeatedly for continuous animations.**** Unlike ``setInterval()``, ``requestAnimationFrame()`` doesn't loop automatically. Always invoke it again inside your callback if you want the next frame.

Practice questions

1. **Theory:** What advantages does `requestAnimationFrame()` have over `setInterval()` for animations? Discuss throttling and timing alignment.
2. **Coding:** Write a function that smoothly fades out an element over two seconds using `requestAnimationFrame()`. The element's opacity should decrease linearly from 1 to 0.
3. **Coding:** Create a basic game loop using `requestAnimationFrame()` that updates a character's position and redraws a canvas. How would you pause and resume the loop?
4. **Theory:** Why is it important to cancel scheduled animation frames when a component is removed from the DOM or a page is hidden? What problems might arise if you forget to do so?
5. **Theory:** Explain how the timestamp parameter passed to a `requestAnimationFrame()` callback can be used to create frame-rate independent animations.

Use these questions to explore efficient animation patterns and to practice writing smooth, efficient browser animations.

Explain IntersectionObserver and MutationObserver APIs

Observing the DOM: IntersectionObserver and MutationObserver

Modern web pages are highly dynamic. Elements appear and disappear, and you may need to react when an element enters the viewport or when the DOM structure changes. Polling the DOM on every scroll or mutation is inefficient. The **IntersectionObserver** and **MutationObserver** APIs provide efficient, event-driven ways to observe these changes.

IntersectionObserver: watching visibility changes

The `IntersectionObserver` API lets you asynchronously detect when a **target element** enters or leaves a specified **root**'s viewport or crosses visibility thresholds. This is useful for lazy loading images, implementing infinite scrolling, triggering animations when content appears, and reporting viewability for ads.

How it works

An `IntersectionObserver` is created with a callback and an optional configuration object:

```
```js
const options = {
 root: null, // defaults to the browser viewport
 rootMargin: "0px", // margins around the root
 threshold: [0, 0.5, 1], // percentages of visibility that trigger the callback
};

const observer = new IntersectionObserver((entries) => {
 entries.forEach((entry) => {
 if (entry.isIntersecting) {
 console.log("Element is visible:", entry.target);
 }
 });
});
```

```

}, options);

// Observe one or more elements
const target = document.querySelector(".lazy-image");
observer.observe(target);
...

```

The `callback` receives an array of `IntersectionObserverEntry` objects. Each entry contains details about a target element's intersection: `isIntersecting` (whether it is visible), `intersectionRatio` (the percentage of the element in view) and bounding rectangles. You can observe multiple elements with a single observer instance.

`IntersectionObserver` does not continuously poll. The browser schedules callbacks when visibility thresholds are crossed, making it efficient compared to listening to scroll events. Configuration options include:

- **`root`**: The element or viewport used as the boundary for testing visibility. If omitted or `null`, the browser viewport is used.
- **`rootMargin`**: Offsets applied to the root's bounding box, specified like CSS margins (e.g., `'0px 0px -50% 0px'` to trigger sooner).
- **`threshold`**: A number or array of numbers between 0 and 1 indicating the intersection ratios that trigger the callback.

## ## MutationObserver: watching structural changes

The `MutationObserver` API allows you to observe changes to the DOM tree itself: additions or removals of nodes, changes to attributes, and text mutations. It replaces the deprecated Mutation Events and runs asynchronously to avoid blocking the main thread.

### ### How it works

To use a `MutationObserver`, create it with a callback and call `observe()` on a target node with options describing what you want to watch:

```
```js
```

```

const list = document.getElementById("myList");

const observer = new MutationObserver((mutationRecords) => {
  mutationRecords.forEach((record) => {
    if (record.type === "childList") {
      record.addedNodes.forEach((node) => console.log("Node added:", node));
      record.removedNodes.forEach((node) => console.log("Node removed:", node));
    } else if (record.type === "attributes") {
      console.log(
        `Attribute ${record.attributeName} changed on`,
        record.target
      );
    }
  });
});

observer.observe(list, {
  childList: true,
  attributes: true,
  subtree: true, // include child nodes
});

// Later, stop observing
// observer.disconnect();
...

```

The options allow you to specify what to watch:

- **childList**: Observe additions or removals of child nodes.
- **attributes**: Observe attribute changes on the target node.
- **subtree**: Extend observation to descendants of the target.

- **`characterData`**: Observe changes to text nodes.
- **`attributeFilter`** and **`attributeOldValue`**: Fine-tune which attributes trigger records and whether to record previous values.

The callback is called with an array of `MutationRecord` objects detailing the changes. Use `disconnect()` to stop observation when it's no longer needed.

Choosing the right observer

Use **`IntersectionObserver`** when you care about an element's visibility relative to the viewport or a scroll container. It's especially useful for:

- Lazy loading images and resources when they come into view.
- Triggering animations or counters when elements become visible.
- Implementing infinite scrolling: detecting when the user reaches the bottom of a list and loading more content.
- Collecting analytics on which sections of a page are seen by the user.

Use **`MutationObserver`** when you need to react to changes in the DOM structure or attributes. Typical use cases include:

- Implementing custom components that react to children being added or removed.
- Observing attribute changes to synchronize state (e.g., watching `data-*` attributes).
- Detecting text changes in content editable areas.
- Building debugging tools that log when the page changes unexpectedly.

Real-world analogy

Imagine you're managing a museum. You have one guard (`IntersectionObserver`) watching whether a visitor enters a specific room, and another guard (`MutationObserver`) watching whether an artwork is moved, swapped out or labelled differently. The first guard reports when something becomes visible; the second reports when something changes its existence or attributes.

Practice questions

1. **Theory:** Describe how `IntersectionObserver` differs from listening to the `scroll` event for detecting when an element enters the viewport. What performance benefits does it offer?
2. **Theory:** What types of mutations can `MutationObserver` detect? How do the options you pass to `observe()` control which mutations are reported?
3. **Coding:** Implement lazy loading of images: write a script that uses `IntersectionObserver` to replace `data-src` with `src` when images scroll into view.
4. **Coding:** Write a `MutationObserver` that logs a message whenever a `<ul>` gains or loses `<li>` items. Then write code to add and remove list items to test it.
5. **Theory:** Why is it important to call `disconnect()` on a `MutationObserver` or stop observing with `unobserve()` on an `IntersectionObserver` when they're no longer needed?

These questions and examples will help you use observers effectively to build performant and reactive user interfaces.

What is the difference between innerHTML and textContent?

`innerHTML` vs. `textContent`: Reading and Writing DOM Content Safely

When manipulating HTML elements, two commonly used properties are `innerHTML` and `textContent`. Both allow you to inspect or update the contents of an element, but they behave differently and should be used for different purposes.

`innerHTML`: working with markup

The `innerHTML` property gets or sets the **HTML markup** contained within an element. Reading `innerHTML` returns a string with the serialized HTML of the element's descendants. Writing to `innerHTML` parses the provided string as HTML and replaces all existing children with the result.

```
```html
<div id="container">Hello, world!</div>

<script>

 const container = document.getElementById("container");

 console.log(container.innerHTML); // "Hello, world!"

 // Insert new markup

 container.innerHTML = "Hi there!";

 // The content now becomes: Hi there!

</script>
```
```

Because `innerHTML` parses and inserts HTML, it is considered an **injection sink**—a potential source of **cross-site scripting (XSS)** vulnerabilities. Never insert untrusted user input via `innerHTML`. Browsers treat the input as HTML, so any `

- **Performance:** Setting `innerHTML` replaces all child nodes, causing the browser to tear down and rebuild the subtree. For small strings this is fine, but for large or frequent updates it can be costly.
- **Security:** As noted, raw HTML insertion can lead to XSS if not sanitized.

Despite these caveats, `innerHTML` is useful when you intentionally want to insert or extract markup—such as templating frameworks, building elements from strings, or copying fragments of HTML.

`textContent`: working with plain text

The `textContent` property returns the **text** content of an element and all its descendants. It ignores HTML tags and scripts, and returns all text including that within `<script>` or `<style>` elements. Setting `textContent` on a node removes all existing children and inserts a single text node with the provided string.

```
```html
<div id="msg">Hi there!</div>
<script>
 const msg = document.getElementById("msg");
 console.log(msg.textContent); // "Hi there!"
 msg.textContent = "Safe?"; // sets literal text, not HTML
 // The div now literally contains: Safe?
</script>
```
```

Key points about `textContent`:

- It **does not parse HTML**. Characters like `<` and `>` are treated as plain text.
- It includes text from `<script>` and `<style>` elements when reading. If you want only human-readable text, consider using `innerText` (which is aware of styling and excludes hidden text), although `innerText` triggers layout reflows and should be used sparingly.
- It is generally **faster and safer** for inserting or retrieving plain text because the browser doesn't have to run the HTML parser or manage potential scripts.

Which one should you use?

Use ``innerHTML`` when you need to work with HTML fragments—adding or retrieving markup. Always sanitize or trust the input. Use ``textContent`` when you only need to handle text, especially if the text may include characters that look like HTML. It prevents injection attacks and avoids triggering a reflow when reading.

Summary of differences

| Aspect | <code>`innerHTML`</code> | <code>`textContent`</code> |
|--|--|--|
| Returns | HTML markup as a string | Only textual content |
| Parses input as HTML | Yes | No; treats input as plain text |
| Includes <code><script></code> text when reading | Not included when reading | Included when reading |
| Performance when writing | Replaces all children and reparses HTML | Replaces children with a single text node |
| XSS risk | High if input is not sanitized | Low, since markup is escaped |
| Common use cases | Rendering templates, copying HTML, complex UIs | Setting or reading user-visible plain text |

Real-world analogy

Think of ``innerHTML`` as giving someone a bowl of ingredients and a recipe—the browser must cook (parse) the recipe into a finished dish. If you hand over spoiled or dangerous ingredients (unsanitized user input), you risk poisoning the dish. ``textContent`` is like handing over a sealed, pre-cooked meal—no interpretation is needed and there's no risk of hidden hazards.

Practice questions

- Theory:** Explain the security implications of using ``innerHTML`` with user-provided input. How can you mitigate these risks?
- Coding:** Write a function `setSafeText(element, str)` that sets the text of `element` using the safest property for displaying plain text. Test it by passing strings containing HTML tags.

3. **Coding:** Create a script that reads the text of every `<p>` element on a page using `textContent` and appends the lengths of these texts after each paragraph.
4. **Theory:** Compare `textContent` with `innerText`. In what scenarios would you prefer one over the other?
5. **Theory:** Why might setting `innerHTML` cause performance issues if used repeatedly in a loop? Suggest a more efficient alternative for appending many elements.

These exercises will help you choose the right property and write secure, efficient DOM manipulation code.

What are custom events and how do you dispatch them?

Custom events and how to dispatch them

Web pages are built around **events**. A click, keypress or network response triggers an event that bubbles through the DOM. While browsers provide many built-in events, you can also define your own to signal that something application-specific has happened. These are called **custom events**.

Why custom events?

Imagine you have a complex widget composed of smaller components. When one component finishes a task (say, a form validates successfully), it needs to tell its parent about it without tightly coupling the two. A custom event provides this channel: the child dispatches a named event on itself; the parent listens for that name and reacts accordingly.

Creating a custom event

You create a custom event with the `CustomEvent` constructor. The first argument is the event type (a string) and the second is an optional object where you can attach data:

```
```js
const todoAdded = new CustomEvent("todoAdded", {
 detail: { id: 42, text: "Learn custom events" },
 bubbles: true, // allow the event to bubble up the DOM
 cancelable: false, // whether the event's default action can be prevented
});
```
```

The `detail` property can contain any serialisable data. The `bubbles` flag determines whether the event propagates up through ancestor elements. If you omit `bubbles`, the event will fire only on the target element.

Dispatching a custom event

Custom events are dispatched using `dispatchEvent()` on any `EventTarget` (elements, `window`, `document`, etc.):

```
```js
const form = document.querySelector("#todo-form");

// After creating the event
form.dispatchEvent(todoAdded);
```
```

When the event is dispatched, any listeners for `'todoAdded'` on the target or its ancestors (if `'bubbles'` is `'true'`) will run. You add listeners with `addEventListener()` just like for native events:

```
```js
document.addEventListener("todoAdded", (event) => {
 console.log("New todo:", event.detail);
});
```
```

The event object passed to the handler has the usual properties (`'type'`, `'target'`, `'currentTarget'`, `'bubbles'`) plus your data on `'detail'`. You can call `stopPropagation()` to prevent the event from bubbling further or `preventDefault()` only if `'cancelable'` is `'true'` and you've defined a default action.

Example: notifying when data loads

Suppose you fetch data in a component and want to notify consumers that loading has finished. Here's an example using a custom event:

```
```js
// data-loader.js

export default class DataLoader extends HTMLElement {
 async connectedCallback() {
```



```

const res = await fetch("/api/users");
this.users = await res.json();
// inform listeners that data is ready
this.dispatchEvent(
 new CustomEvent("data-loaded", {
 detail: { users: this.users },
 bubbles: true,
 })
);
}
}

// parent component
document.body.addEventListener("data-loaded", (event) => {
 console.log("Users:", event.detail.users);
});
...

```

By using a custom event, the `DataLoader` component doesn't need to know who is interested in the data; it simply broadcasts that the data is ready.

### ## Best practices and pitfalls

- **Name events clearly.** Event names are case-sensitive, so `taskDone` and `taskdone` are different events. Use a naming scheme (like kebab-case) to avoid collisions.
- **Use the `detail` property** to transmit data; avoid attaching arbitrary properties to the event object.
- **Consider bubbling** when the event should be handled by ancestors, but avoid unnecessary bubbling on large DOM subtrees to reduce overhead.
- **Don't overuse custom events.** For tightly coupled components, direct method calls or callback functions are simpler. Custom events shine when decoupling components or implementing plugin systems.

## ## Practice questions

1. **Theory:** Explain the difference between a built-in DOM event (such as `click`) and a custom event created with `CustomEvent`.
2. **Theory:** What happens if you dispatch a custom event without setting the `bubbles` option? How does it affect event propagation?
3. **Coding:** Write a function `createStatusEvent(name, status)` that returns a custom event named `name` with a `status` property in its detail. Include the option for the event to bubble.
4. **Coding:** Given a component that validates user input, dispatch a `formValid` event with a boolean indicating success. Demonstrate how a parent element listens for this event.
5. **Theory:** When would you choose to prevent the default action of a custom event? How do `cancelable` and `preventDefault()` work together in the context of custom events?

# Explain microtask queue vs nextTick in Node.js

## # Microtask queue vs `process.nextTick()` in Node.js

JavaScript runtimes use an **event loop** to interleave work so the main thread can handle user interactions, I/O and timers without blocking. Within each loop iteration there are two important queues of callbacks: **macrotasks** (also called tasks) and **microtasks**.

Understanding the distinction between the microtask queue and Node's special `process.nextTick()` queue helps you write efficient asynchronous code without starving I/O.

## ## The microtask queue

Microtasks are scheduled by features such as `Promise` resolutions, `queueMicrotask()` and (in browsers) `MutationObserver`. After the currently executing script finishes and the call stack unwinds, the event loop processes **all** microtasks in FIFO order before moving on to the next macrotask. This guarantees that promise callbacks run as soon as possible, keeping program state consistent:

```
```js
Promise.resolve().then(() => console.log("microtask 1"));
console.log("script end");
// logs:
// script end
// microtask 1
```
```

Because the microtask queue is drained completely before the next task, adding more microtasks within a microtask will delay the event loop until the queue is empty. Overusing microtasks (especially in loops) can delay timers and I/O.

## ## `process.nextTick()`

Node.js provides its own queue processed even **earlier** than the microtask queue. Calling `process.nextTick()` schedules a callback to run **immediately after the current operation completes**, before any pending microtasks or I/O events:

```
``js
console.log("start");
process.nextTick(() => console.log("nextTick"));
Promise.resolve().then(() => console.log("microtask"));
console.log("end");
// output:
// start
// end
// nextTick
// microtask
``
```

Here, the `nextTick` callback runs before the promise's `.then()` handler. Node uses `nextTick()` internally for its own housekeeping (such as emitting the `'exit'` event). Because callbacks scheduled with `process.nextTick()` run before I/O, it's possible to starve the event loop. If a `nextTick` callback recursively schedules itself, it can block the processing of timers or network events.

### ### When to use `process.nextTick()`

You might use `nextTick()` when you need to run code **immediately** after the current function finishes but before any promise handlers or I/O. Common use cases include:

- **Error handling:** Ensure error listeners are attached before emitting an error event.
- **Breaking up synchronous work:** Large synchronous operations can block the event loop. Splitting work across multiple `nextTick()` calls yields control back to Node between chunks.

However, prefer `setImmediate()` or promises when you simply need to schedule code on the next iteration of the event loop. They run after I/O callbacks, avoiding starvation.

### ## Summary of differences

|                  |                                                             |                                                                     |
|------------------|-------------------------------------------------------------|---------------------------------------------------------------------|
| Aspect           | Microtask queue ( <code>Promise</code> )                    | <code>process.nextTick()</code>                                     |
| Execution timing | After the current call stack and before the next task       | After the current call stack and <b>before microtasks and I/O</b>   |
| Standard?        | Part of the HTML and Node specifications                    | Node-specific API                                                   |
| Typical uses     | <code>Promise.then()</code> , <code>queueMicrotask()</code> | Internal Node housekeeping; critical callbacks                      |
| Risk             | Starvation if microtask queue never drains                  | Starvation if callbacks schedule more <code>nextTick()</code> calls |

## ## Practice questions

- Theory:** What is the order of execution between `process.nextTick()`, promise microtasks and timer callbacks? Explain with a simple example.
- Coding:** Write a Node.js script that logs messages in the following order: `script`, `nextTick`, `microtask`, `timer`. Use `process.nextTick()`, `Promise.resolve().then()` and `setTimeout()`.
- Theory:** Why can overusing `process.nextTick()` lead to starvation of I/O? How can you mitigate this risk?
- Coding:** Convert a synchronous loop that blocks the event loop for 100 ms into a non-blocking loop using `process.nextTick()` or `setImmediate()`. Observe the differences.
- Theory:** When would you prefer `queueMicrotask()` over `process.nextTick()` in Node.js? Discuss advantages and trade-offs.

# What is the difference between V8 engine internals and standard JavaScript?

## # Difference between V8 engine internals and standard JavaScript

JavaScript is defined by the **ECMAScript specification**, which describes the syntax and behaviour of the language. A JavaScript engine is an implementation of that specification. **V8** is one of those engines. It powers Google Chrome, Node.js, Deno and many other environments.

Understanding the difference between the two helps explain why some features exist in one environment but not another.

### ## Standard JavaScript

The ECMAScript specification (often shortened to **ECMA-262**) defines how the language behaves: how variables are scoped, how functions are called, how promises work and so on. It deliberately leaves out details about performance, memory layout or integration with operating systems. When you write `const x = 5`, you're using standard JavaScript syntax and semantics that any conforming engine should implement.

The spec also defines *\_built-in\_* objects like `Array`, `Promise` and `Map`. It does **not** define host-specific APIs such as `document`, `console`, `require()` or file system access. Those are provided by the host environment (browser, Node.js) on top of the core language.

### ## V8 engine internals

V8 is written in C++ and implements the ECMAScript spec. It also implements Web APIs (when embedded in Chrome) or Node APIs (when embedded in Node.js) by exposing additional objects. Internally, V8 performs many optimisations to execute JavaScript quickly:

- **Just-in-time (JIT) compilation.** V8 first interprets JavaScript with a baseline interpreter (Ignition). Hot functions are then compiled to machine code by the TurboFan optimizing compiler. V8 collects type feedback at runtime and speculates about types to generate fast code. If speculation fails, it "deoptimises" back to the interpreter.
- **Hidden classes and inline caching.** Unlike statically typed languages, JavaScript objects can change shape at runtime. V8 groups objects with the same property layout into hidden classes. When properties are accessed, V8 can look up their offset quickly using the hidden class and caches the location for subsequent accesses. This avoids expensive dictionary lookups on every property access.

- **Generational garbage collection.** V8 allocates objects in a young generation and assumes most die young. The young generation is collected frequently, and surviving objects are promoted to an old generation. This reduces pause times compared to scanning the entire heap.

These internal features are not visible in standard JavaScript code—there is no API to control hidden classes or trigger JIT compilation—but understanding them can inform best practices: define all properties in your constructor to avoid creating multiple hidden classes, avoid adding new properties to objects on the fly and refrain from mixing types in arrays.

## ## Differences in environment features

Because V8 is embedded in different hosts, it exposes different global objects depending on where it runs:

| Environment    | Globals provided beyond ECMAScript                                                                                    |
|----------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Browser</b> | <code>window</code> , <code>document</code> , <code>alert()</code> , <code>fetch()</code> , <code>setTimeout()</code> |
| <b>Node.js</b> | <code>global</code> , <code>process</code> , <code>Buffer</code> , <code>require()</code> , <code>module</code>       |
| <b>Deno</b>    | <code>Deno</code> , <code>fetch()</code> , <code>Worker</code> (no <code>require()</code> )                           |

These APIs are not part of the ECMAScript spec; they are provided by the host to interact with the environment (DOM, file system, network). This explains why code that uses `document.getElementById()` fails in Node.js and why Node's `require()` is unavailable in browsers.

## ## Practice questions

- Theory:** What aspects of JavaScript are defined by ECMAScript, and what aspects are left to host environments? Give examples of each.
- Theory:** Explain how V8's hidden classes and inline caching improve property access performance. How can code structure influence hidden class generation?
- Theory:** Why is there no standard API to control the JIT compiler or garbage collector in JavaScript? What would be potential issues if such control existed?
- Coding:** Write a constructor function that sets all properties on the instance in the same order each time. Explain why this approach benefits V8's hidden class optimisation.
- Theory:** Compare the global objects available in the browser and Node.js. Why are some APIs like `fetch()` available in both, while others like `require()` are environment-specific?

# How does just-in-time (JIT) compilation work in JavaScript engines?

## # How just-in-time (JIT) compilation works in JavaScript engines

JavaScript began life as an interpreted scripting language. Early engines read source code and executed it directly. To improve performance, modern engines use **just-in-time (JIT) compilation**: they compile frequently executed code to machine instructions at runtime. JIT compilers combine the flexibility of dynamic languages with the speed of compiled languages.

## ## Interpreter, baseline JIT and optimising compiler

Most engines employ a multi-tier pipeline. Taking V8 as an example:

1. **Parser and interpreter**: Source code is parsed and compiled into bytecode for a lightweight interpreter (Ignition). This allows quick start-up and supports dynamic features like `eval()`.
2. **Profiler**: As code runs, the engine records how often functions are called, what types arguments and return values have and which branches are taken. This data is called *\_type feedback\_*.
3. **Optimising compiler**: When a function becomes "hot" (executed many times), the engine compiles it with an optimising compiler (TurboFan in V8, IonMonkey in SpiderMonkey). The compiler uses type feedback to specialise the code: it may assume that a variable is always a number and generate faster machine code for that case.

The optimising compiler performs aggressive transformations: inlining small functions, eliminating bounds checks, constant folding and removing dead code. If the assumptions prove true, the code runs much faster than interpreted code.

## ## Deoptimisation

JavaScript is dynamic. A variable that was always a number may suddenly become a string. If that happens, the assumptions in the compiled code no longer hold. Engines handle this by **deoptimising**: they bail back to the interpreter, patching up the program state, and collect new feedback. The function may be recompiled with updated assumptions or continue in the interpreter.

Deoptimisation makes JIT compilation invisible to developers. You don't need to think about types ahead of time, but writing code with consistent types helps the compiler stay in optimized code paths.



## ## Benefits and trade-offs

- **Speed:** JIT-compiled code can be as fast as native code when the compiler's assumptions hold. Hot loops and heavy calculations benefit the most.
- **Startup cost:** Compiling code takes time. That's why engines interpret code first and only compile hot functions. Very short scripts may not get JIT-compiled at all.
- **Memory:** JIT compilers store multiple versions of compiled code (baseline and optimized) and metadata. On devices with limited memory, engines may limit JIT usage.
- **Security:** Generating executable memory at runtime has security implications (spectre mitigations, write-execute permissions). Browsers implement safeguards like memory page protection and pointer authentication.

## ## JIT compilation in other engines

While the details differ, major engines follow a similar strategy:

- **V8 (Chrome, Node.js):** Ignition interpreter, TurboFan optimising compiler.
- **SpiderMonkey (Firefox):** Baseline Interpreter, Baseline JIT, IonMonkey optimising JIT.
- **JavaScriptCore (Safari):** LLInt (low-level interpreter), Baseline JIT (DFG), and the C Loop interpreter.

Some environments, such as older mobile browsers or embedded systems, may disable JIT for security or resource reasons. In such cases, code runs entirely in the interpreter.

## ## Practice questions

1. **Theory:** Describe the roles of the interpreter, profiler and optimising compiler in a modern JS engine's pipeline. Why is a two-tier strategy used instead of immediately compiling everything?
2. **Theory:** Explain how type feedback enables faster machine code. What happens when an assumption is violated?
3. **Theory:** List some optimisations that an optimising JIT compiler might perform on JavaScript code. How do they improve performance?

4. **Coding:** Write a function that performs a numeric calculation in a tight loop. Run it multiple times and observe the difference in performance with and without type changes (e.g., switching a number to a string midway). Explain why the engine might deoptimise and reoptimise your function.
5. **Theory:** Discuss the security considerations of JIT compilation. What measures do browser vendors take to mitigate risks associated with generating executable code at runtime?

# Explain hidden classes and inline caching in V8

## # Hidden classes and inline caching in V8

JavaScript is a dynamic language: you can add or remove properties from objects at any time. While this flexibility is powerful, it presents challenges for performance. The V8 engine uses two key techniques—**hidden classes** and **inline caching**—to make property access fast without sacrificing dynamism.

### ## Hidden classes (also called shapes)

In languages like C++ each class has a fixed layout that tells the runtime where to find fields. JavaScript objects do not have classes in the same way; each object's properties could be different. V8 bridges this gap by creating an internal *\_hidden class\_* for each distinct object shape. A hidden class records the names and order of an object's properties and maps them to fixed offsets in memory.

When you create an object, V8 assigns it a hidden class based on the properties you define. Adding a new property transitions the object to a new hidden class. If two objects add the same properties in the same order, V8 reuses the same hidden class, enabling them to share inline caches and compiled code.

### ### Why order matters

Because hidden classes track the order in which properties are added, defining properties consistently yields fewer class transitions:

```
``js
// Constructor function creates a predictable shape
function Point(x, y) {
 this.x = x;
 this.y = y;
}

const p1 = new Point(1, 2);
```

```

const p2 = new Point(3, 4);

// p1 and p2 share the same hidden class

// Avoid adding properties later, which would create a new hidden class:
p1.z = 5; // now p1 uses a different hidden class than p2
'''

```

By declaring all properties in the constructor, you help V8 reuse hidden classes, enabling better optimisation and inline caching. Adding properties outside the constructor or in different orders causes V8 to create new hidden classes, preventing code sharing.

## ## Inline caching

When V8 executes a property access like ``obj.x``, it must find where ``x`` lives in memory. Without optimisation, it would perform a dynamic lookup every time. Inline caching (IC) speeds this up by remembering the hidden class of the object and the offset of the requested property.

- The first time ``obj.x`` is executed, V8 does the full lookup and stores the resulting hidden class and offset in a tiny cache attached to the instruction.
- The next time the same instruction runs and the object has the same hidden class, V8 can skip the lookup and jump straight to the stored offset.
- If the object has a different hidden class, V8 falls back to a slower path and may update the inline cache to reflect the new shape (polymorphic inline caching).

Inline caches dramatically speed up property reads and writes when objects are consistently shaped. They also collect type feedback for the JIT compiler; if the cache sees many different shapes, the compiler may treat the operation as polymorphic and generate a more general (slower) code path.

## ## Best practices to help hidden classes and caches

- **\*\*Initialise all instance properties in the constructor\*\***. Avoid adding properties later.
- **\*\*Add properties in the same order\*\*** for all instances of a given "class".
- **\*\*Avoid mixing unrelated types\*\*** in arrays and object fields. Homogeneous arrays enable better optimised code.

- **Delete properties sparingly**, as deletion also changes the hidden class and can deoptimise code.

## ## Practice questions

1. **Theory:** What is a hidden class in V8 and how does it differ from a traditional class in languages like Java or C++?
2. **Theory:** Describe how adding properties in different orders affects hidden classes. Why is it beneficial to initialise all properties in the constructor?
3. **Theory:** Explain the concept of inline caching. How does it speed up repeated property accesses?
4. **Coding:** Create a constructor that defines three properties. Instantiate two objects and demonstrate how adding a property to one instance can lead to different hidden classes and potentially slower code.
5. **Theory:** What are the potential downsides of relying on inline caching? Under what circumstances might an inline cache become polymorphic or megamorphic, and how does that affect performance?

# What are WeakRefs and FinalizationRegistry?

## # Weak references and the `FinalizationRegistry`

Modern JavaScript engines use garbage collection to free memory when objects are no longer reachable. Most of the time you can ignore memory management, but some patterns—such as caches—require you to hold references to objects without preventing their collection. **`Weak references (WeakRef)`** and **`FinalizationRegistry`** provide mechanisms for that.

## ## Weak references

A normal (strong) reference prevents an object from being garbage-collected. A *weak reference* allows you to refer to an object without affecting its liveness. If the object becomes unreachable from the rest of your code, the garbage collector can reclaim it even though a `WeakRef` still exists.

You create a weak reference with `new WeakRef(target)`. The only operation on a `WeakRef` is `.deref()`, which returns the original object if it's still alive or `undefined` if it has been collected:

```
```js
const cache = new Map();

function getUser(id) {
  const ref = cache.get(id);
  let user = ref && ref.deref();
  if (!user) {
    user = loadUserFromDB(id);
    cache.set(id, new WeakRef(user));
  }
  return user;
}
```
```

Here, the cache doesn't prevent `user` objects from being freed if nothing else refers to them. When you call `ref.deref()`, you either get a live object or `undefined`, in which case you reload it.

Weak references are primarily useful for caches, memoisation and other data structures where stale values can be recomputed. You should not use them to manage critical resources; rely on strong references and proper cleanup instead.

### ## The `FinalizationRegistry`

`FinalizationRegistry` lets you register a callback that the engine will call after a particular object is garbage-collected. The constructor takes a cleanup function; you then call `.register(target, heldValue)` to associate a target object with a "held" value (often a resource identifier). When `target` is collected, the registry queues the held value for cleanup:

```
```js
const registry = new FinalizationRegistry((held) => {
  console.log("Cleaning up resource", held);
});

function trackResource(obj, id) {
  // obj is the user-facing object; id identifies an external resource
  registry.register(obj, id);
}

// later
const resource = {
  /* ... */
};

trackResource(resource, "socket:1234");

// when resource becomes unreachable, the cleanup callback runs
```
```

The callback runs at some time *after* the object has been reclaimed. The specification makes no guarantee about when or even if the callback will execute. The callback is invoked in a separate task, so you cannot depend on it for essential finalisation (e.g. closing files or releasing locks). Use it instead to clean up ancillary caches or to log when objects are collected.

## ## Caveats and warnings

- Weak references and finalisation can expose details of the garbage collector. Overusing them may make code unpredictable or brittle.
- There is no guarantee that a finalizer will run in a timely manner; it may never run if the process exits before collection. Always provide explicit cleanup methods when working with critical resources.
- Only use `FinalizationRegistry` when you have no other way to detect object lifecycle events. For example, a `Map` of objects keyed by ID should use explicit `delete()` instead of relying on finalisation.

## ## Practice questions

1. **Theory:** Why might you use a `WeakRef` rather than a normal reference in a cache? What happens when the object referenced by a `WeakRef` is garbage-collected?
2. **Theory:** Describe the lifecycle of a finalizer registered via `FinalizationRegistry`. Why is it not safe to put critical cleanup in a finalizer callback?
3. **Coding:** Implement a memoisation function that caches results using `WeakRef` so that entries can be reclaimed when their keys are no longer used.
4. **Theory:** List potential pitfalls of using `WeakRef` and `FinalizationRegistry`. How would you design your code to avoid these pitfalls?
5. **Coding:** Demonstrate how you can register an object with `FinalizationRegistry` and observe when the cleanup callback runs by creating and discarding objects inside a loop. (Hint: You may need to force garbage collection in a controlled environment for testing.)



# How does debounce–throttle combo optimize performance?

## # How a debounce-throttle combination optimises performance

Many user interactions fire events rapidly: resize, scroll, keyup or window resize can trigger dozens or hundreds of callbacks per second. If each event performs expensive work (rendering, network requests), performance suffers. **Debouncing** and **throttling** are two techniques for limiting how often a function runs. Combining them allows you to tailor responsiveness and resource use.

### ## Quick recap

- **Debouncing** delays execution of a function until a certain time has passed without another trigger. Each new call resets the timer. Debounce is useful when you only care about the final event in a burst—such as submitting a search after the user stops typing.
- **Throttling** allows a function to run at most once every `N` milliseconds. Additional calls within the interval are ignored. Throttle is ideal when you want periodic updates, like repositioning elements while the user scrolls.

### ## Combining debounce and throttle

Sometimes you need the best of both worlds: provide an immediate response, update periodically during a burst and ensure a final update after activity stops. A **debounce-throttle combo** does exactly that. A common implementation runs the function at the beginning and end of a burst, but throttles intermediate calls.

Here is a simple implementation with options for leading (immediate) and trailing (final) calls:

```
```js
function debounceThrottle(fn, delay) {
  let lastCall = 0;
  let timerId;
  return function (...args) {
    const now = Date.now();
    const remaining = delay - (now - lastCall);
```

```

clearTimeout(timerId);
if (remaining <= 0) {
  // Leading: run immediately and update lastCall
  lastCall = now;
  fn.apply(this, args);
} else {
  // Trailing: schedule for after remaining time
  timerId = setTimeout(() => {
    lastCall = Date.now();
    fn.apply(this, args);
  }, remaining);
}
};
}

// Usage: execute immediately and at most every 200ms thereafter
const handleResize = debounceThrottle(() => {
  console.log("Resized at", new Date());
}, 200);
window.addEventListener("resize", handleResize);
...

```

In this example, the function runs immediately on the first resize event and then at most once every `delay` milliseconds. If events stop, a final trailing call ensures the latest state is processed. This pattern keeps the UI responsive while avoiding unnecessary work.

When to use a combo

- **Responsive UI updates.** For scroll or mousemove handlers, you may want to update layout on the first event and periodically thereafter, but still handle the final state.
- **Search suggestions.** Provide an instant suggestion as the user types but avoid hitting the API on every keystroke; throttle intermediate calls and debounce the final call.

- **Window resize recalculations.** Immediately adjust layout, then throttle continuous updates and finally apply finishing touches when resizing stops.

Practice questions

1. **Theory:** Explain the difference between debouncing and throttling. Give an example use case for each.
2. **Theory:** Why might you combine debouncing and throttling rather than use only one? Describe a scenario where a combo is beneficial.
3. **Coding:** Implement a `debounceThrottle` function that accepts options `{ delay, leading, trailing }` to control whether the wrapped function runs at the start of the burst, end of the burst or both.
4. **Coding:** Attach a scroll event listener to an element using your `debounceThrottle` implementation. Log the scroll position immediately and then no more frequently than every 100 ms, with a final log after scrolling stops.
5. **Theory:** What risks arise if you debounce a function that handles essential state updates (e.g. resizing a canvas)? How can a throttle or combo alleviate those risks?

What are ArrayBuffer and TypedArray?

ArrayBuffer and TypedArray

JavaScript strings and objects are convenient for text and structured data, but they are not suitable for handling raw binary data such as images, audio streams or network packets. The **ArrayBuffer** and **TypedArray** APIs provide a way to work with binary data in memory, enabling you to read and write bytes directly.

ArrayBuffer

An **ArrayBuffer** is a fixed-length block of raw memory. It represents a contiguous sequence of bytes but offers no way to interpret those bytes. Think of it as an empty canvas: you need a brush (a `_view_`) to draw on it. You create an `ArrayBuffer` with `new ArrayBuffer(length)`, where `length` is the number of bytes:

```
```js
const buffer = new ArrayBuffer(16); // allocate 16 bytes (128 bits)
console.log(buffer.byteLength); // 16
```
```

You cannot read or write individual bytes directly through the `ArrayBuffer`. Instead, you create a view—either a typed array or a `DataView`—that provides typed access to its contents.

Typed arrays

Typed arrays are array-like objects that view and manipulate binary data in an `ArrayBuffer`. Each typed array type corresponds to a specific numeric format (e.g., `Uint8Array` for 8-bit unsigned integers, `Int32Array` for 32-bit signed integers, `Float64Array` for double-precision floats). Creating a typed array allocates or attaches to a buffer:

```
```js
// Create a new buffer and view it as 8-bit unsigned integers
const bytes = new Uint8Array(4); // allocates a buffer of length 4
bytes[0] = 255;
```
```

```

bytes.set([1, 2, 3], 1);
console.log(bytes); // Uint8Array [255, 1, 2, 3]

// Create a buffer separately and attach a view
const buf = new ArrayBuffer(8);
const ints = new Int16Array(buf); // 16-bit signed integers (2 bytes each)
ints[0] = 42;
ints[1] = -1;
console.log(ints); // Int16Array [42, -1, 0, 0]
...

```

Typed arrays share many methods with normal arrays (`map`, `forEach`, `filter`) but lack some (like `push`, because their length is fixed). They are not true `Array` objects; `Array.isArray()` returns `false` for typed arrays. All typed arrays have a `buffer` property referencing their underlying `ArrayBuffer`, and they view data starting at an **offset** for a **length**. Multiple views can reference the same buffer with different offsets and element types, allowing you to interpret the same bytes in various ways.

DataView

While typed arrays interpret data in fixed element sizes, `DataView` lets you read and write arbitrary numbers of bytes with specific endianness. It is useful when the data format doesn't align neatly with typed array element sizes or when working with network protocols that require big-endian order:

```

```js
const buf = new ArrayBuffer(4);
const view = new DataView(buf);
view.setUint16(0, 0x1234, false); // big-endian
view.setUint16(2, 0xabcd, false);
console.log(view.getUint32(0, false).toString(16)); // '1234abcd'
...

```

## ## Use cases

- **Binary network protocols:** WebSockets and WebRTC can send `ArrayBuffers` directly, allowing efficient binary transmission.
- **Multimedia:** Audio and video APIs use typed arrays for PCM data and pixel buffers.
- **Cryptography:** Cryptographic algorithms operate on byte arrays rather than strings.
- **WebGL:** Vertex buffers and textures are supplied as typed arrays to WebGL for rendering.

## ## Practice questions

1. **Theory:** What is the relationship between an `ArrayBuffer` and a typed array? Why can't you read or write bytes directly on an `ArrayBuffer`?
2. **Theory:** Describe how typed arrays differ from regular JavaScript arrays. What methods do they share and which do they lack?
3. **Coding:** Create an `ArrayBuffer` of length 12 bytes. View it as both `Uint8Array` and `Float32Array` and demonstrate how changing one view affects the other.
4. **Coding:** Use a `DataView` to write a 32-bit big-endian integer at offset 0 and then read it back as two 16-bit unsigned integers. Explain what happens.
5. **Theory:** Why might you choose a typed array over a regular array when working with WebGL or other low-level APIs?

# Explain SharedArrayBuffer and Atomics API

## # SharedArrayBuffer and the `Atomics` API

Web workers allow JavaScript to run in parallel threads, but normally each worker has its own heap. Data is copied between threads via structured cloning. For high-performance scenarios like games, scientific simulations or shared caches, copying large amounts of data becomes a bottleneck. **SharedArrayBuffer** and the **Atomics** API solve this by enabling shared memory and synchronised access.

## ## SharedArrayBuffer

A **SharedArrayBuffer** is like an `ArrayBuffer` but its memory can be shared between multiple execution contexts (the main thread and workers). Any view (typed array or `DataView`) created from the same `SharedArrayBuffer` sees the same underlying bytes. Changing a byte in one view immediately affects all views.

Example of creating a shared buffer and sharing it with a worker:

```
``js
// main.js
const sab = new SharedArrayBuffer(1024); // 1KB shared memory
const sharedInts = new Uint32Array(sab);
sharedInts[0] = 42;

const worker = new Worker('worker.js');
// Transfer a reference to the shared buffer
worker.postMessage(sab);

// worker.js
self.onmessage = (e) => {
 const shared = new Uint32Array(e.data);
 console.log('Initial value:', shared[0]);
 shared[0] = shared[0] + 1;
```

```
// The main thread sees this change instantly
};
...

```

Because `SharedArrayBuffer` allows threads to modify the same memory, it introduces race conditions. Without coordination, updates can interleave unpredictably. To coordinate reads and writes, you must use **`**atomic operations**`**.

## ## The `Atomics` API

The `Atomics` object provides atomic operations on shared typed arrays. Atomic operations are indivisible: no other thread can observe a partial update. `Atomics` includes functions for reading and writing (`load`, `store`), arithmetic (`add`, `sub`, `and`, `or`, `xor`), compare-and-swap (`compareExchange`) and waiting/notification (`wait`, `notify`). These functions operate on integer typed arrays (`Int8Array`, `Uint16Array`, `Uint32Array`, etc.) backed by a `SharedArrayBuffer`.

## ### Synchronisation and locks

`Atomics.wait()` causes the calling thread to block until another thread calls `Atomics.notify()` on the same location or a timeout expires. This provides a simple building block for implementing mutexes, semaphores and other synchronisation primitives. For example, a shared ring buffer can coordinate producer and consumer threads:

```
``js
// main thread: producer
const queue = new Int32Array(sab);
const writeIndex = 0; // index 0 holds write pointer
const readIndex = 1; // index 1 holds read pointer

function produce(value) {
 const i = Atomics.load(queue, writeIndex);
 // write value at position i+2
 Atomics.store(queue, i + 2, value);
 Atomics.store(queue, writeIndex, (i + 1) % 10);
}

```



```

 Atomics.notify(queue, readIndex);
 }

 // worker thread: consumer
 function consume() {
 while (true) {
 let r = Atomics.load(queue, readIndex);
 if (r === Atomics.load(queue, writeIndex)) {
 // nothing to read; wait
 Atomics.wait(queue, readIndex, r);
 } else {
 const value = Atomics.load(queue, r + 2);
 // process value
 Atomics.store(queue, readIndex, (r + 1) % 10);
 }
 }
 }
 ...

```

This example shows how `Atomsics` can implement a circular buffer without busy waiting.

### ## Security considerations

Because SharedArrayBuffers expose new side-channel attacks (Spectre), browsers require pages to opt in to **cross-origin isolation** and run in a secure context (HTTPS) before allowing SharedArrayBuffer usage. You must set the `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp` headers. Without these headers, `SharedArrayBuffer` will throw a `SecurityError`.

### ## Practice questions

1. **Theory:** Explain why copying data between workers can be inefficient. How does a SharedArrayBuffer solve this problem?

2. **Theory:** Why are atomic operations necessary when using `SharedArrayBuffer` across threads? What could go wrong if you access the buffer without `Atomics`?
3. **Coding:** Create a main thread and a worker that share a `SharedArrayBuffer` of four 32-bit integers. Have the worker increment the first value using `Atomics.add()` and notify the main thread using `Atomics.notify()`. The main thread should wait for the update with `Atomics.wait()` and then log the new value.
4. **Theory:** Describe the difference between `Atomics.compareExchange()` and `Atomics.exchange()`. When would you use each?
5. **Theory:** What security headers are required to use `SharedArrayBuffer` in modern browsers? Why are they necessary?

# What is structured concurrency (upcoming spec)?

## # Structured concurrency: an upcoming JavaScript proposal

Asynchronous programming allows a program to perform multiple tasks seemingly at once, but unstructured concurrency can lead to "zombie" tasks that continue running after the function that spawned them has returned. **\*\*Structured concurrency\*\*** is a design principle and proposed specification that aims to tame asynchronous code by ensuring that tasks have a well-defined lifetime and that errors propagate predictably.

## ## The problem with unstructured concurrency

In JavaScript today you can fire off a promise or `setTimeout()` without awaiting it. For example:

```
```js
async function fetchAndLog(url) {
  fetch(url).then((response) => console.log("Fetched", response.status));
  return "returned immediately";
}

const result = fetchAndLog("/api/data");
console.log(result); // logs 'returned immediately' while fetch continues in background
```
```

The `fetchAndLog()` function returns immediately, and the caller has no control over or awareness of the ongoing fetch. If an error occurs, it may go unhandled. If the caller is cancelled or times out, the fetch keeps running. In larger systems, these orphaned tasks can accumulate and cause resource leaks.

## ## What structured concurrency offers

Structured concurrency proposes that asynchronous operations should be scoped to their parent function or block. When the parent completes, any child tasks should automatically be awaited or cancelled. The key ideas are:

- **Task scoping:** Spawned tasks are tied to a context. They cannot outlive it.
- **Failure propagation:** If a child task errors, the error bubbles up to the parent, allowing a single place to catch exceptions.
- **Cancellation:** Cancelling the parent cancels all children.

Other languages implement structured concurrency through constructs like Go's `context.Context`, Python's `asyncio` task groups or Kotlin's coroutines. The JavaScript proposal aims to provide similar capabilities via native APIs.

### ## Proposed API sketch

As of this writing, structured concurrency is an early-stage proposal. One experimental API uses **task functions** and **cancellation tokens**. A simplified sketch might look like this:

```
``js
// hypothetical API
async function parent() {
 const token = new CancellationToken();
 // start child tasks within this scope
 const child1 = startTask(token, async () => doSomething());
 const child2 = startTask(token, async () => doSomethingElse());

 // wait for all children or throw on first error
 try {
 await Promise.all([child1, child2]);
 } finally {
 token.cancel(); // ensure cancellation on exit
 }
}
...

```

In this sketch, `startTask()` spawns a task tied to a token. If one child throws, the token is cancelled and all tasks clean up. The parent waits on children before returning, ensuring no task leaks.

## ## Current alternatives

Until the proposal is standardised, you can achieve similar structure by manually tracking promises and cancellations:

- **`AbortController`** and **`AbortSignal`**: Many web APIs support abort signals for cancellation. Pass the same signal to all concurrent operations and call `abort()` in a `finally` block.
- **Helper libraries**: Libraries like `p-limit` and `taskgroup` implement structured concurrency patterns for Node.js.
- **Error handling wrappers**: Wrap asynchronous functions so that thrown errors or rejections propagate to a common error handler instead of being lost.

## ## Practice questions

1. **Theory:** What problems arise when asynchronous tasks outlive their spawning function? Why is this problematic in long-running applications?
2. **Theory:** Explain the main principles of structured concurrency. How do they differ from ad-hoc spawning of promises?
3. **Coding:** Using `AbortController`, write an `async` function that starts two fetches in parallel and cancels both if either rejects or if the parent function returns early.
4. **Theory:** How do other languages (e.g. Go, Python, Kotlin) implement structured concurrency? Identify similarities and differences with the proposed JavaScript approach.
5. **Theory:** Discuss potential challenges in adding structured concurrency to the existing event loop and promise model. How might backward compatibility and cancellation semantics be handled?

# How does the ECMAScript spec define execution order?

## # How the ECMAScript specification defines execution order

JavaScript's behaviour isn't arbitrary; it's governed by the **ECMAScript specification**. Understanding how the spec defines execution order helps demystify why code runs in a particular sequence and how asynchronous tasks are scheduled.

### ## Synchronous execution order

Within a single script, ECMAScript defines evaluation rules that determine the order in which expressions are executed. Key points include:

- **Left-to-right evaluation**: In most binary expressions (`a + b`, `f(x, y)`), the left operand is evaluated before the right. Function arguments are evaluated from left to right. This means side effects occur in a predictable order:

```
``js
function log(val) {
 console.log(val);
 return val;
}

const result = log("first") + log("second");

// logs: first, then second
``
```

- **Execution context stack**: Each time a function is called, a new execution context is pushed onto the call stack. The context holds local variables, the value of `this` and the point to return to. When a function returns, its context is popped and control resumes at the previous context.

- **Hoisting**: Declarations of variables and functions are processed before code execution begins. Function declarations are hoisted with their body; `var` declarations are hoisted but initialised with `undefined`; `let` and `const` declarations are hoisted but remain in the "temporal dead zone" until initialised.

These rules ensure that synchronous code executes deterministically.

### ## Asynchronous execution: the event loop

JavaScript environments are single-threaded, but they handle asynchronous operations by deferring work to the environment (e.g. the browser or Node's libuv) and then queueing callbacks. The ECMAScript spec doesn't define the event loop itself but references it for tasks like promises and modules. Hosts specify the details, but common behaviour is:

1. **Run-to-completion**: A script or callback runs until it finishes without interruption.
2. **Task queue**: When asynchronous operations (timers, I/O, user events) complete, their callbacks are queued as **tasks** (macrotasks). Examples include `setTimeout`, `setInterval`, DOM events and `requestAnimationFrame`.
3. **Microtask queue**: Promise reaction jobs and `queueMicrotask()` callbacks are queued as **microtasks**. At the end of each task, the environment processes the microtask queue until it is empty.
4. **Rendering**: In browsers, after microtasks, the rendering engine may update the UI. Then the event loop proceeds to the next task.

This order explains why `Promise.then()` runs before `setTimeout()` callbacks, and why microtasks added within microtasks run before the next macrotask.

### ## Property enumeration order

The specification also defines the order in which object properties are iterated. `Object.keys()`, `for...in` and `Object.getOwnPropertyNames()` list:

1. Integer index properties in ascending numeric order.
2. String-keyed properties in insertion order.
3. Symbol-keyed properties in insertion order.

This guarantees predictability when iterating over object keys.

### ## Module evaluation

ECMAScript modules are loaded and executed in dependency order. The spec describes how module records are created, linked and evaluated. The host ensures that modules are fetched and parsed; the spec ensures that imported bindings are initialised before executing module code. Modules execute in top-level scope with their own module context; evaluation is asynchronous when using dynamic `import()`.

## ## Practice questions

1. **Theory:** In what order are function arguments evaluated in JavaScript? Give an example where the evaluation order affects the result.
2. **Theory:** Describe what happens when a function calls itself recursively. How does the execution context stack handle nested calls?
3. **Coding:** Write a script that logs numbers using `setTimeout()`, promises and synchronous logs to demonstrate the order in which tasks and microtasks run. Explain the observed output.
4. **Theory:** Explain the property enumeration order for `for...in` loops. How does it differ for numeric keys versus string keys?
5. **Theory:** How does the event loop ensure that the UI remains responsive while JavaScript code executes? What happens if you queue too many microtasks without yielding control?



# Explain `Function.prototype.toString` and environment internal slots

## # `Function.prototype.toString()` and internal slots

JavaScript functions are objects with special behaviour defined by the ECMAScript specification. They provide useful methods and internal state. Two important aspects are the `Function.prototype.toString()` method—used to obtain a function's source code—and the **internal slots** that hold information like the environment where the function was created.

## ## What does `Function.prototype.toString()` do?

Every function instance inherits a `toString()` method that returns a string representation of its source code. For user-defined functions, this is the exact source text used to define the function, including whitespace and comments. If the function was created via the `Function` constructor, the returned string synthesises a function declaration named `anonymous`. For built-in functions (those implemented by the host), it returns a generic `[native code]` string.

Examples:

```
```js
function add(a, b) {
  return a + b;
}

console.log(add.toString());
// "function add(a, b) { return a + b; }"
```

```
const anon = function (x) {
  /* empty */
};

console.log(`${anon}`); // coercion calls toString
// "function (x) { /* empty */ }"
```

```
const sum = new Function("a", "b", "return a + b");
```

```

console.log(sum.toString());

// "function anonymous(a,b) {\nreturn a + b\n}"

console.log(Math.max.toString());

// "function max() { [native code] }"

...

```

The revision to `Function.prototype.toString()` in ES2018 requires engines to return the *_exact_* source text for user code, ensuring predictable serialisation for tools like formatters and transpilers. Using `eval()` on the returned string of a built-in function is always a syntax error—its body cannot be reconstructed.

Internal slots and environments

The ECMAScript specification defines objects using **internal slots**—hidden properties that cannot be accessed from user code. Function objects have several internal slots, including:

- **[[Environment]]** - the lexical environment (scope) where the function was created. This slot enables closures: the function remembers variables from its defining scope.
- **[[ECMAScriptCode]]** - the parsed bytecode for the function body.
- **[[Realm]]** - the realm (global object and intrinsics) where the function was created. Different realms have different copies of built-in constructors and methods.
- **[[Prototype]]** - the object's prototype, used for inheritance.

These slots are conceptual, not properties you can read or write. They explain how the language works under the hood. When you call a function, the engine creates a new execution context and binds the function's `[[Environment]]` as its outer scope. This allows inner functions to access variables defined outside their body.

Realms and globals

A **realm** is an execution context containing a global object and a set of intrinsic objects (`Array`, `Object`, etc.). When you evaluate code in a different realm (for example, in an `iframe`), functions created there have their `[[Realm]]` pointing to that realm. Their prototypes and constructors come from that realm's intrinsics. This explains why `instanceof` checks can fail across frames: each realm has distinct constructor functions.

Practice questions

1. **Theory:** What does `Function.prototype.toString()` return for user-defined functions, functions created with the `Function` constructor and built-in functions? Give an example of each.
2. **Theory:** Why does calling `eval()` on the result of `Math.max.toString()` throw a syntax error?
3. **Theory:** Describe the purpose of the `[[Environment]]` internal slot. How does it enable closures?
4. **Coding:** Write a function that captures a variable from its outer scope and use `toString()` to show the function's source. Explain why the function still has access to the captured variable when invoked later.
5. **Theory:** Explain what a realm is in JavaScript and how the `[[Realm]]` internal slot affects things like `instanceof` across iframes.

What is the Realms API and why might it matter for sandboxing?

The Realms API and sandboxing

JavaScript code normally runs in a single **realm**—an environment consisting of a global object, a global scope and a set of intrinsic constructors like `Array` and `Object`. In the browser, the default realm is the top-level `window`. If you import code into this realm, it shares your globals and intrinsics. For some applications, this sharing is undesirable: you might want to run third-party scripts in isolation, without giving them access to your global state. That's where the **Realms API** comes in.

What is a realm?

A realm is a separate instance of the JavaScript execution environment. Each realm has its own global object and its own copies of built-ins. Two objects from different realms have different constructors; for example, `Array.prototype` in one realm is not the same object as `Array.prototype` in another. Realms already exist implicitly: every browser iframe or Web Worker is a distinct realm. However, you can't easily create a new realm from code today; you must rely on iframes or workers.

The Realms (ShadowRealm) proposal

The **ShadowRealm** API is a TC39 proposal that allows you to create a new realm programmatically. Its purpose is to enable sandboxing and to execute code with a fresh set of intrinsics. A `ShadowRealm` instance exposes a method `evaluate(code)`, which runs the given string of JavaScript in that realm and returns the result. Functions and values can be passed into and returned from a `ShadowRealm`, but the objects themselves remain in their originating realm. This prevents code in the shadow realm from mutating the host's global object or prototypes.

Example usage:

```
``js
// Create a new shadow realm
const sr = new ShadowRealm();

// Evaluate code in the isolated realm
```

```

const result = sr.evaluate("1 + 2");
console.log(result); // 3

// Define a function in the current realm
function greet(name) {
  return `Hello, ${name}!`;
}

// Wrap it for the shadow realm
const wrappedGreet = sr.wrap(greet);

// Call it from within the shadow realm
const message = sr.evaluate("(" + wrappedGreet + ")('World')");
console.log(message); // "Hello, World!"
...

```

In this example, the code executed in the shadow realm cannot access the main global object or modify built-ins. `sr.wrap()` produces a callable proxy that marshals arguments and return values between realms.

Why realms matter for sandboxing

Sandboxing refers to running untrusted or user-supplied code in a controlled environment where it cannot interfere with or access sensitive data. Without a Realms API, developers resort to workarounds like `iframe` sandboxes`, `vm` modules` in Node.js or library-provided sandboxes (SES, Caja). These solutions add complexity and have performance or compatibility limitations.

The Realms API promises:

- **Isolation:** Code executed in a new realm gets its own global scope and intrinsic constructors, preventing prototype pollution of the host environment.
- **Controlled communication:** Only values explicitly passed in or returned by the realm boundary are shared. Objects remain in their original realm unless explicitly wrapped.

- **No DOM access:** In the browser, a shadow realm has no access to the DOM or Web APIs unless you pass in proxies. This makes it suitable for running untrusted scripts safely.
- **Portability:** Unlike `iframe` tricks, realms work consistently in any JavaScript host (browsers, Node.js). They require no markup or cross-origin restrictions.

As of this writing, the Realms API (specifically the `ShadowRealm` proposal) is still under development. It may evolve before standardisation, but the core idea—creating and isolating execution contexts programmatically—remains the same. When supported, realms could simplify sandbox implementations, plugin systems and testing environments.

Practice questions

1. **Theory:** What is a realm in JavaScript? How do realms in different iframes differ from each other?
2. **Theory:** Explain how the Realms API enables sandboxing. Why is separate global state important when running untrusted code?
3. **Coding:** Use an `iframe` or `Web Worker` to create a separate realm in a browser today. Demonstrate how objects created in one realm fail `instanceof` checks against constructors from another realm.
4. **Theory:** What limitations remain even when using a `ShadowRealm` for sandboxing? Consider things like network requests or CPU usage.
5. **Coding:** Sketch a function `runInShadowRealm(sourceCode)` that evaluates a string of code in a new realm and returns its result. Discuss how you would handle passing functions into and out of the realm safely.

How does the module resolution algorithm work in ESM?

How module resolution works in ECMAScript modules

When you write an `import` statement in JavaScript, the runtime must locate the referenced module. This process is called **module resolution**. The ECMAScript module (ESM) specification defines how module specifiers map to module files, but leaves room for host environments to implement details. Let's explore how browsers and Node.js resolve modules.

Specifier types

Module specifiers come in several flavours:

- **Relative specifiers** start with `./` or `../` and resolve relative to the importing module's location. For example, `import util from './utils.js'` loads a file named `utils.js` in the same folder as the current module.
- **Absolute specifiers** begin with `/` and resolve from the origin (in browsers) or file system root (in Node.js). For example, `import x from '/lib/module.js'` refers to `/lib/module.js` on your server.
- **Bare specifiers** are bare names like `react` or `lodash`. They reference packages installed in `node_modules` or provided by the host. Browsers do not natively resolve bare specifiers; bundlers or import maps are required.
- **Package import specifiers** start with `#` and resolve according to the package's `imports` field in its `package.json`. They are used for internal package modules.

Resolution in browsers

In browsers, relative and absolute specifiers are resolved straightforwardly using URLs. The imported path must include an explicit filename and extension (e.g., `.js`, `.mjs`, `.json`). If you omit the extension, the browser doesn't guess—it will throw an error. To use packages from npm in the browser, you need an **import map** or a bundler that rewrites bare specifiers to full URLs.

Example:

```
```html
<!-- index.html -->
```

```

<script type="module">
 import { sum } from "./math.js";
 console.log(sum(2, 3));
 // Without an import map, the following fails in the browser:
 // import React from 'react';
</script>
...

```

An **import map** lets you specify how bare specifiers should be resolved:

```

<<html
<script type="importmap">
 {
 "imports": {
 "react": "https://cdn.skypack.dev/react@18"
 }
 }
</script>
<script type="module">
 import React from "react";
 console.log(React.version);
</script>
...

```

### ## Resolution in Node.js

Node.js implements a more complex resolution algorithm for ES modules because it must support local files, packages and package exports. At a high level:

1. If the specifier is a file URL or a relative specifier, Node resolves it to a file on disk relative to the importing module's URL. If the file lacks an extension, Node searches for ``.js``, ``.json`` and ``.node`` (native addon) in that order.



2. If the specifier starts with ``#``, Node looks at the importing package's ``imports`` field in its ``package.json`` to find a matching entry. These package import specifiers allow defining internal module names that point to specific files.
3. Otherwise, the specifier is considered a bare specifier. Node resolves it as a package name in ``node_modules``. It finds the package directory, reads its ``package.json`` and checks the ``exports`` field to determine which file to load. If no ``exports`` field exists, Node falls back to CommonJS resolution rules: looking for the ``main`` field, ``index.js`` and so on. Node also honours ``type: "module"`` in ``package.json`` to know whether to treat ``.js`` files as ESM or CommonJS.

If none of these rules resolve the specifier, Node throws a "Module not found" error. Additionally, Node does not allow implicit directory imports: ``import x from './dir';`` fails unless ``.dir/index.js`` is specified or exported via ``package.json``.

## ## Resolution errors

Common resolution errors include:

- **Invalid module specifier:** The specifier doesn't conform to valid URL or package name syntax.
- **Module not found:** The file or package does not exist at the resolved location.
- **Unsupported directory import:** Attempting to import a directory without an explicit file.
- **Package path not exported:** When using package exports, the requested subpath isn't defined in the ``exports`` field.

Understanding the resolution algorithm helps avoid surprises—especially when publishing packages. Always specify your package's exports and imports fields, and avoid relying on implicit resolution rules that may change.

## ## Practice questions

1. **Theory:** Explain the difference between relative, absolute and bare module specifiers. Give examples of each and describe how they are resolved in the browser.
2. **Theory:** Why do browsers require file extensions on import specifiers while Node.js can search for ``.js`` and ``.json``? How do import maps help with bare specifiers in browsers?
3. **Coding:** Create a simple Node.js project with an ``index.mjs`` file and a ``math.js`` module. Import functions using relative and bare specifiers and observe how Node resolves them.

4. **Theory:** What purpose do the `exports` and `imports` fields in `package.json` serve? How do they influence module resolution?
5. **Coding:** Define an import map that maps `@utils` to `/scripts/utils/`. Use it in an HTML file to import a module with a bare specifier.

# What are decorators and how do they extend class behavior?

## # Decorators and extending class behaviour

**\*\*Decorators\*\*** are a proposed JavaScript feature that allows you to modify classes and their members (methods, fields, accessors) declaratively. A decorator is a function that runs at definition time and can observe, replace or extend the thing it decorates. Decorators are widely used in TypeScript and other languages (like Python) for metaprogramming. The ECMAScript decorators proposal is currently at stage 3 and may change before final standardisation.

### ## Basic idea

A decorator is applied using the `@`` syntax preceding a class or class member. For example:

```
``js
@sealed
class Person {
 @logged
 greet(name) {
 return `Hello, ${name}!`;
 }
}
``
```

Here, ``sealed`` and ``logged`` are decorator functions. They are invoked when the class definition is evaluated, **\*\*before\*\*** any instances are created. The decorators receive metadata about the target (the class or method) and can modify its behaviour.

### ## Class decorators

A **\*\*class decorator\*\*** is a function that takes the class constructor and context metadata. It can replace or extend the class by returning a new constructor or adding static properties.

```

```js
function sealed(target, context) {
  Object.seal(target.prototype);
  Object.seal(target);
}

@sealed
class Library {} // instances cannot add new properties
```

```

The decorator seals both the class and its prototype, preventing new properties from being added. If a decorator returns a new class, that class replaces the original in the scope where it was declared.

## ## Method decorators

Decorators on methods can wrap or modify the method. A method decorator receives the method (as a function), its kind (e.g. `'method'`), the name, and a context object with utilities. It can return a replacement function or access the original via `context.access`.

```

```js
function logged(value, context) {
  if (context.kind === 'method') {
    return function (...args) {
      console.log(`Calling ${context.name} with`, args);
      const result = value.call(this, ...args);
      console.log(`Result of ${context.name}:`, result);
      return result;
    };
  }
}

class Calculator {

```

```

@logged
def add(a, b): { return a + b; }
}

const calc = new Calculator();
calc.add(2, 3); // logs arguments and result
'''

```

Decorators can also define accessors (``get`/`set``) or fields. Field decorators can modify initial values or create reactive properties.

Extending class behaviour

Decorators are powerful because they run at definition time, allowing you to:

- * **Wrap methods for cross-cutting concerns.** Logging, memoisation, validation and deprecation warnings can be applied uniformly.

- * **Register metadata.** Decorators can store metadata about classes and members, enabling frameworks to perform dependency injection or routing.

- * **Enforce invariants.** Class decorators can freeze or seal classes, preventing accidental modifications.

- * **Implement mixins.** A decorator can augment a class with additional methods or properties, similar to a mixin, without explicitly inheriting from another class.

Because decorators modify definitions rather than runtime instances, they offer better ergonomics than manually wrapping each method. However, they should be used judiciously; excessive metaprogramming can make code harder to understand.

Practice questions

1. **Theory:** What is the difference between a class decorator and a method decorator? When does each run, and what arguments do they receive?
2. **Coding:** Implement a decorator `@readonly` that makes a method non-writable so that attempts to override it throw an error.

3. **Theory:** How can decorators be used to implement dependency injection or routing in a web framework? Describe the mechanism at a high level.
4. **Coding:** Create a decorator `@memoize` that caches the result of a method based on its arguments. Apply it to a method that performs an expensive calculation.
5. **Theory:** What are some potential downsides of using decorators extensively? How might they affect readability and debugging?

Explain WeakMap-based private fields vs native private fields () in classes

WeakMap-based private fields vs native private fields in classes

Before JavaScript introduced **private class fields**, developers used patterns like closures and **WeakMap** to emulate encapsulation. With the `#` syntax now available in modern JavaScript, it's worth comparing the two approaches.

WeakMap-based private fields

A WeakMap is a collection that maps objects to values without preventing garbage collection. To emulate private data, you create a WeakMap outside the class and use the instance as the key and the private data as the value. Because only code that has access to the WeakMap can retrieve the data, it isn't directly exposed on the instance.

Example:

```
``js
const _balance = new WeakMap();

class Account {
  constructor(initial) {
    _balance.set(this, initial);
  }

  deposit(amount) {
    _balance.set(this, _balance.get(this) + amount);
  }

  getBalance() {
    return _balance.get(this);
  }
}
```

```
const acc = new Account(100);
acc.deposit(50);
console.log(acc.getBalance()); // 150
...

```

Key characteristics:

- Privacy is enforced by closure scope: only the module or function that holds the WeakMap can access the data.
- WeakMap entries are removed when the object (key) is garbage-collected, avoiding memory leaks.
- Accessing the private data requires a `WeakMap.get()` call, which adds some overhead.
- The private data lives outside the instance; it cannot be inspected via reflection APIs like `Object.getOwnPropertyNames()`.

Native private fields (`#`` syntax)

ES2019 introduced **private class fields** using the `#`` prefix. These fields are declared directly in the class body and are truly private: they are not properties on the object and cannot be accessed outside the class.

```
```js
class Account {
 #balance;
 constructor(initial) {
 this.#balance = initial;
 }
 deposit(amount) {
 this.#balance += amount;
 }
 getBalance() {
 return this.#balance;
 }
}

```



```
}
```

```
const acc = new Account(100);
console.log(acc.#balance); // SyntaxError: Private field '#balance' must be declared
...
```

Features of native private fields:

- **Syntax level privacy:** Attempting to access a private field outside its class results in a syntax error. There is no way to circumvent this without modifying the class definition.
- **Performance:** Access to `#balance` compiles down to fast property lookups. There is no need for map lookups, so it is faster than using a `WeakMap`.
- **Encapsulation:** Private fields live on the instance itself (in an internal slot) rather than in an external structure. They are not enumerable, cannot be deleted and are not visible to reflection APIs.
- **Interop with inheritance:** Private fields are not inherited by subclasses. Each class defines its own private fields. If you need to share private state with subclasses, use `protected` patterns with symbols or methods.

## ## Comparison

| Aspect                 | WeakMap-based privacy                       | Native private fields                         |
|------------------------|---------------------------------------------|-----------------------------------------------|
| Definition location    | Outside the class (closure or module)       | Inside the class using <code>#`</code> syntax |
| Access syntax          | Methods call <code>weakMap.get(this)</code> | Use <code>this.#field</code> directly         |
| Garbage collection     | Entries automatically removed when key dies | Stored on instance internal slots             |
| Inheritance            | Data not directly accessible to subclasses  | Private fields not inherited                  |
| Performance            | Slight overhead of map lookups              | Direct access; faster                         |
| Encapsulation strength | Depends on closure and module scoping       | Enforced by the language (syntax error)       |

In modern code, prefer native private fields for clarity, performance and stronger encapsulation. `WeakMap` patterns remain useful when you need private data associated with objects you don't

control (for example, augmenting DOM elements), or when targeting environments without private field support.

## ## Practice questions

1. **Theory:** How does using a WeakMap provide privacy for instance data? What happens to the WeakMap entry when the instance is garbage-collected?
2. **Coding:** Rewrite the WeakMap example above using native private fields. Compare the syntax and readability.
3. **Theory:** Why are private fields not inherited in subclasses? How could you share state between a superclass and subclass while keeping it encapsulated?
4. **Coding:** Show how you might store additional data for a DOM element using a WeakMap. Explain why private fields cannot be used in that context.
5. **Theory:** What are potential downsides to using WeakMap for privacy when compared to native private fields? Consider discoverability, performance and maintenance.

# What are import assertions and why are they used?

## # Import assertions and why they are used

ES modules allow you to import code and data. In addition to JavaScript, runtimes like Node.js and browsers can import JSON, WebAssembly and other resources. Historically, support for these module types has varied and often required non-standard loaders or bundlers. **Import assertions** (also called **import attributes**) provide a standard way to convey metadata alongside the import specifier, ensuring that modules are interpreted correctly.

## ## What are import assertions?

An import assertion is a syntax that attaches an **attributes object** to an `import` statement. The attributes specify how the module should be treated. The most common use is to assert the module type for JSON imports.

Syntax:

```
``js
import data from "./data.json" assert { type: "json" };
``
```

In this example, the `assert { type: 'json' }` clause informs the host that the imported file should be parsed as JSON. If the host does not support JSON modules or if the assertion doesn't match the file's actual type, it throws an error.

Import assertions can also be used with dynamic imports:

```
``js
const module = await import("./strings.po", { assert: { type: "text" } });
``
```

The attributes are available as the second argument to `import()` in environments that implement import attributes. Note that the syntax is still evolving: Node.js 20 and browsers such as Chrome 91

support import assertions, but some environments may require flags or may remove them in favour of the more general **import attributes** proposal.

### ## Why use import assertions?

1. **Unambiguous module formats:** Without assertions, importing a file like `data.json` might accidentally be interpreted as JavaScript or cause conflicts with packages that also export JSON. Import assertions let the developer specify the expected format explicitly.
2. **Security and reliability:** By asserting the expected type, the runtime can reject mismatches early. For example, an attacker cannot trick your program into executing a script disguised as a `.json` file.
3. **Extensibility:** As new module types emerge (translation files, configs, images), import assertions provide a uniform mechanism to attach metadata. Tools and bundlers can use the assertions to choose appropriate loaders.
4. **Interoperability:** Node.js and browsers can unify behaviour around non-JavaScript modules. For example, Node's experimental JSON modules require `assert { type: 'json' }` to avoid breaking existing CommonJS `require('./data.json')` semantics.

### ## Example: importing JSON in Node.js

In Node.js 17 and later (with `type: 'module'` in `package.json`), you can import JSON modules if you provide a type assertion:

```
``js
// package.json contains { "type": "module" }
// data.json => { "name": "Alice", "age": 30 }

import person from "./data.json" assert { type: "json" };
console.log(person.name); // Alice

// Without the assertion, Node throws an error:
// ERR_UNSUPPORTED_ESM_URL_SCHEME: Only URLs relative to ...
...
```

As of Node 22, import assertions have been replaced by **`**import attributes**`** with a slightly different syntax. The concept remains the same: attach metadata to imports to ensure correct loading.

### ## Practice questions

1. **Theory:** What problems do import assertions solve when importing non-JavaScript modules? Why is it not enough to rely on file extensions alone?
2. **Theory:** Describe how import assertions improve security when loading JSON or other data files. What happens if the assertion does not match the actual module type?
3. **Coding:** Create a JSON file and import it in a Node.js module using an import assertion. Log its contents and observe what happens if you omit the assertion.
4. **Theory:** Compare import assertions in static `import` statements and dynamic `import()`. How is the syntax different?
5. **Theory:** Discuss how the import attributes proposal generalises import assertions. What advantages does it have over the earlier syntax?

# What is structuredClone's difference from deep copy via JSON?

## # `structuredClone()` vs deep copy via JSON

Copying objects in JavaScript can be tricky. A **shallow copy** duplicates only the top level of an object, while nested objects are shared. A **deep copy** recursively duplicates all nested structures. Two common approaches are using `JSON.stringify()`/`JSON.parse()` and using the built-in `structuredClone()` function. Although they both produce independent copies, they differ significantly in what they support and how they handle data.

### ## Deep copy via JSON

The simplest way to create a deep copy is to serialise an object to JSON and then parse it back:

```
``js
const original = { a: 1, b: { c: 2 } };
const copy = JSON.parse(JSON.stringify(original));
``
```

This technique works for plain objects containing numbers, strings, booleans, `null` and arrays. However, it has major limitations:

- **Unsupported types:** Functions, `undefined`, `Symbol` properties and objects containing circular references cannot be serialised. Dates become strings; Maps and Sets become empty objects; typed arrays and ArrayBuffers are not preserved.
- **Loss of metadata:** Property descriptors, prototypes and non-enumerable properties are lost. Instances of classes become plain objects.
- **Loss of type fidelity:** Special numeric values like `NaN` and `Infinity` are converted to `null`. BigInt values cause a `TypeError`.

Because of these constraints, JSON cloning is suitable for simple data structures but not for complex objects.

### ## `structuredClone()`

The `structuredClone()` function is a standard API that deeply copies most built-in types using the structured clone algorithm (the same algorithm used for posting messages to Web Workers). It can clone many kinds of data that JSON cannot, including:

- Dates, RegExps, Maps, Sets and other built-ins.
- Typed arrays (`Uint8Array`, `Float64Array`) and `ArrayBuffers`.
- Blobs, Files and `ImageBitmaps` in browser environments.
- Objects with circular references.
- Objects with custom prototypes (the prototype is preserved).

Example:

```
``js
const original = {
 date: new Date(),
 map: new Map([["key", 42]]),
 set: new Set([1, 2, 3]),
 nested: {},
};
original.nested.self = original; // circular reference

const clone = structuredClone(original);
console.log(clone.date instanceof Date); // true
console.log(clone.map.get("key")); // 42
console.log(clone.set.has(2)); // true
console.log(clone.nested.self === clone); // true
``
```

`structuredClone()` performs a deep copy, preserving most built-in types. It throws a `DataCloneError` if you attempt to clone unsupported types, such as DOM nodes, functions or certain host objects. It can also transfer ownership of transferable objects (`ArrayBuffers`, `MessagePorts`) to the cloned object using a `transfer` option, removing them from the original.

## ## When to use each

- **Use JSON cloning** for simple, serialisable data where you don't care about dates, methods or prototypes. It's fast, widely supported and easy to understand.
- **Use `structuredClone()`** for complex data structures, objects with circular references or built-in types beyond basic JSON. It preserves more fidelity and has explicit error handling for unsupported types.
- **Avoid cloning functions and DOM nodes.** If you need to duplicate behaviour, explicitly copy or recreate the function. DOM nodes are tied to their document; create new nodes instead.

## ## Practice questions

1. **Theory:** What limitations does cloning via `JSON.stringify()`/`JSON.parse()` have? Give examples of data that cannot be cloned correctly using this method.
2. **Theory:** Explain how `structuredClone()` handles circular references and built-in types like `Map` and `Set` compared to JSON cloning.
3. **Coding:** Write a function `deepCloneJSON(value)` that clones an object via JSON. Test it on an object containing a `Date` and observe how the date is transformed.
4. **Coding:** Use `structuredClone()` to clone an object with a `Map` and a circular reference. Verify that the clone retains the `Map` entries and that the circular reference points to the clone.
5. **Theory:** When would you choose JSON cloning over `structuredClone()` even though the latter is more capable? Consider browser support and performance.



# Explain lazy vs eager evaluation in iterables

## # Lazy vs. eager evaluation in iterables

Iterables are JavaScript objects you can loop over with `for...of`. Arrays, strings and Maps are examples of **eager** iterables: they compute their values immediately and hold them in memory. Generator functions and other iterator-based constructs are **lazy**: they produce values one at a time as they're requested. Understanding the difference between these strategies helps you choose the right tool for large datasets, infinite sequences or performance-sensitive tasks.

### ## Eager evaluation

- **Definition** - In an eager iterable all values are computed up front. For example, when you call `Array.from()` or the `map()` method, JavaScript creates a new array containing every transformed value immediately.
- **Memory implications** - Because the entire result is realized at once, eager evaluation can consume significant memory if the collection is large. An array of a million numbers occupies memory for all million elements even if you only ever use the first few.
- **When it shines** - Eager evaluation is straightforward and performant when you need the entire result and the dataset is reasonably small. Operations like sorting, reducing and random access (`arr[i]`) are trivial on an array because all values are available.

### ### Example: eager pipeline

```
``js
// doubling then filtering an array eagerly
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map((n) => n * 2); // [2,4,6,8,10]
const evens = doubled.filter((n) => n % 2 === 0); // [2,4,6,8,10] (all are even)
console.log(evens);
``
```

Here `map()` and `filter()` each produce a new array. Even if you ultimately only need the first few values, every intermediate result is computed.

## ## Lazy evaluation

- **Definition** - Lazy evaluation postpones computing a value until it's actually needed. In JavaScript, you implement laziness by returning an iterator: an object with a `next()` method that yields the next value and remembers its state between calls. Generators (`function*`) are the most common way to build lazy iterables.

- **Benefits** - Because lazy iterables produce values on demand, they avoid creating large intermediate arrays. This can reduce memory usage and improve responsiveness when working with streams, infinite sequences or expensive calculations. Lazy evaluation also allows you to compose operations without paying the cost until you consume the result.

- **Trade-offs** - Lazy iterables cannot be randomly indexed; they must be consumed sequentially. Each call to `next()` may involve computation, so repeated consumption of the same sequence requires either caching (memoization) or starting over.

### ### Example: lazy infinite sequence

```
``js
// generator producing an infinite sequence of natural numbers lazily
function* naturalNumbers() {
 let n = 0;
 while (true) {
 yield n++;
 }
}

const iterator = naturalNumbers();
console.log(iterator.next().value); // 0
console.log(iterator.next().value); // 1
console.log(iterator.next().value); // 2
// the sequence can continue indefinitely
...

```

The generator function `naturalNumbers` does not build an array of numbers. Each call to `next()` computes the next value and pauses until the next call. Because of this, you can represent potentially unbounded sequences without exhausting memory.

### ### Chaining lazy transformations

You can create helper functions that accept an iterable and return a new lazy iterable. Each helper yields values on demand, allowing you to build complex pipelines without intermediate arrays:

```
``js
// helper to map lazily
function* mapLazy(iterable, fn) {
 for (const value of iterable) {
 yield fn(value);
 }
}

// helper to filter lazily
function* filterLazy(iterable, predicate) {
 for (const value of iterable) {
 if (predicate(value)) yield value;
 }
}

// compose lazy operations
const numbersLazy = naturalNumbers();
const doubledLazy = mapLazy(numbersLazy, (n) => n * 2);
const evensLazy = filterLazy(doubledLazy, (n) => n % 4 === 0);

// consume only the first five values
for (let i = 0; i < 5; i++) {
 console.log(evensLazy.next().value);
}
```

```
}
...

```

Here the mapping and filtering operations are lazy; they don't compute anything until `next()` is called. The loop stops after five values without computing the rest of the infinite sequence.

### ### Real-world analogy

Imagine a bakery that sells loaves of bread. An **eager** bakery bakes every loaf at 5 a.m. even if only a few customers will buy them, wasting ingredients and shelf space. A **lazy** bakery bakes a loaf only when a customer orders it. If nobody orders bread, no loaves are baked. Similarly, lazy evaluation produces values only when requested, saving resources when you don't need the whole batch.

### ### Common misconceptions and pitfalls

- **"Lazy is always better"** - Lazy evaluation can save memory, but it's not free. Each call to `next()` involves overhead and may trigger a complex calculation. If you need all results anyway, the overhead of laziness can outweigh the benefits.
- **Single-use iterators** - Once you consume a lazy iterable with `for...of` or by calling `next()` repeatedly, it's exhausted. You must create a new iterator if you need to iterate again.
- **Error handling** - Lazy iterables can throw errors during iteration, not at creation time. Make sure to handle errors where you consume the iterator.

### ## Practice questions

1. **Theory:** Explain the difference between eager and lazy evaluation in your own words. What kinds of operations in JavaScript are eager by default, and what constructs enable lazy evaluation?
2. **Coding:** Write a generator function `primes()` that lazily yields prime numbers on demand. Then use it to print the first ten primes.
3. **Theory:** Discuss situations where lazy evaluation might hurt performance compared to eager evaluation. Give an example of an operation where an eager array method is simpler and more efficient than using generators.
4. **Coding:** Implement a lazy version of the `take(n, iterable)` function that returns an iterator yielding the first `n` values from any iterable without forcing the rest of the iterable to compute.

5. **Theory:** What happens if you try to iterate over a generator twice? How could you design an iterable that supports multiple passes over the same lazy sequence?

# What is monkey-patching and why is it discouraged?

## # Monkey patching in JavaScript and why it's discouraged

**\*\*Monkey patching\*\*** is the practice of dynamically modifying or extending existing code at run time. In JavaScript this usually means adding, overriding or deleting properties on built-in objects or modules after they have been loaded. While this flexibility can be powerful for shims, polyfills and test doubles, it also introduces hidden dependencies and fragile code. Understanding how monkey patching works will help you decide when (and when not) to use it.

## ## What is monkey patching?

At its core, JavaScript treats functions and objects as mutable. You can add methods to prototypes, replace existing functions or change modules on the fly:

```
``js
// adding a custom method to Array.prototype (monkey patch)
Array.prototype.sum = function () {
 return this.reduce((acc, val) => acc + val, 0);
};

const nums = [1, 2, 3];
console.log(nums.sum()); // 6

// overriding a built-in method
const originalDateNow = Date.now;
Date.now = () => 42; // pretend the current timestamp is always 42

console.log(Date.now()); // 42

// later restore the original implementation
Date.now = originalDateNow;
``
```

In testing frameworks like Jest or Sinon, monkey patching (also called **\*\*stubbing\*\*** or **\*\*spying\*\***) is used to replace network calls or timers with deterministic mocks. Libraries like [core-js] provide polyfills by monkey patching global objects when native implementations are missing.

## ## Why is it discouraged?

Although monkey patching can solve short-term problems, it carries significant risks:

- **\*\*Namespace pollution\*\*** - Adding methods to built-in prototypes affects all code in the environment. If different libraries define the same method with different semantics, they may conflict with each other.
- **\*\*Fragile upgrades\*\*** - When the JavaScript engine or library authors update their implementations, your patched code may break. A patched method may rely on internal behavior that changes between versions, leading to subtle bugs.
- **\*\*Debugging difficulty\*\*** - Monkey patches change behavior behind the scenes. Future maintainers may not realize that an object has been patched and will spend time chasing unexpected behaviour.
- **\*\*Security concerns\*\*** - Malicious scripts can monkey patch critical functions (e.g., intercepting `fetch()`) to steal data. Browsers and Node.js treat such modifications as part of the page, so there is no sandboxing of patched code.
- **\*\*Performance overhead\*\*** - Patching prototypes can de-optimize JavaScript engines. V8 and other engines optimize property lookups based on hidden classes and inline caches; adding unexpected properties invalidates these optimizations, causing slower property access.

## ### Legitimate uses

There are narrow cases where monkey patching is acceptable:

- **\*\*Polyfills\*\*** - Before ECMAScript 2015, developers used shims to add missing methods like `Array.prototype.forEach` or `String.prototype.startsWith`. These patches detect native support and only modify the prototype when necessary.
- **\*\*Testing and mocking\*\*** - During tests you may override functions like `fetch()`, timers or logging to provide predictable results. Frameworks that monkey patch do so within a controlled scope and restore original behaviour after tests.
- **\*\*Metaprogramming\*\*** - Libraries like Ember.js historically relied on prototype extensions to provide convenience methods. Modern best practice is to avoid global changes and instead import utilities explicitly.

## ## Best practices and safer alternatives

If you need to extend functionality, consider safer patterns:

- **Composition and wrappers** - Rather than modifying a function in place, write a wrapper that adds functionality and delegates to the original:

```
```js
function sumArray(arr) {
  return arr.reduce((acc, val) => acc + val, 0);
}

console.log(sumArray([1, 2, 3])); // 6
...
```
```

- **Subclassing** - For custom collections, derive your own class from `Array` or compose with existing objects instead of changing global prototypes.
- **Dependency injection** - Pass dependencies into functions so they can be replaced in tests without altering global state.
- **Import modifications explicitly** - If you must alter a module's behavior, create a module that wraps and re-exports the modified version rather than patching the original.

## ## Practice questions

1. **Theory:** Define monkey patching in the context of JavaScript. What are some of the potential problems it introduces?
2. **Coding:** Write a polyfill for `Array.prototype.includes` that adds the method only if it does not already exist. Explain why this pattern is safer than unconditionally overwriting the method.
3. **Theory:** Describe a scenario in which monkey patching a native object could lead to a security vulnerability.
4. **Coding:** In a testing environment, override `console.log()` so that it collects log messages into an array instead of printing them. After the test, restore the original implementation. Discuss why this approach is preferable to patching `console.log()` globally in production code.



5. **Theory:** How can dependency injection or higher-order functions eliminate the need for monkey patching when writing unit tests?

# How do JavaScript engines optimize tail calls (TCO)?

## # How JavaScript engines optimize tail calls (TCO)

Recursive functions are elegant but can blow up the call stack if they recurse deeply. **\*\*Tail call optimization\*\*** (TCO) is a technique used by compilers and interpreters to reuse stack frames when a function call occurs in a *\_tail position\_*—the last thing the function does before returning. When a call is tail-recursive, the caller doesn't need to do any further work after the callee returns, so the engine can replace the caller's frame rather than creating a new one. This effectively turns recursion into iteration and prevents stack overflow.

## ## Tail positions

A call is in **\*\*tail position\*\*** if its result is immediately returned by its caller, with no further computation. In JavaScript:

```
```\js
function sum(n) {
  if (n === 0) return 0;
  return n + sum(n - 1); // not a tail call - the addition happens after the call
}

function sumTail(n, acc = 0) {
  if (n === 0) return acc;
  return sumTail(n - 1, acc + n); // tail call - no pending work after the call
}
```\
```

In `sumTail()`, the recursive call is the last operation; there is no multiplication or addition after it. This makes it eligible for TCO.

## ## The ES6 specification and proper tail calls

ECMAScript 2015 introduced the concept of **proper tail calls** (PTC). The specification describes how compliant engines *may* reuse the current call frame when a tail call occurs, thus limiting stack growth. However, the spec does not require tail call optimization, and most mainstream engines (V8 in Chrome/Node, SpiderMonkey in Firefox) have not implemented PTC. Safari's JavaScriptCore briefly supported it but later removed support. As a result, relying on TCO in cross-platform JavaScript is unsafe.

### ### Why hasn't TCO been widely implemented?

Implementing tail call optimization in dynamic languages like JavaScript is challenging for several reasons:

- **Debugging semantics** - Reusing call frames changes the observable call stack. Developers expect stack traces to show each function in a recursion chain; TCO would hide those frames, making debugging harder.
- **Security and backwards compatibility** - Some existing code relies on detecting a call stack's depth or using `arguments.caller`/`Function.prototype.caller`` to inspect callers. Removing frames could break such code.
- **Engine complexity** - JavaScript engines perform many optimizations (inline caching, JIT compilation, de-optimisations). Supporting full TCO requires changes across interpreter and JIT pipelines.

For these reasons, engines have prioritized other optimizations. The ES committee has since reclassified proper tail calls as optional.

### ## Simulating TCO with trampolines or iteration

To avoid stack overflow in recursive algorithms, you can refactor code to be iterative or use a `_trampoline_` to repeatedly call functions without growing the stack:

```
``js
// Trampoline helper runs a function until it returns a non-function
function trampoline(fn) {
 let result = fn;
 while (typeof result === "function") {
 result = result();
 }
}
```

```

 }

 return result;
}

// Tail-recursive factorial using a thunk (function with no arguments)
function factorialThunk(n, acc = 1) {
 if (n === 0) return acc;
 return () => factorialThunk(n - 1, acc * n);
}

console.log(trampoline(() => factorialThunk(5))); // 120
...

```

The `factorialThunk` function returns another function instead of making a direct recursive call. The `trampoline` repeatedly invokes these thunks until it gets a number. This pattern keeps the call stack flat.

### ## Practice questions

1. **Theory:** What is a tail call? Give an example of a recursive function that is *not* in tail form and explain why it cannot be optimized.
2. **Coding:** Rewrite a recursive function to compute the *n*th Fibonacci number using an accumulator so that the recursive call is in tail position.
3. **Theory:** Why do most JavaScript engines not implement proper tail calls? Describe at least two challenges.
4. **Coding:** Implement a trampoline function that can take a self-recursive thunk and produce a result without growing the call stack. Use it to compute a large factorial (e.g., `factorial(1_000)` without a stack overflow).
5. **Theory:** Aside from TCO, what other techniques can you use in JavaScript to avoid stack overflow in recursive algorithms?

# What is generator delegation (yield) and how does it work?

## # Generator Delegation (`yield*`): How It Works

JavaScript's generator functions allow you to produce sequences of values on demand using the `yield` keyword. A generator is paused when it yields and resumes when the caller invokes its `next()` method. In complex scenarios you may want one generator to "hand off" iteration to another generator or iterable. The **`yield*`** (`yield*`) provides that capability: it lets a generator transparently pass through values from another iterable.

## ## What is generator delegation?

Normally, each `yield` in a generator returns a single value:

```
``js
function* numbers() {
 yield 1;
 yield 2;
 yield 3;
}

const it = numbers();
console.log(it.next()); // { value: 1, done: false }
console.log(it.next()); // { value: 2, done: false }
...

```

If you need to yield all values from another generator or iterable without writing a loop, use the `yield*` expression. The syntax is similar to a normal yield, but the asterisk signals delegation:

```
``js
function* g1() {
 yield 2;
}

```

```

yield 3;

yield 4;
}

function* g2() {
 yield 1;
 yield* g1(); // delegate to g1()
 yield 5;
}

```

```

for (const x of g2()) {
 console.log(x);
}

// 1 2 3 4 5
...

```

`yield* g1()` iterates over `g1()` and yields each value as if `g2()` had yielded them directly. When the delegated generator finishes, control returns to `g2()` and it continues yielding its own values. The `yield*` expression itself evaluates to whatever value the delegated iterator's final `return` produced, which can be captured if needed.

### ### Delegating to other iterables

`yield*` works with any iterable object, not just generators. Arrays, strings, Sets, Maps and even the `arguments` object can be delegated:

```

```js
function* g3() {
  yield* [1, 2]; // yields 1, then 2
  yield* "34"; // yields '3', then '4'
  yield* arguments; // yields any extra arguments passed to g3()
}

```

```
const it = g3(5, 6);  
console.log([...it]); // [1, 2, '3', '4', 5, 6]  
...
```

Capturing the return value

A delegated generator can return a value using `return`. The `yield*` expression will evaluate to that returned value, allowing the delegating generator to capture it:

```
``js  
function* inner() {  
  yield 1;  
  yield 2;  
  return "done";  
}  
  
function* outer() {  
  const result = yield* inner();  
  console.log("Inner returned:", result);  
}  
  
[...outer()];  
// logs: Inner returned: done  
...
```

Why use generator delegation?

Composition of generators

`yield*` lets you build complex sequences by composing smaller generators. Instead of manually looping over a subgenerator and yielding each value, you can delegate and let the language handle

iteration. This is particularly helpful when you need to flatten nested generators or forward values from helper functions.

Simplifying iteration logic

Consider a tree traversal generator that needs to walk child nodes. Without delegation you would write an explicit loop inside the parent generator. With ``yield*``, your generator reads more declaratively:

```
``js
function* traverse(node) {
  yield node.value;
  for (const child of node.children) {
    yield* traverse(child);
  }
}

// traverse the tree and print all values
for (const value of traverse(root)) {
  console.log(value);
}
``
```

Real-world analogy

Imagine you are telling a story but need to include a detailed anecdote told by your friend. Instead of retelling the anecdote yourself, you hand the microphone to your friend (delegate) so they can speak directly. When they finish, you take back the microphone and continue your story. That is what ``yield*`` does for generators: it hands control to another iterator and then resumes when it's finished.

Common pitfalls and misconceptions

- **Using `yield` instead of `yield*`** - If you write `yield g1()` instead of `yield* g1()`, you will yield the generator object itself, not the values produced by it. Always use the asterisk when delegating.
- **Delegating to non-iterables** - The operand to `yield*` must be iterable (it must have a `Symbol.iterator` method). Passing a plain object without an iterator will cause a runtime error.
- **Ignoring the return value** - The value of a `yield*` expression is the return value of the delegated iterator. If the subgenerator returns a value you care about, store it; otherwise you can ignore it.

Practice questions

1. **Theory:** In your own words, explain the difference between `yield` and `yield*`. When would you choose one over the other?
2. **Coding:** Write a generator `flatten` that takes a nested array of numbers (e.g., `[1, [2, 3], 4, [5]]`) and yields the numbers in a flat sequence using generator delegation.
3. **Theory:** What will the `yield*` expression evaluate to if the delegated generator does not explicitly return a value? How can you capture this value?
4. **Coding:** Using `yield*`, implement a generator that yields the Fibonacci sequence up to a given count by delegating to a helper generator that yields successive values.

What are async generators and how are they used with for-await-of?

Async Generators and `for await...of`

Most asynchronous operations in JavaScript—such as network requests, file reads or timers—return promises that resolve sometime in the future. While `Promise.all()` and callbacks work for one-off operations, they become unwieldy when dealing with streams of asynchronous values. **Async generators** bridge this gap by combining generator syntax with asynchronous control flow, and **`for await...of`** provides a simple way to consume these asynchronous sequences.

What are async generators?

An async generator is defined with `async function*`. It looks like a regular generator (`function*`), but it can contain `await` expressions and can yield promises. When you call an async generator, it returns an **async iterator**—an object with an asynchronous `next()` method that returns a promise resolving to `{ value, done }` objects. Each `yield` in the generator produces a value, and `await` pauses until the awaited promise settles.

Here's a simple async generator that waits one second between numbers:

```
``js
async function* countWithDelay(n) {
  for (let i = 1; i <= n; i++) {
    // simulate an asynchronous delay
    await new Promise((resolve) => setTimeout(resolve, 1000));
    yield i;
  }
}
...

```

Calling `countWithDelay(3)` returns an object whose `next()` method returns promises. Each call to `next()` waits for the delay before resolving with the next number. Without dedicated syntax, consuming such an async iterator requires promise chaining:

```

```js
const it = countWithDelay(3);

it.next()

 .then(({ value }) => {
 console.log(value); // 1 after 1 s
 return it.next();
 })
 .then(({ value }) => {
 console.log(value); // 2 after 2 s
 });
```

```

`for await...of` - consuming async iterables

ES2018 introduced the `for await...of` loop to simplify consumption of **async iterable** objects. An object is async iterable if it has a `Symbol.asyncIterator` method that returns an async iterator. All async generators are async iterables by default. Inside an async function (or at top level of a module in some environments), you can iterate over them using `for await...of`:

```

```js
async function demo() {
 for await (const value of countWithDelay(3)) {
 console.log(value);
 }
 console.log("Done");
}

demo();

// Logs 1, 2, 3 at one-second intervals, then 'Done'
```

```

The loop pauses after each iteration until the promise returned by `next()` resolves. Unlike `Promise.all()`, which runs operations concurrently, `for await...of` processes values sequentially, making it ideal for streaming APIs or when order matters.

Async iterable sources

Apart from async generators, many browser and Node.js APIs expose async iterables:

- **Readable streams:** The Fetch API's `Response.body` is a `ReadableStream` whose reader is an async iterable. You can iterate over chunks of data as they arrive.
- **File handles:** In Node.js, the `fs` module provides asynchronous iteration over directory entries (`fs.opendir()`) and file contents.
- **Custom async iterables:** Any object implementing `Symbol.asyncIterator` and returning an object with an async `next()` method can be consumed via `for await...of`.

Combining async and sync iterables

`for await...of` can also iterate over synchronous iterables. The specification says that if an object does not have a `Symbol.asyncIterator` method but has a `Symbol.iterator`, the loop will use the synchronous iterator. This means you can write loops that consume values regardless of whether the source is synchronous or asynchronous.

Use cases

- **Streaming data** - reading lines from a network socket or file one by one without loading the entire content into memory.
- **Paginated APIs** - iterating through pages of API results by awaiting each page's promise.
- **Event streams** - yielding values from an event emitter as events occur.

Real-world analogy

Imagine you are waiting for deliveries from an online store. Each package arrives at different times. Instead of standing by the door constantly checking for deliveries, you set up a system where you wait for the courier to ring and then take each package as it comes. `for await...of` behaves like that system: it waits for each promise to resolve (the courier rings) and then yields the value (the package) before moving to the next.

Common pitfalls and misconceptions

- **Only inside async contexts** - You can use `for await...of` only inside an `async` function or at the top level of an ES module. Using it in a normal function causes a syntax error.
- **Sequential execution** - The loop waits for each iteration to finish before proceeding. If you want concurrency, collect promises in an array and await them with `Promise.all()`.
- **Error handling** - Wrap your `for await...of` loop in a `try...catch` to handle rejections. If the async iterator's `next()` method rejects, the rejection will propagate and exit the loop unless caught.

Practice questions

1. **Theory:** What is the difference between a synchronous generator and an async generator? How do their `next()` methods behave?
2. **Coding:** Write an async generator `readLines(url)` that fetches a text file via `fetch()`, splits it by newline, and yields each line after a short delay using `await`.
3. **Theory:** Why must `for await...of` be used inside an async function? What happens if you try to use it in a regular function?
4. **Coding:** Create a custom object with a `Symbol.asyncIterator` method that yields three values with different delays. Iterate over it with `for await...of` and log the order of completion.

Explain top-level await in ES modules

Top-Level `await` in ES Modules

`await` normally appears inside `async` functions. Until recently, if you needed to perform asynchronous initialization at the top of a module, you had to wrap your logic in an async function or chain promises. ****Top-level `await`**** changes that: it allows the `await` keyword at the top level of an ES module, turning the module into an implicitly asynchronous task. This capability simplifies module initialization and dynamic imports but comes with important semantics to understand.

What is top-level `await`?

In a traditional script or CommonJS module, using `await` outside of an `async` function results in a syntax error. Top-level `await` is a feature of ES modules that relaxes this rule. When a module contains a bare `await`, the JavaScript engine treats the module's execution as if it were wrapped in an async function. The module will pause at the `await` expression until the operand settles, then resume execution. Modules that import the awaiting module will wait for its evaluation to complete before continuing.

Consider a module `db.js` that establishes a database connection:

```
```.js
// db.js — note the top-level await
const connection = await connectToDatabase();
export default connection;
...`
```

When another module imports `db.js`, it must wait for `connectToDatabase()` to resolve before it can use the default export. The top-level await ensures that consumers see a fully initialized connection.

## ## How top-level `await` affects module evaluation

ES modules are evaluated in dependency order. Normally, a module's code runs synchronously once its dependencies are linked. When a module contains a top-level await, evaluation becomes asynchronous:

- **Execution pauses at `await`.** The module's evaluation returns a promise that resolves when all awaited promises within the module have settled.
- **Importing modules wait.** A module that imports a top-level-awaiting module must wait until that module's promise resolves before running its own module body. This ensures dependent modules see the awaited exports in their final state.
- **Sibling modules continue.** If two modules import the awaiting module, they both pause at their import statements until the awaited module resolves. Once resolved, each module continues evaluation independently.

This mechanism is similar to treating the entire module as an `async function`—all imports of the module implicitly `await` its promise.

## ## Use cases

Top-level await simplifies a few patterns that previously required workarounds:

### ### Dynamic module loading based on runtime data

```
``js
// i18n.js
const lang = navigator.language;
const strings = await import(`./locale/${lang}.js`);
export default strings.default;
...

```

Without top-level await you'd need to wrap this in an async function and export a promise. Now the module can return the actual data.

### ### One-time asynchronous initialization

Modules often need to read configuration files, open database connections or prefetch data before exporting functions. Top-level await lets you perform those actions inline:

```
``js
```

```
// cache.js
const cache = await fetch("/api/bootstrap.json").then((r) => r.json());
export function getItem(key) {
 return cache[key];
}
...

```

Consumers of `cache.js` know that `getItem()` will always read from an initialized cache.

### ### Fallback imports

You can attempt to import a preferred module and fall back gracefully if it fails:

```
```js
let parser;
try {
  parser = await import("./fast-parser.js");
} catch {
  parser = await import("./slow-parser.js");
}
export default parser;
...

```

Limitations and warnings

- **Modules only.** Top-level await is allowed only in ES modules. It cannot be used in classic `

- ****Unsupported environments.**** Some older browsers and Node.js versions do not support top-level await. Node added support in v14.8.0 behind a flag and in v16 as a default; always check your runtime environment.

Real-world analogy

Picture a cooking show where the host needs to marinate meat before continuing with the recipe. In earlier seasons, the show cut away to a pre-prepared dish while the meat marinated off camera. With top-level await, the host simply waits in real time, and the rest of the kitchen staff (importing modules) pauses until the marination finishes. Once done, everyone proceeds with fully prepared ingredients.

Practice questions

1. ****Theory:**** Why can't you use ``await`` at the top level of a classic script? What must you do to enable it?
2. ****Coding:**** Create an ES module that fetches JSON data at the top level using ``await`` and exports a function that returns a property from the fetched object.
3. ****Theory:**** Describe how top-level await affects the evaluation order of modules that import a module containing it. What happens if multiple modules import it simultaneously?
4. ****Coding:**** Write two ES modules, ``A.js`` and ``B.js``, where ``A.js`` awaits a promise at the top level and ``B.js`` imports and uses ``A.js``. Show how ``B.js`` waits for ``A.js`` to finish evaluation before executing.

How does module caching work in ES modules and CommonJS?

How does module caching work in ES modules and CommonJS?

JavaScript has two primary module systems: **CommonJS (CJS)** used historically in Node.js, and **ECMAScript modules (ESM)** which are part of the language spec and supported in modern browsers and Node. Both systems cache modules after they have been loaded, but the caching mechanisms and semantics differ. Understanding how caching works helps avoid subtle bugs when modules depend on each other or when you want to reload code dynamically.

Module execution and caching in CommonJS

In Node.js, when you `require()` a module for the first time the runtime:

1. **Resolves the specifier** to an absolute file path (e.g. `./utils.js` → `/full/path/utils.js`). File extensions like `.js`, `.json` and `.node` are tried automatically if none is provided.
2. **Loads and executes** the module code in its own wrapper function. During execution the module can populate `module.exports` or `exports` with values to expose.
3. **Caches the result** in `require.cache` keyed by the resolved filename. Subsequent `require()` calls for the same file return the same `module.exports` object without re-executing the module code.

Because the module code runs only once, any side effects (like logging, reading files, connecting to a database) happen a single time. Mutating the exported object also changes the instance that other modules receive because everyone references the same cached object. You can manually clear the cache entry by deleting it from `require.cache`, but this is rarely done in production because it may lead to inconsistent state.

The caching system also enables **cyclic dependencies**. When two modules require each other, Node will return a partially constructed `exports` object for the module still loading. This allows both modules to finish executing, but any values referenced before they are assigned will be `undefined`. See the circular dependencies section for details.

Module caching in ECMAScript modules

ESM was designed with static structure and live bindings. When a module is imported the first time using `import` or `import()`:

1. **Resolving and loading**: The module specifier is resolved using browser rules or Node's ESM resolution algorithm. ESM specifiers do not attempt file extensions unless you configure import maps or package exports; you must specify the full path (`./utils.js`) or rely on a package's `exports` field.
2. **Instantiation**: The module code is parsed, declarations are created and live bindings are set up. Import statements create read-only references to exported values; if the exporting module later changes the value, all importers see the new value.
3. **Evaluation**: The top-level code runs once. Unlike CJS, ESM does not wrap code in a function scope; variables declared at the top level belong to the module. After evaluation, the module is placed in the host's internal module cache.

Like CommonJS, subsequent imports of the same ESM module return the same module namespace object; evaluation never repeats. However, ESM does not expose a public cache that you can modify. Module namespace objects are immutable and there is no standard way to clear or reload a module. Dynamic `import()` returns a promise that resolves to the module namespace; if the module has already been evaluated it resolves immediately. Re-loading code usually requires changing the module's URL (e.g. adding a query string or hash) or restarting the environment.

Key differences

| Aspect | CommonJS | ES modules |
|---------------------|--|---|
| Spec resolution | Node automatically adds extensions and searches <code>node_modules</code> . | Resolution is strict: file extensions and file names must match unless configured via <code>'exports'</code> or import map. |
| Export semantics | <code>'module.exports'</code> is a single object; values are copied by reference when required. | <code>'export'</code> provides live bindings; imported values update when the original changes. |
| Caching API | Cached modules stored in <code>'require.cache'</code> ; can be modified or cleared manually. | Module cache is internal; no standard API to clear entries. |
| Wrapper and context | Module code is wrapped in a function (<code>function(exports, require, module, _filename, _dirname) {...}</code>). | Module code executes in its own scope; top-level <code>'this'</code> is <code>'undefined'</code> . |

Example: demonstrating CommonJS caching

```
```\n// logger.js (CommonJS)\nconsole.log("logger loaded");\nmodule.exports = { time: Date.now() };\n\n// app.js\nconst logger1 = require("./logger");\nsetTimeout(() => {\n  const logger2 = require("./logger");\n  console.log(logger1.time === logger2.time); // true\n}, 1000);\n```\n
```

When `logger.js` is first required, it logs "logger loaded" and exports the current timestamp. Even after one second the second `require()` returns the same object, so comparing the two times yields `true`. The module isn't re-executed because it is retrieved from the cache.

### ## Example: demonstrating ESM caching

```
```\n// utils.js (ESM)\nexport let counter = 0;\nexport function increment() {\n  counter++;\n}\n\n// app.mjs\nimport { counter, increment } from "./utils.js";\nconsole.log(counter); // 0\nincrement();\n```\n
```

```

console.log(counter); // 1 (live binding updates)

const ns = await import("./utils.js");

console.log(ns.counter); // 1 (same module instance)

...

```

`utils.js` is executed only once. The `counter` variable is exported as a live binding. When `increment()` is called, the change is visible to all importers. Using dynamic `import()` later returns the same module namespace object; the module isn't re-run.

Best practices and pitfalls

- **Don't mutate imported bindings** - In ESM, imported variables are read-only. To change shared state, export functions or objects that encapsulate state.
- **Avoid relying on module side effects** - Because modules run once, any side effects occur only during initial load. Avoid modules that perform critical actions at import time; instead, provide explicit functions to call.
- **Be mindful of cycles** - Cyclic dependencies can return incomplete exports or cause deadlocks (especially with top-level await). Refactor to remove cycles or defer access until after modules have finished initializing.
- **Hot reloading** - There's no portable way to unload and reload ESM modules. Development tooling often works by spinning up new workers or altering import URLs with cache-busting query strings.

Practice questions

1. **Theory:** What happens if you require the same CommonJS module multiple times in different files? Explain why the module's code is not executed again.
2. **Theory:** How do live bindings in ES modules differ from the values exported by CommonJS modules?
3. **Coding:** Create two CommonJS modules (`a.js` and `b.js`) that require each other. Show how partial exports are returned during loading and how you can safely use the values after both modules finish executing.
4. **Coding:** Write a small ESM program where one module exports a counter and another imports it twice—once statically and once via dynamic `import()`. Demonstrate that both imports refer to the same counter value.
5. **Theory:** Why is there no standard API to clear the ESM module cache, and what strategies exist for reloading code in an ESM environment?

What happens in circular module dependencies in JavaScript?

What happens in circular module dependencies in JavaScript?

A **circular dependency** occurs when two or more modules depend on each other, directly or indirectly. For example, `module A` imports something from `module B`, and `module B` imports something back from `module A`. Circular dependencies are legal in both CommonJS and ECMAScript modules, but they behave differently. Understanding these differences helps you avoid `undefined` values, deadlocks and other subtle bugs.

Circular dependencies in CommonJS

In Node's CommonJS system, modules execute synchronously when first `require()`d. When a module encounters a `require()` call to another module, it pauses execution, resolves and loads the other module, and then resumes. If two modules require each other, Node breaks the cycle by providing a **partially filled `exports` object** to the dependent module while the first module continues loading.

Example

```
``js
// a.js
console.log("a starting");
const b = require("./b");
module.exports = {
  name: "module A",
  bName: b.name,
};
console.log("a loaded");

// b.js
console.log("b starting");
const a = require("./a");
```

```

module.exports = {
  name: "module B",
  aName: a.name,
};

console.log("b loaded");

// main.js
const a = require("./a");
const b = require("./b");
console.log(a, b);
...

```

When `a.js` requires `b.js`, Node starts evaluating `b.js`. In turn, `b.js` requires `a.js`, but `a.js` hasn't finished executing. Node returns the current incomplete `exports` object for `a.js` (which only contains `name: 'module A'`). Once both files finish executing, the exports are fully populated. Running `main.js` logs:

```

``text
a starting
b starting
b loaded
a loaded
{ name: 'module A', bName: 'module B' } { name: 'module B', aName: 'module A' }
...

```

Notice that both modules eventually receive the correct names, but if you tried to access `b.bName` inside `b.js` before `b.js` finished executing, it would be `undefined`. To avoid issues, design modules so that any values imported from other modules are not needed during initial execution. Typically you should export functions or classes rather than direct values that need to exist immediately.

Circular dependencies in ES modules

ES modules handle cycles differently. Import statements are hoisted and evaluated before any code runs; the module loader sets up **live bindings** for all imported and exported names. When a module imports another module that imports it back, both modules initialise their export bindings to `undefined` and then execute their top-level code. As exports are assigned, the imported bindings update automatically. Because of this design, most cycles resolve correctly as long as you don't use imported variables before they are initialised.

Example

```
``js
// a.mjs
import { getName as getBName } from "./b.mjs";
export function getName() {
  return "module A";
}
export const bName = getBName();

// b.mjs
import { getName as getAName } from "./a.mjs";
export function getName() {
  return "module B";
}
export const aName = getAName();

// main.mjs
import { bName } from "./a.mjs";
import { aName } from "./b.mjs";
console.log({ aName, bName });
...

```

This example works because the imported functions are called after both modules assign their exports. Each module exports a function (`getName`) and uses the other module's function to define a constant. When `getName()` is called, the function refers to its own module's code, which has already been defined. The output is:

```
``text
{ aName: 'module A', bName: 'module B' }
``
```

If you instead imported a variable that hasn't been assigned yet, you'd get `undefined`. So it's important to export functions or objects and defer calling them until after module initialisation.

Top-level await and cycles

Starting with modern JavaScript, ES modules can use top-level `await`. This allows asynchronous code at the top level of a module but also introduces a new form of deadlock. When a module uses top-level `await`, importing modules must **`wait`** for it to resolve before continuing execution. If two modules with top-level `await` import each other and each awaits a promise that depends on the other module's completion, neither will ever resolve. The environment throws a `TypeError: Circular dependency detected while resolving promise` in these cases.

To avoid deadlocks, avoid using top-level `await` in mutually dependent modules. Instead, move asynchronous operations into functions that the importer can call after initialisation or restructure the module dependencies to remove the cycle.

Best practices

- **Minimise cycles** - Circular dependencies make code harder to reason about and can lead to undefined values or deadlocks. Refactor code into smaller modules or move shared code into a third module both depend on.
- **Export functions and classes** - Avoid exporting computed values that depend on the other module's exports. Use functions to provide values when needed.
- **Be cautious with top-level code** - In CommonJS, any code at the top level runs immediately; in ES modules it runs once at import time. Don't perform heavy work or rely on imported values before they exist.
- **Prefer dynamic import in asynchronous cycles** - If you must load a module lazily to break a cycle, use `import()` within a function rather than a static `import`. This defers loading until runtime and avoids the static cycle.

Practice questions

1. **Theory:** How does Node.js handle a circular dependency between two CommonJS modules? What will be returned when one module requires the other before it has finished executing?
2. **Theory:** In ES modules, why does importing a function from a module involved in a cycle work, while importing a yet-to-be-initialised variable often results in `undefined`?
3. **Coding:** Write two CommonJS modules that depend on each other and demonstrate a case where accessing a value too early results in `undefined`. Then refactor the code to avoid the issue.
4. **Coding:** Create a pair of ES modules with top-level `await` that import each other and intentionally cause a deadlock. Observe the error and then modify the code to remove the deadlock by moving the awaits into a function.
5. **Theory:** Explain how top-level `await` can lead to deadlocks in circular dependencies and list strategies to prevent such situations.

Explain bare imports and import maps in browsers

Explain bare imports and import maps in browsers

When writing modular JavaScript in browsers, you typically use **relative** or **absolute** URLs for your module specifiers:

```
``js
import helper from "./utils/helper.js";
import library from "https://cdn.example.com/lib.js";
``
```

These specifiers tell the browser exactly where to fetch a module. However, many Node.js or bundler-based codebases use **bare import specifiers** like `react` or `lodash` that do not contain `./`, `../`, or a URL. Historically, browsers could not resolve these names on their own because there was no mapping from the bare name to a network path. **Import maps** solve this problem by letting developers tell the browser how to resolve bare specifiers to actual URLs.

What are bare imports?

A **bare import** is an import specifier that is not relative or absolute. Examples include:

```
``js
import React from "react";
import { useState } from "preact/hooks";
import theme from "app/theme";
``
```

Browsers don't know where to fetch `react`; they need a URL. Tools like webpack, Vite or Rollup handle this during bundling by resolving module names to files in `node_modules`. For browser-native modules, import maps provide a declarative way to achieve similar resolution.

What is an import map?

An **import map** is a JSON object embedded in a `<script type="importmap">` tag. It defines one or more maps—`imports` and optionally `scopes`—that associate bare specifiers with URLs. At page load, the browser reads the import map and uses it whenever it resolves module specifiers in subsequent `import` statements or `import()` calls.

Basic example

```
``html
<!DOCTYPE html>

<html>

  <head>

    <!-- Define the import map before any modules load -->

    <script type="importmap">

      {
        "imports": {
          "react": "https://cdn.skypack.dev/react@18.2.0",
          "react-dom": "https://cdn.skypack.dev/react-dom@18.2.0",
          "app/": "/static/app/"
        }
      }
    </script>

    <script type="module" src="/static/app/main.js"></script>

  </head>

  <body></body>

</html>
``
```

In this example, any `import 'react'` in `main.js` or its dependencies will resolve to `https://cdn.skypack.dev/react@18.2.0`. Imports starting with `app/` are resolved relative to `/static/app/`. The import map must be declared **before** the module scripts that depend on it; otherwise the browser won't apply it.

Scoped import maps

You can define different resolutions depending on which module imports the specifier using the optional ``scopes`` property. A scope is keyed by a parent module URL and contains its own ``imports`` mapping. This is useful when two parts of your application need different versions of the same dependency.

```
```html
<script type="importmap">
{
 "imports": {
 "lit": "https://unpkg.com/lit@2/index.js"
 },
 "scopes": {
 "/admin/": {
 "lit": "https://unpkg.com/lit@1/index.js"
 }
 }
}
</script>
```
```

Modules under ``/admin/`` will receive version 1 of Lit, while others receive version 2.

When to use import maps

Import maps are useful when you want to:

- **Use bare specifiers in the browser** without a bundler.
- **Pin versions of external libraries** hosted on a CDN.
- **Alias local paths** to shorter specifiers (e.g. map ``@components/`` to ``/src/components/``).
- **Experiment with CDNs** or new package versions without changing source code.

They work well for small or medium projects, demos and quick prototypes. For large applications with hundreds of modules, bundlers still offer performance benefits (tree shaking, bundling multiple files into a single HTTP request).

Limitations and considerations

- **Single import map:** Only the first import map is processed. Additional import maps are ignored.
- **Must appear before modules:** The import map must be defined before any `<script type="module">` that uses bare specifiers; otherwise the specifiers won't resolve.
- **Spec applies only to module imports:** Import maps affect `import` statements and dynamic `import()`. They do not apply to `<script src="...">`, `worker new Worker()`, or other non-module resource loads.
- **Browser support:** As of 2025, major Chromium-based browsers support import maps. Firefox has partial support behind a flag, while Safari support is experimental. Use a feature detection or a polyfill for wider compatibility.

Practice questions

1. **Theory:** What is a bare import specifier, and why can't browsers resolve it without help?
2. **Theory:** Describe the purpose of an import map's `imports` and `scopes` sections. When might you use a scoped import map?
3. **Coding:** Write a small HTML page that defines an import map mapping `lodash` to a CDN URL. Then write a module that imports `lodash` and uses it to merge two objects.
4. **Coding:** Modify the previous example to use a scoped import map so that `/admin/dashboard.js` imports a different version of `lodash` than the rest of your site.
5. **Theory:** List some limitations of import maps and explain why a bundler might still be preferred for large applications.

What are custom error classes and how do you create them?

What are custom error classes and how do you create them?

JavaScript's built-in error types—such as `Error`, `TypeError` and `RangeError`—represent common programming mistakes. In real-world applications you often need to distinguish different kinds of failure conditions: for example, an error when a user cannot be found is different from an error when a network request fails. **Custom error classes** let you define your own error types with meaningful names and additional information, making error handling more precise and expressive.

Why create custom errors?

- **Clarity:** A specific error class conveys intent better than a generic `Error`. It's easier to catch and handle `ValidationError` or `PermissionError` separately from unrelated issues.
- **Pattern matching:** You can use `instanceof` to check for a particular error type and recover accordingly.
- **Additional context:** Custom errors can carry extra properties (like an error code or user ID) to help debugging or return more information to callers.

Extending the `Error` class

To define a custom error, create a class that extends the built-in `Error` and call `super(message)` in the constructor. Set the `name` property to the class name so that stack traces show the correct type. You can also define additional properties.

```
```js
class ValidationError extends Error {
 constructor(message, code) {
 super(message); // call the parent constructor with the message
 this.name = "ValidationError";
 this.code = code; // custom property
 }
}
```



```

function processUser(user) {
 if (!user.email) {
 throw new ValidationError("Email is required", "MISSING_EMAIL");
 }
 // process user...
}

try {
 processUser({ name: "Bob" });
} catch (err) {
 if (err instanceof ValidationError) {
 console.error(`Invalid input: ${err.message} (code ${err.code})`);
 } else {
 throw err; // rethrow unexpected errors
 }
}
...

```

In the example above, throwing a `ValidationError` allows the caller to distinguish input validation problems from other errors. The custom property `code` holds a machine-readable error code.

### ## Using the `cause` property

ES2022 introduced an optional `cause` property on errors. You can provide another error as the cause when constructing your custom error:

```

```js
class DatabaseError extends Error {
  constructor(message, options) {
    super(message, options);
    this.name = "DatabaseError";
  }
}

```

```
}
```

```
async function saveRecord(record) {  
  try {  
    await db.insert(record);  
  } catch (err) {  
    throw new DatabaseError("Failed to save record", { cause: err });  
  }  
}  
...  
}
```

Now, `DatabaseError` instances include a `cause` property referencing the original error, preserving the error chain for debugging.

Best practices for custom errors

- **Always extend `Error`:** This ensures your custom error has a proper stack trace and can be caught using `instanceof Error`.
- **Set the `name` property:** By default the `Error` constructor sets `name` to `"Error"`. Overwrite it with your class name for clearer stack traces.
- **Add context carefully:** Include only relevant information (like HTTP status codes or identifiers). Don't put large objects on errors because they may be logged or sent over the wire.
- **Document expected errors:** Make it clear which error types a function can throw so callers know what to handle.

Practice questions

1. **Theory:** Why might you prefer throwing a `NotFoundError` instead of a generic `Error` when a database record is missing?
2. **Theory:** What is the purpose of the `cause` property on an error, and how can it help debugging?
3. **Coding:** Create a `PermissionError` class that extends `Error`. Have it take a `userId` and required `role` in its constructor and include them on the instance. Demonstrate how you would catch and log this error differently from other errors.

4. **Coding:** Rewrite a function that throws generic errors to instead use specific custom error classes (`ValidationError`, `NetworkError`, etc.). Show how the caller can distinguish between them using `instanceof`.
5. **Theory:** When defining a custom error class, why is it important to call `super(message)` and set the `name` property?

How do try-catch-finally blocks behave with async/await?

How do try-catch-finally blocks behave with async/await?

`async/await` lets you write asynchronous code that looks synchronous. But asynchronous operations can still fail, and errors must be handled carefully. The familiar `try...catch...finally` construct works with `await` just like it does with synchronous code, but there are some nuances to be aware of.

Basic use with `await`

`await` pauses the execution of an async function until the awaited promise settles (fulfills or rejects). If the promise fulfills, the value becomes the result of the `await` expression. If it rejects, the `await` expression throws the rejection reason as an exception. This means you can wrap an `await` in a `try...catch` block to handle promise rejections:

```
```js
async function fetchData() {
 try {
 const response = await fetch("/api/data");
 if (!response.ok) {
 throw new Error("HTTP error: " + response.status);
 }
 return await response.json();
 } catch (err) {
 console.error("Failed to fetch data:", err);
 // Optionally rethrow or return a fallback value
 throw err;
 }
}
```
```

In this example, any error thrown by `fetch()` or by manual checks inside the `try` block is caught in the `catch` block. If the error is rethrown, callers can catch it further up the chain.

The `finally` clause

The `finally` block executes **after** the `try` and `catch`, regardless of whether an exception occurred. This is useful for cleanup operations that must run no matter what (closing files, releasing locks, showing or hiding loaders). If the function returns or throws inside the `try` or `catch`, the `finally` block still executes before control leaves the function:

```
```js
async function withLock(lock, work) {
 await lock.acquire();
 try {
 return await work();
 } catch (err) {
 // handle error or rethrow
 throw err;
 } finally {
 lock.release(); // always release lock
 }
}
```
```

Even if `work()` throws an error or returns a value, `lock.release()` is guaranteed to run.

Catching multiple awaits

A single `try` block can contain multiple `await` statements. If any awaited promise rejects, control jumps to the nearest enclosing `catch` block. To handle errors from individual awaits separately, place each in its own `try...catch` or use `Promise.allSettled()` to collect results without throwing:

```
```js
```

```

async function fetchMany(urls) {
 const results = [];
 for (const url of urls) {
 try {
 const res = await fetch(url);
 results.push(await res.json());
 } catch (err) {
 console.warn("Error fetching", url, err);
 results.push(null);
 }
 }
 return results;
}

```

// Or using Promise.allSettled to handle all rejections at once

```

async function fetchManyParallel(urls) {
 const promises = urls.map((u) => fetch(u).then((r) => r.json()));
 const settled = await Promise.allSettled(promises);
 return settled.map((res) => (res.status === "fulfilled" ? res.value : null));
}
...

```

### ## Beware of unhandled rejections

If an awaited promise rejects outside of any `try...catch`, the error will propagate to the next enclosing async function or to the global environment. In Node.js, an unhandled rejection triggers the `unhandledRejection` event and may terminate the process. In browsers, it triggers an `unhandledrejection` event on `window`. Always either catch or return errors.

### ## Finally and returned values

If you return a value from inside ``try`` or ``catch``, the ``finally`` block still runs **\*\*before\*\*** the return. If you return a value in ``finally``, that value overrides any earlier return or thrown error. Overwriting return values in ``finally`` is discouraged because it makes control flow hard to understand.

```
```js
async function tricky() {
  try {
    return 1;
  } finally {
    return 2; // overrides the 1
  }
}
// tricky().then(console.log); // logs 2
```
```

## ## Practice questions

1. **\*\*Theory:\*\*** How does ``await`` interact with ``try...catch`` when a promise rejects? What happens if the rejection is not caught?
2. **\*\*Theory:\*\*** Why is the ``finally`` block useful in asynchronous code? Give two scenarios where you'd put code in a ``finally`` clause.
3. **\*\*Coding:\*\*** Write an async function that reads from two APIs sequentially using ``await``. Use one ``try...catch`` to handle errors from both calls, and then modify the code to handle errors from each call separately.
4. **\*\*Coding:\*\*** Demonstrate what happens when a ``finally`` block returns a value different from the one returned in the ``try`` block. Explain why doing this can be confusing.
5. **\*\*Theory:\*\*** In Node.js, what global event is emitted when a promise rejection has no handler, and how can you listen for it?

# What is unhandledrejection and how can it crash your app?

## # What is `unhandledrejection` and how can it crash your app?

Promises allow you to chain asynchronous operations and handle errors with `.catch()` or `try...catch` and `await`. If a promise is rejected and **no rejection handler** is attached, it is considered **unhandled**. Different environments react to unhandled rejections in different ways, and failing to handle them can cause your application to behave unpredictably or even crash.

## ## Unhandled rejections in browsers

In browsers, when a promise rejection is not handled at the time it occurs, the JavaScript engine dispatches a global `unhandledrejection` event on the `window` object. You can listen for this event to log or handle unexpected rejections:

```
```js
window.addEventListener("unhandledrejection", (event) => {
  console.error("Unhandled rejection:", event.reason);
  event.preventDefault(); // prevents the default logging to console
});

// Example of an unhandled rejection
Promise.reject(new Error("Something went wrong"));
```
```

If you don't listen for `unhandledrejection`, the browser typically logs the error to the console. Unhandled rejections do not crash the page by default, but they can leave your application in an inconsistent state.

If a rejection is later handled (e.g. by attaching a `.catch()`), some browsers also emit a `rejectionhandled` event. However, relying on late handlers is discouraged; always attach `.catch()` on promises or wrap `await` in `try...catch`.

## ## Unhandled rejections in Node.js



Node.js emits an `unhandledRejection` event on the `process` object when a promise rejection is not handled during the current turn of the event loop. By default, Node prints a stack trace and, depending on the `--unhandled-rejections` flag, may terminate the process. Unhandled rejections can therefore crash your application if left unchecked.

You can handle these events globally:

```
```js
process.on("unhandledRejection", (reason, promise) => {
  console.error("Unhandled rejection at:", promise, "reason:", reason);
  // Application specific: decide whether to exit
  process.exit(1);
});

// Example of an unhandled rejection
async function run() {
  throw new Error("Database unavailable");
}

run();
```
```

In recent versions of Node, the default behaviour is configurable. You can set `--unhandled-rejections=strict` to make any unhandled rejection terminate the process, `warn` to log a warning but keep running, or `none` to ignore. The default is `warn`.

### ## Why unhandled rejections are dangerous

- **Silent failures:** If errors are not caught, parts of your app may silently fail and leave inconsistent state. For example, a failed API call might leave UI in a loading state forever.
- **Crashes in Node:** Leaving a promise rejection unhandled in Node can crash your server process, resulting in downtime.
- **Debugging difficulty:** Without catching errors near where they occur, stack traces may be harder to follow.

## ## Best practices

- **\*\*Always handle promise rejections:\*\*** Use `.catch()` when chaining promises or wrap `await` calls in `try...catch`.
- **\*\*Use `Promise.allSettled()` for multiple operations:\*\*** When running many asynchronous tasks concurrently, `Promise.all()` fails fast on the first rejection. If you want to handle all rejections gracefully, use `Promise.allSettled()` or handle errors individually.
- **\*\*Global handlers for fallback logging:\*\*** Set up a `window.unhandledrejection` listener in browsers and a `process.on('unhandledRejection')` handler in Node to log unexpected rejections and prevent the process from crashing.
- **\*\*Fail fast in critical services:\*\*** In backend services it is often preferable to exit on unhandled rejections so that a process manager can restart the service in a clean state.

## ## Practice questions

1. **\*\*Theory:\*\*** What is an "unhandled promise rejection," and how do browsers and Node.js differ in their default handling?
2. **\*\*Theory:\*\*** Describe the dangers of leaving promise rejections unhandled in a Node.js server application.
3. **\*\*Coding:\*\*** Write a piece of code in the browser that rejects a promise without a `.catch()`. Add an `unhandledrejection` listener to log the error message and prevent the default behaviour.
4. **\*\*Coding:\*\*** In Node.js, demonstrate how to configure unhandled rejection behaviour to `'strict'`, then show how to handle unhandled rejections globally to avoid process termination.
5. **\*\*Theory:\*\*** Why is it good practice to attach `.catch()` to every promise or handle errors with `try...catch` when using `await`?

# What are tagged template literals used for in libraries like styled-components?

## # What are tagged template literals used for in libraries like styled-components?

JavaScript's **tagged template literals** combine template strings with a function. When you place a function name immediately before a template literal—``tag`...``—JavaScript doesn't create a string directly. Instead, it calls the tag function with the literal's text fragments and substitution values. The function can return any value. Tagged templates provide powerful metaprogramming capabilities for parsing, transforming and contextualising string data.

Libraries like **styled-components** use tagged template literals to implement CSS-in-JS. Styled-components lets you write actual CSS inside JavaScript and produces React components with encapsulated styles. Here's how it works and why tagged templates are essential.

## ## Basics of tagged templates

A tag function is called with two parameters: an array of **string segments** and a list of **interpolated values**. The first argument is an array of the literal portions of the template, and subsequent arguments correspond to the embedded expressions. For example:

```
``js
function myTag(strings, ...values) {
 console.log(strings); // ['Hello ', ', you are ', '!']
 console.log(values); // ['Alice', 30]
 return strings[0] + values[0] + strings[1] + values[1] + strings[2];
}

const result = myTag`Hello ${"Alice"}, you are ${30}!`;
console.log(result); // 'Hello Alice, you are 30!'
``
```

The tag function can transform or even ignore the template parts. Because tags receive raw strings, they can perform advanced processing like syntax highlighting, sanitisation, or constructing data structures.

## ## How styled-components uses tags

Styled-components is a popular library for styling React components. You create a styled component by calling `styled.tagName`` followed by a template literal containing CSS. The `styled`` object is a function that returns another function (the tag). When you write:

```
```\njs\nimport styled from "styled-components";\n\nconst Title = styled.h1`\n  color: ${props => (props.primary ? "hotpink" : "black")};\n  font-size: 2rem;\n`;\n\nfunction App() {\n  return <Title primary>Hello world</Title>;\n}\n```\n
```

several things happen:

1. **The tag function processes the template:** The styled-components tag function receives the static CSS strings and the interpolation functions (here `(props) => (props.primary ? 'hotpink' : 'black')`).
2. **Dynamic interpolation:** At runtime, when the `Title`` component is rendered, styled-components calls the interpolation functions with the component's props to compute dynamic values. In the example above, the color becomes `'hotpink'` when the `primary`` prop is true.
3. **Generates a unique class name:** Styled-components compiles the resulting CSS and injects it into a `<style>` tag. It generates a unique class name (e.g. `sc-a1234``) and applies it to the rendered component so that styles are encapsulated.
4. **Returns a React component:** The result of the tagged template call is a React component (`Title``) that accepts props. You can use it like any other component in your JSX.

Because tagged template literals allow styled-components to capture both static CSS and dynamic JavaScript expressions in a single construct, they are ideal for CSS-in-JS libraries. Without tags, you'd have to build CSS strings manually and apply them to elements, losing readability and syntax highlighting.

Other uses of tagged templates

Tagged template literals are used beyond styling libraries:

- **Translation (i18n):** Tags can implement string interpolation for translations, handling pluralisation and locale-specific formatting.
- **Safe HTML escaping:** A tag function can escape user input to prevent XSS attacks when constructing HTML.
- **Domain-specific languages:** Tags can parse custom mini-languages embedded in template literals (e.g. GraphQL queries).

Best practices and cautions

- **Keep tag functions pure:** A tag should be a pure function of its arguments; avoid side effects like DOM manipulation within the tag.
- **Performance considerations:** Tags are executed each time the template literal is evaluated. In performance-critical code paths, avoid complex computation in tag functions.
- **Beware of injection:** When using tagged templates to generate CSS or HTML, make sure to sanitise interpolated values to avoid injecting malicious content.

Practice questions

1. **Theory:** How do tagged template literals differ from normal template literals, and what are their parameters?
2. **Theory:** Describe the steps styled-components performs when it encounters a tagged template literal.
3. **Coding:** Write a simple tag function named `sanitizeHtml` that escapes `<`, `>`, `&` and `""` in interpolated values. Use it to safely insert user input into an HTML string.
4. **Coding:** Create a basic React component using styled-components that sets the background color based on a `danger` prop. Explain how the interpolation function uses props.

5. **Theory:** Besides styling, name two other domains where tagged template literals are useful and explain how they help.

Explain the difference between lazy evaluation and eager evaluation in iterables

Explain the difference between lazy evaluation and eager evaluation in iterables

In programming, **evaluation strategy** describes when expressions are executed. Two common strategies are **eager (strict) evaluation** and **lazy (deferred) evaluation**. Although JavaScript is an eagerly evaluated language by default, it provides constructs—like generator functions—that allow lazy evaluation. Understanding the difference helps you write more efficient code, especially when dealing with large data sets or infinite sequences.

Eager evaluation

With eager evaluation, expressions are evaluated as soon as they are bound to a variable or passed as arguments. Arrays in JavaScript are **eager collections**. When you map, filter or reduce an array, the entire array is traversed and a new array is created immediately, even if you ultimately only need a few values.

Example of eager evaluation:

```
```\njs\nconst numbers = [1, 2, 3, 4, 5];\nconst doubled = numbers.map((n) => n * 2); // [2, 4, 6, 8, 10]\n\n// All values are computed up front\n```\n
```

Eager evaluation is simple and predictable. However, it can be wasteful when operating on huge collections or when chaining many operations, because intermediate arrays are created and every element is processed regardless of whether you use them.

### ## Lazy evaluation

Lazy evaluation (also called deferred or call-by-need) delays computing a value until it is actually needed. In JavaScript, **generator functions** and **iterators** enable lazy evaluation. A generator is defined with `function*` and yields values one at a time. Values are produced on demand when you iterate over the generator with `for...of` or call `.next()`.

```

``js
function* countUpTo(n) {
 for (let i = 1; i <= n; i++) {
 console.log("generating", i);
 yield i;
 }
}

const seq = countUpTo(3);
for (const num of seq) {
 console.log(num);
}

// Logs "generating 1" then 1, "generating 2" then 2, etc.

```

The generator produces each value only when requested by the loop. If you break out of the loop early, unused values are never generated. This makes lazy evaluation efficient for potentially large or infinite sequences, and for pipelines where intermediate results might be filtered out.

## ## Comparing lazy and eager evaluation

| Feature                         | Eager evaluation                                        | Lazy evaluation                                         |
|---------------------------------|---------------------------------------------------------|---------------------------------------------------------|
| <b>When values are computed</b> | Immediately                                             | On demand, when iterated                                |
| <b>Memory usage</b>             | May allocate intermediate arrays                        | Minimal; stores only current state                      |
| <b>Suitable for</b>             | Small to medium collections; when you need all results  | Large or infinite sequences; when you may stop early    |
| <b>Debugging</b>                | Straightforward; easier to inspect intermediate results | Requires understanding of generator state and iteration |

### ### Example: chaining operations

Suppose you want to take the first 3 even numbers greater than 10 from a list of numbers. Using eager arrays:

```

``js
const result = numbers

```



```

.filter((n) => n > 10)
.filter((n) => n % 2 === 0)
.slice(0, 3);
...

```

All filters run on the entire array before slicing the first three results, even if the list is huge. With a lazy pipeline using generators you can short-circuit:

```

```js
function* filter(iterable, predicate) {
  for (const item of iterable) {
    if (predicate(item)) yield item;
  }
}

function* take(iterable, n) {
  let count = 0;
  for (const item of iterable) {
    if (count++ < n) yield item;
    else return;
  }
}

const lazyResult = take(
  filter(
    filter(numbers, (n) => n > 10),
    (n) => n % 2 === 0
  ),
  3
);

// Values are produced only until the third match is found

```

...

Benefits and trade-offs

- **Performance:** Lazy sequences can reduce CPU and memory usage because they avoid creating intermediate collections and stop generating values once you have enough. However, for small data sets or when you need all values, the overhead of generator functions can make eager evaluation faster.
- **Composability:** You can build pipelines of generator functions (``map``, ``filter``, etc.) that mirror functional programming libraries. Libraries like [Iterables](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Iteration_protocols) or third-party packages provide helper functions.
- **Infinite sequences:** Lazy evaluation makes it trivial to represent infinite sequences (e.g. the Fibonacci sequence) since values are generated on demand.
- **Statefulness:** Generators maintain internal state between iterations. If you need random access or multiple passes over data, you must recreate the generator or collect its output into an array.

Practice questions

1. **Theory:** Define lazy evaluation in your own words. How does it differ from eager evaluation?
2. **Theory:** What advantages does a generator offer when processing large data sets compared to using array methods like ``map`` and ``filter``?
3. **Coding:** Write a generator function ``range(start, end)`` that lazily yields all integers from ``start`` (inclusive) up to ``end`` (exclusive). Use it to sum the first ten positive integers without creating an array.
4. **Coding:** Refactor a chain of ``Array`` methods (``filter``, ``map``, ``reduce``) into a lazy pipeline using generator functions. Measure performance differences with a large array.
5. **Theory:** When might eager evaluation be preferable to lazy evaluation? Give an example where laziness does not provide a benefit.

How does tail-call optimization (TCO) work, and is it supported in JavaScript engines?

How does tail-call optimization (TCO) work and is it supported in JavaScript engines?

Tail-call optimization (TCO) is a compiler or interpreter feature that allows certain kinds of recursive functions to execute without growing the call stack. In languages that implement TCO, a call to a function in **tail position**—meaning the call is the final action in the function—can reuse the current stack frame instead of allocating a new one. This enables deep recursion without causing stack overflows and can improve performance.

What is a tail call?

A **tail call** is a function call that happens as the last operation of another function. There is no additional work to do after the call returns. For example, in this function the recursive call to `factorial` is in tail position:

```
```js
function factorialTail(n, acc = 1) {
 if (n === 0) return acc;
 return factorialTail(n - 1, acc * n); // tail call
}

factorialTail(5); // 120
```
```

Because nothing happens after `factorialTail(n - 1, ...)` returns, the function could, in theory, replace its stack frame with that of the new call. In contrast, this naive implementation has work to do after the recursive call (multiplying by `n`), so it is **not** in tail position:

```
```js
function factorial(n) {
 if (n === 0) return 1;
 return n * factorial(n - 1); // not a tail call
}
```

```
}
...

```

## ## How TCO works

When TCO is implemented, the runtime recognises that the last action of a function is calling another function (possibly itself). Instead of pushing a new stack frame, it reuses the current one by updating parameters and jumping to the start of the called function. This eliminates stack growth for tail-recursive functions and allows them to run as efficiently as loops.

Languages like Scheme, some implementations of Python, and functional languages like Haskell support TCO. In JavaScript, proper tail calls (PTC) were specified in ECMAScript 2015, but support has been spotty.

## ## TCO support in JavaScript engines

- **Safari / WebKit:** Safari's JavaScriptCore engine implements proper tail calls in strict mode. If you write a tail-recursive function and run it in Safari with `'use strict'`, deep recursion won't overflow the stack. This makes Safari the only major browser with native TCO support as of 2025.
- **V8 (Chrome, Node.js), SpiderMonkey (Firefox) and JSC (other modes):** V8 and SpiderMonkey removed their experimental TCO implementations due to debugging complexity and compatibility concerns. As a result, tail-recursive functions still consume stack frames and may overflow on deep recursion. Developers must convert recursive algorithms to loops or manually maintain their own stacks.
- **Embedded engines:** Smaller engines like Kinoma XS6 and Duktape support TCO. These engines are used in embedded contexts and emphasise memory efficiency.

Because TCO is not universally supported, it's best to avoid relying on it in production code. Use iterative constructs or self-managed stacks when writing algorithms that could recurse deeply.

## ## Simulating TCO with trampolines

If you need recursion without stack growth in environments that lack TCO, you can employ the **trampoline** pattern. A trampoline repeatedly calls a function that returns either a value or another function to call. This turns recursion into iteration:

```

``js
function sumRange(n, acc = 0) {
 if (n === 0) return acc;
 return () => sumRange(n - 1, acc + n);
}

function trampoline(fn) {
 let result = fn;
 while (typeof result === "function") {
 result = result();
 }
 return result;
}

const result = trampoline(() => sumRange(100000));
console.log(result); // 5000050000 without stack overflow
...

```

The `sumRange` function returns a new function instead of making a direct recursive call. The trampoline repeatedly invokes functions until a final value is returned, effectively turning recursion into a loop.

## ## Practice questions

1. **Theory:** Explain what a tail call is and why it allows tail-call optimization. Contrast a tail call with a non-tail call in the context of recursion.
2. **Theory:** Describe why proper tail call support has not been widely implemented in JavaScript engines despite being part of the ECMAScript specification.
3. **Coding:** Write a tail-recursive function to compute the *n*th Fibonacci number. Test it in a browser that supports TCO (if available) and in Node.js. Observe whether stack overflow occurs.
4. **Coding:** Implement the trampoline pattern to compute the factorial of a large number (e.g. 50 000) without causing a stack overflow. Explain how the trampoline avoids stack growth.

5. **Theory:** In which environments might you safely rely on native TCO? How would you handle deep recursion in environments without TCO?

# What are Record and Tuple proposals, and how do they differ from Objects and Arrays?

## # What are record and tuple proposals and how do they differ from objects and arrays?

The ECMAScript **Record & Tuple** proposal introduces two new compound primitive types to JavaScript: **records** and **tuples**. These types aim to provide deeply immutable, value-based data structures as alternatives to mutable objects and arrays. As of 2025 the proposal sits at Stage 2 of the TC39 process, meaning it is still experimental. Even though the proposal may change, its core ideas are worth understanding.

### ## Motivation

JavaScript objects and arrays are mutable and compared by reference. To manage state immutably, developers often use libraries (like Immutable.js or Immer) that create read-only proxies or wrapper objects. These libraries add overhead and do not integrate seamlessly with the language. Records and tuples would provide **built-in immutable data structures** with predictable behaviour:

- **Deep immutability:** Once created, records and tuples cannot be changed. Nested data is also immutable.
- **Value equality:** Two records or tuples are considered equal (`===`) if their contents are identical, unlike objects and arrays which compare by reference.
- **Contain only primitives:** Records and tuples may only contain primitives (including other records and tuples). They cannot contain objects, functions or mutable arrays.

### ## Syntax

Although syntax is still under discussion, the proposal uses a `#`` prefix to distinguish records and tuples from objects and arrays. For example:

```
``js
// Record: similar to an object but immutable
const point = #{ x: 10, y: 20 };

// Tuple: similar to an array but immutable
```

```

const coords = #[1, 2, 3];

// Nested record with a tuple property
const shape = #{
 name: "triangle",
 vertices: #[
 [0, 0],
 [1, 0],
 [0, 1],
],
};

// Equality by value
#{ x: 1, y: 2 } === #{ x: 1, y: 2 }; // true
#[] === #[]; // true
...

```

In these examples, the `#` prefix signals that the structure is a record or tuple. Because they are deeply immutable, trying to modify a record or tuple would throw or have no effect.

## ## Differences from objects and arrays



| Aspect      | Objects / Arrays                                                                                          | Records / Tuples                                                                                                    |
|-------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Aspect      | Mutable by default; properties can be                                                                     | Deeply immutable; no modifications after creation                                                                   |
| Equality    | Compared by value; two objects are                                                                        | Compared by value; two records or tuples with identical contents are <code>===</code> equal.                        |
| Contents    | May contain any type (primitives, objects, functions).                                                    | Can contain only primitives (including other records/tuples).                                                       |
| Methods     | Objects and arrays have methods ( <code>push</code> , <code>splice</code> , <code>toString</code> , etc.) | Records and tuples have no methods of their own; standard operators and helper functions operate on them            |
| Hashability | Objects cannot be used as keys in Maps without reference identity.                                        | Records and tuples can be used as map keys and lookups by value (proposal includes supporting Map/'Set' operations) |

## ## Interoperability

Records and tuples are designed to interoperate smoothly with existing language features:

- They support destructuring and property access just like objects and arrays:

```
``js
const #{ x, y } = point; // destructure a record
const [first, second] = coords; // destructure a tuple
...
```

- Records and tuples are intended to be serialisable and interoperable with JSON and structured cloning. Because they are immutable, transferring them across workers would not require copying internal state.

- Since they are primitives, records and tuples do not have prototypes. They are not affected by changes to `Object.prototype` or `Array.prototype`.

## ## Current status and future

The Record & Tuple proposal is still evolving. At Stage 2, the details of syntax, semantics, and interoperability may change. There was an earlier proposal that used `{| ... |}` and `[| |]` syntax; the

current version uses a ``#`` prefix but this is not final. Some concerns include complexity of value equality, limitations of only allowing primitives, and interaction with existing APIs.

If the proposal advances, records and tuples could reduce reliance on external immutability libraries and make it easier to reason about state. Until then, libraries like Immer remain the best way to work with immutable data in JavaScript.

### ## Practice questions

1. **Theory:** Describe the key differences between records/tuples and objects/arrays in JavaScript. Why are records and tuples compared by value instead of by reference?
2. **Theory:** Why can't records and tuples contain non-primitive values? What benefits does this restriction provide?
3. **Coding:** Declare a record representing a user with ``name`` and ``age`` properties, and a tuple representing the user's favorite colors. Show how you might destructure these structures.
4. **Coding:** Create two records with the same contents and demonstrate that they are ``===`` equal. Then create two plain objects with the same properties and demonstrate that they are not strictly equal.
5. **Theory:** What challenges might arise when adding records and tuples to JavaScript? Consider syntax, backward compatibility and performance.

# What is pattern matching in JavaScript (proposal stage)?

## # What is pattern matching in JavaScript (proposal stage)?

**\*\*Pattern matching\*\*** is a proposal to add a powerful conditional construct to JavaScript, inspired by languages like Haskell, Rust and Swift. The proposal introduces a `match` expression that can test a value against multiple patterns—such as literal values, destructured objects, arrays, and even guard conditions—and execute code based on the first matching case. As of 2025 the pattern matching proposal is at Stage 3, meaning its design is complete but it is not yet part of the language.

### ## Motivation

JavaScript developers often rely on nested `if...else` statements or verbose `switch` statements to handle different shapes of data. Pattern matching aims to make such logic concise and expressive. It builds on existing destructuring patterns, enabling you to match and extract values in one expression while avoiding fall-through behaviour and improving readability.

### ## Basic syntax

Here's an example of a proposed pattern matching syntax:

```
```js
const response = { status: 200, data: { message: 'OK' } };

match (response) {
  { status: 200, data } => console.log('Success:', data.message),
  { status: 404 }      => console.log('Not found'),
  { status: 500 }      => console.log('Server error'),
  _                    => console.log('Unknown status'),
}
```
```

Key features:

- **Patterns** can be object patterns (`{ status: 200, data }`), array patterns (`[x, y]`), literal values (`42`, `'hello'`), or identifier patterns (`_` for a wildcard). Patterns can use nested destructuring, default values and computed keys.
- **Arrow syntax** (`pattern => expression`) associates each pattern with an expression or block to evaluate when the match succeeds. Only the first matching branch executes—there is no fall-through like in `switch`.
- **Exhaustiveness** is encouraged but not enforced. A wildcard `_` at the end acts as a catch-all case.

## ## Guards and fallthrough

Patterns can include **guard conditions** using `if` to add additional checks:

```
```js
match (value) {
  [x, y] if x === y => console.log('a symmetric pair'),
  [x, y]      => console.log('a pair of numbers'),
  _          => console.log('something else'),
}
```
```

In this example, the first branch matches arrays with two elements and then checks the guard `x === y`. If the guard fails, execution continues to the next branch.

## ## Advantages over `switch` and `if...else`

- **Concise destructuring and matching:** You can match against nested shapes and extract values without separate destructuring statements.
- **No accidental fall-through:** Each branch is independent; there is no need for `break` statements as in `switch`.
- **Safer defaults:** A wildcard case encourages handling unknown values explicitly.
- **Readability:** For complex conditionals with different data shapes (e.g. discriminated unions, tagged responses), `match` can be easier to follow than nested `if` statements.

## ## Current status and tooling

The pattern matching proposal is still experimental. Tooling like Babel plugins and TypeScript transforms allow you to experiment with pattern matching today. Because the feature is not yet standard, syntax and semantics may change. You should avoid using it in production without transpilation.

## ## Practice questions

1. **Theory:** How does the proposed `match` expression improve upon existing `switch` statements? Discuss fall-through and destructuring.
2. **Theory:** What are guard conditions in pattern matching? Provide an example.
3. **Coding:** Write a `match` expression that takes a variable which may be a string, number, array of two numbers or any other value, and logs different messages for each case.
4. **Coding:** Convert a nested `if...else` chain that checks `response.status` and `response.data` into a pattern matching expression.
5. **Theory:** Why is the pattern matching proposal still at Stage 3, and what considerations should developers keep in mind before using experimental syntax?

# How does the pipeline operator improve function chaining?

## # How does the pipeline operator improve function chaining?

Modern JavaScript encourages composition: building complex behaviour by combining small, pure functions. When you compose functions using nested calls, however, the code often reads **\*\*inside-out\*\***. Consider calculating a result by doubling a number, adding three and then negating it:

```
```js
const result = negate(add(3, double(5)));
...
```
```

To understand this expression you have to start in the middle (`double(5)`), then move to the left (`add(3, ...)`), and finally apply `negate`. The proposed **\*\*pipeline operator\*\*** (`|>`) flips this around. It takes the value on its left and feeds it as an argument into the function on its right, allowing the chain to be read **\*\*left-to-right\*\***:

```
```js
const result = 5 |> double |> ((n) => add(3, n)) |> negate;
...
```
```

Each pipeline stage receives the output of the previous stage. The above code is easier to scan because the order of operations matches the order of the lines.

## ## Syntax and semantics

At the time of writing the pipeline operator is an ECMAScript proposal (still under development), so its exact syntax may change. The most widely discussed variant, sometimes called the "smart pipeline", allows two forms:

1. **\*\*Bare function form\*\*** - When the right-hand side is an identifier, it must refer to a function. The pipeline operator calls that function with the value on the left as its only argument:

```

```js
const triple = (x) => x * 3;

const value = 2 |> triple; // same as triple(2)

...

```

2. **Topic reference form** - For more complex expressions, a special placeholder (often `#`) represents the value being piped. This lets you call functions that take more than one argument or access methods:

```

```js
const result = 10

 |> # + 1 // increment

 |> Math.pow(#, 2) // square

 |> (n) => n.toString();

// result === "121"

...

```

Each pipeline stage is evaluated one after the other. Unlike Unix pipes, the JavaScript pipeline operator does not automatically read from or write to streams; it simply passes values between functions.

## ## Benefits of pipelines

- **Readability**: Pipelines allow code to read from top to bottom, matching the order in which data flows through the functions. This reduces the cognitive overhead of nested parentheses.
- **Composability**: Small functions become building blocks that can be chained together. This style is common in functional programming languages (F#, Elm) and libraries like Ramda or RxJS.
- **Reduced boilerplate**: Without pipelines, you often create temporary variables to break up complex expressions. Piping reduces the need for temporary names and clarifies the transformation steps.
- **Consistency with other features**: Pipelines pair well with arrow functions, optional chaining and pattern matching proposals, promoting a declarative style.

## ## Practical examples

### ### Transforming values

Suppose you need to normalise a string by trimming whitespace, converting it to lowercase and replacing spaces with hyphens. Without pipelines, you might write:

```
```js
const normalise = (str) => replaceSpaces(toLowerCase(str.trim()));
```
```

With the pipeline operator, each step becomes its own line:

```
```js
const normalise = (str) =>
  str
    |> String.prototype.trim.call(#)
    |> ((s) => s.toLowerCase())
    |> ((s) => s.replace(/\s+/g, '-'));
```
```

This makes it clear that the input flows through trimming, lower-casing and replacing spaces. The ``call`` method is necessary because ``trim`` is a method, not a standalone function. Future versions of the proposal may allow a more ergonomic syntax for method calls.

### ### Avoiding nested parentheses

Pipelines particularly shine when composing many functions:

```
```js
// Without pipeline
const result = decodeURIComponent(atob(data)).split(',').map(Number).reduce((a, b) => a + b, 0);
```
```



```
// With pipeline
const result = data

 |> atob

 |> decodeURIComponent

 |> (#.split(','))

 |> (#.map(Number))

 |> ((arr) => arr.reduce((a, b) => a + b, 0));
...

```

Each step is isolated, and you can easily add `console.log()` calls between stages for debugging.

### ## Status and considerations

The pipeline operator is still in the proposal stage and not yet part of the ECMAScript standard. Different variations are under discussion, so the syntax may change before it is finalised. When using experimental language features, transpilers like Babel and TypeScript can compile pipelines down to supported JavaScript.

Be mindful that pipeline operators are **not** magic performance boosters; they simply provide syntactic sugar. Each stage still creates a function call, so deeply chained pipelines could have a small overhead compared to a single dedicated function.

### ## Real-world analogy

Think of a pipeline like an assembly line in a factory. An unfinished product moves along a conveyor belt and stops at each station. At each stop, a worker performs a specific transformation (paints the part, attaches a piece, tests functionality) before sending it to the next station. Similarly, the pipeline operator sends a value through a series of functions, each of which transforms it slightly.

### ## Practice questions

1. **Theory:** What problem does the pipeline operator aim to solve when composing functions? How does the code read differently when using pipelines compared to nested function calls?

2. **Theory:** Describe the two forms of the proposed pipeline operator. When do you need to use a topic reference placeholder?
3. **Coding:** Rewrite the following expression using the pipeline operator (assume the operator and topic reference are supported):

```
```js
const result = toTitleCase(removeStopWords(cleanText(rawString)));
```
```

where `cleanText`, `removeStopWords` and `toTitleCase` are functions.

4. **Coding:** Use the pipeline operator to compute the mean of an array of numbers by chaining `Array.prototype.reduce()` and a function that divides by the array length. Compare the pipeline version to a traditional implementation.
5. **Theory:** The pipeline operator proposal is still evolving. What are some considerations you should have before using it in production code?

# What is `Array.prototype.groupBy` and how is it used?

## # What is `groupBy` and how is it used?

Grouping items is a common task: you might need to organise products by category, group students by grade, or partition events by month. Traditionally, you would reach for `Array.prototype.reduce()` or an external library like Lodash. The **grouping proposal** introduces built-in methods that make grouping more convenient and expressive.

Although early drafts suggested adding a `groupBy` method directly on arrays, the current design provides two **static methods**:

- `Object.groupBy(iterable, callback)` - Returns a plain object with string or symbol keys.
- `Map.groupBy(iterable, callback)` - Returns a `Map` keyed by arbitrary values.

Both methods take an iterable (such as an array) and a callback that returns a key for each element. Elements that return the same key are placed into the same group.

## ## How `Object.groupBy()` works

`Object.groupBy()` accepts two arguments: the iterable of values to group and a callback function. The callback is called once per element and should return a value that can be coerced into a property key (a string or a symbol). The returned object has one property for each unique key, mapping to an array of the corresponding elements.

```
``js
const inventory = [
 { name: "asparagus", type: "vegetables", quantity: 9 },
 { name: "bananas", type: "fruit", quantity: 5 },
 { name: "goat", type: "meat", quantity: 23 },
 { name: "cherries", type: "fruit", quantity: 12 },
 { name: "fish", type: "meat", quantity: 22 },
];
```

```
const byType = Object.groupBy(inventory, (item) => item.type);

/* byType is:
{
 vegetables: [{ name: 'asparagus', type: 'vegetables', quantity: 9 }],
 fruit: [{ name: 'bananas', type: 'fruit', quantity: 5 },
 { name: 'cherries', type: 'fruit', quantity: 12 }],
 meat: [{ name: 'goat', type: 'meat', quantity: 23 },
 { name: 'fish', type: 'meat', quantity: 22 }]
}
*/
...

```

Because the returned object has a `null` prototype, it does not inherit methods like `toString()` — reducing the risk of name collisions. Note that the grouped elements are the **same objects** as the ones in the original array; modifying a grouped element also modifies the original.

If the callback returns values that are not strings or symbols (for example, numbers or booleans), they are coerced to strings when used as property keys. If you need to use arbitrary objects or values (like dates or complex objects) as keys, use `Map.groupBy()` instead.

### ## How `Map.groupBy()` works

`Map.groupBy()` works like `Object.groupBy()`, but instead of an object, it returns a `Map` where keys can be **any value**. This is useful when grouping by values that should not be coerced to strings. For example, you can group by a boolean or even a reference:

```
```js
const numbers = [5, 7, 8, 12, 15, 20];
const parity = Map.groupBy(numbers, (n) => n % 2 === 0);

/* parity is a Map:
  key: false → [5, 7, 15]
  key: true  → [8, 12, 20]
*/

```

```

const mapKey1 = {};
const mapKey2 = {};
const objects = [
  { key: mapKey1, value: 1 },
  { key: mapKey2, value: 2 },
  { key: mapKey1, value: 3 },
];
const grouped = Map.groupBy(objects, (obj) => obj.key);
// grouped.get(mapKey1) → [ { key: mapKey1, value: 1 }, { key: mapKey1, value: 3 } ]
// grouped.get(mapKey2) → [ { key: mapKey2, value: 2 } ]
...

```

The keys in a `Map` preserve identity: two objects with the same content but different references will be treated as different keys.

Use cases and benefits

- **Simpler grouping:** Grouping values used to involve writing a `reduce()` loop or using external libraries. `Object.groupBy()` and `Map.groupBy()` make the intent explicit and reduce boilerplate.
- **Better type safety:** Group names are determined by the callback; you avoid accidental collisions with inherited object properties (because the returned object has a null prototype).
- **Flexible keys:** `Map.groupBy()` lets you use any JavaScript value as a key, which is useful when grouping by booleans, dates or other non-string values.
- **Readable code:** Grouping logic can now be expressed declaratively at the top level of your code instead of being buried in loops.

Comparison with a manual reducer

Before these methods existed, you might have grouped values like this:

```

```js
function groupByManual(arr, getKey) {

```

```

return arr.reduce((groups, item) => {
 const key = getKey(item);
 (groups[key] = groups[key] || []).push(item);
 return groups;
}, {});
}

const byType = groupByManual(inventory, (item) => item.type);
...

```

The built-in `Object.groupBy()` does the same job but improves readability and avoids mutating a pre-initialised accumulator.

### ## Practice questions

1. **Theory:** What is the difference between `Object.groupBy()` and `Map.groupBy()`? When would you choose one over the other?
2. **Theory:** How does the grouping proposal handle property keys? Why does the object returned by `Object.groupBy()` have a `null` prototype?
3. **Coding:** Use `Object.groupBy()` to group an array of words by their first letter. Show the resulting object.
4. **Coding:** Given an array of transactions with a `date` property, use `Map.groupBy()` to group them by month (hint: use `new Date(tx.date).getMonth()` as the key). How could you achieve the same result without `Map.groupBy()`?
5. **Theory:** What precautions should you take when grouping objects that may later be mutated? How do the grouping methods handle deep copies?

## What is the purpose of Symbol.dispose and the using statement proposal?

### # What is the purpose of `Symbol.dispose` and the `using` statement proposal?

Resource management—opening and closing files, connecting to databases, acquiring locks—is a common source of bugs. In languages like C++ and Python, deterministic destruction (`RAII` / `with` statements) ensures resources are released when they go out of scope. JavaScript has historically lacked a built-in mechanism to automatically dispose of resources. The **explicit resource management** proposal introduces two new features to fill this gap: the **well-known symbol** `Symbol.dispose` (along with its asynchronous counterpart `Symbol.asyncDispose`) and the `using` statement.

### ## Disposable objects

An object is considered *\_disposable\_* if it defines a method keyed by `Symbol.dispose`:

```
```js
class FileHandle {
  constructor(name) {
    this.name = name;
    this.open = true;
  }
  read() {
    if (!this.open) throw new Error("File is closed");
    /* ...read from file... */
  }
  [Symbol.dispose]() {
    // clean up resources here
    console.log(`Closing ${this.name}`);
    this.open = false;
  }
}
```
```

The method must be synchronous and should perform any necessary cleanup. Calling it multiple times should not throw. For asynchronous resources (e.g. network sockets), define `[Symbol.asyncDispose]()` returning a promise.

## ## The `using` declaration

The `using` statement automatically calls an object's disposal method when the surrounding block is exited. It ensures that cleanup happens regardless of whether the block completes normally or due to an exception. Here is the previous `FileHandle` class used inside a `using` block:

```
```js
{
  using fh = new FileHandle('log.txt');
  fh.read();
  // leaving the block calls fh[Symbol.dispose]()
}
// logs "Closing log.txt"
```
```

Because `Symbol.dispose` is looked up on the initializer of the `using` variable, any object that implements this symbol can participate. If the object also defines `[Symbol.asyncDispose]()`, you can use `await using` to wait for asynchronous cleanup:

```
```js
class Socket {
  /* ... */
  async [Symbol.asyncDispose]() {
    await this.closeConnection();
  }
}

async function sendRequest() {
```



```

    await using sock = new Socket();

    await sock.send('hello');
}
...

```

``await using`` guarantees that disposal of asynchronous resources is awaited before control leaves the scope.

Why explicit resource management matters

- **Reliability:** Without automatic disposal, it's easy to forget to close a file or release a lock—leading to memory leaks, file descriptor exhaustion, or inconsistent state.
- **Composability:** When functions encapsulate resource acquisition inside themselves, they obscure the cost and lifetimes of resources. ``using`` makes resource lifetimes explicit at the call site.
- **Error safety:** By tying disposal to scope, even exceptions cannot bypass cleanup. The ``using`` syntax ensures deterministic finalisation, similar to ``try...finally`` but with less boilerplate.

Comparisons and pitfalls

- **``try...finally``:** You can always manage resources with ``try...finally``. The ``using`` statement is syntactic sugar that calls the appropriate disposal method for you. It reduces the risk of missing cleanup in error paths.
- **One-time use:** ``Symbol.dispose`` should only be called by the runtime. Calling it manually will not prevent it from being called again at the end of the scope.
- **Not widely supported:** As of 2025, explicit resource management is still a proposal. It is implemented in some environments (e.g. TypeScript has experimental support). Don't use it in production without a transpiler/polyfill.
- **Cannot await synchronous dispose:** The ``Symbol.dispose`` method must be synchronous. Use ``[Symbol.asyncDispose]`` and ``await using`` for asynchronous cleanup.

Real-world analogy

Imagine borrowing a library book. When you finish reading, you should return it to avoid late fees and free the book for others. If you forget, the library eventually contacts you—similar to how garbage collection eventually frees memory but without releasing the library's copy. The ``using`` statement is like a policy that automatically returns the book when you leave the library.

Practice questions

1. **Theory:** What methods must an object implement to participate in the `using` mechanism? How does `Symbol.dispose` differ from `Symbol.asyncDispose`?
2. **Theory:** Explain how the `using` statement improves reliability compared to managing resources with `try...finally` or manually calling cleanup methods.
3. **Coding:** Create a `Timer` class that starts measuring time in its constructor and prints the elapsed time in its `[Symbol.dispose]()` method. Use `using` to measure the duration of a code block.
4. **Coding:** Implement an asynchronous database connection class with `[Symbol.asyncDispose]()` that closes the connection. Use `await using` inside an `async` function to ensure the connection is closed even if an error occurs.
5. **Theory:** What potential issues might arise if you forget to declare `await using` for an object with an `[Symbol.asyncDispose]()` method?

What is Error.cause — new error message cause — and when is it useful?

What is the `cause` property on errors and when is it useful?

When something goes wrong deep inside your code, the error that bubbles up often lacks context. You might catch an error from a low-level function, wrap it with a higher-level message, and then lose the original exception. The `cause` property introduced in ES2022 solves this problem by letting you ****chain errors together****.

Creating an error with a cause

All built-in error constructors (like `Error`, `TypeError`, `RangeError`) accept an optional second argument: an options object with a `cause` property. You can pass the original error (or any other value) to this property. The new error stores the cause on its `.cause` property but otherwise behaves like a normal error.

```
```js
function parseSettings(json) {
 try {
 return JSON.parse(json);
 } catch (err) {
 // Add context and preserve the original error
 throw new SyntaxError("Invalid configuration file", { cause: err });
 }
}

try {
 parseSettings("not valid JSON");
} catch (err) {
 console.error(err.message); // "Invalid configuration file"
 console.error(err.cause.message); // "Unexpected token n in JSON at position 0"
}
```
```

In this example, the `cause` property holds the original `SyntaxError` thrown by `JSON.parse()`. When debugging, you can inspect `err.cause` to see what went wrong at the deeper level.

Error chaining across layers

Suppose you have a function that reads a file and parses its contents. If reading fails, you want to know whether the error came from the file system or from parsing:

```
``js
const fs = require("fs/promises");

async function readConfig(path) {
  try {
    const data = await fs.readFile(path, "utf8");
    return JSON.parse(data);
  } catch (err) {
    throw new Error(`Failed to load configuration: ${path}`, { cause: err });
  }
}

readConfig("config.json").catch((err) => {
  console.error(err.message);
  // Walk the chain of causes
  let current = err.cause;
  while (current) {
    console.error("caused by:", current);
    current = current.cause;
  }
});
``
```

The `cause` property lets you build an error chain. Each error in the chain can store its own message and metadata while pointing to the lower-level error that triggered it. In Node.js, some APIs (such as `fs/promises`) already include useful causes when they throw.

When and why to use `cause`

- **Add context:** When rethrowing an error, wrap it with a message describing what the higher-level operation was doing. The original error remains available via `.cause`.
- **Preserve stack information:** Although the top-level error has its own stack trace, the cause retains its own stack trace and properties. This makes debugging easier than logging plain strings.
- **Structured error handling:** Frameworks and logging libraries can inspect the error chain and present a hierarchical view of failures.
- **Flexibility:** The `cause` property can hold any value—another error, a string, or a structured object. This allows creative uses such as attaching additional metadata.

Be mindful that the `cause` property is not automatically printed when calling `console.error(err)`. You need to log it explicitly or use utilities that traverse causes. When creating custom error classes, pass the `cause` option to `super()` so that the base error stores it:

```
```js
class ValidationError extends Error {
 constructor(message, options) {
 super(message, options);
 this.name = "ValidationError";
 }
}

try {
 // Simulate a lower-level error
 throw new Error("Field length exceeded");
} catch (e) {
 throw new ValidationError("Invalid user input", { cause: e });
}
```
```

Real-world analogy

Imagine a chain of customer service escalations. The frontline agent documents the issue and escalates it to a supervisor. The supervisor adds context ("customer attempted to reset password") and escalates further. Each step attaches additional information but references the previous record. When the issue is resolved, you can trace back through all notes to understand the root cause. The ``cause`` property plays a similar role: each layer adds context while preserving the underlying error.

Practice questions

1. **Theory:** Why is it beneficial to include the original error as the ``cause`` when rethrowing an error? How does this improve debugging?
2. **Theory:** What types of values can the ``cause`` property hold? What happens if you set ``cause`` to a non-error value?
3. **Coding:** Write a function `fetchJson(url)` that fetches data from a URL and parses the response as JSON. If the fetch fails or the JSON is invalid, throw a new error with an appropriate message and set the original error as the ``cause``.
4. **Coding:** Create a custom error class `DatabaseError` that takes a ``cause`` option. Demonstrate catching a low-level error (e.g. connection timeout) and wrapping it in a `DatabaseError`. How would you traverse the chain of causes to log each error?
5. **Theory:** Why doesn't logging an error automatically display its cause? How can you design a logging utility that prints the entire error chain?

What are the phases of the event loop (timers, poll, check, close)?

What are the phases of the event loop (timers, poll, check, close)?

JavaScript environments like browsers and Node.js are **single-threaded** at the language level, yet they can perform non-blocking I/O. This is achieved through an **event loop**: a mechanism that schedules and runs callbacks in a predictable order. While browsers and Node share the same high-level idea, Node's event loop exposes more phases. Understanding these phases helps you reason about callback ordering and avoid surprises when mixing timers, I/O and microtasks.

High-level overview

The event loop continuously cycles through a set of **queues** and **phases**. In each cycle (often called a `_tick_`), it performs work in the following order:

1. **Timers phase** - Executes callbacks scheduled by `setTimeout()` and `setInterval()` whose time has expired.
2. **Pending callbacks** - Executes I/O callbacks deferred to the next loop iteration.
3. **Idle, prepare** - Internal phases used by Node.js (not exposed to user code).
4. **Poll phase** - Retrieves new I/O events (such as network or file events) and executes their callbacks. If there are no timers due and the poll queue is empty, this phase can block until I/O arrives.
5. **Check phase** - Executes callbacks scheduled by `setImmediate()`.
6. **Close callbacks** - Executes callbacks for closed resources, such as sockets whose `"close"` event is emitted.

After the close callbacks phase completes, the event loop checks the **microtask queue**, runs all queued microtasks (e.g. promise callbacks via `.then()`, `queueMicrotask()`, `process.nextTick()` in Node) until it is empty, and then begins the next tick.

Timers phase

The timers phase handles functions scheduled by `setTimeout()` and `setInterval()`. A timer is executed **once** when its delay has elapsed. If multiple timers expire at the same tick, they are executed in order of scheduling. Intervals (`setInterval()`) re-queue themselves after each execution.

Pending callbacks

Some operating system operations, such as certain types of TCP errors, run their callbacks here. This phase is rarely used directly by JavaScript developers.

Poll phase

The poll phase is responsible for retrieving new I/O events (e.g. data arriving on a network socket, file system events) and executing their callbacks. If the poll queue is empty, Node.js will either:

- Block and wait for incoming I/O if there are no timers scheduled, or
- Immediately proceed to the check phase if timers are due or the poll has been idle for a maximum timeout.

This behaviour prevents the poll phase from ****starving**** the rest of the loop. Under the hood, the libuv library imposes a maximum blocking time so that other phases still get a chance to run.

Check phase

Callbacks passed to `setImmediate()` execute in the check phase, ****after**** the poll phase and before closed resources are handled. Because `setImmediate()` always runs after the poll phase, it can be used to schedule work to happen after I/O events in the same tick. This distinguishes it from `setTimeout(fn, 0)`, which runs in the timers phase and may occur earlier.

Close callbacks phase

When a stream or socket closes, Node emits a `'close'` event. Listeners attached to `'close'` fire in this phase. For example, closing a TCP socket triggers its `'close'` callback here.

Microtasks and next ticks

Within each phase, after a callback runs, the runtime processes the microtask queue. Microtasks include:

- **Promise callbacks:** `.then()`, `.catch()` and `.finally()` handlers.
- **`queueMicrotask()` callbacks:** A way to enqueue microtasks manually.
- **`process.nextTick()` callbacks (Node only):** Executed even before other microtasks and can starve the loop if misused.

After all microtasks are processed, the event loop either continues with the current phase (if there are more callbacks) or moves on to the next phase. This guarantees that microtasks run *before* the next macrotask (e.g. timer or I/O callback).

Illustrative example

Consider the following Node.js code:

```
``js
setTimeout(() => {
  console.log("timeout");
}, 0);

setImmediate(() => {
  console.log("immediate");
});

fs.readFile(__filename, () => {
  console.log("file read");
});

Promise.resolve().then(() => {
  console.log("promise");
});

process.nextTick(() => {
```

```
console.log("nextTick");
});
...
```

When this script runs:

1. `process.nextTick()` executes immediately after the current operation finishes (`nextTick`).
2. The resolved promise adds a microtask, which runs next (`promise`).
3. I/O callbacks run in the poll phase (`file read`).
4. `setImmediate()` callbacks run in the check phase (`immediate`).
5. `setTimeout()` with 0 delay runs in the timers phase of the **next** tick (`timeout`).

Actual ordering can vary if the file read finishes before the timer is ready, but microtasks (`nextTick` and `promise`) always run before any of the macrotasks.

Practice questions

1. **Theory:** List the main phases of the Node.js event loop and briefly describe what happens in each phase.
2. **Theory:** How does `setImmediate(fn)` differ from `setTimeout(fn, 0)` in terms of when their callbacks are executed?
3. **Coding:** Write a Node.js script that uses `setTimeout`, `setImmediate`, `Promise.resolve().then`, and `fs.readFile()` to illustrate the order in which their callbacks run. Compare your output to the explanation in this section.
4. **Theory:** What is the purpose of the poll phase's ability to block? How does libuv avoid starving the event loop when waiting for I/O?
5. **Theory:** Explain the difference between microtasks (e.g. promise handlers, `queueMicrotask()`, `process.nextTick()`) and macrotasks (e.g. timer callbacks, I/O events). Why is it important that microtasks run after each macrotask?

What is event-loop starvation and how can you prevent it?

What is event loop starvation and how can you prevent it?

JavaScript's event loop allows your code to perform I/O and other asynchronous operations without blocking execution. However, it's possible for a program to hog the event loop so badly that other tasks never get a chance to run. This situation is called ****event loop starvation**** or ****starving the event loop****. Understanding what causes starvation and how to prevent it will help you write responsive, non-blocking applications.

What is event loop starvation?

The event loop cycles through macrotask queues (timers, I/O events, setImmediate, etc.) and the microtask queue (promise callbacks, `queueMicrotask()`, `process.nextTick()`). Event loop starvation occurs when code keeps the loop busy for too long, preventing other queued tasks from executing. Symptoms include:

- User interfaces freezing because long-running JavaScript blocks re-rendering.
- Network or file I/O callbacks delayed indefinitely.
- High CPU usage with little progress on other tasks.

There are two common causes of starvation:

1. ****Synchronous blocking loops:**** A while loop that runs for seconds will block the event loop. During this time the browser cannot respond to user input, and Node.js cannot process incoming requests.

```
```js
// Bad: blocks the event loop
function crunchNumbers() {
 let sum = 0;
 for (let i = 0; i < 1e10; i++) {
 sum += i;
 }
}
```

```

 return sum;
}
crunchNumbers();
// Nothing else runs until the loop finishes
...

```

2. **\*\*Unbounded microtask queues:\*\*** Promise callbacks and `process.nextTick()` run after the current macrotask but before the next one. If you schedule more microtasks in each microtask, the loop can get stuck processing microtasks forever and never return to macrotasks or I/O.

```

```js
function spinMicrotasks() {
    Promise.resolve().then(spinMicrotasks);
}
spinMicrotasks();
// The above code continually queues microtasks; timers and I/O starve
...

```

In both cases, the CPU is busy executing JavaScript while other callbacks starve waiting for a chance to run.

How to prevent starvation

1. ****Break up long tasks:**** Instead of doing all the work in one synchronous chunk, divide it into smaller pieces and schedule each piece with `setTimeout()` or `setImmediate()`. This yields control back to the event loop, allowing other tasks to run.

```

```js
function crunchInChunks(start, end, chunkSize, callback) {
 let sum = 0;
 function processChunk(i) {
 const max = Math.min(i + chunkSize, end);
 for (; i < max; i++) {

```

```

 sum += i;
 }
 if (max < end) {
 setTimeout(() => processChunk(max), 0); // yield to event loop
 } else {
 callback(sum);
 }
}
processChunk(start);
}
crunchInChunks(0, 1e7, 1e5, (result) => {
 console.log("Sum:", result);
});
...

```

By using `setTimeout(..., 0)`, the loop continues processing timers, I/O and microtasks between chunks. For Node.js-specific code, you can use `setImmediate()` to schedule the next chunk after the poll phase.

2. **\*\*Limit microtask recursion:\*\*** Avoid recursive chains of promises that queue additional promises inside `.then()` callbacks or `queueMicrotask()`. If you need to process a large number of items with promises, batch them or use asynchronous loops that yield:

```

```js
async function processItems(items) {
  for (const item of items) {
    await doAsyncWork(item);

    // microtask queues drain between iterations
  }
}
...

```

``await`` yields control back to the event loop after each iteration, preventing microtask starvation.

3. ****Use workers for CPU-heavy tasks:**** Modern browsers support Web Workers, and Node.js supports worker threads. Offload CPU-intensive computation to a worker so that the main event loop remains responsive.

4. ****Monitor and test:**** Use performance profiling tools to detect long tasks and microtask churn. When debugging, sprinkle ``console.log()`` statements or timers to observe if certain callbacks aren't firing when expected.

Real-world analogy

Imagine a single-lane bridge controlled by a traffic light. Cars (tasks) can cross only when the light is green. If one driver parks their car on the bridge and refuses to move, they block everyone behind them—this is like a blocking loop. Alternatively, if a stream of endless motorcycles (microtasks) keeps the light green for themselves, cars waiting at the other side can never cross. Preventing event loop starvation means ensuring everyone gets a fair turn.

Practice questions

1. ****Theory:**** What is event loop starvation? Describe two common scenarios that can lead to it.
2. ****Theory:**** Why does scheduling microtasks recursively cause starvation? How are microtasks prioritised relative to macrotasks?
3. ****Coding:**** Modify the ``crunchNumbers()`` function to compute the sum of $1..1e8$ without blocking the event loop. Use either ``setTimeout()`` or ``setImmediate()`` to yield between chunks and measure how it affects responsiveness.
4. ****Coding:**** Write code that creates an unbounded microtask chain using promises. Run it and observe how it affects the execution of a ``setTimeout(() => console.log('timeout'), 0)`` call. Then fix the program by batching work with an asynchronous loop.
5. ****Theory:**** Explain how Web Workers (in browsers) or worker threads (in Node.js) help prevent event loop starvation when performing heavy computations.

What is requestIdleCallback and when should you use it?

What is `requestIdleCallback()` and when should you use it?

Web applications often need to perform work that isn't critical to the next frame: analytics, preloading data, cleaning caches, or other housekeeping tasks. Running these tasks on the main thread at the wrong time can cause visible **jank**, resulting in dropped frames or delayed input. The **`requestIdleCallback()`** API allows you to schedule non-urgent work to run during periods when the browser is idle.

How it works

`requestIdleCallback(callback, options?)` tells the browser, "call this function when you have spare time." The browser schedules the callback at a point when it's not busy handling user input, layout, or rendering. When the callback runs, it receives an **`IdleDeadline`** object that provides two methods:

- `timeRemaining()` - Returns the estimated number of milliseconds remaining before the browser needs to yield to high-priority work. You can check this inside your callback and split your work into chunks if it runs long.
- `didTimeout` - Indicates whether the callback is running because a timeout specified in `options.timeout` expired. If you provide a timeout, the browser guarantees that the callback will run within that many milliseconds, even if the tab isn't idle.

For example:

```
```js
function heavyTask(deadline) {
 while (deadline.timeRemaining() > 0 && tasks.length > 0) {
 const task = tasks.shift();
 // perform a small piece of work
 process(task);
 }
 if (tasks.length > 0) {
```

```

 requestIdleCallback(heavyTask); // schedule next chunk
 }
}
requestIdleCallback(heavyTask);
...

```

In this pattern, you repeatedly process items until the deadline runs out. If work remains, you schedule another idle callback. This keeps the main thread responsive while eventually finishing all tasks.

## ## When to use it

Use `requestIdleCallback()` for tasks that:

- **\*\*Are low priority:\*\*** analytics, reporting, warm caches, or non-critical UI updates.
- **\*\*Can be broken into small chunks:\*\*** if your work needs more time than a single idle period, split it across multiple calls.
- **\*\*Don't need to run on every frame:\*\*** animation and layout should use `requestAnimationFrame()` instead.

Avoid using `requestIdleCallback()` for tasks that must run at specific times (e.g. right before paint) or require deterministic timing. Because idle time is unpredictable, your callback might run sooner or later depending on user interactions and system load.

## ## Browser support and fallbacks

`requestIdleCallback()` is available in Chromium-based browsers and some versions of Firefox, but not universally supported. To use it safely, provide a fallback to `setTimeout()` so that your code still runs after a reasonable delay:

```

```js
const scheduleIdleTask =
  window.requestIdleCallback ||
  function (cb) {

```



```
// Run the callback after 200 ms if requestIdleCallback isn't available
return setTimeout(
  () => cb({ timeRemaining: () => 0, didTimeout: true }),
  200
);
};
```

```
const cancelIdleTask = window.cancelIdleCallback || clearTimeout;
...
```

With this fallback, older browsers will execute your idle tasks after a small delay instead of waiting for a true idle period.

Real-world analogy

Imagine an office worker who needs to file some paperwork but also answers the phone and greets visitors. Filing is important but not urgent. The worker does it during lulls between phone calls and walk-ins, stopping whenever someone needs attention. Likewise, `requestIdleCallback()` lets the browser handle non-urgent tasks without disrupting critical user interactions.

Practice questions

1. **Theory:** What kinds of tasks are good candidates for `requestIdleCallback()`? Why shouldn't you use it for animation or layout work?
2. **Theory:** Describe the purpose of the `IdleDeadline` object. How can you use `timeRemaining()` and `didTimeout` to split work into chunks?
3. **Coding:** Implement a polyfill for `requestIdleCallback()` using `setTimeout()`. Then write a function that processes an array of 100,000 items during idle periods without freezing the UI.
4. **Coding:** Use `requestIdleCallback()` to prefetch a list of images when the page is idle. Include a timeout so the prefetching still occurs if the browser never goes idle.
5. **Theory:** What happens if you schedule an idle callback with no timeout and the tab remains busy? How might you mitigate that situation?

How do garbage collection triggers and mark-and-sweep impact performance?

How do garbage collection triggers and mark-and-sweep impact performance?

JavaScript frees memory automatically through **garbage collection** (GC). While automatic memory management saves developers from calling `free()`, it isn't free of cost. Understanding what triggers GC and how the **mark-and-sweep** algorithm works can help you write code that performs better and avoids unexpected pauses.

What triggers garbage collection?

In most JavaScript engines, you cannot explicitly invoke garbage collection (though some hosts provide debugging APIs). Instead, the runtime decides when to run GC based on heuristics:

- **Allocation thresholds:** When a certain number of objects have been allocated or a memory threshold is reached, the engine schedules a GC cycle to reclaim space.
- **Memory pressure:** If there isn't enough free memory to satisfy new allocations, the engine immediately runs GC. This often happens on devices with limited RAM.
- **Idle periods:** Modern engines run incremental collection during idle times to reduce pauses. For example, a browser might perform some marking work between frames.
- **Generational heuristics:** Many engines divide objects into "young" and "old" generations. Short-lived objects (like those inside a function) are collected more frequently (minor GC), while long-lived objects trigger less frequent but more expensive major collections.

You cannot predict exactly when GC will run, but you can influence how often it runs by controlling how many objects you create and how long you retain them.

The mark-and-sweep algorithm

The classic GC algorithm used in JavaScript engines is **mark-and-sweep**. It runs in two phases:

1. **Mark phase:** Starting from a set of `_roots_` (global variables, the call stack, closures), the collector recursively **marks** all objects reachable through references. Reachable objects are considered "live".

2. **Sweep phase:** The collector then scans the heap and **sweeps away** any objects that weren't marked. These unreachable objects are reclaimed and their memory becomes available for future allocations.

Modern collectors add variations such as **generational** (separate young and old generations), **incremental** (break GC into small chunks), and **concurrent** (perform work on a background thread) collection. These techniques reduce the duration of "stop-the-world" pauses where the entire JavaScript thread is halted.

Impact on performance

Garbage collection can impact your program in two ways:

1. **Pause times (jank):** During a GC cycle, the engine may stop executing JavaScript to safely traverse and modify memory. Long GC pauses can cause animations to stutter or block user input. Minor collections are usually fast, but major collections can cause noticeable pauses.
2. **CPU usage:** Marking and sweeping take CPU time. If your code creates many short-lived objects (for example, allocating new arrays inside a tight loop), the GC may run frequently, consuming cycles that could be used for application logic.

Example: allocation patterns

Creating a large number of objects in a loop can trigger frequent minor collections:

```
```\nfunction allocateMany() {\n  const arr = [];\n  for (let i = 0; i < 100000; i++) {\n    arr.push({ index: i });\n  }\n  return arr;\n}\n\n// Each object becomes a candidate for GC once arr goes out of scope\n```\n
```

If `allocateMany()` is called repeatedly, the engine may interleave your code with GC cycles, slowing down the overall throughput.

### ## Mitigating GC overhead

While you cannot stop GC, you can write code that makes it less intrusive:

- **Reuse objects:** Instead of creating new objects inside loops, reuse existing ones where possible. This reduces allocation pressure.
- **Avoid retaining unnecessary references:** Let references go out of scope when you're done. Storing objects in global variables or long-lived closures prevents them from being collected.
- **Be careful with large data structures:** Holding onto large arrays or Maps can delay GC. Clear them (`array.length = 0` or `map.clear()`) when you're finished.
- **Prefer primitives and typed arrays:** Plain numbers, strings and typed arrays are simpler for the engine to manage than nested object graphs.
- **Use `WeakMap` and `WeakSet` for caches:** Weak collections hold references that do not prevent objects from being collected, which is useful for memoisation caches.
- **Measure memory usage:** Browser developer tools and Node's `--inspect` flag provide memory profiling to identify leaks and high allocation sites. Use them to verify that your changes reduce GC overhead.

### ## Real-world analogy

Imagine your house's cleaning service. The cleaners show up when the place is cluttered, stop everything, and tidy up. If you accumulate clutter quickly by buying things and never throwing anything away, the cleaners have to come more often, interrupting your routine. By buying less and discarding what you don't need, you reduce the frequency and duration of cleanings. Similarly, controlling allocations and releasing references reduces GC interruptions.

### ## Practice questions

1. **Theory:** Explain the two phases of the mark-and-sweep algorithm. Why does the collector need to stop executing JavaScript code during certain parts of the process?
2. **Theory:** What are generational garbage collectors, and how do they improve performance compared to a single heap?

3. **Coding:** Write a function that allocates a large number of objects inside a loop. Use performance tools to observe how GC frequency changes when you reuse a single object instead.
4. **Theory:** Name three things you can do in your code to reduce the frequency of garbage collection cycles.
5. **Coding:** Implement a simple cache using `WeakMap` to store the results of an expensive function. Explain how using `WeakMap` helps with memory management.

# What causes detached DOM node memory leaks and how to avoid them?

## # What causes detached DOM node memory leaks and how to avoid them?

Memory leaks occur when your application stores references to objects longer than necessary, preventing the garbage collector from reclaiming them. In web applications, a common source of leaks is **\*\*detached DOM nodes\*\***: elements that have been removed from the page but are still referenced by JavaScript.

## ## What are detached DOM nodes?

A **\*\*detached node\*\*** is an element that exists in memory but is no longer part of the document's active tree. This can happen if you remove an element from the DOM (for example, calling `element.remove()` or `parent.innerHTML = ""`) but keep a reference to it in a variable, array or closure. Because the garbage collector sees the reference, it considers the node reachable and doesn't free the memory associated with it.

Over time, detached nodes accumulate and consume memory, leading to degraded performance, especially on long-running pages like single-page applications.

## ## Common causes

1. **\*\*Storing DOM references globally\*\***: Assigning elements to global variables, object properties, or arrays and never clearing them can keep nodes alive after they are removed from the DOM.

```
``js
// Bad: storing removed nodes in an array
const cachedItems = [];

function addItem() {
 const item = document.createElement("li");
 item.textContent = "Item";
 document.body.appendChild(item);
 cachedItems.push(item);

 // Later, remove the item from the DOM
```

```

 item.remove();

 // item remains in cachedItems and cannot be collected
}
...

```

2. **\*\*Unremoved event listeners:\*\*** Adding an event listener to an element creates a reference from the event loop to that element via the closure that contains the callback. If you remove the element but never call `element.removeEventListener()`, the reference chain may keep it alive.

3. **\*\*Closures capturing DOM nodes:\*\*** Functions that enclose DOM variables can keep them alive even after removal. If you store such functions or pass them around, the enclosed nodes persist.

4. **\*\*Framework bugs:\*\*** Complex UI libraries may sometimes retain references to removed components due to internal caches or subscriptions.

## ## Strategies to avoid detached node leaks

- **\*\*Remove references:\*\*** Set your references to `null` or `undefined` when the node is no longer needed. For collections, call `.splice()` or `.clear()`.
- **\*\*Use event delegation:\*\*** Attach listeners to a common ancestor instead of individual elements. This reduces the number of listeners and avoids attaching callbacks directly to soon-to-be-removed elements.
- **\*\*Detach event listeners:\*\*** Always call `element.removeEventListener()` before removing an element. For class-based components, clean up listeners in a `destroy` or `unmount` method.
- **\*\*Weak references:\*\*** Use `WeakMap` or `WeakRef` to store metadata or caches keyed by elements. Weak references do not prevent the target from being collected.
- **\*\*Avoid storing DOM nodes globally:\*\*** If you need to cache values, store only data (like IDs or attributes) rather than element references.
- **\*\*Monitor memory:\*\*** Use browser developer tools (Performance or Memory panels) to identify detached nodes. Many tools highlight detached DOM trees and allow you to track their references.

## ### Example: fixing a leak

```

```js
const items = [];

```

```

function addAndRemove() {
  const el = document.createElement("div");
  el.textContent = "Hello";
  document.body.appendChild(el);
  // simulate some work
  setTimeout(() => {
    el.remove();
    // fix: remove references
    const idx = items.indexOf(el);
    if (idx !== -1) items.splice(idx, 1);
  }, 1000);
  items.push(el);
}
```

```

Alternatively, use a `WeakSet` so that removed elements don't prevent collection:

```

```js
const items = new WeakSet();
function addAndRemove() {
  const el = document.createElement("div");
  document.body.appendChild(el);
  items.add(el);
  setTimeout(() => {
    el.remove();
    // no need to delete from WeakSet
  }, 1000);
}
```

```

**## Real-world analogy**



Imagine removing an old piece of furniture from your living room but leaving a note in your home inventory that it's still there. Every time you move or clean, you think the furniture exists and allocate space for it. The physical item is gone, but because your records say otherwise, you never free the space. Clearing or updating your inventory is like clearing references to detached DOM nodes.

## ## Practice questions

1. **Theory:** What is a detached DOM node? How do references in your JavaScript code prevent it from being garbage-collected?
2. **Theory:** Why are unremoved event listeners a common source of memory leaks? How does event delegation help?
3. **Coding:** Write a function that creates a list of 1,000 `<li>` elements and then removes them. Use browser dev tools to verify whether any detached nodes remain. Modify the function to ensure that no leaks occur.
4. **Coding:** Refactor a component that caches DOM nodes in an array to instead use a `WeakMap` keyed by the node's ID. Explain why this prevents memory leaks.
5. **Theory:** Besides detached nodes, what other patterns can cause memory leaks in web applications? How would you identify them?

# What is the difference between microtasks, macrotasks, and animation frames?

## # What is the difference between microtasks, macrotasks and animation frames?

JavaScript executes code through an **event loop** that processes different kinds of tasks. Understanding the distinction between **microtasks**, **macrotasks** and **animation frames** helps you choose the right scheduling API and predict execution order.

### ## Macrotasks (tasks)

Macrotasks are the basic units of work in the event loop. They include:

- Callbacks scheduled with `setTimeout()` and `setInterval()`.
- I/O events (network, file system).
- `setImmediate()` (Node.js).
- User interactions (clicks, keypresses, etc.).

When the event loop dequeues a macrotask, it executes it from start to finish. After the macrotask completes, the browser drains the microtask queue (see below) and then may paint a frame. Each macrotask is also called a **tick**.

### ## Microtasks

Microtasks are higher-priority callbacks that run **after the current macrotask but before the next one**. They allow fine-grained scheduling so that state updates can occur before the browser performs any rendering or the next event is handled. Examples include:

- **Promise** callbacks attached via `.then()`, `.catch()` or `.finally()`.
- `queueMicrotask(callback)`: a way to enqueue a microtask manually.
- **MutationObserver** callbacks (in browsers).
- `process.nextTick()` (in Node.js), which runs even before other microtasks.

Microtasks are executed in order until the queue is empty. If a microtask schedules more microtasks, they run immediately after, which can starve the event loop if not used carefully.

### ## Animation frames

`requestAnimationFrame(callback)` schedules a function to run **right before the next repaint**. Browsers typically aim for 60 frames per second, so animation frame callbacks run about every 16 ms. The callback receives a timestamp and is executed after the microtask queue is empty but before painting. Use animation frames for smooth animations and layout reads/writes that should occur once per frame.

`requestAnimationFrame()` is neither a microtask nor a regular macrotask; it's part of the browser's rendering pipeline. If the page is in the background or hidden, the browser may throttle or stop calling animation frame callbacks to save resources.

### ## Execution order

During each tick of the event loop:

1. A macrotask is taken from the queue and executed.
2. Once the macrotask finishes, **all microtasks** are processed in order.
3. The browser may perform layout and paint.
4. If there's a pending animation frame and it's time to render, the browser calls the `requestAnimationFrame()` callbacks.
5. The loop proceeds to the next macrotask.

### ### Example

```
``js
console.log("start");

setTimeout(() => console.log("timeout"), 0);

Promise.resolve().then(() => {
```

```

 console.log("promise");
 queueMicrotask(() => console.log("microtask"));
 });

 requestAnimationFrame(() => console.log("animation frame"));

 console.log("end");
 ...

```

Possible output (exact order can vary slightly across environments, but the sequence of categories is consistent):

```

...
start
end
promise
microtask
animation frame
timeout
...

```

Explanation:

1. The synchronous code (`start`, `end`) runs immediately.
2. The promise callback is a microtask and runs after the current macrotask.
3. Within the promise, another microtask is queued and runs right away.
4. Next, before rendering, `requestAnimationFrame()` runs.
5. Finally, `setTimeout(..., 0)` runs on the next macrotask.

## ## Choosing the right scheduler

- **Microtasks** (`queueMicrotask`, `promises`): Use for short, high-priority work that must run before the browser paints, such as updating component state after a DOM event handler. Don't schedule long loops here or you might block rendering.
- **Macrotasks** (`setTimeout`, `setInterval`, `I/O callbacks`): Use for tasks that can wait until after rendering, such as logging, analytics, or deferring heavy computations.
- **Animation frames** (`requestAnimationFrame`): Use for work that coordinates with the next frame—reading layout, updating CSS transforms, performing animations. The callback runs only when the browser is ready to paint, ensuring smooth motion.

## ## Practice questions

1. **Theory:** Explain the difference between a microtask and a macrotask. Why do microtasks run before the next event loop tick?
2. **Theory:** Where does `requestAnimationFrame()` fit in the event loop relative to microtasks and macrotasks? What happens if the page is hidden?
3. **Coding:** Write code that schedules a `setTimeout()`, a resolved promise, and a `requestAnimationFrame()` call. Use `console.log()` statements to observe the order of execution.
4. **Coding:** Refactor an animation that uses `setTimeout()` into one that uses `requestAnimationFrame()`. Why does the latter produce smoother motion?
5. **Theory:** What problems can occur if you queue too many microtasks? How can you prevent microtask starvation?

# How do `Performance.now` and `PerformanceObserver` help in profiling?

## # How do `performance.now()` and `PerformanceObserver` help in profiling?

Measuring the performance of your JavaScript code often requires more precision than `Date.now()` can provide. The **Performance API** exposes high-resolution timestamps and tools for collecting timing data, enabling developers to profile code and identify bottlenecks. Two key pieces of this API are `performance.now()` and the `PerformanceObserver` interface.

### ## `performance.now()` - high-resolution timing

`performance.now()` returns a **DOMHighResTimeStamp**, a floating-point number representing the number of milliseconds elapsed since a performance "time origin" (typically when the page was created). Unlike `Date.now()`, which has millisecond resolution and can be affected by system clock changes, `performance.now()` provides sub-millisecond resolution and monotonically increasing values.

Example: measuring the duration of a function call:

```
``js
const t0 = performance.now();
doHeavyComputation();
const t1 = performance.now();
console.log(`Heavy computation took ${t1 - t0} ms`);
``
```

You can use `performance.now()` multiple times to measure different parts of your code. The high resolution helps detect even small performance regressions.

### ## User marks and measures

The Performance API also allows you to create **marks** and **measures**:

- `performance.mark(name)`: records a timestamp with a given name.
- `performance.measure(name, startMark, endMark)`: records the duration between two marks.

For example, to measure how long it takes to fetch data:

```
``js
performance.mark("fetch-start");
const response = await fetch("/api/data");
performance.mark("fetch-end");
performance.measure("fetch", "fetch-start", "fetch-end");

const entries = performance.getEntriesByType("measure");
const fetchEntry = entries.find((e) => e.name === "fetch");
console.log(`Fetch took ${fetchEntry.duration} ms`);
``
```

You can view these entries in the browser's performance tooling or access them programmatically.

## ## Observing performance entries with `PerformanceObserver`

`PerformanceObserver` lets you subscribe to performance entry events as they happen. You create an observer with a callback that receives a list of new entries. The callback runs asynchronously, but you can specify `buffered: true` to receive past entries as well.

```
``js
const obs = new PerformanceObserver((list) => {
 for (const entry of list.getEntries()) {
 console.log(entry.name, entry.entryType, entry.duration);
 }
});

obs.observe({ type: "measure", buffered: true });
```

```
// Now whenever performance.measure() is called, the observer callback logs the entry
...

```

You can observe different types of entries:

- **resource**: network requests (images, scripts, stylesheets).
- **paint**: first paint and first contentful paint.
- **longtask**: tasks that block the event loop for more than 50 ms, recorded by the Long Tasks API.
- **measure**, **mark**, **navigation**, etc.

This makes `PerformanceObserver` a powerful tool for monitoring your application in real time and integrating performance metrics into your logging or analytics systems.

## ## Practical use cases

1. **Profiling code sections**: Use `performance.now()` or marks/measures to time how long certain functions or operations take. This helps identify slow code paths.
2. **Monitoring page load**: Observe `navigation` and `paint` entries to see when the page started loading and when it rendered meaningful content (First Contentful Paint).
3. **Detecting long tasks**: Observe `longtask` entries to find parts of your code that block the event loop. Breaking long tasks into smaller chunks can improve responsiveness.
4. **Custom metrics**: Define your own marks and measures around user interactions or API calls. Send these metrics to your analytics endpoint to monitor performance in production.

## ## Real-world analogy

`performance.now()` is like using a stopwatch with microsecond precision instead of a wall clock. The `PerformanceObserver` is like having a reporter who notes every milestone during a race—start, halfway, finish—and hands you a detailed timeline at the end. With these tools, you can confidently optimise your application.

## ## Practice questions



1. **Theory:** Why is `performance.now()` preferred over `Date.now()` for measuring short time intervals? What characteristics make it more suitable?
2. **Theory:** Explain the difference between `performance.mark()` and `performance.measure()`. How are they used together?
3. **Coding:** Use marks and measures to time how long it takes to sort a large array of random numbers. Log the duration using a `PerformanceObserver`.
4. **Coding:** Observe `longtask` entries with `PerformanceObserver` and write code that intentionally blocks the event loop for 200 ms. Verify that a long task entry is recorded.
5. **Theory:** How can you integrate performance metrics collected via `PerformanceObserver` into your application's analytics or monitoring dashboard?

# What is the difference between `queueMicrotask`, `setTimeout`, and `requestAnimationFrame`?

## # What is the difference between `queueMicrotask`, `setTimeout` and `requestAnimationFrame`?

JavaScript offers several ways to schedule callbacks. Choosing the right one requires understanding how they interact with the event loop and rendering pipeline. Here's how `queueMicrotask()`, `setTimeout()` and `requestAnimationFrame()` differ.

### ## `queueMicrotask(callback)`

`queueMicrotask()` schedules a function to run **at the end of the current macrotask**, after the current call stack unwinds but before the browser or Node processes the next event. It adds the callback to the **microtask queue**, alongside resolved promise callbacks and `MutationObserver` notifications.

Microtasks run in **FIFO** order and are executed continuously until the queue is empty. Because they run before the next render, microtasks are ideal for small, immediate updates (e.g. updating component state) that should happen before the user sees the result. However, you must not queue heavy loops here or you risk starving the event loop.

Example:

```
```js
console.log("start");
queueMicrotask(() => console.log("microtask"));
console.log("end");
// Output: start, end, microtask
```
```

### ## `setTimeout(callback, delay)`

`setTimeout()` schedules a **macrotask**. The callback runs after at least the specified delay (in milliseconds) has elapsed. Even `0` doesn't guarantee immediate execution; the callback will run on the next tick after all current microtasks and rendering are done.

Use `setTimeout()` to defer work until after the browser repaints or to break large tasks into smaller pieces. Timers have a minimum clamping delay (usually 1-4 ms) in modern browsers and may be throttled in background tabs.

```
```js
setTimeout(() => console.log("timeout"), 0);
Promise.resolve().then(() => console.log("promise"));
// Output: promise, timeout
```
```

### ## `requestAnimationFrame(callback)`

`requestAnimationFrame()` schedules a callback to run **\*\*just before the next repaint\*\***. The browser passes a timestamp to the callback, which you can use to synchronise animations. The callback is executed after the microtask queue is empty but before painting. If the page is hidden, many browsers throttle or suspend rAF callbacks to conserve resources.

Use rAF for any code that updates animations, reads layout, or writes styles. Scheduling animation code here ensures it runs at the right time for smooth visuals.

```
```js
requestAnimationFrame((timestamp) => {
  // Update position based on timestamp
  box.style.transform = `translateX(${timestamp / 10}px)`;
});
```
```

### ## Putting it all together

Consider the following:

```
```js
```

```

queueMicrotask(() => console.log("microtask"));
setTimeout(() => console.log("timeout"), 0);
requestAnimationFrame(() => console.log("raf"));
...

```

Possible output:

```

...
microtask
raf
timeout
...

```

Explanation:

1. The microtask runs first because it is scheduled at the end of the current task.
2. On the next tick, before painting, the browser invokes the rAF callback.
3. Finally, the timer callback runs. Since its delay is 0, it fires on the next event loop iteration after microtasks and rAF.

Choosing between them

| Method cases | When it runs | Typical use |
|---------------------------|---|---|
| ----- | ----- | ----- |
| `queueMicrotask()` | After the current call stack, before rendering | Updating state that must occur immediately, chaining promises |
| `setTimeout()` | In a future macrotask, after a delay | Deferring work, throttling or debouncing, breaking up heavy tasks |
| `requestAnimationFrame()` | Before the next repaint, synced with frame rate | Animations, reading/writing layout, smooth UI updates |

Remember that microtasks should be short and non-blocking. If you need to schedule heavy computation, split it with `setTimeout()` or `setImmediate()`. Use rAF to coordinate with the browser's rendering loop.

Practice questions

1. **Theory:** How does `queueMicrotask()` differ from `setTimeout(fn, 0)` in terms of when the callback runs?
2. **Theory:** Why is `requestAnimationFrame()` better suited for animations than `setTimeout()` or `setInterval()`?
3. **Coding:** Write code that logs messages scheduled with `queueMicrotask()`, a resolved promise, `setTimeout(fn, 0)`, and `requestAnimationFrame()`. Observe the order of execution in your browser.
4. **Coding:** Use `requestAnimationFrame()` to animate an element smoothly across the screen. Then modify the code to use `setTimeout()` instead and compare the smoothness.
5. **Theory:** What precautions should you take when scheduling microtasks to avoid starving the event loop? How can you break large microtasks into smaller chunks?

How do hidden classes and inline caching affect JS performance internally?

How do hidden classes and inline caching affect JavaScript performance internally?

JavaScript is a dynamic language: objects can have properties added and removed at any time, and property names are strings. Yet modern engines like V8 execute property access at speeds comparable to statically typed languages. Two key techniques make this possible: ****hidden classes**** and ****inline caching****.

Hidden classes

When you create an object, the engine internally assigns it a ****hidden class**** (also called a *_shape_* or *_map_*). A hidden class describes the layout of the object's properties—what properties it has and in what order they were added. This allows the engine to determine the memory offsets of properties and access them quickly.

Consider the following constructor:

```
``js
function Person(name, age) {
  this.name = name;
  this.age = age;
}
const alice = new Person("Alice", 30);
const bob = new Person("Bob", 25);
``
```

Both ``alice`` and ``bob`` share the same hidden class because the same properties are added in the same order. The engine can treat them like instances of a "class" with known offsets for ``name`` and ``age``. Accessing ``alice.age`` becomes a simple offset lookup instead of a hash table search.

However, if you add a new property after creation, the engine creates a new hidden class:

```
``js
alice.location = "NYC";
``
```

Now `alice` has a different hidden class from `bob`, and property access on one cannot use the assumptions made for the other. Similarly, adding properties in different orders (`obj.a = 1; obj.b = 2` vs `obj.b = 2; obj.a = 1`) yields different shapes. Frequent shape changes inhibit optimisation.

****Best practices for shape stability:****

- Initialise all properties in the constructor or object literal to keep objects sharing the same shape.
- Avoid adding or deleting properties after creation. Instead, set unused properties to `null` to preserve the shape.
- Use classes or factory functions consistently so that similar objects follow the same property order.

Inline caching

Even with hidden classes, property lookup still requires checking the object's class and then calculating the offset. ****Inline caching (IC)**** speeds up repeated property accesses by caching the location of a property for a given hidden class directly in the machine code.

When the engine first executes `obj.foo`, it doesn't know the hidden class of `obj`. It generates a ****monomorphic inline cache**** that records the hidden class encountered and the offset of `foo`. The next time it sees `obj.foo` with an object of the same class, it skips lookup and reads the property directly. If it later encounters an object with a different class, the inline cache becomes ****polymorphic**** and stores multiple class-offset pairs. Too many shapes can degrade performance and cause the engine to fall back to slower generic lookup.

Inline caches are also used for method calls (`obj.method()`), array indexing, and binary operators. They allow the engine to optimise dynamic code by specialising for the most common types it sees.

Impact on performance

Hidden classes and inline caching turn dynamic property access into predictable, fast operations. When your code uses consistent object shapes and avoids shape changes, the engine can generate optimised machine code that performs at near-native speed. In contrast, code that adds properties

dynamically or stores heterogeneous objects in arrays may force the engine into megamorphic inline caches, reducing optimisation opportunities.

Example of consistent shapes

```
``js
// Good: consistent shape
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }
}

const points = [];
for (let i = 0; i < 10000; i++) {
  points.push(new Point(i, i));
}

// Accessing points[i].x and points[i].y is fast due to stable shapes

// Bad: inconsistent shapes
const objects = [];
for (let i = 0; i < 10000; i++) {
  const obj = {};
  if (i % 2 === 0) {
    obj.a = i;
    obj.b = i;
  } else {
    obj.b = i;
    obj.a = i;
  }
}
```



```
objects.push(obj);  
}  
// Accessing objects[i].a forces the engine to handle multiple shapes  
...
```

Real-world analogy

Hidden classes are like blueprints for houses in a new subdivision. If every house follows the same blueprint, you know where each room is located. Inline caching is like keeping a floor plan in your pocket so you can walk straight to the kitchen without looking around. If each house is rearranged differently, the floor plan becomes useless, and you have to explore each time you visit.

Practice questions

1. **Theory:** What is a hidden class (or shape) in a JavaScript engine? How does property order affect hidden classes?
2. **Theory:** Describe the difference between monomorphic and polymorphic inline caches. What causes an inline cache to become megamorphic?
3. **Coding:** Write two constructor functions that add properties in different orders. Create many objects from each constructor and measure the time it takes to access a common property. Compare the performance.
4. **Coding:** Refactor code that adds properties to an object after creation so that all properties are initialised in the constructor. How does this change affect hidden classes and potential optimisation?
5. **Theory:** Besides property access, what other operations in JavaScript engines use inline caching? Why do engines cache these operations?

What is ResizeObserver and how is it different from MutationObserver?

What is `ResizeObserver` and how is it different from `MutationObserver`?

Modern web layouts often need to respond to changes in element size. Responsive components, grids, and charts should adjust when their containers resize, even if the window itself doesn't change. The **ResizeObserver** API solves this problem by notifying you when an element's size changes. It complements the existing **MutationObserver**, which watches for changes in the DOM tree but not for size changes. Understanding the difference helps you choose the right observer for your task.

`ResizeObserver` - observing element size

`ResizeObserver` lets you watch the dimensions of one or more elements. You create an observer with a callback that receives a list of `ResizeObserverEntry` objects whenever the observed element's **border box** (including padding and border) or **content box** (excluding padding and border) changes size. The callback runs asynchronously—after layout but before paint—so repeated changes are batched.

Example:

```
```js
const box = document.querySelector(".container");
const ro = new ResizeObserver((entries) => {
 for (const entry of entries) {
 const { width, height } = entry.contentRect;
 console.log(`Size changed: ${width} × ${height}`);
 // Update layout or perform calculations here
 }
});
ro.observe(box);
```
```

When the `.container`` element grows or shrinks, your callback will run with the new dimensions. You can call `ro.unobserve(element)`` to stop watching or `ro.disconnect()`` to remove all observations.

Important notes:

- The callback may fire multiple times as the element changes (for example, when animating width). Use throttling or logic inside the callback to avoid expensive recalculations on every pixel change.
- `ResizeObserver`` runs before paint, so you should read sizes and apply layout adjustments but avoid heavy DOM mutations that might trigger additional reflows.

`MutationObserver` - observing DOM changes

`MutationObserver`` watches for **structural changes** in the DOM, such as:

- Adding or removing child nodes.
- Changing attributes or text content.
- Moving nodes within the tree.

It does **not** fire when an element's size changes due to CSS, flexbox, or grid adjustments. To use it, you create a `MutationObserver`` with a callback and call `observe()`` on a target node, specifying which mutations to watch:

```
```js
const observer = new MutationObserver((mutationList) => {
 mutationList.forEach((mutation) => {
 console.log(mutation.type);
 });
});

observer.observe(document.body, { childList: true, subtree: true });
```
```

This callback will run whenever nodes are added or removed anywhere in the document body, but it won't run if a node's size changes due to CSS.

Key differences

| Feature | `ResizeObserver` | `MutationObserver` |
|-------------------------------|--|--|
| Watches size changes | ✓ Yes | ✗ No |
| Watches DOM structure changes | ✗ No | ✓ Yes (additions, removals, attribute changes) |
| Callback timing | After layout, before paint | Microtask queue (after the current macrotask) |
| Typical use cases | Responsive components, chart resizing Updating UI when nodes are added/removed, syncing attributes | |

Choosing the right observer

- Use **`ResizeObserver`** when you need to react to changes in element dimensions, such as adjusting a canvas when its container resizes, or repositioning tooltips when their target grows.
- Use **`MutationObserver`** when you need to track changes to the DOM tree or attributes, like updating a counter when items are added to a list, or running logic when an attribute is toggled.
- You can use both observers together: a mutation could add a new element, which you then observe with a `ResizeObserver` to watch its size.

Practice questions

1. **Theory:** What events trigger a `ResizeObserver` callback? Why won't a `MutationObserver` fire when an element's width changes due to flexbox or CSS transitions?
2. **Theory:** Compare the timing of `ResizeObserver` callbacks with `MutationObserver` callbacks. When in the render cycle do each run?
3. **Coding:** Create a resizable panel and use a `ResizeObserver` to update a label showing its current dimensions. Resize the panel using CSS and verify that the observer fires.
4. **Coding:** Use a `MutationObserver` to log whenever items are added to or removed from a `<ul>` element. Then add a `ResizeObserver` to each `<li>` to log when a list item's size changes.

5. **Theory:** In what scenarios might you combine a `MutationObserver` and a `ResizeObserver`? Give an example where reacting to both structure and size changes is necessary.

How does the Clipboard API work for copying and pasting programmatically?

How does the Clipboard API work for copying and pasting programmatically?

Copying and pasting data is an essential part of user interaction. Modern browsers expose the **Clipboard API** to allow web pages to read from and write to the system clipboard asynchronously, in a secure and user-friendly way. Using the Clipboard API, you can implement features such as "Copy to clipboard" buttons, rich text editors and custom paste handlers.

Basic operations

The Clipboard API lives on the `navigator.clipboard` object. It provides four primary methods:

- **`writeText(text)`** - Copies the given string to the clipboard. Returns a promise that resolves when the text has been written.
- **`readText()`** - Reads plain text from the clipboard. Returns a promise that resolves with a string.
- **`write(data)`** - Copies arbitrary data (like images or rich text) represented as an array of `ClipboardItem` objects.
- **`read()`** - Reads arbitrary clipboard data and returns an array of `ClipboardItem` objects.

Example: copying text

```
``js
async function copyEmail() {
  try {
    await navigator.clipboard.writeText("user@example.com");
    console.log("Email copied!");
  } catch (err) {
    console.error("Failed to copy:", err);
  }
}
```

```
document.querySelector("#copy-btn").addEventListener("click", copyEmail);  
...
```

Example: reading text

```
```js  
async function pasteText() {
 try {
 const text = await navigator.clipboard.readText();
 console.log("Pasted:", text);
 } catch (err) {
 console.error("Failed to read clipboard:", err);
 }
}

document.querySelector("#paste-btn").addEventListener("click", pasteText);
...
```

### ### Copying images and rich content

To copy rich data, create `ClipboardItem` objects with MIME types and associated `Blob` data:

```
```js  
async function copyImage(imgBlob) {  
  const item = new ClipboardItem({ "image/png": imgBlob });  
  await navigator.clipboard.write([item]);  
}  
...
```

When reading, iterate over the returned items and call `getType(mime)` to retrieve the blob. Note that reading arbitrary clipboard data requires additional permissions (see below).

Permissions and security considerations

Because the clipboard can contain sensitive data, browsers restrict access to it:

- **Secure context:** Clipboard API methods are only available on HTTPS pages (or `localhost`).
- **User gesture:** Writing to the clipboard generally requires a user gesture, such as clicking a button. Reading from the clipboard often prompts the user to grant permission.
- **Permissions API:** You can check and request clipboard permissions:

```
```js
const status = await navigator.permissions.query({ name: "clipboard-read" });
console.log(status.state); // 'granted', 'prompt' or 'denied'
...
```
```

In many browsers, reading from the clipboard will trigger a permission prompt the first time. Writing text usually succeeds silently if called in response to a user action.

Fallbacks for older browsers

Some browsers don't support the asynchronous Clipboard API. A common fallback is to create a temporary ``, set its value, select it and use the now-deprecated `document.execCommand('copy')` to perform the copy. However, this method doesn't support images or rich content and is blocked in many contexts. When writing production code, detect support for `navigator.clipboard` and provide a fallback only for text copying when necessary.</p></div><div data-bbox="114 721 292 738" data-label="Section-Header"><h2>## Real-world analogy</h2></div><div data-bbox="114 774 869 826" data-label="Text"><p>The Clipboard API is like having a virtual clipboard in your app. You can politely ask the user to hand you a note (read), or you can hand them a note to keep (write), but only when they're paying attention (user gesture) and only if you're in the right room (secure context).</p></div><div data-bbox="114 862 284 879" data-label="Section-Header"><h2>## Practice questions</h2></div><div data-bbox="478 938 516 955" data-label="Page-Footer"><p>539</p></div><div data-bbox="353 949 641 979" data-label="Page-Footer"><p>Upamanyu Deka</p></div>

1. **Theory:** What restrictions do browsers impose on clipboard access? Why are these restrictions necessary?
2. **Theory:** Explain the difference between `writeText()` and `write()`. When would you use each?
3. **Coding:** Implement a "Copy code" button that writes the contents of a `<pre>` element to the clipboard and provides visual feedback when the operation succeeds or fails.
4. **Coding:** Write code that reads text from the clipboard and displays it in an input field. Handle any errors or permission prompts gracefully.
5. **Theory:** How would you handle clipboard copying in a browser that does not support `navigator.clipboard`? Discuss the limitations of your approach.

What is the Notification API and how can you request user permission?

What is the Notification API and how can you request user permission?

The **Notification API** allows websites to display system-level notifications to the user, even when the page is not in focus. These notifications can inform users about chat messages, reminders, updates or other events. Because they can be intrusive, the browser requires explicit user permission before a site can show notifications.

Requesting permission

Before creating a notification, check whether permission has been granted. The permission status can be `default`, `granted` or `denied`.

```
``js
if (Notification.permission === "granted") {
  showNotification();
} else if (Notification.permission !== "denied") {
  Notification.requestPermission().then((permission) => {
    if (permission === "granted") {
      showNotification();
    }
  });
}
```

```
function showNotification() {
  const notification = new Notification("Hello!", {
    body: "You have a new message.",
    icon: "/images/icon.png",
  });
  notification.onclick = () => {
    window.focus();
  }
}
```

```
notification.close();  
};  
}  
...
```

`requestPermission()` returns a promise that resolves with the user's decision. You should call it in response to a user gesture (like clicking a button) rather than immediately on page load, as browsers may block unprompted permission requests.

Notification options

When constructing a notification, you can specify various options:

- `body`: The main text of the notification.
- `icon`: A small icon displayed with the notification.
- `badge`: A monochrome symbol for small contexts (like Android status bars).
- `image`: A large image displayed below the text.
- `actions`: An array of action buttons, each with a title and an action identifier.
- `tag`: A string that groups notifications. Notifications with the same tag replace each other instead of stacking.
- `requireInteraction`: If `true`, the notification stays visible until the user dismisses it.

Different platforms support different subsets of options, so test on your target devices.

Notifications from service workers

In Progressive Web Apps, you can show notifications from a **service worker**. This is essential for push notifications received when the site isn't open. Inside the service worker, use `self.registration.showNotification(title, options)` instead of creating a `Notification` object directly. A push event might look like this:

```
``js  
self.addEventListener("push", (event) => {
```

```

const data = event.data?.json() ?? {};
const { title, message } = data;
event.waitUntil(
  self.registration.showNotification(title, {
    body: message,
    icon: "/images/icon.png",
  })
);
});
...

```

Service worker notifications behave similarly, but you don't need to request permission inside the worker; permission is granted or denied globally for your origin.

Best practices

- **Ask at the right time:** Request permission after the user performs an action that justifies notifications. Asking on page load often leads to denial.
- **Be respectful:** Don't spam users. Send notifications only when they're useful. Provide clear ways to manage preferences.
- **Handle denied and default states:** Some users will deny permission or ignore prompts. Gracefully degrade by updating the UI without notifications.
- **Use tags:** Group notifications with the same tag to avoid cluttering the notification tray.

Real-world analogy

Requesting notification permission is like asking someone if it's okay to send them text messages. If they agree, you can send messages when something important happens. If they decline, you respect their choice and find other ways to communicate.

Practice questions

1. **Theory:** What are the possible values of `Notification.permission`, and what do they mean? When is it appropriate to call `Notification.requestPermission()`?
2. **Theory:** Describe some of the options you can specify when creating a new notification. Which ones might not be supported on all platforms?
3. **Coding:** Implement a button that asks the user for notification permission and, upon approval, displays a custom notification with an action button. Handle the action by opening a specific page.
4. **Coding:** Write service worker code that listens for a push event and shows a notification using `self.registration.showNotification()`. Include a `tag` so new notifications replace older ones.
5. **Theory:** Why should you avoid requesting notification permission immediately when the page loads? Suggest a user flow that leads to higher opt-in rates.

Explain the Battery Status and Network Information APIs

Explain the Battery Status and Network Information APIs

Web applications are becoming more aware of their host environment. Two browser APIs—the **Battery Status API** and the **Network Information API**—allow your app to adapt its behaviour based on device battery level and network conditions. While both APIs are subject to privacy and support limitations, understanding them can help you design more responsive, energy-efficient applications.

Battery Status API

Overview

The Battery Status API exposes information about the system's battery through the `navigator.getBattery()` method. This method returns a promise that resolves to a `BatteryManager` object with the following properties:

- **charging** - `true` if the battery is currently charging.
- **level** - Battery charge level as a number between 0 and 1 (e.g. `0.5` for 50%).
- **chargingTime** - Approximate time (in seconds) until the battery is fully charged (0 when already full).
- **dischargingTime** - Approximate time (in seconds) until the battery is empty (Infinity if charging).

The `BatteryManager` object also emits events when these values change: `chargingchange`, `levelchange`, `chargingtimechange`, and `dischargingtimechange`. Example usage:

```
``js
async function monitorBattery() {
  const battery = await navigator.getBattery();

  function update() {
    console.log(`Battery level: ${Math.round(battery.level * 100)}%`);
    console.log(`Charging: ${battery.charging}`);
  }
}
```

```

update();

battery.addEventListener("levelchange", update);

battery.addEventListener("chargingchange", update);
}

monitorBattery();
'''

```

With battery information, you can postpone non-essential tasks when power is low, reduce network usage, or warn the user before a long download.

Privacy and support

Due to fingerprinting concerns (battery state could uniquely identify a device), many browsers have removed or restricted access to this API. Chrome and Edge dropped support, while some mobile browsers still implement it. Always check browser compatibility and provide fallbacks. If `navigator.getBattery`` isn't available, avoid gating core functionality on it.

Network Information API

Overview

The Network Information API exposes network quality and connection type via `navigator.connection`` (also known as `navigator.networkInformation``). The `connection`` object includes properties such as:

- **effectiveType`** - An estimate of the current connection quality (e.g. `'slow-2g'``, `'2g'``, `'3g'``, `'4g'``).
- **downlink`** - Estimated effective bandwidth in megabits per second.
- **rtt`** - Estimated round-trip time in milliseconds.
- **saveData`** - `'true'`` if the user has enabled a "reduce data usage" setting.

The API also emits a `'change'`` event when the connection changes. Example:

```

``js
function handleConnectionChange() {
  const conn = navigator.connection;
  console.log(
    `Network type: ${conn.effectiveType}, downlink: ${conn.downlink} Mb/s`
  );
  if (conn.saveData) {
    console.log("User prefers reduced data usage.");
    // Perhaps fetch lower-resolution assets
  }
}

if ("connection" in navigator) {
  handleConnectionChange();
  navigator.connection.addEventListener("change", handleConnectionChange);
}
...

```

You can use this information to adjust media quality, defer updates, or choose between offline and online modes. For example, a video player might automatically lower resolution when the connection is poor.

Limitations and privacy

Like the Battery API, the Network Information API is not universally supported (mainly available in Chrome and some Android browsers). Some browsers deliberately provide coarse or randomised values to prevent fingerprinting. Always check for feature presence and use reasonable defaults when it's missing.

Real-world analogy

Think of your application as a restaurant. The Battery Status API is like knowing how much fuel is left in your delivery truck; you might postpone long deliveries if the tank is nearly empty. The Network

Information API is like checking road conditions; you might choose a slower route or defer deliveries when traffic is heavy. By adapting to energy and network conditions, you provide a better service without wasting resources.

Practice questions

1. **Theory:** What information does the `BatteryManager` object provide? Why might a browser choose to restrict access to this API?
2. **Theory:** What does the `effectiveType` property of `navigator.connection` represent? How could a video streaming application use this information?
3. **Coding:** Write a function that uses the Battery API (if available) to postpone a large file download when the battery level is below 20% and the device isn't charging.
4. **Coding:** Use the Network Information API to adjust the quality of images loaded on a page. If the connection type is `'slow-2g'` or `'2g'`, load low-resolution images; otherwise, load high-resolution images.
5. **Theory:** What are some potential privacy risks of exposing battery and network information to websites? How can developers design applications that respect user privacy while still benefiting from these APIs?

What is the Fetch streaming API and how can you consume a streamed response?

What is the Fetch Streaming API and how can you consume a streamed response?

The **Fetch API** is the modern way to make network requests in JavaScript. By default, high-level methods like `response.json()` or `response.text()` buffer the entire response into memory before returning it. For large downloads or real-time data, this can be inefficient or cause jank. The **Fetch Streaming API** exposes the response body as a **ReadableStream**, allowing you to consume the data chunk by chunk as it arrives.

When streaming is useful

- **Large files:** Streaming prevents large responses from blocking memory and allows you to process parts of the file immediately (e.g. show progress while downloading a video).
- **Progressive rendering:** You can progressively display content (like streaming HTML or logs) instead of waiting for the full response.
- **Real-time feeds:** Server-sent events or chat messages can be delivered over long-lived HTTP requests that stream updates.

Accessing the stream

When you call `fetch()`, the returned `Response` object has a `body` property—an instance of `ReadableStream`—if the response supports streaming. There are two main ways to consume it: using a reader or using async iteration.

Consuming with a reader

```
``js
async function streamText(url) {
  const response = await fetch(url);
  const reader = response.body.getReader();
  const decoder = new TextDecoder();
  let received = "";
  while (true) {
```

```

const { value, done } = await reader.read();
if (done) break;
received += decoder.decode(value, { stream: true });
console.log("Received chunk:", value.length);
}
console.log("Full text:", received);
}
streamText("/large-file.txt");
```

```

`reader.read()` returns a promise that resolves with an object containing a `Uint8Array` of bytes and a `done` flag. Use a `TextDecoder` to convert bytes into strings. By processing each chunk, you can update progress indicators or parse partial content.

### ### Consuming with async iteration

Some environments support iterating over a `ReadableStream` directly:

```

```js
async function streamJSONLines(url) {
  const response = await fetch(url);
  const reader = response.body;
  const decoder = new TextDecoder();
  let buffer = "";
  for await (const chunk of reader) {
    buffer += decoder.decode(chunk, { stream: true });
    let lines = buffer.split("\n");
    buffer = lines.pop();
    for (const line of lines) {
      if (line) {
        const data = JSON.parse(line);
        console.log("Received item:", data);
      }
    }
  }
}
```

```

```

 }
 }
}
if (buffer) {
 console.log("Remaining:", buffer);
}
}
...

```

This example reads a newline-delimited JSON feed. Using `for await...of` yields each `Uint8Array` chunk. You accumulate partial lines until a newline and then parse JSON objects as they arrive.

### ### Streaming to other APIs

Because a response body is a `ReadableStream`, you can pipe it to other streams or build new `Response` objects from it. For example, you can stream a large file directly into a `WritableStream` provided by the [File System Access API] or into a custom transform stream that compresses data.

### ## Handling errors and cancellation

Streaming requests can be cancelled using `AbortController`. If the network connection drops, an exception will be thrown when reading from the stream. Surround your streaming logic with `try...catch` and handle partial data accordingly. Always close or cancel the reader if you stop consuming early.

### ## Real-world analogy

Imagine downloading a long podcast. Instead of waiting for the entire file to download before you start listening, streaming lets you begin listening while the rest of the file arrives. Similarly, the Fetch Streaming API allows you to process data progressively rather than waiting for the entire response.

### ## Practice questions

1. **Theory:** Why is streaming a response more memory-efficient than using `response.text()` or `response.json()` on large files?
2. **Theory:** What are the advantages of processing a streamed response as it arrives? Give two real-world scenarios where this approach is beneficial.
3. **Coding:** Write a function that downloads a large binary file using `fetch()` and streams it into a `WritableStream` that writes to the browser's IndexedDB or File System Access API. Show how to display progress.
4. **Coding:** Implement a function that consumes a newline-delimited JSON response using `ReadableStreamDefaultReader` and logs each parsed object as it arrives.
5. **Theory:** How can you cancel an ongoing fetch streaming operation? What happens to the `ReadableStream` when you call `abort()` on the `AbortController`?

# What is the Web Share API and when is it useful?

## # What is the Web Share API and when is it useful?

Sharing content—links, text, images—is a common action on mobile devices. The **Web Share API** allows websites to trigger the native sharing interface of the operating system, letting users seamlessly share data to social apps, messaging apps or other targets installed on their device. This API bridges the gap between web and native apps by providing a simple JavaScript interface for invoking the system share sheet.

## ## Basic usage

The API is exposed on `navigator.share()`. It accepts an object with up to three properties: `title`, `text` and `url`.

```
```js
async function shareArticle() {
  if (navigator.share) {
    try {
      await navigator.share({
        title: "Interesting article",
        text: "Check out this blog post on Web APIs!",
        url: "https://example.com/blog/web-share-api",
      });
      console.log("Content shared successfully");
    } catch (err) {
      console.error("Share failed:", err);
    }
  } else {
    alert("Web Share API not supported on this browser.");
  }
}

document.querySelector("#share-button").addEventListener("click", shareArticle);
```

...

Calling `navigator.share()` returns a promise that resolves when the user completes or cancels the share action. The method must be invoked in response to a user gesture (like a button click) and only works in **secure contexts** (`https`` or `localhost`).

Sharing files

An extension to the API (Level 2) allows sharing files using the `files`` property. This lets you share images, videos or arbitrary blobs. You create `File`` objects and pass them in an array:

```
```js
async function shareScreenshot(canvas) {
 const blob = await new Promise((resolve) =>
 canvas.toBlob(resolve, "image/png")
);
 const file = new File([blob], "screenshot.png", { type: "image/png" });
 await navigator.share({ files: [file], title: "Screenshot" });
}
...

```

File sharing is supported only on certain platforms (primarily Android and Chrome OS) and may require user gestures.

### ## Use cases and benefits

- **User convenience:** Users are familiar with the system share sheet. Integrating it provides a native feel and reduces friction compared to copying a link manually.
- **Consistency:** You don't need to implement your own share UI or integrate with each social platform's SDK. The OS handles the user's preferred apps.
- **Deep linking:** The `url`` property encourages sharing canonical links, improving SEO and discoverability.
- **Progressive enhancement:** You can detect support for `navigator.share`` and fall back to copying to clipboard or showing a share modal when unsupported.

## ## Limitations and considerations

- **Platform support:** The Web Share API is primarily supported on mobile browsers (Chrome, Edge, Samsung Internet). Desktop support is limited and may require experimental flags.
- **User gesture required:** Browsers restrict `navigator.share()` to user-initiated events. You can't auto-trigger sharing on page load.
- **No preview control:** Unlike custom share widgets, you can't customise the appearance of the share sheet. The OS decides how to present options.
- **Privacy:** The API doesn't provide feedback about which app the user chose or the content of the share. You only know whether it succeeded or failed.

## ## Real-world analogy

Imagine you're at a party and want to pass along a photo. You could either ask each friend what messaging app they use or hand them your phone and let them choose the app themselves. The Web Share API is like handing over your phone—it uses the person's preferred channel without you needing to know anything about it.

## ## Practice questions

1. **Theory:** What security and user interaction requirements must be met for `navigator.share()` to work? Why are these restrictions in place?
2. **Theory:** Describe the differences between Web Share API Levels 1 and 2. When would you use the `files` property?
3. **Coding:** Implement a share button that attempts to use `navigator.share()`. If unsupported, fall back to copying the current page's URL to the clipboard and informing the user.
4. **Coding:** Create a function that captures a screenshot of a canvas element and shares it via the Web Share API (if supported). Handle errors gracefully.
5. **Theory:** What are some scenarios where using the Web Share API enhances user experience compared to a custom in-app share modal?



# How does the Web Crypto API provide secure randomness and hashing?

## # How Does the Web Crypto API Provide Secure Randomness and Hashing?

The **Web Crypto API** is a set of low-level cryptographic primitives built into modern browsers. Its goal is to provide secure, performant operations—such as key generation, encryption/decryption, signing and hashing—without exposing the raw implementation details. Two of the most commonly used features are `_secure random number generation_` and `_cryptographic hashing_`. This article explains how the Web Crypto API implements these features and how you can use them safely.

### ## Secure randomness

In JavaScript there are two primary ways to generate random values:

- **`Math.random()`** - returns a floating-point number between `0` and `1`. It is *not* suitable for security-sensitive applications because the underlying algorithm is designed for speed, not unpredictability. Attackers can sometimes predict outputs or reconstruct seeds.
- **`crypto.getRandomValues()`** - part of the Web Crypto API. This function fills a typed array (e.g. `Uint8Array`, `Uint32Array`) with values drawn from the operating system's cryptographically secure pseudo-random number generator (CSPRNG). These values are unpredictable and suitable for generating keys, nonces, tokens or identifiers.

Using `crypto.getRandomValues()` is straightforward:

```
``js
// Generate 16 cryptographically secure random bytes
const bytes = new Uint8Array(16);
crypto.getRandomValues(bytes);
console.log(bytes);

// Convert bytes to a hexadecimal string (e.g. for a token)
const token = Array.from(bytes)
 .map((b) => b.toString(16).padStart(2, "0"))
 .join("");
```

```
console.log(token);
...
```

The API guarantees that every call returns new random values. You must provide your own typed array; the method does not allocate memory for you. Because it relies on the operating system's CSPRNG, it is unsuitable for use in a deterministic environment such as seeded pseudo-random simulations.

### ### Generating UUIDs

The Web Crypto API makes it easy to generate [UUID version 4](<https://www.rfc-editor.org/rfc/rfc4122>) values. A UUID is a 128-bit identifier with specific bits set according to the standard. Here's a helper function:

```
```js  
function generateUUIDv4() {  
  const bytes = crypto.getRandomValues(new Uint8Array(16));  
  // Set version (byte 6) to `0100` and variant (byte 8) to `10xx`  
  bytes[6] = (bytes[6] & 0x0f) | 0x40;  
  bytes[8] = (bytes[8] & 0x3f) | 0x80;  
  const parts = [  
    bytes.slice(0, 4),  
    bytes.slice(4, 6),  
    bytes.slice(6, 8),  
    bytes.slice(8, 10),  
    bytes.slice(10, 16),  
  ].map((arr) =>  
    Array.from(arr)  
      .map((b) => b.toString(16).padStart(2, "0"))  
      .join("")  
  );  
  return parts.join("-");  
}
```

```
console.log(generateUUIDv4());  
...
```

This example shows how to manipulate the random bytes to comply with the UUID format and then convert them into a human-readable string. Because the randomness comes from `crypto.getRandomValues()`, the resulting UUIDs are safe to use as identifiers in security-sensitive contexts (e.g. session IDs).

Cryptographic hashing

Hash functions map arbitrary data to fixed-length digests. A secure hash function must be deterministic, fast to compute, and computationally infeasible to invert or find collisions for. The Web Crypto API exposes hash functions via the asynchronous `SubtleCrypto` interface:

```
```js  
const data = new TextEncoder().encode("Hello, world!");
const digestBuffer = await crypto.subtle.digest("SHA-256", data);
const digestArray = Array.from(new Uint8Array(digestBuffer));
const digestHex = digestArray
 .map((b) => b.toString(16).padStart(2, "0"))
 .join("");
console.log(digestHex);
// Output: a591a6d40bf420404a011733cfb7b190...
```
```

The `digest` method takes an algorithm name (e.g. `"SHA-256"`, `"SHA-1"`, `"SHA-384"`, `"SHA-512"`) and an `ArrayBuffer` or typed array. It returns a `Promise` that resolves with an `ArrayBuffer` containing the digest. Converting this buffer into a hex or base64 string requires manual processing as shown above.

Hashing files

You can compute a hash of a large file without reading it all at once by streaming it through the `ReadableStream` API and updating a hash incrementally. However, `SubtleCrypto` does not currently support incremental hashing. To hash large files efficiently you can read chunks into memory, append them to a `Uint8Array`, and hash the entire buffer when done. For truly incremental hashing you need a third-party library.

Important considerations

- **Asynchrony** - Many `SubtleCrypto` methods return Promises because cryptographic operations may offload work to dedicated hardware or separate threads. Always `await` the returned values.
- **Algorithm support** - Browsers may support different sets of algorithms. Common hashes (SHA-1/2) and random value generation are widely available, while more advanced algorithms like Argon2 or BLAKE3 are not supported. Check `window.crypto.subtle` for available methods.
- **No direct encryption API for randomness** - The Web Crypto API exposes low-level building blocks. You still need to design protocols correctly (e.g. choose unique nonces, manage keys securely) when performing encryption or hashing operations.
- **Security of random values** - Always use `crypto.getRandomValues()` rather than `Math.random()` for security-critical randomness. Do not attempt to seed or re-seed the CSPRNG.

Practice questions

1. **Theory:** Why is `Math.random()` unsuitable for generating session tokens or cryptographic keys? What properties does a cryptographically secure random number generator provide?
2. **Coding:** Write a function that generates a 32-byte cryptographically secure random token and returns it as a base64 string.
3. **Theory:** Describe how the `crypto.subtle.digest()` method works. Why does it return a Promise? What steps do you need to perform to convert the digest into a hexadecimal string?
4. **Coding:** Implement a function that takes a user-provided password string, salts it with a random 16-byte salt, hashes it using SHA-256, and returns both the salt and the hash in hex format.

What is `crypto.getRandomValues` and why is it safer than `Math.random`?

What Is `crypto.getRandomValues()` and Why Is It Safer Than `Math.random()`?

When writing client-side JavaScript, you often need random values—for example to generate session tokens, unique IDs, salts, or keys. JavaScript provides two different randomness sources: the built-in `Math.random()` function and the `crypto.getRandomValues()` method from the Web Crypto API. They differ in design, purpose and security. Understanding those differences is essential when building secure applications.

`Math.random()` — fast but not secure

The `Math.random()` function returns a pseudo-random floating-point number between `0` (inclusive) and `1` (exclusive). Its implementation is deliberately simple and fast, which makes it appropriate for simulation, games and general-purpose randomness. However, because it is not designed for cryptographic use, the values it produces can be predicted given knowledge of the underlying algorithm and state. This predictability is the reason `Math.random()` **should never** be used for security-critical purposes such as generating passwords, authentication tokens or encryption keys.

For example, consider generating a six-digit code using `Math.random()`:

```
```js
// A simple but insecure code generator
function generateCode() {
 return Math.floor(Math.random() * 1_000_000)
 .toString()
 .padStart(6, "0");
}

console.log(generateCode());
```
```

While this code might appear to work, someone with access to the underlying pseudo-random sequence could predict or reproduce the same codes. Attackers have exploited predictable randomness in the past to break authentication schemes.

`crypto.getRandomValues()` — cryptographically secure

`crypto.getRandomValues()` is part of the Web Crypto API and uses the browser's operating system to generate cryptographically secure random values. It fills a supplied typed array (e.g. `Uint8Array`, `Uint32Array`) with random bytes. These values are drawn from a secure source and are unpredictable by design.

Here is how you might use it to generate a token:

```
``js
// Generate a 20-byte random token encoded in base64
function generateSecureToken() {
  const array = new Uint8Array(20);
  crypto.getRandomValues(array);
  // Convert bytes to base64
  const binary = String.fromCharCode(...array);
  return btoa(binary);
}

console.log(generateSecureToken());
...

```

Why it's safer

1. ****Unpredictability:**** The output cannot be feasibly predicted or reproduced without access to the underlying OS entropy source. Cryptographically secure random number generators (CSPRNGs) are designed so that even with partial knowledge of the internal state, an attacker cannot predict future outputs.

2. ****Uniform distribution:**** Values returned by `crypto.getRandomValues()` are uniformly distributed across the available range of the typed array. This avoids biases that can occur when manually mapping random floats to integers.
3. ****System entropy:**** Browsers delegate randomness to the operating system (e.g. `/dev/urandom` on Unix, `CryptGenRandom` on Windows), which collects entropy from hardware events. These sources are continually re-seeded.
4. ****Independent instances:**** Calling `crypto.getRandomValues()` in one tab does not affect the internal state for other calls or other tabs. By contrast, some implementations of `Math.random()` maintain a single global state that persists across calls.

Best practices

- ****Always prefer `crypto.getRandomValues()` for any sensitive random data.**** Use it when generating CSRF tokens, password reset codes, salts, encryption keys or unique identifiers.
- ****Do not attempt to re-seed or seed the CSPRNG yourself.**** The system manages its own entropy pool; manual seeding can weaken security.
- ****Use typed arrays appropriate for your needs.**** For example, use a `Uint32Array` to generate random 32-bit integers, or a `Uint8Array` for byte sequences. Convert to other formats (hex, base64) as needed.
- ****Combine with a cryptographic hash or HMAC when necessary.**** Sometimes you'll use `crypto.getRandomValues()` to create a secret, then derive keys using `crypto.subtle.importKey()` and `crypto.subtle.deriveKey()` for stronger key derivation and management.

Practice questions

1. ****Theory:**** Explain why values generated by `Math.random()` can sometimes be predicted. What kind of problems might this cause?
2. ****Coding:**** Write a function that generates an 8-character hexadecimal string using `crypto.getRandomValues()`. Why is this method safer than one based on `Math.random()`?
3. ****Theory:**** What types of arrays can be passed to `crypto.getRandomValues()`? What happens if you pass an unsupported typed array?
4. ****Coding:**** Modify the secure token generator to return a URL-safe base64 string (replace `+` with `-` and `/` with `_` and remove padding).

Explain WebRTC basics — data channels and peer connections

WebRTC Basics: Data Channels and Peer Connections

WebRTC (Web Real-Time Communication) is a set of browser APIs that enables peer-to-peer audio, video and data communication without requiring plugins. It allows two (or more) devices to exchange streams directly, making features like video conferencing, file sharing and collaborative editing possible in the browser. This article focuses on the core pieces—**RTCPeerConnection** and **RTCDataChannel**—and walks through an example of establishing a simple data channel.

RTCPeerConnection

The `RTCPeerConnection` interface represents a connection between two peers. It handles:

- **Signaling and negotiation** - exchanging session descriptions (SDP) and network information (ICE candidates) through an external signalling channel of your choice (WebSocket, HTTP, WebSocket server). WebRTC does not define signaling; you must implement your own mechanism to transfer negotiation messages between peers.
- **Network traversal** - using STUN and optionally TURN servers to determine public endpoints and traverse NAT/firewalls.
- **Media and data streams** - creating and managing media tracks (audio/video) and data channels over the connection.

Creating a peer connection is simple:

```
```js
const pc = new RTCPeerConnection({
 iceServers: [
 { urls: "stun:stun.l.google.com:19302" },
 // Optional: TURN servers for relaying if direct connection fails
],
});

// When the browser gathers a new ICE candidate (network info), send it to the other peer
```



```
pc.onicecandidate = (event) => {
 if (event.candidate) {
 sendToSignalingServer({ type: "candidate", candidate: event.candidate });
 }
};
```

```
// Handle negotiation needed event
pc.onnegotiationneeded = async () => {
 const offer = await pc.createOffer();
 await pc.setLocalDescription(offer);
 sendToSignalingServer({ type: "offer", sdp: pc.localDescription });
};
...

```

### ## RTCDataChannel

While WebRTC is often associated with video and audio, it also includes **data channels** — bidirectional communication channels for arbitrary data. Data channels use the same underlying transport as media streams (SRTP over UDP) and can be configured for reliability (`ordered`) and congestion control. They are ideal for sending small messages or file chunks in real time.

To create a data channel, call `pc.createDataChannel(name, options)` **before** negotiation begins on the initiating peer. On the remote peer you listen to the `ondatachannel` event to receive the channel:

```
```js
// Caller (creates the channel)
const channel = pc.createDataChannel("chat");
channel.onopen = () => console.log("Data channel open");
channel.onmessage = (e) => console.log("Received:", e.data);

// Callee (receives the channel)

```

```
pc.ondatachannel = (event) => {
  const channel = event.channel;
  channel.onopen = () => console.log("Data channel open");
  channel.onmessage = (e) => console.log("Received:", e.data);
};
...

```

Once the channel is open, you can send and receive strings or binary data:

```
```js
// Sending from the caller
channel.send("Hello from caller");

// Sending an ArrayBuffer
const buffer = new Uint8Array([1, 2, 3]).buffer;
channel.send(buffer);
...

```

### ### Establishing a connection: full flow

To establish a connection between two peers you need a signalling mechanism to exchange offers, answers and ICE candidates. A simplified flow:

1. **\*\*Create peer connections\*\*** on both peers and set up event handlers for ``icecandidate``, ``onnegotiationneeded`` and ``ondatachannel``.
2. **\*\*Create a data channel\*\*** on one peer before negotiation. This triggers an ``onnegotiationneeded`` event.
3. **\*\*Caller creates offer\*\***: call ``createOffer()``, set it locally with ``setLocalDescription()``, then send the offer to the other peer through your signalling server.
4. **\*\*Callee receives offer\*\***: call ``setRemoteDescription()``, then call ``createAnswer()``, set it locally, and send the answer back.

5. **\*\*Exchange ICE candidates\*\***: as each peer generates candidates, send them to the other peer and call `addIceCandidate()` to add them. Once enough candidates are gathered, a direct or relay connection is established.

6. **\*\*Use the data channel\*\***: when the channel's `onopen` event fires on both peers, you can call `send()` and handle `onmessage` events.

### ### Use cases for WebRTC data channels

- **\*\*Peer-to-peer chat or file sharing\*\*** - Users can send messages or transfer files directly without routing data through a server, reducing latency and bandwidth costs.
- **\*\*Collaborative editing\*\*** - Real-time collaborative applications can synchronize state by exchanging JSON patches or operational transforms.
- **\*\*Game networking\*\*** - Fast, low-latency peer communication for multiplayer games.

### ### Limitations and considerations

- **\*\*Signalling is not included\*\*** - You must implement or use an external signalling server to exchange negotiation messages. Without signalling, peers cannot discover each other.
- **\*\*Firewall/NAT traversal\*\*** - WebRTC attempts a direct connection, but may fall back to relaying via TURN servers. This can affect latency and requires additional server infrastructure.
- **\*\*Security\*\*** - Data channels are encrypted. However, you still need to handle authentication and access control at the application level.
- **\*\*Browser support\*\*** - All major browsers support WebRTC, but some features differ. Mobile browsers can have additional limitations. Always feature-detect and provide fallbacks.

### ## Practice questions

1. **\*\*Theory:\*\*** What purpose do STUN and TURN servers serve in WebRTC? Why are they necessary?
2. **\*\*Coding:\*\*** Implement a simple signalling mechanism using WebSockets that exchanges offers, answers and ICE candidates between two peers.
3. **\*\*Theory:\*\*** Explain the difference between a data channel created with `ordered: true` and one with `ordered: false`. When might you use an unordered channel?
4. **\*\*Coding:\*\*** Write a function that sends a file over a data channel by splitting it into chunks and reassembling it on the receiving peer.



# What is AbortController and how do you use it to cancel fetch requests?

## # What Is `AbortController` and How Do You Use It to Cancel Fetch Requests?

Long-running network requests can become unnecessary if the user navigates away, changes input or performs a different action. Canceling these requests frees up resources and improves responsiveness. The **AbortController API** provides a standardized way to cancel asynchronous operations, including `fetch()` calls, by signalling an abort event.

### ## The AbortController API

`AbortController` is a built-in interface that creates an **abort signal** (`AbortSignal`). The signal can be passed to APIs that support abortable operations—most notably `fetch()`, `ReadableStream`, and Web Crypto methods. When the controller's `abort()` method is called, it triggers an `abort` event on the signal, allowing the consuming API to cancel the operation.

Here's how to create and use an `AbortController`:

```
```js
// 1. Create a controller and obtain its signal
const controller = new AbortController();
const signal = controller.signal;

// 2. Pass the signal to fetch
fetch("https://jsonplaceholder.typicode.com/posts/1", { signal })
  .then((response) => response.json())
  .then((data) => console.log("Received:", data))
  .catch((err) => {
    if (err.name === "AbortError") {
      console.log("Fetch aborted");
    } else {
      console.error("Fetch error:", err);
    }
  })
}
```

```
});
```

```
// 3. Later, abort the request (e.g. user cancels)
```

```
setTimeout(() => {  
  controller.abort();  
}, 200);  
``
```

When `controller.abort()` is called, the fetch operation rejects with an `AbortError`. Catching this error allows your application to distinguish an intentional abort from a network failure or other exception.

Integrating AbortController in UI interactions

Abort controllers are often tied to user actions. For example, cancel a search request when the user types a new query:

```
``js  
let searchController;  
  
async function search(query) {  
  // Abort previous search if one exists  
  if (searchController) searchController.abort();  
  searchController = new AbortController();  
  
  try {  
    const response = await fetch(`/search?q=${encodeURIComponent(query)}`, {  
      signal: searchController.signal,  
    });  
    const results = await response.json();  
    displayResults(results);  
  } catch (err) {
```

```

    if (err.name !== "AbortError") {
      showError(err);
    }
  }
}

// Bind search function to input event
document.querySelector("#search").addEventListener("input", (e) => {
  search(e.target.value);
});
...

```

In this example, each time the user modifies the search box, the previous request is cancelled. Only the latest request continues, preventing outdated results from racing with newer ones.

Combining with timeouts

You can implement request timeouts using `AbortController` without `setTimeout()` race conditions. Here's a helper function that wraps `fetch()` with a timeout:

```

```js
async function fetchWithTimeout(url, options = {}, timeout = 5000) {
 const controller = new AbortController();
 const id = setTimeout(() => controller.abort(), timeout);
 try {
 const response = await fetch(url, {
 ...options,
 signal: controller.signal,
 });
 clearTimeout(id);
 return response;
 } catch (err) {

```

```

clearTimeout(id);

throw err;
}
}

// Usage
fetchWithTimeout("https://example.com/data", {}, 2000)
 .then((response) => response.json())
 .then((data) => console.log(data))
 .catch((err) => {
 if (err.name === "AbortError") {
 console.log("Request timed out");
 }
 });
...

```

This pattern ensures that the fetch is aborted if it takes longer than the specified timeout. You can also pass an external `AbortSignal` via `AbortSignal.timeout()` (in environments that support it) to simplify this logic.

## ## Limitations and notes

- **API support** - Only certain web APIs support abort signals. Many modern ones do (fetch, some streams, some crypto operations), but older APIs do not. Always refer to documentation.
- **One-time use** - An `AbortController` can only be aborted once. After calling `abort()`, the signal stays aborted and cannot be reused. Create a new controller for each new operation.
- **Propagation** - Abort signals can be combined using `AbortSignal.any()` or `AbortSignal.timeout()` (where available) to handle multiple cancellation conditions.
- **Server cooperation** - Canceling a fetch aborts the client-side processing, but the server may still process the request. For long-running server operations, implement server-side cancellation mechanisms (e.g. using websockets or custom abort signals).

## ## Practice questions



1. **Theory:** What happens when a `fetch()` with an associated `AbortSignal` is aborted? How can your code distinguish between an abort and another type of error?
2. **Coding:** Modify the search example so that it shows a spinner while the request is in progress and hides it when the request finishes or is aborted.
3. **Theory:** Why can a single `AbortController` only cancel once? Describe a situation where you would use `AbortSignal.any()`.
4. **Coding:** Write a wrapper around `fetch()` that accepts an optional `AbortSignal`. It should return a promise that rejects with a custom error if the fetch is aborted or times out.

# What is Content Security Policy (CSP) and why is it important?

## # What Is Content Security Policy (CSP) and Why Is It Important?

**\*\*Content Security Policy (CSP)\*\*** is a security standard that helps prevent a broad class of injection attacks—such as cross-site scripting (XSS), clickjacking and data injection—by controlling which resources the browser is allowed to load and execute. CSP is delivered via HTTP headers or `<meta>` tags and instructs the browser to enforce restrictions on scripts, styles, images, fonts, frames and other assets. By defining a strict policy, you significantly reduce the attack surface of your application.

## ## Why CSP matters

Injection attacks occur when an attacker can inject arbitrary HTML or JavaScript into a page. The browser executes this code with the same privileges as trusted code, allowing attackers to steal cookies, deface pages or perform actions on behalf of the user. Even with input validation and output encoding, mistakes can happen. CSP acts as a *\_second line of defense\_* by preventing the browser from executing untrusted code even if it makes its way into the DOM.

Without CSP, a single unescaped user comment containing `<script>alert('xss')</script>` could trigger an alert or worse. With a properly configured CSP that disallows inline scripts and only allows scripts from trusted domains, the browser will block the injection.

## ## Basic syntax

A CSP is defined using the `Content-Security-Policy` header or the `<meta http-equiv="Content-Security-Policy">` tag. A policy consists of **\*\*directives\*\***, each specifying allowed sources for a type of resource. Some common directives include:

- `default-src` - fallback for resources not covered by other directives.
- `script-src` - allowed sources for JavaScript. Accepts URLs, `'self'`, `'none'`, `'unsafe-inline'`, `'unsafe-eval'`, hashes, and nonces.
- `style-src` - allowed sources for CSS.
- `img-src`, `font-src`, `frame-src`, etc. - allowed sources for images, fonts, frames.
- `object-src`, `base-uri`, `form-action`, `connect-src` - restrict other behaviours.

A simple CSP might look like this:

```
```http
Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.example.com; style-src 'self'
'unsafe-inline'; object-src 'none'
```
```

This policy allows content to be loaded from the site's own origin ('self'). Scripts are permitted from the same origin and a trusted CDN. Inline styles are allowed (though this weakens protection) and all plugins (<object>, <embed>) are disallowed.

### ### Nonces and hashes

To allow specific inline scripts or styles while blocking arbitrary inline code, CSP supports **nonces** and **hashes**:

- **Nonces** - A nonce is a random string generated on each request and included as a `nonce` attribute on `<script>` or `<style>` tags. The policy specifies `script-src 'nonce-<value>'`. Only scripts with the matching nonce are executed; all others are blocked.
- **Hashes** - You can include a hash of the exact script contents in the policy (`script-src 'sha256-<hash>'`). The browser computes hashes of inline scripts and executes them only if they match an allowed hash. This prevents script tampering.

Nonces and hashes allow inline scripts without opening up vulnerabilities to arbitrary injection. They are preferable to `'unsafe-inline'`, which disables inline script protections entirely.

### ### Reporting

CSP can be configured to report violations via the `report-uri` or `report-to` directives. When a violation occurs, the browser sends a JSON report to the specified endpoint containing details like the blocked URI and violated directive. You can enable **report-only mode** by using the `Content-Security-Policy-Report-Only` header. This is useful for testing a policy before enforcing it.

### ## Best practices for implementing CSP

1. **\*\*Start in report-only mode.\*\*** Apply a strict policy with the ``-Report-Only`` header to capture violations in production without blocking users. Review and adjust the policy based on reports, then switch to enforcement mode.
2. **\*\*Avoid broad sources.\*\*** Using ``*`` or allowing entire external domains (e.g. ``https://example.com/*``) increases risk if those domains are compromised. Permit only trusted, specific hostnames.
3. **\*\*Disable inline execution.\*\*** Remove ``unsafe-inline`` and ``unsafe-eval`` from your ``script-src`` directive. Use external scripts or nonces/hashes for inline code.
4. **\*\*Use per-request nonces.\*\*** Generate a new random nonce on each request to prevent reuse and make it difficult for attackers to guess.
5. **\*\*Combine with other defenses.\*\*** CSP complements but does not replace input validation, output encoding and proper authentication/authorization. Use CSP as part of a layered security strategy.

### ## Practice questions

1. **\*\*Theory:\*\*** Describe how CSP can prevent XSS even when an attacker manages to inject a ``<script>`` tag into your page.
2. **\*\*Coding:\*\*** Write an Express.js middleware that sets a CSP header blocking inline scripts but allowing scripts from ``cdnjs.cloudflare.com``. Add a report endpoint that logs violations.
3. **\*\*Theory:\*\*** What is the difference between ``Content-Security-Policy`` and ``Content-Security-Policy-Report-Only``? Why might you use the latter during deployment?
4. **\*\*Coding:\*\*** Explain how you would use a random nonce to permit a single inline script to run while still blocking other inline scripts.

# What is cross-site scripting (XSS) and how can JavaScript prevent it?

## # What Is Cross-Site Scripting (XSS) and How Can JavaScript Prevent It?

**\*\*Cross-Site Scripting (XSS)\*\*** is a security vulnerability that allows attackers to inject malicious scripts into web pages viewed by other users. When a vulnerable page displays unsanitised user input as HTML, an attacker can inject a `<script>` tag, event handler or other code that runs in the victim's browser. This malicious code can steal cookies, read sensitive data, or perform actions on behalf of the user. XSS is one of the most common web vulnerabilities and comes in three main forms: stored, reflected and DOM-based.

## ## Types of XSS

1. **\*\*Stored XSS (persistent)\*\*** - The attacker stores malicious code on the server (e.g. in a comment field). Each time a user loads the page, the stored script runs.
2. **\*\*Reflected XSS\*\*** - Malicious code is included in a URL or form parameter and immediately reflected back in the response. Users who click the link trigger the script.
3. **\*\*DOM-based XSS\*\*** - The client-side JavaScript manipulates the DOM using untrusted input (e.g. `location.hash`) and inserts it into the page using `innerHTML`, causing the injection to run without a new HTTP response.

Regardless of type, XSS occurs because **\*\*untrusted data is treated as code\*\***. Preventing XSS requires ensuring that data is safely encoded or sanitised before it reaches the DOM.

## ## Prevention techniques

### ### 1. Output encoding and escaping

When inserting user content into the DOM, avoid APIs that interpret input as HTML. For example, instead of using `innerHTML`, use `textContent` or `innerText` to insert untrusted strings. These methods treat the input as plain text and do not parse markup:

```
``js
```

```
// Unsafe: This will parse HTML and execute any embedded scripts
```

```
element.innerHTML = `<p>${comment}</p>`;
```

```
// Safe: Inserts text without interpreting it as markup
```

```
element.textContent = comment;
```

```
// If you must insert HTML, sanitise it using a library
```

```
import DOMPurify from "dompurify";
```

```
element.innerHTML = DOMPurify.sanitize(userHtml);
```

```
...
```

When generating HTML on the server, ensure you properly escape special characters (`<`, `>`, `&`, `"`, `'`) before inserting user content. Most templating engines have built-in mechanisms to do this automatically when using safe interpolation.

### ### 2. Use Content Security Policy (CSP)

CSP can mitigate XSS by blocking inline scripts and restricting the domains from which scripts can be loaded. Even if an attacker manages to inject markup, the browser will refuse to run it if it violates the policy. Use nonces or hashes to allow only known inline scripts and disallow `'unsafe-inline'`.

### ### 3. Validate and sanitise input

Never trust user input. Validate data on both the client and server for type, length and format. Reject or sanitise unexpected values. For rich text editors or user-generated HTML, use a robust sanitiser like [DOMPurify](<https://github.com/cure53/DOMPurify>) to remove dangerous tags and attributes.

### ### 4. Avoid dangerous JavaScript APIs

Using ``eval()``, ``Function()``, ``setTimeout()``/``setInterval()`` with string arguments, or dynamically generating HTML via string concatenation increases risk. Instead, work with data structures and functions directly. Where dynamic evaluation is unavoidable (e.g. configuration), strictly control the input and environment.

### ### 5. Escaping within JavaScript templates

If you're using client-side templating libraries (e.g. React, Vue, Handlebars), avoid dangerously setting HTML. React automatically escapes content when using JSX. Only use ``dangerouslySetInnerHTML`` when you have sanitised input.

### ### 6. HTTP-only and secure cookies

Mark session cookies as ``HttpOnly`` and ``Secure`` so that they cannot be accessed via JavaScript and are transmitted only over HTTPS. This reduces what an attacker can do even if they manage to inject a script.

## ## Practice questions

1. **Theory:** Explain the difference between stored and reflected XSS. Give an example of each.
2. **Coding:** Refactor the following vulnerable code to prevent XSS:

```
``js
// Vulnerable: inserts user input directly into the DOM
const name = location.search.slice(1);
document.getElementById("welcome").innerHTML = `

Hello ${name}!</h2>`; ``


```

3. **Theory:** How does using a CSP header complement other XSS prevention techniques? Can CSP alone eliminate all XSS vulnerabilities?
4. **Coding:** Implement a function that safely displays user comments, escaping any HTML tags and attributes before inserting them into the page.

# What is cross-site request forgery (CSRF) and how can JS help mitigate it?

## # What Is Cross-Site Request Forgery (CSRF) and How Can JavaScript Help Mitigate It?

**\*\*Cross-Site Request Forgery (CSRF)\*\*** is an attack that tricks a user's browser into making unwanted requests to a different site where the user is authenticated. Since browsers automatically include cookies (and sometimes HTTP authentication headers) with each request, an attacker can cause the victim's browser to perform actions—like changing account settings or transferring money—without their knowledge. Unlike XSS, CSRF exploits the trust a site has in the user's browser rather than the trust the user has in the site.

For instance, suppose you're logged into your bank at `bank.example.com`. If you visit a malicious page that contains `

## ## Defenses against CSRF

### ### 1. Synchronizer tokens (anti-CSRF tokens)

The most common defense is to include a **\*\*unique, unpredictable token\*\*** in each state-changing request. The server generates this token and associates it with the user's session. The token is embedded in the page (e.g. in a hidden form field) and must be submitted with the request. Because an attacker cannot read the token (due to same-origin policy), they cannot construct a valid request.

When using `fetch()` or AJAX requests, your JavaScript can read the token from a meta tag or cookie and include it in the `X-CSRF-Token` header:

```
```html
<meta name="csrf-token" content="abc123" />
```

```js
async function postData(url, data) {
  const token = document.querySelector('meta[name="csrf-token"]').content;
```



```

const response = await fetch(url, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "X-CSRF-Token": token,
  },
  credentials: "include",
  body: JSON.stringify(data),
});
return response.json();
}
...

```

The server validates the `X-CSRF-Token` against the session. If missing or incorrect, the request is rejected.

2. SameSite cookie attribute

Modern browsers support the `SameSite` cookie attribute which controls whether cookies are sent on cross-site requests. Setting `SameSite=Lax` or `SameSite=Strict` on session cookies restricts the circumstances in which browsers include cookies for cross-site requests, mitigating CSRF. For example:

```

```http
Set-Cookie: sessionId=...; Path=/; Secure; HttpOnly; SameSite=Lax
...

```

With `SameSite=Lax`, cookies are sent for top-level navigations but not for third-party requests like hidden images or iframes. `SameSite=Strict` prevents cookies from being sent for any cross-site request. Combined with `Secure` and `HttpOnly`, this greatly reduces CSRF risk.

### ### 3. Double submit cookies

Another technique is the **\*\*double submit cookie\*\*** pattern: the server sets a CSRF token cookie (accessible to JavaScript) and requires the same value in a request header or form field. Since JavaScript must explicitly read the cookie and include its value, an attacker cannot supply it from a different site.

#### ### 4. Custom headers and CORS preflight

If your API requires clients to include a custom header (e.g. ``X-Requested-With: XMLHttpRequest`` or ``Authorization: Bearer <token>``), browsers will automatically trigger a CORS preflight for cross-site requests. The preflight uses an ``OPTIONS`` request that does **\*\*not\*\*** include cookies. The server can reject such requests based on the ``Origin`` header. This technique is often used for JSON APIs where the client uses ``fetch()`` with ``credentials: 'include'`` only on same-origin interactions.

#### ### 5. Frame busting and UI considerations

Clickjacking attacks can pair with CSRF to trick users into clicking hidden buttons inside iframes. Use frame busting (``X-Frame-Options: DENY`` or ``Content-Security-Policy: frame-ancestors 'none'``) to prevent your site from being embedded in iframes. Provide clear UI feedback and confirm critical actions with additional credentials (e.g. reenter password).

### ## Role of JavaScript

While CSRF protections are primarily implemented on the server, **\*\*JavaScript plays a supporting role\*\***:

- **\*\*Including tokens in requests\*\*** - Client-side code can read the CSRF token from a meta tag or cookie and attach it to every ``fetch()`` or AJAX request. Without this, SPA frameworks may inadvertently omit the token.
- **\*\*Using ``credentials: 'same-origin'``\*\*** - When using ``fetch()``, set ``credentials`` appropriately (``'same-origin'`` or ``'include'``). This ensures cookies are sent only when permitted by the server and respects ``SameSite`` restrictions. It also prevents unintended credential leakage to third-party origins.
- **\*\*Avoiding cross-site requests from untrusted pages\*\*** - Do not fetch cross-origin resources with credentials in contexts you cannot control, such as dynamic scripts or untrusted iframes.

### ## Practice questions

1. **Theory:** Explain how a CSRF attack might work against a user who is logged in to a banking site. What conditions are necessary for the attack to succeed?
2. **Coding:** Write a helper function that reads a CSRF token from a `<meta>` tag and includes it in a JSON POST request using `fetch()`.
3. **Theory:** Describe how the `SameSite` cookie attribute helps mitigate CSRF. Compare `SameSite=Lax` with `SameSite=Strict` in terms of user experience.
4. **Coding:** Implement a server-side anti-CSRF middleware (in Express.js or similar) that verifies an incoming `X-CSRF-Token` against the value stored in the session. Ensure that the token cannot be guessed.

# Explain sandboxed iframes and the same-origin policy in browsers

## # Sandboxed ``<iframe>``s and the Same-Origin Policy in Browsers

Embedding content from other sites has always been a core part of the web. ``<iframe>`` elements allow you to display another webpage inside your page, but doing so raises security concerns. Browsers enforce a **Same-Origin Policy (SOP)** that restricts how documents or scripts loaded from one origin can interact with resources from another origin. Additionally, ``<iframe>``s support a ``sandbox`` attribute that can further restrict the capabilities of the embedded content. Understanding these mechanisms helps you embed third-party content safely.

### ## The Same-Origin Policy

The same-origin policy is a critical security feature implemented by browsers to prevent malicious sites from interacting with the sensitive data of other sites. Two URLs share the same origin if they have the **same protocol**, **host** and **port**. Under the SOP:

- A page loaded from ``https://example.com`` cannot read or modify the DOM of a page from ``https://another.com``. It also cannot access cookies, localStorage or IndexedDB from another origin.
- Scripts are allowed to send requests to any domain, but they cannot read responses from cross-origin requests unless the server explicitly allows it via [Cross-Origin Resource Sharing (CORS)](<https://developer.mozilla.org/en-US/docs/050-what-is-cors-and-how-does-it-work>).
- Forms and image tags are not restricted and can submit cross-origin requests, but they cannot read responses.

This policy prevents a malicious site from reading a user's email or banking details by embedding or loading the site and accessing its content via JavaScript.

### ### Cross-origin communication

Sometimes you need to communicate with an iframe loaded from a different origin (e.g. a payment widget). You cannot call its functions directly due to SOP, but you can use the **postMessage API**. Both the parent and iframe can send messages using `window.postMessage(message, targetOrigin)`, and listen for `message` events with `window.addEventListener('message', handler)`. Ensure you verify `event.origin` to make sure messages come from a trusted domain.

## ## Sandboxed `<iframe>`'s

The `<iframe>` element includes a boolean `sandbox` attribute that places the embedded document in a **\*\*unique, restricted browsing context\*\***, regardless of its origin. The restrictions remove many of the iframe's capabilities, and you can selectively re-enable some via **\*\*sandbox flags\*\***.

By default, `sandbox` does the following:

- Disallows form submission.
- Disables scripts from running.
- Prevents the iframe from opening new windows (`window.open()`).
- Blocks `alert()`, `prompt()` and other dialogs.
- Forces the iframe to be treated as cross-origin—even if it has the same origin as the parent—meaning it cannot access the parent's DOM or cookies.

You can relax these restrictions by adding a space-separated list of tokens:

- `allow-scripts` - permits JavaScript execution.
- `allow-same-origin` - treats the document as same origin with respect to SOP, enabling DOM access if the origin matches.
- `allow-forms` - permits form submission.
- `allow-popups` - allows `window.open()` and target-blank links.
- `allow-modals`, `allow-pointer-lock`, `allow-top-navigation`, etc. - enable specific capabilities.

Example:

```
```html
<!-- Completely sandboxed: no scripts, no forms -->
<iframe src="https://thirdparty.example.com" sandbox></iframe>

<!-- Allows scripts but still treats the iframe as cross-origin -->
<iframe src="https://thirdparty.example.com" sandbox="allow-scripts"></iframe>
```

```
<!-- Allows scripts and same-origin if the iframe is from the same domain -->
<iframe src="/widget.html" sandbox="allow-scripts allow-same-origin"></iframe>
...
```

The `sandbox` attribute is useful when embedding potentially untrusted content, such as user-generated HTML or third-party widgets. It prevents the embedded page from manipulating your page or stealing data, even if it comes from the same domain.

Practical example: Embedding a third-party widget

Suppose you integrate a third-party comments widget. You want the widget to run its own scripts but not interfere with your site. You can embed it with a sandbox:

```
```html
<iframe
 src="https://comments.example.com/embed"
 sandbox="allow-scripts allow-forms"
 title="Comments Widget"
></iframe>
...
```
```

This allows the widget to execute its scripts and submit forms (e.g. comment submissions) but still isolates it from your origin. Even if the widget is compromised, it cannot access your DOM or cookies. If you need to receive events from the widget, use `postMessage()` with an explicit `targetOrigin` check:

```
```js
window.addEventListener("message", (event) => {
 if (event.origin === "https://comments.example.com") {
 console.log("Received message:", event.data);
 }
});
```
```

```
// Send a message to the iframe when ready

const iframe = document.querySelector("iframe");

iframe.contentWindow.postMessage("init", "https://comments.example.com");

...

```

Summary

Sandboxed iframes and the same-origin policy are powerful tools for isolating untrusted content. The SOP prevents scripts from one origin accessing resources from another, while the `sandbox` attribute can restrict even same-origin content to a safe environment. Use them to protect your users and your site when embedding external content.

Practice questions

1. **Theory:** Describe the difference between the same-origin policy and the `` attribute. How do they complement each other?&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;2. &amp;amp;lt;b&amp;amp;gt;Coding:&amp;amp;lt;/b&amp;amp;gt; Create a page with an embedded iframe that loads a trusted same-origin page. Use `sandbox` to disable forms and prevent the iframe from navigating the top window but allow scripts. Demonstrate sending a message from the iframe to the parent using `postMessage()`.&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;3. &amp;amp;lt;b&amp;amp;gt;Theory:&amp;amp;lt;/b&amp;amp;gt; What risks could arise if you include `allow-same-origin` in a sandboxed iframe pointing to an untrusted third-party domain?&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;4. &amp;amp;lt;b&amp;amp;gt;Coding:&amp;amp;lt;/b&amp;amp;gt; Write a helper function that listens for `message` events and verifies the origin before processing the data. Explain why origin checks are important.&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;/ol&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="479 938 516 955" data-label="Page-Footer"&amp;amp;gt;&amp;amp;lt;p&amp;amp;gt;586&amp;amp;lt;/p&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="353 949 641 979" data-label="Page-Footer"&amp;amp;gt;&amp;amp;lt;p&amp;amp;gt;Upamanyu Deka&amp;amp;lt;/p&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;

What is the Trusted Types API and how does it defend against XSS?

What Is the Trusted Types API and How Does It Defend Against XSS?

Browser vendors continually evolve security features to mitigate cross-site scripting (XSS). **Trusted Types** is a new API designed to eliminate a whole class of DOM-based XSS vulnerabilities by forcing developers to explicitly create and approve HTML, script URLs and other critical DOM values. When enabled, the browser rejects assignments of raw strings to dangerous sinks (e.g. `element.innerHTML` or `eval()`) unless the value is a *Trusted Type* object created through a registered policy. This approach ensures that only sanitised or vetted content can reach these sinks.

Why traditional measures fall short

Developers often mitigate XSS using input validation, output encoding and a Content Security Policy (CSP) that blocks inline scripts. However, client-side code can still inadvertently create DOM-based XSS vulnerabilities by concatenating untrusted strings and assigning them to properties like `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `setAttribute('src', ...)`, or by calling `eval()` on untrusted input. Attackers can exploit these to inject scripts even if the server sanitises input. As projects grow and involve multiple contributors or dependencies, it becomes hard to audit every string assignment.

How Trusted Types works

Trusted Types introduces three object types: **TrustedHTML**, **TrustedScript**, and **TrustedScriptURL**. The browser refuses to use regular strings in critical DOM sinks when Trusted Types is enforced. Instead, you must provide a Trusted Type object. These objects can only be created through **policies** that you define. A policy is a function that processes input and returns a Trusted Type if it is safe. If it cannot guarantee safety, it should throw an error.

Enabling Trusted Types

Trusted Types is activated via CSP by adding the directive `require-trusted-types-for 'script'`. You can also use `trusted-types <policy-names>` to whitelist policies by name. Without a policy, assignments to dangerous sinks throw a `TypeError`.

Example header:


```
```http
```

```
Content-Security-Policy: require-trusted-types-for 'script'; trusted-types default; report-uri /csp-report
```

```
```
```

This tells the browser to require Trusted Types for all script-related sinks and defines a single policy called `default`.

Defining a policy

You define a policy using `trustedTypes.createPolicy(name, handlers)`. The handlers are functions that take a string and return a Trusted Type. For HTML, a common approach is to sanitise or escape dangerous markup.

```
```js
```

```
// Register a default policy that sanitises HTML using DOMPurify
```

```
import DOMPurify from "dompurify";
```

```
const policy = trustedTypes.createPolicy("default", {
 createHTML: (input) => DOMPurify.sanitize(input),
 createScriptURL: (url) => {
 const allowed = url.startsWith("https://trusted.cdn.example.com/");
 if (!allowed) throw new TypeError("Untrusted script URL");
 return url;
 },
});
```

```
// Later: set HTML using the TrustedHTML object
```

```
element.innerHTML = policy.createHTML(userComment);
```

```
// Set a script element's src using TrustedScriptURL
```

```
scriptElement.src = policy.createScriptURL(
 "https://trusted.cdn.example.com/lib.js"
);
...
```

The `createHTML` handler sanitises or transforms untrusted user input into a safe HTML string, then wraps it as a `TrustedHTML` object. If the string contains disallowed content, it may remove or encode it. Similarly, `createScriptURL` ensures that script sources come only from approved domains.

### ### Violations and reporting

When Trusted Types is enabled and a script attempts to assign a string to a restricted sink without converting it into a Trusted Type, the browser throws a `TypeError`. You can capture such violations using CSP report endpoints and adjust your code accordingly.

### ## Advantages of Trusted Types

- **Enforced sanitisation** - You cannot accidentally introduce DOM XSS via string concatenation. All assignments to sensitive sinks must go through your policy, centralising sanitisation logic.
- **Integrates with CSP** - Policies are configured via CSP, so they work in tandem with `script-src` restrictions and other headers.
- **Library support** - Frameworks and libraries can create their own policies to generate Trusted Types, making it easy to adopt.
- **Fine-grained control** - You can create multiple policies with different behaviours for different parts of your application.

### ## Migration considerations

Because Trusted Types is opt-in and can break legacy code, migrate incrementally:

1. Add `Content-Security-Policy: require-trusted-types-for 'script'` in report-only mode. Inspect violation reports to find code paths that need fixing.
2. Create a default policy that sanitises or escapes user input. Replace assignments to `innerHTML`, `outerHTML`, `src` attributes and other sinks with calls to `policy.createHTML()` or similar methods.

3. Gradually tighten your policy or split it into multiple policies as needed.
4. Switch to enforcement mode once all violations are resolved.

## ## Practice questions

1. **Theory:** Explain why using `innerHTML` with untrusted data can introduce DOM-based XSS. How does Trusted Types help prevent this?
2. **Coding:** Write a Trusted Types policy that allows only images hosted on `example.com` to be assigned to `img.src`. Show how to apply it when creating an `<img>` element.
3. **Theory:** How do `require-trusted-types-for` and `trusted-types` directives in CSP differ? Why is a report-only mode useful when adopting Trusted Types?
4. **Coding:** Suppose your application uses a rich text editor that outputs HTML. Show how to sanitise the HTML using your policy before inserting it into the DOM.

# What is the Cache Storage API and how does it relate to Service Workers?

## # What Is the Cache Storage API and How Does It Relate to Service Workers?

Modern web applications often need to operate reliably under poor network conditions or offline. The **Cache Storage API** provides a way to store and retrieve network request/response pairs in named caches. It is most commonly used in combination with **service workers** to intercept network requests and serve cached resources. Understanding how this API works empowers you to implement robust caching strategies and improve the performance and resilience of your site.

## ## The Cache Storage API

The Cache Storage API is exposed globally through the `caches` object (in window and workers) and allows you to create and manage separate caches, each identified by a name. Each cache holds entries mapping requests to responses. A cache is like a persistent key-value store for HTTP requests and responses:

```
``js
// Open or create a cache named 'static-assets'
const cache = await caches.open("static-assets");

// Add individual entries
await cache.put(
 "/styles/main.css",
 new Response("body { color: red; }", {
 headers: { "Content-Type": "text/css" },
 })
);

// Retrieve an entry
const response = await cache.match("/styles/main.css");
if (response) {
 const text = await response.text();
}
```

```

 console.log(text); // 'body { color: red; }'
 }

 // Delete an entry
 await cache.delete("/styles/main.css");

 // Iterate over cached requests
 for (const request of await cache.keys()) {
 console.log(request.url);
 }
 ...

```

The `cache` object itself has methods such as `cache.keys()` (list cache names), `cache.delete(name)` (remove a cache), and `cache.match(request)` (search all caches).

### ### Important points

- **Entries are opaque** - Responses are stored exactly as returned from the network. If you store a response for a `GET` request, the response must be `_basic_` or `_CORS-allowed_`; opaque responses cannot be read due to cross-origin restrictions but can still be cached.
- **Order matters** - `cache.match()` searches caches in reverse order of creation; the first matching response is returned. Consider the order when layering caches.
- **Storage limits** - Browsers impose storage quotas for caches (often a percentage of available storage). Manage your caches by removing outdated entries during service worker activation.

### ## Integrating with service workers

A **service worker** is a background script that intercepts network requests via the `fetch` event. By listening to `fetch`, you can respond with cached resources or fall back to the network. The Cache Storage API is the primary mechanism for storing those cached responses.

Here's a basic service worker that caches assets during installation and serves them from the cache on subsequent visits:

```

``js
// sw.js

const PRECACHE = "precache-v1";

const PRECACHE_ASSETS = ["/", "/index.html", "/styles.css", "/app.js"];

// Install event: populate the cache
self.addEventListener("install", (event) => {
 event.waitUntil(
 caches.open(PRECACHE).then((cache) => cache.addAll(PRECACHE_ASSETS))
);
});

// Fetch event: respond from cache or network
self.addEventListener("fetch", (event) => {
 event.respondWith(
 caches.match(event.request).then((cachedResponse) => {
 if (cachedResponse) {
 return cachedResponse;
 }

 // Otherwise fetch from network and optionally cache the result
 return fetch(event.request).then((networkResponse) => {
 return caches.open(PRECACHE).then((cache) => {
 // Cache a clone of the response; streams can be read once
 cache.put(event.request, networkResponse.clone());
 return networkResponse;
 });
 });
 })
);
});

```

```
// Activate event: clean up old caches
self.addEventListener("activate", (event) => {
 const currentCaches = [PRECACHE];
 event.waitUntil(
 caches.keys().then((cacheNames) => {
 return Promise.all(
 cacheNames.map((name) => {
 if (!currentCaches.includes(name)) {
 return caches.delete(name);
 }
 })
);
 })
);
});
...

```

This code caches essential assets on install, serves them from the cache if available on fetch, and removes old caches during activation. You can implement more sophisticated strategies, such as **network-first** (try the network, fallback to cache), **stale-while-revalidate** (serve from cache and update in background), or **cache-first** depending on the resource type.

### ### Cache vs HTTP cache

The browser's built-in HTTP cache stores responses based on headers like `Cache-Control` and `ETag`. You cannot directly control or inspect it from JavaScript. The Cache Storage API is separate; you decide what to store and how to respond. When using the service worker cache, you should set appropriate headers on responses you generate (e.g. `Cache-Control: no-store`) to prevent duplicate caching in the HTTP cache. You can also use both: rely on the HTTP cache for short-term caching and the service worker cache for offline fallbacks.

### ## Practice questions

1. **Theory:** Explain the difference between the browser's HTTP cache and the Cache Storage API. Why would you choose to use the latter in a service worker?
2. **Coding:** Write a service worker handler that implements a network-first strategy for API requests while caching static assets with a cache-first strategy.
3. **Theory:** What happens if you call `cache.put()` with a response whose body has already been read? How do you avoid this issue?
4. **Coding:** Implement a function that iterates over all stored caches, deletes those whose names do not start with a given prefix, and logs the number of deleted entries.



# How do Progressive Web Apps (PWAs) leverage Service Workers for offline access?

## # How Do Progressive Web Apps (PWAs) Leverage Service Workers for Offline Access?

**Progressive Web Apps (PWAs)** are web applications that behave like native apps: they load fast, work offline, and can be installed on a user's device. A key technology enabling these capabilities is the **service worker**, a background script that intercepts network requests and can return cached responses when the network is unavailable or slow. This article explores how service workers empower PWAs to provide offline access and improved performance.

### ## Service worker basics

A service worker is a JavaScript file that runs separately from the main page and is registered by the application. Once registered and activated, it can:

- **Intercept network requests** through the `fetch` event.
- **Cache resources** using the Cache Storage API.
- **Receive push notifications** and display them.
- **Synchronise data in the background** via the Background Sync API.

Because service workers run outside the page context, they do not have direct access to the DOM, but they can communicate with the page via the `postMessage` API.

### ## Caching strategies for offline access

To enable offline access, PWAs use service workers to pre-cache resources and serve them when the network is unavailable. There are several common caching strategies:

#### ### 1. Cache-first

When an asset is requested, the service worker first looks in the cache. If found, it returns the cached response. If not, it fetches from the network, caches the response and returns it. This strategy ensures quick loads for static assets like CSS and JavaScript bundles.

```

```js
self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.match(event.request).then((cached) => {
      return (
        cached ||
        fetch(event.request).then((response) => {
          return caches.open("dynamic").then((cache) => {
            cache.put(event.request, response.clone());
            return response;
          });
        })
      );
    })
  );
});
```

```

### ### 2. Network-first

For dynamic content (e.g. API responses), you may prefer the freshest data. The service worker fetches from the network first and falls back to the cache if the network fails. Optionally it caches the response for future offline use.

```

```js
self.addEventListener("fetch", (event) => {
  if (event.request.url.startsWith("/api/")) {
    event.respondWith(
      fetch(event.request)
        .then((response) => {
          const cloned = response.clone();
          caches.open("api").then((cache) => cache.put(event.request, cloned));
        })
    );
  }
});
```

```

```

 return response;
 })
 .catch(() => caches.match(event.request))
);
}
});
...

```

### ### 3. Stale-while-revalidate

This hybrid strategy returns the cached response immediately (stale) while simultaneously fetching an updated version from the network. When the network response arrives, the cache is updated for next time. Users get instant responses but still see fresh content on subsequent visits.

```

```js
self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.match(event.request).then((cached) => {
      const networkFetch = fetch(event.request).then((response) => {
        caches
          .open("dynamic")
          .then((cache) => cache.put(event.request, response.clone()));
        return response;
      });
      return cached || networkFetch;
    })
  );
});
...

```

4. Offline fallback

You can provide an offline HTML page as a fallback when the network is unavailable. During installation, cache an `offline.html` file. In the fetch handler, if both network and cache miss, return the offline page:

```
```js
self.addEventListener("fetch", (event) => {
 event.respondWith(
 fetch(event.request)
 .catch(() => caches.match(event.request))
 .then((response) => {
 return response || caches.match("/offline.html");
 })
);
});
```
```

Web App Manifest and installation

Besides service workers, PWAs use a **Web App Manifest** (`manifest.json`) to provide metadata—name, icons, start URL and theme colors—allowing browsers to install the app on the home screen. When combined with an active service worker, the browser will prompt users to install the PWA. Installed PWAs launch in a standalone window and use cached assets even when offline.

Additional capabilities

- **Background sync** - Service workers can queue failed network requests and retry them later when connectivity returns. This is useful for sending form data or API calls while offline.
- **Push notifications** - Service workers handle push events to display notifications even when the site isn't open.
- **Periodic sync and triggers** - Emerging APIs allow service workers to periodically update cached data or react to triggers (e.g. network status changes).

Considerations and challenges

- **Storage limits** - Offline storage quotas vary by browser and may be cleared if the user's device runs low on space. Use caching judiciously and provide mechanisms to clear old caches.
- **Update lifecycle** - Service workers are versioned. Updating a service worker requires careful handling of the `install` and `activate` events to avoid serving stale assets. Use `skipWaiting()` and `clients.claim()` appropriately to control when the new worker takes over.
- **Testing offline** - Simulate offline conditions using browser dev tools. Ensure that your PWA behaves gracefully when network requests fail.

Practice questions

1. **Theory:** Compare and contrast the cache-first and network-first caching strategies. When would you choose each?
2. **Coding:** Write a service worker that serves a cached image gallery from the cache if available, fetches from the network otherwise, and updates the cache in the background.
3. **Theory:** Why is an `offline.html` fallback page important in a PWA? How would you cache and serve it?
4. **Coding:** Describe how you would implement background sync using the service worker to ensure a form submission succeeds even when the user goes offline after submitting.

What is the difference between deep clone and structuredClone for complex objects?

What Is the Difference Between Deep Clone and `structuredClone()` for Complex Objects?

Copying objects in JavaScript seems straightforward, but under the hood it can be tricky. Shallow copies (e.g. using spread syntax `{...obj}`) duplicate only the top level, leaving nested objects shared between the original and the copy. **Deep cloning** aims to create a complete, independent copy of the entire data structure. Historically, developers wrote custom deep copy functions or used `JSON.parse(JSON.stringify(obj))` with all its limitations. The `structuredClone()` method, introduced in modern browsers and Node.js, offers a built-in alternative for duplicating complex objects. This article compares these approaches and highlights their differences.

Custom deep cloning

Deep cloning is usually implemented by recursively copying properties. The simplest approach serializes to JSON and parses back:

```
```js
const original = { name: "Alice", nested: { arr: [1, 2] } };
const copy = JSON.parse(JSON.stringify(original));
copy.nested.arr.push(3);

console.log(original.nested.arr); // [1, 2] - unaffected
```
```

However, this method has significant drawbacks:

- **Loss of non-JSON types** - `undefined`, functions, symbols, dates, regexes, `Map`, `Set`, `Error` objects, and custom prototypes are lost or converted to plain objects. For example, dates become strings and cannot be converted back automatically.
- **Circular references** - If an object references itself or contains cycles, `JSON.stringify()` throws a `TypeError`. You must write custom logic to track references and rebuild cycles.
- **Prototypes and descriptors** - JSON serialization ignores property descriptors, getters/setters, and prototype chains. The copied object becomes a simple object literal.

- ****Performance**** - Large objects are serialized into strings, which can be slow and memory-intensive.

To overcome these limitations, developers often write or use deep-clone utilities that handle special cases, but these can be error-prone and heavy.

`structuredClone()`

The global `structuredClone()` method implements the ****structured clone algorithm****, which is the same algorithm used by `postMessage()` to copy data between windows or workers. It supports a wide range of built-in types, including:

- Objects, arrays and nested primitives
- Dates, RegExps, `Map`, `Set`
- Typed arrays, ArrayBuffers, DataViews
- Errors, Blob/File objects, ImageBitmaps
- Circular references and cyclic graphs

It does *not* support functions, DOM nodes, WeakMaps/Sets, or objects with private properties. The cloned object has the same prototype as the original (except where prototypes are not cloneable).

Example:

```
``js
const original = {
  date: new Date(),
  regex: /hello/gi,
  map: new Map([[1, "one"]]),
  set: new Set([1, 2, 3]),
  buffer: new Uint8Array([1, 2, 3]).buffer,
};
```

```

const clone = structuredClone(original);
console.log(clone.date instanceof Date); // true
console.log(clone.regex instanceof RegExp); // true
console.log(clone.map instanceof Map); // true
console.log(clone.set instanceof Set); // true
console.log(clone.buffer instanceof ArrayBuffer); // true
...

```

Notice that the cloned values retain their constructors and behave like the originals. Circular references are also handled gracefully:

```

```js
const obj = {};
obj.self = obj;
const clone = structuredClone(obj);
console.log(clone.self === clone); // true
...

```

### ### Transferable objects

`structuredClone()` also supports transferring ownership of certain objects—such as `ArrayBuffer`, `MessagePort`, `OffscreenCanvas`—rather than cloning them. To transfer an object, pass it via the second argument:

```

```js
const buffer = new ArrayBuffer(8);
const clone = structuredClone(buffer, { transfer: [buffer] });
// After transfer, `buffer` is detached and cannot be used
console.log(buffer.byteLength); // 0
console.log(clone.byteLength); // 8
...

```


This is useful when moving large buffers between workers to avoid expensive copying.

Choosing the right approach

- **Use `structuredClone()` whenever available** - It handles many built-in types, supports cycles, and preserves prototypes. It is now supported in modern browsers and Node.js.
- **Avoid JSON cloning for complex data** - Only use `JSON.stringify()`/`JSON.parse()` when the data consists of simple, JSON-safe values and performance is not critical.
- **Custom deep cloning when necessary** - If you need to clone functions, classes with custom prototypes, or special objects not supported by `structuredClone()`, you may need a bespoke solution.

Practice questions

1. **Theory:** What limitations does `JSON.stringify()` have when used for deep cloning? Provide at least three examples of data types that are not preserved.
2. **Coding:** Write a function that uses `structuredClone()` to clone an object containing a `Map` and a `Set`. Verify that the cloned object's values are of the correct types.
3. **Theory:** Explain what happens when you attempt to structured-clone an object containing a function property. How would you handle cloning such objects?
4. **Coding:** Demonstrate how to use the `transfer` option of `structuredClone()` to move an `ArrayBuffer` to a web worker. Show that the original buffer becomes detached.

How do custom iterators work and how can you build your own iterable object?

How Do Custom Iterators Work and How Can You Build Your Own Iterable Object?

JavaScript's iteration protocols allow objects to define their own iteration behaviour so that they can be used in `for...of` loops, spread syntax, destructuring and other iterable contexts. Understanding how these protocols work enables you to create custom iterable data structures such as ranges, linked lists or streams.

The Iterable and Iterator protocols

Two related protocols govern iteration:

1. **Iterable** - An object is iterable if it has a method keyed by `Symbol.iterator` that returns an `_iterator_`.
2. **Iterator** - An iterator is an object with a `next()` method that returns an object of the form `{ value, done }`. Each call to `next()` returns the next value in the sequence and sets `done` to `true` when iteration is complete.

The `for...of` loop calls the iterable's `[Symbol.iterator]()` method to get an iterator and then repeatedly calls `next()` until `done` is `true`.

Building a custom iterable

Consider creating a simple range object that yields numbers from `start` (inclusive) to `end` (exclusive) in steps of 1:

```
``js
// Range constructor
function Range(start, end) {
  this.start = start;
  this.end = end;
}
```

```
// Define the iterator on the prototype
Range.prototype[Symbol.iterator] = function () {
  let current = this.start;
  const end = this.end;
  return {
    next() {
      if (current < end) {
        return { value: current++, done: false };
      }
      return { done: true };
    },
  };
};
```

```
const range = new Range(3, 7);
for (const n of range) {
  console.log(n); // 3, 4, 5, 6
}
...
```

Explanation

- The `Range` constructor stores the start and end values.
- `Range.prototype[Symbol.iterator]` returns an iterator object with a `next()` method.
- Inside `next()`, we keep track of the current value. Each call returns an object with the next value and `done: false`, then increments the value. When we reach the end, `done` becomes `true` and the `value` property can be omitted or set to `undefined`.

This pattern allows `range` to be used with any construct that consumes iterables:

```
```js
```

```

console.log([...range]); // [3, 4, 5, 6]

const [first, ...rest] = range;

console.log(first, rest); // 3 [4, 5, 6]

...

```

## ## Using generator functions

Writing iterators by hand can be verbose. **Generator functions** (`function*`) simplify the process by managing the internal state for you. If a generator function includes a `yield` expression, it implicitly implements the iterator protocol:

```

``js

function* rangeGenerator(start, end) {
 for (let i = start; i < end; i++) {
 yield i;
 }
}

const genRange = rangeGenerator(3, 7);

for (const n of genRange) {
 console.log(n); // 3, 4, 5, 6
}

// Generators can be called again to start over

console.log([...rangeGenerator(1, 4)]); // [1, 2, 3]

...

```

The `yield` keyword suspends the generator, preserving its state. Each call to `next()` resumes execution until the next `yield` and returns the yielded value. When the function completes, the iterator indicates `done: true`.

## ## Designing your own iterable objects

When designing custom iterables, consider:

- **\*\*State management\*\*** - Decide how you will track progress (e.g. an index, a pointer to a node, etc.). Avoid modifying the iterable object itself if you want to allow multiple simultaneous iterators.
- **\*\*Reusability\*\*** - Objects with a `[Symbol.iterator]` method that returns a *\_new iterator\_* each time allow multiple independent iterations. If you return the same iterator instance, repeated iterations will pick up where the previous one left off.
- **\*\*Infinite sequences\*\*** - You can model potentially infinite sequences (e.g. Fibonacci numbers, random values) with iterators. Consumers decide when to stop iterating (e.g. using `'break'` in a loop).
- **\*\*Error handling and cleanup\*\*** - Iterators can implement `'return()'` and `'throw()'` methods that are called when iteration terminates early or encounters an error. These methods let you release resources or propagate errors properly.

## ## Practice questions

1. **\*\*Theory:\*\*** Explain the roles of `'Symbol.iterator'` and `'next()'` in enabling iteration over an object. Why does the iterator return an object with `'value'` and `'done'` properties?
2. **\*\*Coding:\*\*** Implement an iterable that yields only the even numbers within a given inclusive range. Provide both a hand-written iterator and a generator implementation.
3. **\*\*Theory:\*\*** Discuss the trade-offs between implementing an iterator manually and using a generator function. When might you choose one approach over the other?
4. **\*\*Coding:\*\*** Design a custom iterable that walks a binary tree in in-order traversal. Use either an explicit stack or a generator to implement the traversal.

# What are ArrayBuffer and TypedArray, and how are they different from Arrays?

## # What Are `ArrayBuffer` and Typed Arrays and How Are They Different from Arrays?

Working with raw binary data in JavaScript is common in applications such as audio/video processing, cryptography, file parsing and WebGL. Standard JavaScript arrays are designed for general-purpose use and store references to any type of value. **`ArrayBuffer`** and **`typed array`** views provide a way to handle fixed-length binary data efficiently and interoperably with native APIs.

### ## `ArrayBuffer` — a raw block of memory

An `ArrayBuffer` represents a generic, fixed-length block of memory. It contains bytes of data but has no knowledge of how to interpret them. You cannot read or write bytes directly on the `ArrayBuffer`; instead, you use one of the typed array classes or a `DataView` to access the bytes:

```
```\js
const buffer = new ArrayBuffer(8); // 8 bytes (64 bits)

console.log(buffer.byteLength); // 8

// Create a view to interpret the buffer as 32-bit integers
const int32View = new Int32Array(buffer);
int32View[0] = 42;
int32View[1] = -1;
console.log(int32View); // Int32Array [ 42, -1 ]

// Underlying memory is shared across views
const uint8View = new Uint8Array(buffer);
console.log(uint8View); // Uint8Array [ 42, 0, 255, 255, 0, 0, 0, 0 ]
```\
```

In this example, writing to the `Int32Array` view affects the underlying buffer, and the `Uint8Array` view reveals the same data as bytes. This shared memory model makes typed arrays efficient for processing binary data.

## ## Typed arrays — strongly typed views on buffers

Typed arrays are array-like objects that view an `ArrayBuffer` through a specific numeric type. There are multiple typed array classes, each corresponding to a C-style numeric type:

- `Int8Array`, `Uint8Array`, `Uint8ClampedArray`
- `Int16Array`, `Uint16Array`
- `Int32Array`, `Uint32Array`
- `Float32Array`, `Float64Array`

When you create a typed array, you specify either an existing `ArrayBuffer` and an optional byte offset/length, or a length in elements (in which case a new buffer is created). Typed arrays:

- Have a fixed length; you cannot change their size after creation.
- Store numbers in a compact binary representation (1, 2, 4 or 8 bytes per element), offering performance and memory advantages over regular arrays.
- Provide a subset of array methods (`map`, `forEach`, `reduce`, etc.) but do not support methods that add or remove elements (`push`, `pop`, `shift`, `unshift`, `splice`).
- Are not real arrays: `Array.isArray(typedArray)` returns `false` and they do not inherit from `Array.prototype`.

Example of creating and using a typed array:

```
```js
const floats = new Float32Array(3); // Creates a buffer of 12 bytes (3 × 4 bytes)
floats[0] = Math.PI;
floats[1] = Math.E;
floats[2] = 1 / 3;
floats.forEach((f) => console.log(f.toFixed(3))); // 3.142, 2.718, 0.333
```

```
// Create a subarray that views part of the original buffer

const sub = new Float32Array(floats.buffer, 4, 2);

console.log(sub); // Float32Array [ 2.718, 0.333 ]

...

```

Because typed arrays interpret binary data as numbers, they are useful for interacting with Web APIs that require specific binary formats, such as WebGL textures or audio buffers.

Differences from normal arrays

1. **Type enforcement and byte length** - Regular arrays can store any type of value, and each element is a reference. Typed arrays store only numbers of a specific size and type, which allows for contiguous memory allocation and faster processing by the JS engine.
2. **Fixed size** - Typed arrays have a fixed length and cannot be resized. Arrays can grow and shrink.
3. **Methods** - Typed arrays support only a subset of array methods and do not allow structural changes like `push`/`pop`. Many methods return `_new_` typed arrays of the same type, not generic arrays.
4. **Sharing memory** - Multiple typed array views can point to the same `ArrayBuffer`, enabling you to interpret the same bytes in different ways. Regular arrays do not share underlying storage.
5. **Use cases** - Typed arrays are ideal for binary data, whereas arrays are general-purpose collections.

Using `DataView` for arbitrary layouts

If you need to read or write data types not covered by typed arrays (e.g. 64-bit integers, non-aligned values, or bitfields), use the `DataView` interface. It provides methods like `getUint32()`, `getFloat64()`, `setUint8()`, etc., with explicit byte offsets and endianness control.

```
``js
const buffer = new ArrayBuffer(10);
const view = new DataView(buffer);
view.setUint8(0, 0xff);

```



```
view.setInt16(1, -32768, true); // little-endian  
console.log(view.getInt16(1, true)); // -32768  
...
```

Practice questions

1. **Theory:** Explain why typed arrays cannot change size after creation. How does this property benefit performance?
2. **Coding:** Create a 16-bit PCM audio buffer of length 44100 (1 second at 44.1 kHz) using `Int16Array`. Fill it with a sine wave at 440 Hz.
3. **Theory:** Why would you use a `DataView` instead of a typed array? Give an example scenario.
4. **Coding:** Write a function that takes a buffer containing little-endian 32-bit floats and returns an array of the corresponding JavaScript numbers using `DataView`.

What are SharedArrayBuffer and Atomics, and how do they enable thread safety?

What Are `SharedArrayBuffer` and `Atomics` and How Do They Enable Thread Safety?

JavaScript was historically single-threaded, meaning that web developers didn't need to think about data races. With the introduction of **Web Workers**, the language gained multi-threading. However, without shared memory each worker could only communicate via message passing. **SharedArrayBuffer** and the **Atomics** API introduce a new model: multiple threads (workers and the main thread) can share a common block of memory, and atomic operations ensure safe concurrent access. This enables new patterns, such as shared memory buffers for high-performance computing, but also requires careful programming to avoid race conditions.

SharedArrayBuffer — shared memory

`SharedArrayBuffer` is similar to `ArrayBuffer` but its contents are shared between workers. When you post a `SharedArrayBuffer` to another thread (e.g. via `postMessage()`), both threads hold a reference to the same underlying memory; modifying it in one thread immediately reflects in the other. Unlike ordinary buffers, transferring a `SharedArrayBuffer` does not detach it.

```
``js
// Main thread

const shared = new SharedArrayBuffer(4); // 4 bytes
const int32 = new Int32Array(shared);
const worker = new Worker("worker.js");
worker.postMessage(shared);
int32[0] = 42;

// worker.js
self.onmessage = (e) => {
  const shared = e.data;
  const int32 = new Int32Array(shared);
  console.log("Received value:", int32[0]); // 42
};
...

```

Because memory is shared, two threads can modify the same index concurrently. Without coordination, this leads to race conditions (lost updates, inconsistent state). To safely synchronise reads and writes, you must use the ``Atomsics`` API.

Atomics — atomic operations and memory fencing

The ``Atomsics`` namespace provides low-level functions that perform **atomic** operations on shared typed arrays. Atomic operations complete as a single, indivisible step: no other thread can observe an intermediate state. Additionally, these functions perform **memory fencing**, which ensures that reads and writes occur in the intended order across threads.

Common operations

- ``Atomsics.load(typedArray, index)`` - Reads a value from a shared typed array with a memory fence.
- ``Atomsics.store(typedArray, index, value)`` - Writes a value atomically.
- ``Atomsics.exchange(typedArray, index, value)`` - Atomically replaces a value and returns the old value.
- ``Atomsics.add(typedArray, index, value)`` / ``sub()`` / ``and()`` / ``or()`` - Performs read-modify-write atomically.
- ``Atomsics.compareExchange(typedArray, index, expected, replacement)`` - Compares the current value to ``expected``; if equal, writes ``replacement`` and returns the old value. Otherwise, returns the current value.
- ``Atomsics.wait(typedArray, index, value[, timeout])`` and ``Atomsics.notify(typedArray, index, count)`` - Provide blocking/wake mechanisms, allowing threads to sleep until a value changes.

These operations only work on integer typed arrays (``Int8Array``, ``Uint8Array``, ``Int16Array``, ``Uint16Array``, ``Int32Array``, ``Uint32Array``) backed by a ``SharedArrayBuffer``. They do not work on floating-point arrays.

Example: Shared counter

Consider two workers incrementing a shared counter. Without atomic operations, increments might be lost if both read the old value simultaneously. With ``Atomsics.add()``, each increment is thread-safe:

```

``js
// main.js

const shared = new SharedArrayBuffer(4);
const counter = new Int32Array(shared);
const workers = [new Worker("incr.js"), new Worker("incr.js")];
workers.forEach((w) => w.postMessage(shared));


// incr.js
self.onmessage = (e) => {
  const counter = new Int32Array(e.data);
  for (let i = 0; i < 1_000_000; i++) {
    Atomics.add(counter, 0, 1);
  }
  // Notify main thread
  postMessage("done");
};


// Back in main.js: wait for workers to finish
let finished = 0;
workers.forEach((w) => {
  w.onmessage = () => {
    finished++;
    if (finished === workers.length) {
      console.log("Final count:", counter[0]); // 2,000,000
    }
  };
});
...

```

Here, `Atomics.add(counter, 0, 1)` ensures each worker's increment is applied atomically and no updates are lost. Without `Atomics`, you might see a value less than two million.

Blocking with `wait` and `notify`

In addition to arithmetic operations, `Atoms.wait()` allows a thread to block until a specific memory location changes. This is analogous to condition variables or futexes in other languages. Only web workers can block; the main thread cannot call `Atoms.wait()` because it would freeze the UI.

```
```js
// waitingWorker.js
self.onmessage = (e) => {
 const arr = new Int32Array(e.data);
 // Wait until arr[0] becomes non-zero
 Atoms.wait(arr, 0, 0);
 console.log("Value changed:", Atoms.load(arr, 0));
};

// notifyingWorker.js
self.onmessage = (e) => {
 const arr = new Int32Array(e.data);
 Atoms.store(arr, 0, 123);
 Atoms.notify(arr, 0, 1);
};
```
```

Security and cross-origin isolation

Due to vulnerabilities like Spectre, browsers restrict the usage of `SharedArrayBuffer` and `Atoms`. To use them, your site must be served over HTTPS and set the response headers:

```
...
```

Cross-Origin-Embedder-Policy: require-corp

Cross-Origin-Opener-Policy: same-origin

...

These headers enable a *_cross-origin isolated_* environment, preventing certain side-channel attacks. Without them, ``SharedArrayBuffer`` will be unavailable.

Practice questions

1. **Theory:** Why can race conditions occur when multiple workers modify the same ``SharedArrayBuffer`` without atomics? Provide an example scenario.
2. **Coding:** Implement a simple producer/consumer queue using a ``SharedArrayBuffer`` and ``Atoms.wait()`/`Atoms.notify()``. The producer writes numbers to the buffer, and the consumer waits for new data.
3. **Theory:** What are the security requirements for using ``SharedArrayBuffer`` on the web? Why are they necessary?
4. **Coding:** Demonstrate using ``Atoms.compareExchange()`` to implement a lock mechanism that allows only one worker at a time to enter a critical section.

How do WeakRefs and FinalizationRegistry help manage memory?

How Do `WeakRef` and `FinalizationRegistry` Help Manage Memory?

JavaScript's garbage collector automatically reclaims memory occupied by objects that are no longer reachable. However, certain patterns—such as caches, memoization and object graphs with cycles—can lead to unintended memory retention. The ECMAScript **`WeakRef`** and **`FinalizationRegistry`** APIs provide a way to hold "weak" references to objects so that they don't prevent garbage collection, and to run cleanup code when objects are reclaimed. These features are advanced and should be used sparingly, but they can help manage memory in long-running applications.

Strong vs weak references

A **`strong reference`** is the default—if an object is referenced by a variable, property, array element or any reachable data structure, it cannot be garbage collected. Memory leaks occur when you store objects in global caches or maps and never remove them.

A **`weak reference`** does not prevent an object from being collected. If the object is no longer strongly reachable elsewhere, the garbage collector may reclaim it and the weak reference becomes invalid. This is where `WeakRef` and `FinalizationRegistry` come into play.

`WeakRef`

`WeakRef` wraps an object to create a weak reference. You can call `.deref()` to obtain the original object if it hasn't been collected. If the object has been collected, `.deref()` returns `undefined`.

```
```js
class Cache {
 constructor() {
 this.map = new Map();
 }

 get(key) {
 const ref = this.map.get(key);
```

```

 return ref && ref.deref();
}
set(key, value) {
 this.map.set(key, new WeakRef(value));
}
}

const cache = new Cache();
let obj = { data: "expensive" };
cache.set("exp", obj);
console.log(cache.get("exp")); // { data: 'expensive' }
obj = null; // Remove strong reference
// At some point later, obj may be garbage collected
...

```

In this cache example, the map holds only weak references. If the original strong reference (`obj`) is dropped, the value may be reclaimed, freeing memory.

### ### Caveats

- **Timing is unpredictable** - There is no guarantee *\_when\_* or *\_if\_* the garbage collector will run. You cannot rely on weak references to release resources at a specific time.
- **Use for caches only** - `WeakRef` is intended for caches or memoization, where you can recompute values if they disappear. Do not use weak references to manage critical resources (e.g. files or sockets).
- **Check `deref()`** - Always check the return value of `deref()` for `undefined` to avoid errors.

### ## `FinalizationRegistry`

`FinalizationRegistry` lets you register a callback to be called **after** an object has been garbage collected. This allows you to clean up associated resources (e.g. remove entries from a Map) without preventing the object from being reclaimed.



```

```js
const registry = new FinalizationRegistry((token) => {
  console.log("Object with token", token, "was collected");
});

function createResource(id) {
  const resource = { id };
  registry.register(resource, id); // Register for finalization
  return resource;
}

let res = createResource("ABC");
res = null; // Release strong reference
// Later, GC collects res and calls the finalizer with 'ABC'
```

```

### ### Important notes

- **\*\*No deterministic finalization\*\*** - The callback may run long after the object is unreachable, or not at all before the program ends. Never rely on it for critical logic or resource release that must happen promptly.
- **\*\*Potential memory leak\*\*** - If you register an object but forget to unregister it, the registry holds a reference to the cleanup callback and token, which can itself be a source of leaks.
- **\*\*Use with WeakRefs\*\*** - Pair `WeakRef` and `FinalizationRegistry` to create caches that automatically purge entries when objects are collected.

### ## Example: Automatic cache eviction

```

```js
class AutoCache {
  constructor() {
    this.cache = new Map();
  }
}

```

```

    this.registry = new FinalizationRegistry((key) => {
        this.cache.delete(key);
    });
}

set(key, value) {
    this.cache.set(key, new WeakRef(value));
    this.registry.register(value, key, this.cache);
}

get(key) {
    const ref = this.cache.get(key);
    return ref && ref.deref();
}
}

let data = { content: "heavy" };
const autoCache = new AutoCache();
autoCache.set("item", data);
data = null; // Release strong reference
// When GC runs, the entry is automatically removed from autoCache.cache
...

```

This pattern avoids unbounded cache growth. When `data` is collected, the finalization callback deletes its entry from the map.

Practice questions

1. **Theory:** What is the difference between a strong reference and a weak reference? How does this affect garbage collection?
2. **Coding:** Implement a memoization function that uses `WeakMap` internally. What advantages does it provide over a normal `Map`?
3. **Theory:** Explain why `FinalizationRegistry` callbacks should not contain critical logic or depend on timely execution.

4. **Coding:** Write a small class that holds objects weakly and automatically cleans up its internal map when objects are collected. Demonstrate its use with a test object.

What are transferable objects and how do they improve performance in Workers?

What Are Transferable Objects and How Do They Improve Performance in Workers?

When you use Web Workers or the `postMessage()` API to communicate across threads or windows, data is **structured-cloned**: the browser creates a deep copy of the object and sends it to the other context. Copying large amounts of data (e.g. huge arrays, images, files) can be slow and memory-intensive. **Transferable objects** provide a way to *transfer ownership* of certain data types between contexts without copying, enabling efficient high-performance applications.

Structured cloning versus transferring

By default, when you call `worker.postMessage(data)`, the `data` is cloned using the structured clone algorithm. This supports most built-in types (objects, arrays, typed arrays, maps, sets, Blobs, etc.), but the entire contents are copied. For small objects this is fine, but for large buffers copying can dominate runtime.

Transferable objects avoid this overhead by *moving* the underlying memory from one context to another. After a transfer, the sender's reference becomes **detached**—attempting to access its contents throws an error or yields zero length—while the receiver gains ownership.

Types of transferable objects

The set of transferable objects includes:

- `ArrayBuffer` and the underlying buffers of typed arrays
- `MessagePort` objects
- `OffscreenCanvas`
- `ImageBitmap`
- `AudioData`, `VideoFrame` and other media objects (in browsers that support them)
- `ReadableStream` and `WritableStream` instances (in some environments)

Transferring data to a worker

To transfer, you pass the object in the second argument of `postMessage()`:

```
``js
// main.js
const worker = new Worker("worker.js");
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);
uint8.set([1, 2, 3, 4, 5, 6, 7, 8]);

// Transfer the buffer to the worker
worker.postMessage(buffer, [buffer]);
// At this point, `buffer.byteLength` is 0; it has been detached
console.log(buffer.byteLength); // 0

// worker.js
self.onmessage = (e) => {
  const received = e.data; // ArrayBuffer of length 8
  const view = new Uint8Array(received);
  console.log(view); // Uint8Array [1,2,3,4,5,6,7,8]
};
``
```

Because the memory is moved rather than copied, transferring a large array buffer is nearly instantaneous and does not duplicate data.

Using `structuredClone()` with transfer

The global `structuredClone()` function also accepts a `transfer` option that works similarly:

```
``js
const buf = new Uint8Array([10, 20, 30]).buffer;
```

```
const clone = structuredClone(buf, { transfer: [buf] });  
console.log(buf.byteLength); // 0 (detached)  
console.log(clone.byteLength); // 3  
...
```

Transfer versus SharedArrayBuffer

Transferable objects **move** the data from sender to receiver. After transfer, the sender cannot access it. In contrast, `SharedArrayBuffer` lets multiple contexts share the same memory simultaneously. Choose transfer when you need to hand off ownership to a worker and avoid copying, and choose shared memory when multiple threads need concurrent access with synchronisation via `Atomics`.

Performance benefits and use cases

- **Large array processing** - Offload heavy computations (e.g. image processing, cryptography) to a worker by transferring the underlying `ArrayBuffer` of a typed array. The worker modifies the buffer and optionally transfers it back.
- **Streaming media** - Transfer `ReadableStream` or `AudioData` objects to dedicated workers for decoding or playback without duplicating the data.
- **Message ports** - Transfer `MessagePort` objects to set up complex messaging topologies (e.g. one worker communicates with another via a transferred port).

When using transferables, always ensure that the sender does not expect to read the object after transfer. Trying to access a detached buffer will result in errors or zero lengths.

Practice questions

1. **Theory:** Compare the effects of posting an `ArrayBuffer` to a worker with and without listing it in the transfer array. How does this affect `byteLength` on the sender's side?
2. **Coding:** Write a program that creates a large `Float64Array` on the main thread, transfers it to a worker that multiplies every element by 2 and then transfers it back. Confirm that the main thread sees the updated values.
3. **Theory:** Describe a scenario where using a `SharedArrayBuffer` would be more appropriate than transferring an `ArrayBuffer`.

4. **Coding:** Demonstrate how to transfer a `MessagePort` from the main thread to a worker and use it to send messages back and forth.

What is structured concurrency (upcoming spec) and how might it change async patterns?

What Is Structured Concurrency (Upcoming Spec) and How Might It Change Async Patterns?

JavaScript has embraced asynchronous programming through callbacks, promises and `async/await`. However, these patterns don't enforce any relationship between a parent task and the asynchronous work it spawns—promises can outlive the function that created them, leading to "dangling" operations that continue running after they're no longer needed. **Structured concurrency** is a paradigm that aims to address this by ensuring that asynchronous operations are *_nested_* within a well-defined scope, so that they start and finish together. The ECMAScript proposal for structured concurrency (still at an early stage) introduces new APIs to formalise these relationships and improve cancellation and error handling.

Motivation for structured concurrency

Current asynchronous patterns allow you to launch operations and forget about them. For example:

```
``js
async function fetchUser() {
  // Fire off two requests concurrently
  const userPromise = fetch("/user.json");
  const postsPromise = fetch("/posts.json");
  // Return the user data and ignore posts
  const user = await userPromise;
  return user;
}

// If fetchUser() returns early, the posts request continues running
...

```

The second fetch continues even though its result is never used. This wastes resources and complicates error handling—if the posts request fails, where should the error go? Structured concurrency frameworks in other languages (like Kotlin's coroutines or Swift's tasks) solve this by

automatically canceling child tasks when the parent task ends and propagating errors in a controlled manner.

Proposed API: Task groups and cancellation tokens

The JavaScript proposal (often referred to as **"Structured Tasks"** or **"Cancellation API"**) introduces concepts such as **TaskGroup**, **CancellationController** and **CancellationToken**. Though the exact names and semantics may change, the core ideas include:

- **Task groups** - A task group represents a set of asynchronous operations that are tied to a parent function. You create a task group and then start tasks within it. The group waits for all tasks to complete before it resolves.
- **Cancellation tokens** - A token signals cancellation to any operation that observes it. When a parent task is canceled, all child tasks receive the cancellation signal and should abort their work.
- **Automatic propagation** - If a child task throws an error, the error is propagated to the parent group. Other tasks are canceled automatically, so no work continues silently after a failure.

Here's a conceptual example of how a task group might look:

```
``js
async function loadUserData() {
  const controller = new CancellationController();
  const token = controller.token;

  const group = new TaskGroup();
  group.run(() => fetchWithAbort("/user.json", token));
  group.run(() => fetchWithAbort("/posts.json", token));

  try {
    const [user, posts] = await group.join();
    return { user, posts };
  } catch (err) {
    // If either fetch fails, the other is canceled automatically
  }
}
```

```

    throw err;
  }
}

async function fetchWithAbort(url, token) {
  const controller = new AbortController();
  token.addEventListener("cancel", () => controller.abort());
  const response = await fetch(url, { signal: controller.signal });
  return response.json();
}
...

```

In this pseudo-API:

- `TaskGroup.run()` starts a task and registers it with the group.
- `group.join()` waits for all tasks to complete. If one task rejects, the group cancels other tasks and throws the error.
- A `CancellationToken` orchestrates cancellation requests.

Actual API details may differ, but structured concurrency ensures that all tasks launched within a scope are either completed or canceled when the scope ends. This avoids orphaned asynchronous work and centralises error handling.

How it might change async patterns

Structured concurrency would streamline patterns that currently require manual tracking and cleanup:

- **Error propagation** - When using `Promise.all()`, if one promise rejects, others continue executing. With a task group, the remaining tasks would be canceled.
- **Cancellation** - Developers often attach `AbortController` manually to fetch calls. Structured concurrency could provide integrated cancellation signals for any async operation (fetch, timers, custom tasks) without bespoke boilerplate.

- **Resource management** - You can ensure that all spawned tasks finish before returning from an async function, reducing leaks. This is especially important for operations like file handles, database connections or sensors.
- **Readability** - Code reflects the logical structure of tasks: tasks are children of their scope, rather than floating promises that may outlive their parent.

It's important to note that the structured concurrency proposal is still evolving. Adoption will require changes to browser and Node.js APIs to accept cancellation tokens, and developers will need to learn new patterns. But the end result promises more predictable, maintainable asynchronous code.

Practice questions

1. **Theory:** Describe the problem structured concurrency aims to solve. How do current patterns like `Promise.all()` fall short?
2. **Coding:** Given two asynchronous operations (e.g. fetching user and comments), write a function using existing tools (e.g. `AbortController`, `Promise.race()`) that cancels the second request if the first one fails. How might structured concurrency simplify this?
3. **Theory:** What are the benefits of propagating cancellation and errors from child tasks to their parent? Can you think of situations where you might *not* want automatic cancellation?
4. **Coding:** Sketch a custom "task group" class in today's JavaScript that runs multiple async functions concurrently, cancels all on error, and returns their results. Discuss how your implementation differs from the proposal.

Explain monkey patching, why it's discouraged, and alternatives

Explain Monkey Patching, Why It's Discouraged and Alternatives

****Monkey patching**** refers to the practice of modifying or extending code at runtime—especially the behaviour of existing classes, functions or modules—without changing the original source. In JavaScript, monkey patching often means overriding a method on a built-in prototype (like `Array.prototype`) or third-party library object to change how it works. While sometimes convenient, monkey patching is generally discouraged due to the potential for unexpected side effects and maintenance issues.

What is monkey patching?

Monkey patching stems from dynamic languages allowing you to replace methods on objects or prototypes. For example, you might patch `Array.prototype.sort()` to log how many times it's called:

```
``js
// Monkey patching Array.prototype.sort
const originalSort = Array.prototype.sort;
let callCount = 0;

Array.prototype.sort = function (...args) {
  callCount++;
  console.log("sort called", callCount, "times");
  return originalSort.apply(this, args);
};

[3, 1, 2].sort();
[10, 5].sort();
// Output: sort called 1 times, sort called 2 times
...

```

Another common example is polyfilling methods missing in older environments by defining them on prototypes. For instance, adding `Array.prototype.flat()` in browsers that don't support it.

Why monkey patching is discouraged

1. **Unpredictable behaviour** - Modifying built-in objects changes behaviour for *all* code running in the same environment. Libraries and frameworks that assume standard semantics may break if you alter prototypes.
2. **Conflicts** - If multiple modules monkey patch the same method in different ways, they may conflict. The last one to patch wins, potentially breaking the others.
3. **Maintenance burden** - Overriding functions makes it harder to upgrade libraries or the runtime because the patch might not work with new versions. Debugging becomes difficult when behaviour differs from the documented standard.
4. **Global side effects** - Even local patches (e.g. temporarily changing `Date.now`) can leak into other parts of your app if not restored properly.
5. **Security and stability** - Patching might introduce vulnerabilities or degrade performance if not done carefully.

Legitimate use cases

There are scenarios where monkey patching is acceptable or even necessary:

- **Polyfills/shims** - Adding missing methods according to the ECMAScript specification for older environments (e.g. `String.prototype.startsWith`). Polyfills should check for the method's existence before defining it and follow the spec precisely.
- **Instrumentation** - Temporarily wrapping functions to log usage or performance metrics during development or testing. Such patches should be removed in production.
- **Bug fixes in dependencies** - If a library has a bug and you cannot modify its source or wait for an update, a targeted patch may be required. Document the patch clearly and remove it once the bug is fixed upstream.

Alternatives to monkey patching

1. **Composition and wrappers** - Instead of modifying a method, wrap it in another function or create a helper function. For example, rather than patching `Array.prototype.sort()` to count calls, write a `countedSort()` function that calls `sort()` internally and maintains its own counter.

```

```js
function countedSort(arr) {
 countedSort.calls = (countedSort.calls || 0) + 1;
 return arr.sort();
}

const numbers = [3, 1, 2];
countedSort(numbers);
```

```

2. ****Subclassing or extending**** - In class-based code, derive a subclass that overrides specific methods instead of patching the base class. For built-ins, consider using composition (e.g. wrap an array) rather than extending `Array` directly.
3. ****Dependency injection**** - Instead of monkey patching global objects, pass dependencies (functions, modules) into your code. This makes behaviour explicit and testable.
4. ****Decorators and higher-order functions**** - Wrap functions in higher-order functions that add behaviour (e.g. logging) without changing the original function or its prototype.

Practice questions

1. ****Theory:**** Explain how monkey patching could cause two independently developed libraries to interfere with each other. Provide a hypothetical example.
2. ****Coding:**** Write a wrapper function that logs calls to `Array.prototype.map()` without modifying the prototype itself. Use your wrapper on an array and verify that other code using `map()` is unaffected.
3. ****Theory:**** In what situations is polyfilling a built-in method acceptable? What precautions should you take when writing a polyfill?
4. ****Coding:**** Suppose a third-party library uses `Date.now()` internally, and you need to test time-based behaviour. Show how you can replace `Date.now()` with a fake implementation during a test and restore it afterward without affecting other tests.

What is the Realms API and why might it matter for sandboxing?

What Is the Realms API and Why Might It Matter for Sandboxing?

Executing untrusted code safely in JavaScript is challenging. Existing techniques include ``<iframe>`` sandboxes, ``vm`` contexts in Node.js, or third-party libraries like SES. These solutions each have limitations and complexities. The **Realms API** is a proposed ECMAScript feature that aims to provide a built-in mechanism for creating **isolated execution contexts**—called *realms*—that allow code to run without affecting or being affected by the surrounding environment. This could make sandboxing more robust, secure and ergonomic.

What is a realm?

A **realm** is essentially a separate global environment with its own **global object**, **global scope** and **intrinsic objects** (`Object`, `Array`, etc.). In browsers today, each top-level window or ``<iframe>`` has its own realm. When you evaluate code in an iframe, it gets its own global environment and prototypes distinct from the parent page. However, using iframes for sandboxing has drawbacks: they require DOM elements, they may be blocked by Content Security Policy (CSP), and they load an entire browsing context including document and network access.

The Realms API (and ``ShadowRealm``)

The proposed Realms API aims to allow developers to create new realms without a visual browsing context and to control how values are shared across realms. At the time of writing, the API being championed in TC39 is ``ShadowRealm``:

```
``js
const realm = new ShadowRealm();
realm.evaluate("globalThis.foo = 'bar'");
// The new realm's globalThis has foo, but the current realm's does not
console.log(globalThis.foo); // undefined

// To import a function from the current realm into the shadow realm
function add(a, b) {
  return a + b;
}
```

```

}

const wrappedAdd = realm.importValue(() => add, "add");

// Now call the function inside the shadow realm
realm.evaluate("add(1, 2)"); // 3
...

```

`ShadowRealm` provides:

- **Code evaluation** via `realm.evaluate(code)`, which runs code in the new realm's global scope. It cannot access DOM APIs or other host-specific objects by default.
- **Importing functions/values** using `importValue()`. This creates a callable function in the shadow realm that proxies back to the original function. Data passed between realms is structured-cloned to prevent object graph sharing unless wrapped explicitly.

Because each realm has its own intrinsics, built-in prototypes cannot be tampered with from the outside. This isolation prevents prototype pollution (modifying `Object.prototype`) in one realm from affecting code in another.

Why realms matter for sandboxing

1. **Isolation of intrinsics** - If third-party code mutates `Array.prototype` or `Object.prototype`, those changes are confined to its own realm. The host realm's built-ins remain pristine.
2. **No DOM or network access** - Unlike iframes, a `ShadowRealm` does not automatically include the DOM or fetch APIs. Unless you explicitly import functions, the sandboxed code has no capabilities. This reduces the attack surface and makes it easier to audit what is exposed.
3. **Controlled interoperability** - You decide which functions or values to share with the sandbox via `importValue()`. Passing objects across realms uses structured cloning, so there is no shared memory by default.
4. **Lightweight** - Creating a new realm does not require constructing a full iframe or Node VM. It runs in the same process and thread, making it efficient for short-lived computations.

Considerations and limitations

- **Not yet standardized** - As of today, `ShadowRealm` is at Stage 3 in TC39 and subject to change. Browser and Node support is limited, so experiments must include feature detection.

- **Limited host API access** - A shadow realm cannot access the DOM, timers or fetch unless you provide those functions explicitly. This is by design, but may limit use cases.
- **Security still requires care** - Although realms isolate intrinsics, you must carefully vet what you import or execute. Code can still run infinite loops or exhaust CPU resources unless you add timeouts or worker isolation.
- **Alternative approaches** - Projects like [Ses](<https://github.com/endojs/endo>) and [vm2](<https://github.com/patriksimek/vm2>) implement secure sandboxes today by rewriting code or using Node's `vm` module. Realms may eventually provide a browser-native alternative.

Practice questions

1. **Theory:** Describe how a `ShadowRealm` differs from an `` sandbox. What advantages does it offer for sandboxing untrusted code?&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;li&amp;amp;gt;2. &amp;amp;lt;b&amp;amp;gt;Coding:&amp;amp;lt;/b&amp;amp;gt; Write code that creates a `ShadowRealm`, defines a global variable inside it, and demonstrates that the variable is not visible in the parent realm. Then, use `importValue()` to call a function defined in the parent realm from the shadow realm.&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;li&amp;amp;gt;3. &amp;amp;lt;b&amp;amp;gt;Theory:&amp;amp;lt;/b&amp;amp;gt; Why do each realm's intrinsics need to be separate? What attacks can occur if two realms share the same `Object.prototype`?&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;li&amp;amp;gt;4. &amp;amp;lt;b&amp;amp;gt;Coding:&amp;amp;lt;/b&amp;amp;gt; Imagine you're building a plugin system where plugins run in their own realms. Show how you would expose only a specific API to the plugin while keeping the rest of your application's functions inaccessible.&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;/ol&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="479 938 516 955" data-label="Page-Footer"&amp;amp;gt;
&amp;amp;lt;p&amp;amp;gt;636&amp;amp;lt;/p&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="353 949 641 979" data-label="Page-Footer"&amp;amp;gt;
&amp;amp;lt;p&amp;amp;gt;Upamanyu Deka&amp;amp;lt;/p&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;

How does the ECMAScript spec define execution order at the spec level?

How Does ECMAScript Specify Execution Order at the Spec Level?

JavaScript's behaviour is defined by the **ECMAScript specification**, a precise document describing syntax, types, control flow and semantics. When you write an expression like `a() + b() * c()`, the specification dictates *exactly* how and in what order each part is evaluated. Understanding how the spec formalises execution order helps demystify language quirks and clarify why certain code behaves the way it does.

Evaluation order in expressions

In ECMAScript, most operators evaluate their operands **left to right**. The specification expresses this via abstract operations such as `Evaluate` that return **completion records** (containing result, normal/abrupt completion and value). For example, the grammar for an addition expression is described as evaluating the left operand, then the right operand, then applying the `+` operator:

```
``js
function left() {
  console.log("left");
  return 1;
}
function right() {
  console.log("right");
  return 2;
}
console.log(left() + right());
// Output order: 'left', 'right', 3
``
```

Even though `+` has the same precedence for both operands, the left function runs before the right. Similarly, for `a() && b() && c()`, evaluation stops as soon as one operand yields falsy. The spec defines this via short-circuit evaluation for logical operators.

Argument evaluation

Function call argument expressions are evaluated from left to right. Consider:

```
``js
function f(x, y) {
  return x + y;
}

function a() {
  console.log("a");
  return 1;
}

function b() {
  console.log("b");
  return 2;
}

f(a(), b()); // logs 'a', then 'b'
``
```

Even if the function uses only one argument, the other argument expression is still evaluated before the call. This matters when expressions have side effects.

Property access and assignments

Order also applies to property access and assignments. When evaluating `obj[prop] = value`, the spec evaluates `obj` first, then `prop`, then `value`. If evaluating `prop` or `value` has side effects (e.g. calling a getter), those occur in that sequence. Example:

```
``js
const obj = {
  get key() {
    console.log("getter called");
  }
}
```

```

    return "k";
  },
};
function value() {
  console.log("value");
  return 42;
}
obj[obj.key] = value();
// Output: 'getter called', 'value'
...

```

Execution order between synchronous and asynchronous code

Beyond expression evaluation, the ECMAScript specification describes the event loop and **job queues** for asynchronous execution. There are two main task types:

- **Macro tasks (tasks)** - Scheduled by events such as timers, I/O, `setTimeout`, `setInterval`, user interactions and script execution.
- **Microtasks** - Scheduled by promise resolution and `queueMicrotask()`. The spec mandates that after executing a task, the runtime must empty the microtask queue before running the next task.

Example:

```

```js
console.log("script start");
setTimeout(() => console.log("timeout"), 0);
Promise.resolve().then(() => console.log("promise"));
console.log("script end");
// Output: script start, script end, promise, timeout
...

```

The specification ensures that the microtask (the promise callback) runs after the current task (the script) but before the timer callback. This deterministic ordering enables developers to reason about asynchronous code.

## ## Completion records and abrupt completion

The spec introduces the concept of **completion records** to track how evaluation proceeds. A completion record can be `_normal_` (returning a value), `_throw_` (throwing an exception) or `_return_` (exiting from a function). The `try...catch...finally` construct uses completion records to determine how control flow interacts with cleanup code. For example, `finally` clauses always run regardless of whether an exception was thrown or a return occurred.

```
``js
function test() {
 try {
 return "value";
 } finally {
 console.log("finally runs");
 }
}

console.log(test()); // logs 'finally runs', then 'value'
``
```

## ## Practice questions

1. **Theory:** In the expression `foo() || bar() && baz()`, which functions are called and in what order? Explain using the specification's short-circuit rules.
2. **Coding:** Write code demonstrating the evaluation order of the operands in `obj[prop] = value` when `prop` and `value` are functions with side effects. Explain the output order.
3. **Theory:** How do microtasks differ from macro tasks in the ECMAScript event loop? Why does the specification require clearing the microtask queue before moving on to the next macro task?
4. **Coding:** Show that `finally` clauses run even when a `return` statement is executed in the `try` block. Provide an example with side effects in the `finally` block and explain what happens.



# What are the pitfalls of floating-point arithmetic ( $0.1 + 0.2 \neq 0.3$ )?

## # What Are the Pitfalls of Floating-Point Arithmetic ( $0.1 + 0.2 + 0.3$ )?

Many developers have encountered surprising results when performing arithmetic with decimal fractions in JavaScript:

```
``js
console.log(0.1 + 0.2); // 0.30000000000000004
console.log(0.1 + 0.2 + 0.3); // 0.6000000000000001
console.log(0.1 * 0.2); // 0.020000000000000004
``
```

Why doesn't `0.1 + 0.2` equal exactly `0.3`? The answer lies in the binary representation of floating-point numbers. Understanding these pitfalls is essential when dealing with currencies, measurements or any calculations that require precision.

## ## IEEE-754 double precision

JavaScript's `Number` type follows the **IEEE-754 double-precision binary format**, which uses 64 bits: 1 for the sign, 11 for the exponent and 52 for the mantissa. Unlike decimal fractions, many simple decimal values cannot be represented exactly in binary. For example:

- `0.5` ( $1/2$ ) has an exact binary representation: `0.12`.
- `0.25` ( $1/4$ ) has an exact binary representation: `0.012`.
- `0.1` ( $1/10$ ) does **not** have a finite binary representation. Its binary expansion is infinite: `0.000110011001100...2`.

When storing `0.1` as a double, the binary representation is truncated to fit into 52 bits of mantissa. This introduces a **rounding error**. Adding two numbers with rounding errors compounds the error, which is why `0.1 + 0.2` is slightly more than 0.3.

## ### Accumulating error

Errors accumulate with repeated operations:

```
``js
let sum = 0;
for (let i = 0; i < 10; i++) {
 sum += 0.1;
}
console.log(sum); // 0.9999999999999999, not 1
``
```

Each addition introduces a tiny error; ten times that error produces a noticeable difference.

### ## Comparison pitfalls

Due to rounding errors, direct comparisons can yield unexpected results:

```
``js
const a = 0.1 + 0.2;
console.log(a === 0.3); // false
// Instead use a tolerance
function nearlyEqual(x, y, epsilon = Number.EPSILON) {
 return Math.abs(x - y) < epsilon;
}
console.log(nearlyEqual(a, 0.3)); // true
``
```

`Number.EPSILON` is the difference between 1 and the smallest floating-point number greater than 1. Using an epsilon allows you to compare numbers within an acceptable margin of error.

### ## Other quirks and pitfalls



- **Large and small numbers** - Representable numbers range from approximately  $\pm 1.8 \times 10^{308}$  down to  $\pm 5 \times 10^{-324}$ . Numbers beyond this range underflow to `0` or overflow to `Infinity`.
- **NaN propagation** - Operations like `0 / 0` produce `NaN` (Not a Number) and propagate through calculations.
- **Rounding modes** - JavaScript rounds ties to the nearest even value (banker's rounding) in some operations.
- **String conversions** - Converting a number to a string (`number.toString()`) may produce a long decimal, but `Number.parseFloat()` can sometimes produce a slightly different binary representation. Use caution when converting back and forth.

## ## Practice questions

1. **Theory:** Explain why `0.1` cannot be represented exactly in binary floating-point format. Illustrate the binary expansion of  $1/10$ .
2. **Coding:** Write a function that sums an array of decimal numbers while minimising floating-point errors. One approach is to sort the numbers before adding them.
3. **Theory:** What is `Number.EPSILON`, and how can it help when comparing floating-point numbers? Why might a custom epsilon be required in some situations?
4. **Coding:** Demonstrate how repeated subtraction can lead to floating-point drift. Subtract `0.1` from `1` ten times, print each intermediate value, and discuss why the final result is not exactly `0`.

# How can you achieve precise decimal arithmetic in JavaScript?

## # How Can You Achieve Precise Decimal Arithmetic in JavaScript?

Due to the limitations of floating-point representation, performing precise decimal arithmetic in JavaScript can be tricky. This is especially relevant for financial calculations (e.g. currency, taxes), scientific measurements, and any domain where rounding errors are unacceptable. Fortunately, there are techniques and tools to achieve higher precision.

### ## Use integers to represent fixed-point values

One common approach is to **scale** decimal values into integers by multiplying by a power of ten. Perform arithmetic on the integers and then scale back. For currency, working in cents avoids fractional cents:

```
```js
// Represent dollars as integer cents
function addMoney(a, b) {
  const centsA = Math.round(a * 100);
  const centsB = Math.round(b * 100);
  return (centsA + centsB) / 100;
}

console.log(addMoney(0.1, 0.2)); // 0.3
console.log(addMoney(0.1, 0.2) === 0.3); // true
```
```

This method eliminates rounding errors as long as the scaled values are integers. However, you must choose a scale factor large enough to capture the maximum number of decimal places you expect.

### ## BigInt with scaling

For even larger ranges or more precise fractions, you can use **`BigInt`**. `BigInt` can represent arbitrarily large integers but cannot represent decimals. By storing amounts as `_scaled_` integers, you can perform exact arithmetic without overflow. For example:

```
```js
const scale = 10_000n; // support four decimal places

function addScaled(a, b) {
  return (a + b) / scale;
}

const amount1 = 123_45n; // 1.2345
const amount2 = 50_00n; // 0.5000
console.log(addScaled(amount1 * scale, amount2 * scale)); // 1.7345n
```
```

You can encapsulate this logic in a class that stores amounts as scaled `BigInt` and implements addition, subtraction, multiplication and division with appropriate rounding modes.

## ## Decimal libraries

Several libraries implement arbitrary-precision decimal arithmetic. They represent numbers in a decimal base and handle rounding precisely:

- **`decimal.js`** and **`decimal.js-light`** - Provide a `Decimal` type with configurable precision and rounding modes. You can perform arithmetic using methods like `plus`, `minus`, `times` and `div`. Example:

```
```js
import { Decimal } from "decimal.js";

const x = new Decimal("0.1");
const y = new Decimal("0.2");
console.log(x.plus(y).toString()); // '0.3'
```

...

- **big.js** - Similar to decimal.js but with a smaller footprint. You create `Big` objects and call methods to perform arithmetic.

These libraries track decimal places internally and avoid binary rounding errors. They can be slower than native numbers, but for finance or scientific work the correctness often outweighs performance concerns.

ECMAScript Decimal proposal

TC39 has a Stage 1 [Decimal proposal](<https://github.com/tc39/proposal-decimal>) that introduces a new primitive type `decimal64`. It aims to integrate decimal arithmetic into the language with similar semantics to `Number` but using a decimal representation. Once finalised and implemented, you will be able to write:

```
``js
const a = 0.1m; // decimal literal
const b = 0.2m;
console.log(a + b); // 0.3m exactly
``
```

Until then, rely on other methods described here.

Practice questions

1. **Theory:** Explain why multiplying by 100 and rounding helps when adding two decimal numbers like 0.1 and 0.2. What limitations does this technique have?
2. **Coding:** Implement a `Money` class that stores amounts as integer cents using `BigInt`. Provide methods for addition, subtraction and multiplication by a scalar.
3. **Theory:** What advantages do arbitrary-precision decimal libraries offer compared to using scaled integers? When might you choose one over the other?
4. **Coding:** Use a decimal library (e.g. decimal.js or big.js) to compute `(0.1 + 0.2) \* 0.3` exactly. Compare the result with the native `Number` computation.

How do Intl.DateTimeFormat and Intl.NumberFormat support localization?

How Do `Intl.DateTimeFormat` and `Intl.NumberFormat` Support Localization?

Web applications often need to display dates, times and numbers in a way that matches the user's locale—formats vary widely across cultures. The **Intl API** provides built-in internationalisation services for these tasks. In particular, **Intl.DateTimeFormat** and **Intl.NumberFormat** allow you to format dates, times and numbers according to locale conventions and user preferences.

`Intl.DateTimeFormat`

`Intl.DateTimeFormat` formats `Date` objects or timestamps into human-readable strings. You specify a locale and options describing which parts of the date/time to include and how. For example:

```
```js
const date = new Date("2025-12-31T18:30:00Z");

// US English uses month/day/year order and 12-hour time
const usFormatter = new Intl.DateTimeFormat("en-US", {
 year: "numeric",
 month: "long",
 day: "numeric",
 hour: "numeric",
 minute: "2-digit",
 timeZoneName: "short",
});

// French uses day/month/year order and 24-hour time
const frFormatter = new Intl.DateTimeFormat("fr-FR", {
 year: "numeric",
 month: "long",
```

```

 day: "numeric",
 hour: "numeric",
 minute: "2-digit",
 timeZoneName: "short",
 });

```

```

console.log(usFormatter.format(date)); // December 31, 2025 at 1:30 PM GMT
console.log(frFormatter.format(date)); // 31 décembre 2025 à 18:30 UTC
...

```

Key aspects:

- **Locale:** A BCP 47 language tag (e.g. `en-US`, `de-DE`) influences language and ordering. You can specify an array of locales; the browser picks the best match.
- **Options:** Select which date/time components to include (`year`, `month`, `day`, `hour`, `minute`, `second`) and their styles (`numeric`, `2-digit`, `short`, `long`). Additional options include `timeZone`, `timeZoneName`, `hourCycle` and `calendar`.
- **Time zones:** If you don't specify `timeZone`, the user's local time zone is used. You can set `timeZone` to convert UTC times into specific zones.

`Intl.RelativeTimeFormat` complements `DateTimeFormat` by formatting relative times (e.g. "in 2 hours", "3 days ago").

### ## `Intl.NumberFormat`

`Intl.NumberFormat` formats numbers, currencies and percentages according to locale conventions. It handles different decimal separators (comma vs dot), thousands separators, currency symbols and placement, and percentage signs.

#### ### Basic usage

```

```js
const num = 1234567.89;

```

```
// Format number in German (comma as decimal separator)
const deFormat = new Intl.NumberFormat("de-DE");
console.log(deFormat.format(num)); // 1.234.567,89
```

```
// Format currency in Japanese Yen (no fractional digits)
const yenFormat = new Intl.NumberFormat("ja-JP", {
  style: "currency",
  currency: "JPY",
});
console.log(yenFormat.format(1234)); // ¥1,234
```

```
// Format percentage with minimum fraction digits
const percentFormat = new Intl.NumberFormat("en-US", {
  style: "percent",
  minimumFractionDigits: 2,
});
console.log(percentFormat.format(0.1234)); // 12.34%
...

```

Important options:

- ``style`` - ``decimal`` (default), ``currency``, ``percent``, ``unit``.
- ``currency`` - ISO 4217 code required when ``style`` is ``currency``.
- ``currencyDisplay`` - ``symbol``, ``code``, ``name``, ``narrowSymbol``.
- ``minimumFractionDigits`` and ``maximumFractionDigits`` - control the number of decimal places.
- ``notation`` - ``standard``, ``scientific``, ``engineering`` and ``compact`` (for 1K, 1M, etc.).
- ``unit`` and ``unitDisplay`` - format measurements like degrees, litres and metres using `[`Intl.NumberFormat`](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\_Objects/Intl/NumberFormat)`.

Custom grouping and formatting

You can also use locale-sensitive formatting for other purposes. For example, to format file sizes in a human-friendly way using the compact notation:

```
```js
const sizeFormatter = new Intl.NumberFormat("en-US", {
 notation: "compact",
 style: "unit",
 unit: "byte",
 unitDisplay: "narrow",
});
console.log(sizeFormatter.format(1500)); // 1.5 KB
```
```

Best practices

1. ****Always specify a locale**** - Avoid relying on the browser's default locale, as it may vary between users. You can detect the user's locale via ``navigator.language`` or allow user choice.
2. ****Use options appropriately**** - Choose the right style and fraction digits based on the data you're formatting. Avoid manual string concatenation for currencies and percentages; let the API handle it.
3. ****Consider international variations**** - Be aware that some locales have unusual conventions (e.g. using non-Western numerals or right-to-left scripts). Test formatting in your target markets.

Practice questions

1. ****Theory:**** Describe how ``Intl.DateTimeFormat`` handles time zones and how you can format a UTC timestamp in another time zone.
2. ****Coding:**** Write a function that formats a given number into a currency string for the user's locale. It should accept an ISO currency code and use ``navigator.language`` as the locale.
3. ****Theory:**** What are the differences between formatting a number in ``scientific`` notation versus ``compact`` notation using ``Intl.NumberFormat``? Give examples.
4. ****Coding:**** Format the current date and time in Japanese using the Japanese calendar (``calendar: 'japanese'``) and include the era name and day of the week. Explain each option you use.

What are pluralRules and segmenter in the Intl API?

What Are `Intl.PluralRules` and `Intl.Segmenter` in the `Intl` API?

The **Intl` API** in JavaScript provides tools for locale-aware formatting. Two lesser-known but powerful features are **Intl.PluralRules**, which helps decide plural categories for numbers in different languages, and **Intl.Segmenter**, which breaks text into meaningful units like graphemes, words or sentences. These APIs enable applications to handle pluralisation and text segmentation correctly across languages.

`Intl.PluralRules`

Pluralisation varies dramatically across languages. English has two plural categories (singular and plural), but some languages have one, three or more categories, and the rules can be complex. `Intl.PluralRules`` determines which plural category a given number belongs to for a specific locale and plural type.

Usage

```
```js
// English plural rules for cardinal numbers
const prEn = new Intl.PluralRules("en-US", { type: "cardinal" });
console.log(prEn.select(1)); // 'one'
console.log(prEn.select(2)); // 'other'

// Arabic plural rules (cardinal)
const prAr = new Intl.PluralRules("ar", { type: "cardinal" });
console.log(prAr.select(0)); // 'zero'
console.log(prAr.select(1)); // 'one'
console.log(prAr.select(2)); // 'two'
console.log(prAr.select(3)); // 'few'
console.log(prAr.select(11)); // 'many'
...
```
```

The `select()` method returns a category string such as `'one'`, `'few'`, `'many'`, `'other'` (categories vary by locale). Using this, you can choose the appropriate translation for the number:

```
``js
const messages = {
  en: {
    one: "There is one item",
    other: "There are {n} items",
  },
  ru: {
    one: "Есть {n} элемент",
    few: "Есть {n} элемента",
    many: "Есть {n} элементов",
    other: "Есть {n} элемента",
  },
};

function formatCount(n, locale) {
  const pr = new Intl.PluralRules(locale);
  const key = pr.select(n);
  return messages[locale][key].replace("{n}", n);
}

console.log(formatCount(1, "en")); // There is one item
console.log(formatCount(3, "ru")); // Есть 3 элемента
...

```

You can also specify `'type: 'ordinal''` to handle ordinal numbers (1st, 2nd, 3rd) since the plural categories differ.

Resolved options

You can query ``pr.resolvedOptions()`` to see which locale and plural categories are actually in use. This is useful when fallback locales are applied.

`Intl.Segmenter`

Different languages use different rules for dividing text into characters, words and sentences. Simply splitting on spaces or letters is insufficient for scripts with complex grapheme clusters (like emoji or combining characters), or languages like Thai and Japanese where words are not separated by spaces. `Intl.Segmenter` provides locale-aware text segmentation.

Usage

```
```.js

// Segmenter for words in Japanese
const segJa = new Intl.Segmenter("ja", { granularity: "word" });
const textJa = "庭には二羽鶏がいる";
for (const { segment, isWordLike } of segJa.segment(textJa)) {
 if (isWordLike) console.log(segment);
}

// Output: 庭, には, 二羽, 鶏, が, いる

// Segmenter for grapheme clusters (user-perceived characters) in English emoji
const segGrapheme = new Intl.Segmenter("en", { granularity: "grapheme" });
const emojis = "👤 🌀 🚀 ❤️";
console.log([...segGrapheme.segment(emojis)].map((s) => s.segment));

// Output: ['👤', '🌀', '🚀', '❤️']

// Segmenter for sentences in German
const segSentence = new Intl.Segmenter("de", { granularity: "sentence" });
const textDe = "Hallo Welt! Wie geht es dir? Gut.";
console.log([...segSentence.segment(textDe)].map((s) => s.segment));

// Output: ['Hallo Welt!', ' Wie geht es dir?', ' Gut.']
```

...

The `segment()` method returns an iterable of objects with properties:

- `segment` - the extracted substring
- `index` - starting index in the original string
- `input` - the original string
- `isWordLike` - boolean indicating if the segment behaves like a word (for `granularity: 'word'`)

Using `Intl.Segmenter` ensures that text is split according to the conventions of the specified locale, which is vital for cursor movement, word counting, text wrapping and search indexing.

### ## When to use these APIs

- **Localising messages** - Use `Intl.PluralRules` to select the right plural form for quantities in user interfaces.
- **Internationalising text input** - Use `Intl.Segmenter` to count characters or words properly, highlight selections, or implement text-wrapping algorithms.
- **Searching and indexing** - Segment text into words or sentences before indexing for search to improve accuracy across languages.

### ## Practice questions

1. **Theory:** Why is splitting text on spaces insufficient for languages like Thai or Japanese? How does `Intl.Segmenter` address this problem?
2. **Coding:** Implement a function that pluralises a message like "You have {n} new messages" in both English and Russian using `Intl.PluralRules`.
3. **Theory:** What differences exist between cardinal and ordinal plural rules? Give examples of locales where they differ.
4. **Coding:** Use `Intl.Segmenter` to count the number of user-perceived characters (graphemes) in a string containing emojis and combining characters. Explain how this differs from `string.length`.

# How does Temporal API improve date–time management compared to Date?

## # How Does the Temporal API Improve Date/Time Management Compared to `Date`?

The built-in `Date` object has been part of JavaScript since the beginning, but it comes with numerous flaws: it is mutable, combines absolute time with local time zones, lacks support for non-Gregorian calendars and time zones, and has confusing methods for parsing and formatting. The **Temporal API** is a modern proposal (at Stage 3 as of this writing) that aims to replace `Date` with a set of explicit, immutable date and time types. By separating concepts like absolute time, civil time, and time zones, Temporal avoids many pitfalls of `Date` and makes date-time manipulation easier and less error-prone.

### ## Problems with `Date`

- **Mutability** - `Date` instances are mutable. Methods like `setFullYear()` and `setHours()` modify the existing object, which can lead to unintended side effects if a reference is shared.
- **Implicit time zone** - A `Date` stores a timestamp in milliseconds since the Unix epoch internally, but its getter and setter methods interpret that timestamp in the *system's local time zone*. Converting between time zones requires manual calculations.
- **Limited calendar support** - `Date` supports only the proleptic Gregorian calendar. There's no way to use other calendars like the Japanese or Islamic calendars.
- **Parsing and formatting** - `Date.parse()` accepts many formats, but they are inconsistent across implementations. `Date` also lacks built-in formatting; you must use libraries or `Intl.DateTimeFormat`.
- **Daylight saving time (DST) traps** - Adding 24 hours to a `Date` around a DST transition may produce a local time off by an hour due to a 23- or 25-hour day.

### ## Temporal's approach

Temporal introduces several new types, each representing a different concept:

- **Temporal.Instant** - Represents an exact point in time on the timeline (like a `Date`'s internal representation) but immutable.
- **Temporal.PlainDate** - Represents a calendar date (year, month, day) without a time or time zone.
- **Temporal.PlainTime** - Represents a time of day without a date or time zone.

- **Temporal.PlainDateTime** - Combines a `PlainDate` and a `PlainTime` without a time zone.
- **Temporal.ZonedDateTime** - Combines an `Instant` with a time zone and calendar; automatically handles DST and conversions.
- **Temporal.Duration** - Represents spans of time (e.g. 3 days, 2 hours) and supports arithmetic like addition and subtraction.
- **Temporal.Now** - Provides access to the current time in different forms (`now.instant()`, `now.zonedDateTime()`), respecting calendars and time zones.

### ### Immutability and explicitness

Temporal objects are immutable—every method returns a new instance rather than mutating the original. This prevents accidental changes and makes reasoning about time values easier. Moreover, Temporal forces you to be explicit about calendars and time zones, avoiding hidden assumptions.

Example: adding one day across a DST transition with `Temporal.ZonedDateTime`:

```
```js
// Create a ZonedDateTime for March 13, 2022 1:30 AM in America/New_York
const zdt = Temporal.ZonedDateTime.from({
  year: 2022,
  month: 3,
  day: 13,
  hour: 1,
  minute: 30,
  timeZone: "America/New_York",
});
// Add one day
const nextDay = zdt.add({ days: 1 });
console.log(zdt.toString()); // 2022-03-13T01:30-05:00[America/New_York]
console.log(nextDay.toString()); // 2022-03-14T01:30-04:00[America/New_York]
// Notice the offset changed (-05:00 vs -04:00) due to DST
...
```
```



The Temporal API automatically adjusts for DST and time zones when adding or subtracting durations. With `Date`, you would need to manually account for DST boundaries or use a library like `Moment.js`.

### ### Conversions and formatting

Temporal objects provide straightforward conversion methods:

```
```js
const plainDate = Temporal.PlainDate.from("2025-12-31");
const plainTime = Temporal.PlainTime.from("18:45:00");
const dateTime = plainDate.toPlainDateTime(plainTime);
console.log(dateTime.toString()); // 2025-12-31T18:45:00

// Convert from ZonedDateTime to Instant and back
const nowZoned = Temporal.Now.zonedDateTimeISO();
const instant = nowZoned.toInstant();
console.log(instant.toString()); // e.g. 2025-11-07T13:00:00.123456789Z
const sameZoned = instant.toZonedDateTime({ timeZone: "America/New_York" });
console.log(sameZoned.toString());
```
```

Formatting Temporal objects is still done via `Intl.DateTimeFormat` by passing `Temporal` objects directly (or by converting to `Date` if necessary). Temporal focuses on data representation and arithmetic; for display, you still use `Intl`.

## ## Migration and adoption

The Temporal API is available in some environments behind flags or polyfills. Eventually it is expected to be natively supported in browsers and Node.js. Until then, you can experiment with the [proposal-polyfill](<https://github.com/tc39/proposal-temporal>). Once adopted, Temporal could simplify date/time handling and reduce reliance on heavy third-party libraries.

## ## Practice questions

1. **Theory:** Why does adding 24 hours to a `Date`` object around a DST transition sometimes give unexpected local times? How does Temporal avoid this issue?
2. **Coding:** Use `Temporal.ZonedDateTime`` to compute the difference in hours between two time zones (e.g. New York and Tokyo) on a given date. Compare with doing the same using `Date`` and manual calculations.
3. **Theory:** What is the difference between `Temporal.PlainDateTime`` and `Temporal.ZonedDateTime``? When would you use each?
4. **Coding:** Write a function that, given a `Temporal.PlainDate`` and a number of business days to add, returns the resulting date, skipping weekends. How would you implement the same with `Date``?

# What are the limitations of `Math.random()` and how to get cryptographically secure randomness?

## # What Are the Limitations of `Math.random()` and How to Get Cryptographically Secure Randomness?

Generating random numbers is a common requirement—from games and simulations to IDs and cryptography. JavaScript offers the `Math.random()` function as a simple source of randomness, but it has limitations that make it unsuitable for security-sensitive contexts. This article explores these limitations and shows how to obtain cryptographically secure random numbers in both browser and Node.js environments.

### ## Limitations of `Math.random()`

`Math.random()` returns a pseudorandom floating-point number between 0 (inclusive) and 1 (exclusive). Its drawbacks include:

1. **Lack of cryptographic security** - `Math.random()` is designed for simulations and casual randomness. Its internal algorithm is not intended to withstand prediction. Attackers with knowledge of its implementation or initial seed may predict future values. For example, earlier versions of some engines used linear congruential generators that could be reverse-engineered.
2. **No seed control** - The API does not allow you to provide or retrieve the generator's seed. This makes it unsuitable for repeatable random sequences (e.g. in unit tests) and also prevents you from reseeding the generator after a compromise.
3. **Floating-point output** - `Math.random()` outputs a float in  $[0, 1)$ , which you often have to scale and round to obtain integers or bytes. Converting floats to integers can introduce biases if not done carefully. For example, `Math.floor(Math.random() * 10)` does not uniformly distribute numbers because of floating-point rounding.
4. **Engine differences** - Implementations differ slightly between engines, though ECMAScript specifies statistical requirements. Still, for cryptography you need stronger guarantees.

### ## Obtaining cryptographically secure randomness

#### ### Browser: `crypto.getRandomValues()` and `crypto.randomUUID()`

The Web Crypto API exposes `crypto.getRandomValues()` to fill a typed array with cryptographically secure random bytes. It uses the underlying operating system's entropy sources. See

[file 145](../145-what-is-crypto-getrandomvalues-and-why-is-it-safer-than-math-random.md) for details.

Example:

```
``js
// Generate a 32-bit random integer between 0 and 2^32 - 1
const array = new Uint32Array(1);
crypto.getRandomValues(array);
const randomInt = array[0];
// Map to range [0, 999]
const result = randomInt % 1000;
console.log(result);

// Generate a UUID v4 (browser support)
const uuid = crypto.randomUUID();
console.log(uuid); // e.g. '3dfd9c15-8e8a-4f89-ae37-a4d7a8f0d9bb'
...

```

The `randomUUID()` method generates RFC 4122 version 4 UUIDs using secure random values. It is widely supported in modern browsers.

### ### Node.js: `crypto.randomBytes()` and `crypto.randomUUID()`

In Node.js, the `crypto` module provides similar functionality. Use `crypto.randomBytes(size)` to generate a buffer of secure random bytes. Node 14.17+ also offers `crypto.randomUUID()`.

```
``js
const { randomBytes, randomUUID } = require("crypto");

// Generate 16 random bytes
const buf = randomBytes(16);

```

```

console.log(buf.toString("hex"));

// Create a random number between 0 and 9 inclusive
function randomInt10() {
 // Rejection sampling to avoid modulo bias
 while (true) {
 const byte = randomBytes(1)[0];
 if (byte < 250) return byte % 10;
 }
}
console.log(randomInt10());

// Generate a UUID v4
console.log(randomUUID());
...

```

### ### Rejection sampling to avoid modulo bias

When mapping random bytes to a smaller range, naive modulus operations can introduce bias because the range of the random generator may not divide evenly. To avoid this, use **\*\*rejection sampling\*\***: discard values that would skew the distribution.

```

```js
function randomInRange(max) {
  const { randomBytes } = require("crypto");
  const range = 256 % max; // values >= 256 - range would skew the result
  let val;
  do {
    val = randomBytes(1)[0];
  } while (val >= 256 - range);
  return val % max;
}

```

```
console.log(randomInRange(10));
```

```
...
```

Best practices

- **Use CSPRNGs** - For anything security-sensitive (tokens, keys, salts, nonces), always use `crypto.getRandomValues()` in the browser or `crypto.randomBytes()` in Node.js. Never fall back to `Math.random()`.
- **Beware of predictability** - Do not seed your own pseudorandom generator unless you fully understand cryptography. Rely on the system's entropy.
- **Use `randomUUID()` when available** - It simplifies generating UUIDs and ensures correct version and variant bits.
- **Be cautious about modulo bias** - Use rejection sampling or specialized functions like `crypto.randomInt()` in Node.js 14.10+ to generate integers uniformly.

Practice questions

1. **Theory:** Explain why `Math.random()` is unsuitable for cryptographic purposes. What properties must a cryptographically secure random number generator have?
2. **Coding:** Write a browser function that returns a random 6-digit numeric code using `crypto.getRandomValues()`. Ensure that each digit is uniformly random between 0 and 9.
3. **Theory:** What is modulo bias? Why can mapping a byte to a smaller range using `%` introduce bias? Describe how rejection sampling avoids this problem.
4. **Coding:** In Node.js, implement a function that generates a secure random password of length 12 consisting of uppercase letters, lowercase letters and digits. Use `crypto.randomBytes()` and rejection sampling.

What triggers a reflow vs a repaint and how to minimize them?

What triggers a reflow vs a repaint and how to minimize them

Modern browsers break rendering into multiple phases. Two of the most expensive phases are **reflow** (sometimes called layout) and **repaint**. Understanding what triggers each phase and how to avoid unnecessary work is key to writing smooth web applications.

Repaint vs reflow

- **Repaint** happens when an element's *appearance* changes in a way that does not affect its size or position. Examples include changing `color`, `background-color`, `visibility` or `opacity`. The browser does not need to recalculate layout; it simply redraws the affected pixels.
- **Reflow** (layout) occurs when an element's *geometry* changes or when the browser needs to recalculate positions. Actions such as adding or removing DOM nodes, changing element `display`/`position`/`float`, modifying dimensions (`width`, `height`, `padding`, `margin`, `border`) or resizing the window trigger reflow. Reflow can cascade—changing one element's size may require recalculating its ancestors and descendants.

Reflow is typically much more expensive than repaint because the browser must recompute the layout tree and recalculate positions before repainting.

Common triggers

Repaint triggers

- Changing CSS properties that affect only the look: `color`, `background-image`, `background-color`, `text-shadow`, `visibility`, etc.
- Adding or removing classes that change only visual styles (e.g. toggling a dark mode class).
- Changing element attributes such as `class`, `id` or `style` if the resulting styles do not affect layout.

Reflow triggers

- Inserting or removing DOM elements, or changing their order.

- Changing styles that influence layout: ``display``, ``position``, ``top/left/right/bottom``, ``margin``, ``padding``, ``height``, ``width``, ``font-size``, ``line-height``, etc.
- Changing the content of an element (e.g. adding text) which affects its size.
- Querying layout information after a change. Reading properties like ``offsetWidth``, ``offsetHeight``, ``scrollTop``, ``getComputedStyle()`` or ``clientWidth`` after modifying styles forces the browser to flush pending changes and perform a synchronous reflow so that it can return up-to-date values.

Minimizing reflow and repaint

1. **Batch DOM mutations.** Instead of making multiple DOM changes one by one, group them together. For example, use a document fragment to build a set of elements and append it once, or toggle a class that encapsulates multiple style changes instead of changing each property separately.
2. **Avoid layout thrashing.** Layout thrashing occurs when your code alternates between reading layout properties and writing them. Each read forces a reflow, and each write invalidates the layout. To avoid this, perform all reads first, then all writes. For complex animations, consider ``requestAnimationFrame()`` and CSS transitions.
3. **Use transform and opacity.** CSS ``transform`` and ``opacity`` properties can animate without triggering reflow because they operate on a layer composited by the GPU. For example, use ``transform: translateX()`` instead of changing ``left`/`top``.
4. **Simplify the DOM structure.** Deeply nested elements can make reflow more expensive because changes propagate through many ancestors. Keep the DOM shallow where possible.
5. **Debounce or throttle resizing.** Window resize events can trigger continuous reflows. Use a debounce or throttle function to limit the frequency of layout recalculations.
6. **Use ``will-change`` sparingly.** The ``will-change`` property hints to the browser that an element is likely to change. It can promote the element to its own layer, reducing repaint costs during animations. Use it only when necessary because it increases memory usage.

Example: layout thrashing

```
```js
// Poor practice: repeatedly forces reflow
```



```

const items = document.querySelectorAll(".item");

items.forEach((item) => {
 // Writing: changes layout
 item.style.width = item.offsetWidth + 10 + "px";

 // Reading: forces a reflow because the browser must compute offsetWidth
 console.log(item.offsetHeight);
});

// Better: separate reads and writes
const heights = [];
items.forEach((item) => {
 heights.push(item.offsetHeight); // read first
});

items.forEach((item, i) => {
 item.style.width = item.offsetWidth + 10 + "px"; // write after
});
...

```

In the poor example, each call to `offsetWidth` after a write forces a synchronous reflow. In the improved version, all reads happen before writes, eliminating unnecessary reflows.

### ## Practice questions

1. **Theory:** What is the difference between a repaint and a reflow? Provide examples of CSS properties that trigger each.
2. **Theory:** Why does reading `offsetHeight` immediately after changing `style.top` cause a performance hit?
3. **Coding:** Write a function that appends 100 list items to a `<ul>` and minimizes reflow. Explain why your approach is efficient.
4. **Coding:** Use `requestAnimationFrame()` to animate an element horizontally without triggering reflows. Explain how `transform: translateX()` differs from changing `left`.

## 172. How does compositing work in modern browsers?

### # How does compositing work in modern browsers

Rendering a web page involves multiple stages: style calculation, layout, paint and **compositing**. While layout determines where elements go and paint draws them into bitmaps, compositing is the process of assembling these painted pieces into the final on-screen image. Understanding compositing helps explain why certain CSS properties trigger GPU acceleration and how to optimize animations.

### ## The rendering pipeline overview

1. **Style calculation.** The browser converts CSS rules into computed styles for each element.
2. **Layout (reflow).** The browser calculates the geometry of each element—its size and position—based on the computed styles and the DOM tree.
3. **Paint.** The browser paints each element into a bitmap. Many small paint operations may be combined into larger ones.
4. **Compositing.** The browser merges these bitmaps (often called **layers**) into the final image displayed on the screen.

### ## Layers and the compositor

Not all elements are painted into the same layer. Some elements—due to CSS properties like `position: fixed`, `transform`, `filter`, `opacity`, `clip-path` or `will-change`—are promoted to separate compositor layers. The reasons for layer promotion include:

- **Isolation:** Elements with effects like 3D transforms or filters cannot be easily merged with other layers during painting. Placing them in their own layer avoids redrawing their neighbors.
- **Performance:** When an element moves or fades, only its layer needs to be repainted and composited, not the entire page. GPU compositing can combine layers efficiently.

The compositor runs on a separate thread from the main rendering thread in many modern browsers. This separation allows smooth animations even when JavaScript on the main thread is busy, provided the animation uses properties that only affect compositing (e.g., `transform`, `opacity`).

### ## Compositing steps

1. **Layer creation.** During painting, the browser decides which elements should be on their own layer. Each layer is essentially a texture.
2. **Blending order.** Layers are stacked based on z-index and stacking contexts. The compositor orders the layers and determines how they overlap.
3. **Effects and clipping.** The compositor applies CSS effects such as `transform`, `opacity` or `filter` to each layer. Because these operations are GPU-accelerated, they are very fast compared to reflow or repaint.
4. **Composition.** Finally, the compositor blends all the layers into a single image using the GPU. This step is usually synchronized with the display's refresh rate.

### ## Example: GPU-accelerated animation

```
```html
<style>

.box {
  width: 100px;
  height: 100px;
  background: crimson;
  transition: transform 1s;
}

.move {
  transform: translateX(300px);
}

</style>
<div class="box"></div>
<button id="toggle">Animate</button>
<script>
  const box = document.querySelector(".box");
  document.getElementById("toggle").addEventListener("click", () => {
    box.classList.toggle("move");
  });
</script>
```
```

...

In this example, clicking the button toggles a class that changes `transform`. Because `transform` creates a separate compositor layer, the main thread does not need to recalculate layout or repaint the box for each frame. The GPU composites the box's layer at its new position smoothly, leading to a fluid animation.

## ## Best practices

- **Leverage compositor-friendly properties.** Animate `transform` and `opacity` instead of `top`, `left`, `width` or `height` when possible. This avoids triggering layout and paint.
- **Avoid unnecessary layer promotion.** Adding `will-change: transform` or other properties indiscriminately can increase memory consumption. Use layer promotion only on elements that will animate or change frequently.
- **Monitor layers.** Browser dev tools (Chrome's "Layers" panel, Firefox's "Paint" options) allow you to inspect which elements are on their own layers. Use these tools to diagnose performance issues.
- **Minimize paint area.** Even with GPU compositing, large paint areas can cause jank. Keep moving elements isolated and avoid overdraw by using `contain: paint` where appropriate.

## ## Practice questions

1. **Theory:** Describe the difference between painting and compositing. Why does the compositor run on a separate thread?
2. **Theory:** Name three CSS properties that usually cause an element to be promoted to its own layer. Why might this be beneficial?
3. **Coding:** Create a card stack where each card lifts above others on hover using `transform: translateZ()`. Explain how layer promotion prevents jank.
4. **Coding:** Using browser developer tools, inspect a page with multiple animations and identify which elements are on their own layers. Summarize what you observe.

# What are layout thrashing and forced synchronous layouts?

## # What are layout thrashing and forced synchronous layouts

Writing performant JavaScript often requires an understanding of how browser layout works. Two common performance pitfalls are **layout thrashing** and **forced synchronous layouts**. Both arise when code interleaves reads and writes to the DOM in a way that triggers excessive reflows.

### ## Forced synchronous layout

When you modify styles, the browser may defer layout recalculation until the next animation frame. However, if your code subsequently reads a layout property—such as `offsetWidth`, `offsetHeight`, `scrollTop`, `clientTop`, `getBoundingClientRect()`, or `getComputedStyle()`—the browser is forced to flush its queued changes and perform a synchronous reflow to return an up-to-date value. This flush halts the main thread and can lead to jank, especially in loops.

Example:

```
```js
const element = document.querySelector("#box");
element.style.width = "200px";

// Reading layout right after writing forces a synchronous reflow
const currentHeight = element.offsetHeight;
console.log(currentHeight);
```
```

Here, reading `offsetHeight` forces the browser to synchronously recalculate the layout before returning the value. Doing this repeatedly inside a loop can degrade performance.

### ## Layout thrashing

Layout thrashing occurs when code repeatedly alternates between reading layout information and modifying it. Each read triggers a reflow, and each write invalidates the layout, causing the next read

to trigger another reflow. This "ping-pong" effect can result in dozens or hundreds of reflows per frame.

Example of layout thrashing:

```
``js
const items = document.querySelectorAll(".item");
// Increase the width of each item by its current height
items.forEach((item) => {
 item.style.width = item.offsetHeight + 10 + "px"; // write + read repeatedly
});
``
```

For each item, this loop writes to `style.width` (invalidating layout) and then immediately reads `offsetHeight` for the next iteration. On a large list, this can produce many reflows.

## ## Avoiding these pitfalls

1. **\*\*Separate reads and writes.\*\*** Read all required layout values first, store them in variables, then perform all writes. This batches reflows and reduces thrashing.
2. **\*\*Use `requestAnimationFrame()`.** When animating, schedule DOM updates within the same animation frame. The browser will perform at most one reflow per frame. Avoid reading layout properties in between writes during the same frame.
3. **\*\*Cache layout values.\*\*** If possible, compute values once and reuse them instead of repeatedly querying the DOM.
4. **\*\*Use CSS for complex animations.\*\*** CSS transitions and animations keep layout calculations on the browser side and reduce the need for JavaScript layout reads.
5. **\*\*Avoid synchronous API calls.\*\*** Some APIs like `getComputedStyle()` can trigger reflow. Use them sparingly and outside loops.

## ## Example: batching reads and writes

```
```\js
const items = document.querySelectorAll(".item");
const heights = [];
// Read phase
items.forEach((item) => {
  heights.push(item.offsetHeight);
});
// Write phase
items.forEach((item, i) => {
  item.style.width = heights[i] + 10 + "px";
});
```\
```

By separating the read and write phases, this code triggers only a single reflow rather than one per item.

## ## Practice questions

1. **Theory:** Explain what causes a forced synchronous layout and give three methods that can trigger it.
2. **Theory:** Define layout thrashing and describe why it can harm performance.
3. **Coding:** Rewrite the following loop to avoid layout thrashing:

```
```\js
const cards = document.querySelectorAll(".card");
cards.forEach((card) => {
  const h = card.offsetHeight;
  card.style.height = h + 20 + "px";
});
```\
```

4. **Coding:** Use `requestAnimationFrame()` to smoothly animate a progress bar's width without causing layout thrashing. Explain how you schedule reads and writes.





# What is IntersectionObserver and how can it be used for lazy loading?

## # What is IntersectionObserver and how can it be used for lazy loading

`IntersectionObserver` is a browser API that lets you asynchronously observe changes in the intersection of a target element with an ancestor element or the viewport. It allows you to run code when an element enters or leaves the viewport without polling on scroll events. This makes it a perfect tool for **lazy loading** images, infinite scrolling, or triggering animations when content becomes visible.

### ## Key concepts

- **Thresholds:** You can specify an array of intersection ratios (0 to 1) at which to trigger callbacks. For example, a threshold of `0.1` means the callback is invoked when 10 % of the target's area becomes visible.
- **Root:** By default, intersections are relative to the viewport. You can pass a different root element to observe intersections within a scrollable container.
- **Entries:** The callback receives a list of `IntersectionObserverEntry` objects. Each entry provides properties like `isIntersecting`, `intersectionRatio`, `boundingClientRect` and `target`.

### ## Basic usage

```
```js
// Create an observer with a callback
const observer = new IntersectionObserver(
  (entries, obs) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        // Element is visible; perform an action
        console.log("Visible:", entry.target);
        // If we don't need to observe further, unobserve
        obs.unobserve(entry.target);
      }
    });
  }
);
```

```

    });
  },
  {
    root: null, // relative to viewport
    threshold: 0.1, // trigger when 10 % visible
  }
);

// Observe elements
document.querySelectorAll(".observe").forEach((el) => observer.observe(el));
...

```

When any `.observe` element is at least 10 % visible, the callback runs. Using `obs.unobserve()` stops observing that element.

Lazy loading images

One popular use case is loading images only when they are about to enter the viewport. This saves bandwidth and improves page load time.

HTML

```

<<html

...

```

Here, `src` is a placeholder image (tiny or blank) and `data-src` contains the real image URL.

JavaScript

```
```js
const lazyImages = document.querySelectorAll(".lazy-load");
const imgObserver = new IntersectionObserver(
 (entries, observer) => {
 entries.forEach((entry) => {
 if (entry.isIntersecting) {
 const img = entry.target;
 img.src = img.dataset.src;
 observer.unobserve(img);
 }
 });
 },
 { threshold: 0.25 }
);

lazyImages.forEach((img) => imgObserver.observe(img));
```
```

This code observes each `.lazy-load`` image. When an image is at least 25 % visible, its actual ``src`` is assigned, triggering the download. Once loaded, the observer stops observing it. This technique drastically reduces the number of images loaded initially.

Infinite scrolling

IntersectionObserver can also implement endless scrolling. You place a sentinel element at the bottom of the content. When it becomes visible, you fetch more data and append it to the list.

```
```js
const list = document.querySelector("#list");
```

```

const sentinel = document.querySelector("#sentinel");

const infiniteObserver = new IntersectionObserver(
 async (entries) => {
 if (entries[0].isIntersecting) {
 // Fetch next page of data
 const data = await fetchNextPage();
 data.forEach((item) => list.append(createListItem(item)));
 }
 },
 { rootMargin: "200px" }
);

infiniteObserver.observe(sentinel);
...

```

Setting `rootMargin` to 200 px ensures the next page is fetched slightly before the user reaches the end, avoiding perceived delays.

## ## Benefits of IntersectionObserver

- **Efficiency:** It operates asynchronously and is optimized by the browser. Unlike scroll event listeners, it avoids calling JavaScript on every pixel of scroll.
- **Flexibility:** Can observe multiple elements with varying thresholds and root containers.
- **Non-invasive:** You can easily stop observing or add new elements on the fly.

## ## Practice questions

1. **Theory:** What properties can you access from an `IntersectionObserverEntry`? How do `threshold` and `rootMargin` influence when callbacks fire?
2. **Theory:** Explain how lazy loading images using IntersectionObserver improves performance and user experience.

3. **Coding:** Implement lazy loading for background images using `data-bg`` attributes and `IntersectionObserver`.
4. **Coding:** Create an infinite scrolling list that loads more items when the sentinel becomes 50 % visible. Ensure each new batch attaches its own sentinel for continued loading.

# How do custom events improve component communication?

## # How do custom events improve component communication

In modern web development, applications are often built from reusable components. Components should ideally be self-contained and loosely coupled. **Custom events** provide a way for components to communicate without tight coupling, leading to cleaner architectures.

### ## Native events vs custom events

The browser dispatches many built-in events such as `click`, `input`, `scroll` and `submit`. Sometimes you need to signal something that doesn't fit any built-in event, such as "todo item completed" or "user logged in". That's where custom events come in. You can create and dispatch your own events using the `CustomEvent` constructor:

```
```js
const event = new CustomEvent("todo-completed", {
  detail: { id: 123, title: "Buy milk" },
  bubbles: true, // allow the event to bubble up the DOM
  composed: true, // allow it to cross Shadow DOM boundaries
});
someElement.dispatchEvent(event);
```
```

The `detail` property carries any data you want to pass along. The `bubbles` option specifies whether the event should bubble up through ancestor elements, and `composed` allows crossing shadow DOM boundaries so that events can propagate out of Web Components.

### ## Decoupling components

Consider a to-do app built from separate components: a `TodoItem` and a `TodoList`. Instead of the parent directly calling a method on the child or vice versa, the child can dispatch an event when something interesting happens. The parent listens and responds.

```

`html
<!-- parent component -->
<ul id="todo-list">
<script type="module">
 // Create a custom element for TodoItem
 class TodoItem extends HTMLElement {
 connectedCallback() {
 this.innerHTML = `<label><input type="checkbox"> <slot></slot></label>`;
 this.querySelector("input").addEventListener("change", (e) => {
 if (e.target.checked) {
 // dispatch a custom event upward
 this.dispatchEvent(
 new CustomEvent("complete", {
 detail: { text: this.textContent.trim() },
 bubbles: true,
 })
);
 }
 });
 }
 }
 customElements.define("todo-item", TodoItem);

 const list = document.getElementById("todo-list");
 list.addEventListener("complete", (e) => {
 console.log("Todo completed:", e.detail.text);
 // Remove the item or mark it done
 e.target.classList.add("done");
 });

 // Add items
 list.innerHTML = `<todo-item>Buy milk</todo-item><todo-item>Walk dog</todo-item>`;

```



</script>

'''

Here, each `<todo-item>` dispatches a `complete` event when its checkbox is checked. The `todo-list` listens for the event and reacts. The child doesn't need to know about the parent, and the parent doesn't need a reference to each child. This loose coupling improves maintainability and testability.

## ## Event bubbling and delegation

Because custom events can bubble, you can attach a listener on a high-level container and respond to events from many descendants. This is known as **event delegation**. It reduces the number of listeners and makes it easy to handle dynamically added elements.

If you dispatch an event from within a Shadow DOM and want it to cross the boundary, set `composed: true`. Without this, the event stops at the shadow root.

## ## Benefits of custom events

- **Loose coupling:** Components communicate via events instead of direct method calls.
- **Scalability:** Event delegation scales to many child components with minimal listeners.
- **Reusability:** A component can be used in different contexts. It simply emits events; consumers decide how to handle them.
- **Lifecycle awareness:** Events can signal component lifecycle milestones (mounted, destroyed, updated) for instrumentation or cleanup.

## ## Practice questions

1. **Theory:** How does setting the `bubbles` and `composed` options affect a custom event's propagation? When would you set them to `false`?
2. **Theory:** Explain why custom events promote loose coupling in component architecture.
3. **Coding:** Create a `counter-button` custom element that dispatches `increment` and `decrement` events when clicked. Show how a parent component listens to update a total.
4. **Coding:** Build a modal component that dispatches a `close` custom event when the user clicks outside or presses the escape key. Use event bubbling so the parent can remove the modal.



# What is the difference between innerHTML, outerHTML, and textContent?

## # What's the difference between `innerHTML`, `outerHTML` and `textContent`

JavaScript provides several properties for reading and modifying the content of DOM elements: `innerHTML`, `outerHTML` and `textContent`. Although they seem similar, each serves a different purpose and has different performance and security implications.

### ## `innerHTML`

`innerHTML` returns or sets the **HTML markup inside** an element. When you read it, you get a string containing the serialized contents of the element. When you assign to it, the browser parses the string as HTML and replaces the element's children.

```
```js
const div = document.querySelector("#container");
console.log(div.innerHTML); // "<p>Hello <strong>world</strong></p>"
div.innerHTML = "<span>New content</span>";
```
```

Because `innerHTML` parses HTML, it can be expensive on large fragments. It also poses a security risk: setting `innerHTML` from untrusted data can introduce cross-site scripting (XSS) vulnerabilities. Always sanitize untrusted input before assigning to `innerHTML`.

### ## `outerHTML`

`outerHTML` returns or sets the **HTML markup of the element itself and its contents**. Reading it serializes the entire element, including its opening and closing tags. Assigning to `outerHTML` parses the string and replaces the element itself with new content.

```
```js
const heading = document.querySelector("h1");
console.log(heading.outerHTML); // "<h1>Title</h1>"
```
```

```
heading.outerHTML = "<h2>New Title</h2>"; // replaces <h1> entirely
```

```
...
```

Use ``outerHTML`` when you need to replace an element altogether. Like ``innerHTML``, it should not be used with untrusted content.

## ## ``textContent``

``textContent`` returns or sets the **raw text** contained within an element and its descendants. It strips any HTML tags. Assigning to it escapes any HTML characters, inserting text exactly as provided.

```
```js
```

```
const div = document.querySelector("#message");
```

```
div.textContent = "<em>Not parsed</em>"; // inserts literal characters, not HTML
```

```
...
```

Since ``textContent`` does not parse HTML, it is much faster than ``innerHTML`` and safe to use with untrusted input. It is the preferred way to set plain text.

Summary and performance

| Property | Reads/returns | Writes/sets | Use cases |
|----------------------------|-------------------------|-----------------------------------|---------------------------------------|
| <code>`innerHTML`</code> | HTML string of children | Parses string to replace children | Injecting fragments of trusted markup |
| <code>`outerHTML`</code> | HTML string of element | Parses string to replace element | Replacing entire element |
| <code>`textContent`</code> | Plain text of children | Inserts plain text | Setting or reading text safely |

Because ``innerHTML`` and ``outerHTML`` involve parsing and serializing HTML, they are slower than ``textContent``. ``innerHTML`` may trigger a reflow if the new markup affects layout. ``textContent`` simply updates text nodes and is more efficient.

Practice questions

1. **Theory:** Explain why assigning user input directly to ``innerHTML`` can be dangerous. How can you mitigate this risk?

2. **Theory:** What happens when you set `textContent = "<strong>hi</strong>"`? Why is this useful?
3. **Coding:** Write a function `setSanitizedHTML(element, html)` that safely inserts HTML by stripping `<script>` tags and event attributes before setting `innerHTML`.
4. **Coding:** Given an element `<div id="comments"></div>`, demonstrate how to append a new comment as plain text using `textContent` without replacing existing comments.

What is the difference between CORS preflight and simple requests?

What's the difference between CORS preflight and simple requests

Cross-Origin Resource Sharing (CORS) allows a web application on one origin to request resources from a different origin. Because cross-origin requests can pose security risks, the browser follows strict rules before sending them. Requests fall into two categories: **simple requests** and **preflighted requests**.

Simple requests

A request is considered *_simple_* if it meets all of the following criteria:

1. **Method** is one of `GET`, `HEAD` or `POST`.
2. **Headers** are only simple request headers (e.g. `Accept`, `Accept-Language`, `Content-Language`, `Content-Type` with a value of `text/plain`, `application/x-www-form-urlencoded` or `multipart/form-data`).
3. **No custom headers**: It does not include authorization headers like `Authorization` or `X-Custom-Header`.
4. **No `content-type` other than the three permitted values** for POST.

For simple requests, the browser directly sends the HTTP request with the requested method and headers. If the server's response includes appropriate CORS headers (e.g. `Access-Control-Allow-Origin`), the browser makes the response available to the JavaScript code. There is **no extra network round trip** before the actual request.

Preflighted requests

If a request uses methods other than `GET`, `HEAD` or `POST`, or it includes custom headers or uses a disallowed `Content-Type`, the browser must perform a **preflight**. A preflight is an HTTP `OPTIONS` request sent to the target server with the following headers:

- `Access-Control-Request-Method`: The actual HTTP method to be used.
- `Access-Control-Request-Headers`: A comma-separated list of non-simple headers the request will include.

- ``Origin``: The origin of the calling page.

The server must respond to the preflight with ``Access-Control-Allow-Methods`` and, if necessary, ``Access-Control-Allow-Headers`` and ``Access-Control-Allow-Origin``. Only if the server allows the requested method and headers will the browser proceed to send the actual request. If the preflight fails (e.g. the method is not allowed), the browser aborts the request and your JavaScript code receives an error.

Why preflights exist

Preflights protect users and servers by requiring explicit permission before potentially dangerous or unexpected requests. For example, a malicious site might try to send a ``PUT`` request with a custom ``Authorization`` header to an API. Without preflight, the browser would allow the request, potentially leaking sensitive information. With preflight, the API must explicitly opt in, preventing unauthorized access.

When to avoid preflights

Preflights add an extra round trip and latency. To avoid them:

- Stick to simple methods (``GET`` and ``POST``) with permitted content types.
- Avoid custom headers unless absolutely necessary. Use standard headers or encode data in the URL or request body.
- Configure the server to respond with the right CORS headers. If you control the server, you can allow specific methods and headers so preflight responses succeed.

Example

Imagine a frontend at ``https://app.example`` calling an API at ``https://api.example`` with a ``PUT`` request containing JSON and a custom header:

```
```js
fetch("https://api.example/users/123", {
 method: "PUT",
 headers: {
```

```

 "Content-Type": "application/json",
 "X-Auth-Token": "abc123",
 },
 body: JSON.stringify({ name: "Alice" }),
});
'''

```

Because the method is `PUT` and includes a custom header, the browser will:

1. Send a preflight `OPTIONS` request to `https://api.example/users/123` with `Access-Control-Request-Method: PUT` and `Access-Control-Request-Headers: X-Auth-Token, Content-Type`.
2. If the server responds with `Access-Control-Allow-Methods: PUT` and `Access-Control-Allow-Headers: X-Auth-Token, Content-Type`, the browser will send the actual `PUT` request.
3. The server must also include `Access-Control-Allow-Origin: https://app.example` or `\*` in both responses.

### ## Practice questions

1. **Theory:** What are the conditions for a request to be considered simple? Why do these requests skip the preflight?
2. **Theory:** Explain how prefledged requests prevent some types of attacks.
3. **Coding:** Write a fetch request that triggers a CORS preflight and describe the sequence of network requests.
4. **Coding:** Modify the request to avoid a preflight if possible. Explain what changes you made.



# How does sandbox attribute in iframes affect script execution?

## # How does the `sandbox` attribute in iframes affect script execution

The ``<iframe>`` element allows you to embed another page inside your site. Without restrictions, the embedded page can run scripts, submit forms or navigate the parent. The `sandbox` attribute provides a way to constrain what an embedded page can do. By default, applying `sandbox` creates a *\_locked-down\_* environment; you then selectively enable capabilities via the `allow-\*` tokens.

### ## Default sandbox restrictions

When you add the `sandbox` attribute with no value (or with an empty string), the iframe loses many privileges:

- **\*\*No script execution:\*\*** JavaScript and other scripts are disabled. The page cannot run inline scripts or load external scripts.
- **\*\*No forms:\*\*** The page cannot submit forms.
- **\*\*No popups or new windows:\*\*** The page cannot open new windows via `window.open()`.
- **\*\*No same-origin privileges:\*\*** Even if the iframe's content is from the same origin, it is treated as if it were cross-origin. The parent cannot access the iframe's DOM via `contentWindow` and vice versa.
- **\*\*No modal dialogs:\*\*** The page cannot call `alert`, `confirm` or `prompt`.
- **\*\*No pointer lock, geolocation, or top-level navigation.\*\***

These restrictions make sandboxed iframes ideal for isolating untrusted content.

### ## Relaxing restrictions with sandbox tokens

You can gradually restore specific capabilities by adding allow tokens in the `sandbox` attribute. Some common tokens:

- `allow-scripts`: Allows executing JavaScript. If you include this token but not `allow-same-origin`, the script runs in a unique origin and cannot access cookies or localStorage.

- ``allow-same-origin``: Treats the iframe as same origin. Use with caution—if you also allow scripts, the iframe could break out of confinement.
- ``allow-forms``: Allows form submission.
- ``allow-popups``: Allows opening new windows or tabs.
- ``allow-popups-to-escape-sandbox``: Allows popups opened by the sandboxed document to not inherit the sandbox restrictions.
- ``allow-modals``, ``allow-top-navigation``, ``allow-pointer-lock``: Enable dialogs, navigating top-level browsing context, or pointer locking.

Example:

```
```html
<iframe
  src="https://untrusted.example"
  sandbox="allow-scripts allow-forms"
></iframe>
```
```

Here, the iframe can run scripts and submit forms but cannot navigate its top-level frame or access cookies because ``allow-same-origin`` is absent.

## ## Interaction with Content Security Policy (CSP)

Even if you allow scripts via ``sandbox``, CSP rules on the parent or the embedded document still apply. For example, if the embedded page's own CSP forbids inline scripts, they will still be blocked. Likewise, if the parent page uses a CSP with ``frame-ancestors`` directive, it can prevent other sites from embedding it at all.

## ## Shadow DOM and sandbox

Sandboxing isolates the iframe from the parent DOM but is separate from the Shadow DOM concept. The ``sandbox`` attribute controls privileges at the browsing context level, whereas Shadow DOM encapsulates styles and markup within a component. You can nest sandboxed iframes inside Shadow DOM, but they remain independent mechanisms.

## ## Practice questions

1. **Theory:** What happens when you add `sandbox` with no tokens to an iframe? List at least three restrictions it imposes.
2. **Theory:** Why is combining `allow-scripts` and `allow-same-origin` potentially dangerous? Give an example scenario.
3. **Coding:** Create an iframe that embeds a third-party comments widget. Allow scripts to run but prevent access to cookies and localStorage. Explain which tokens you use and why.
4. **Coding:** Demonstrate how a parent page can use postMessage to communicate with a sandboxed iframe. Highlight security considerations.

# What are cross-origin resource policies (CORP, COEP, COOP) and why do they matter?

## # What are cross-origin resource policies (CORP, COEP, COOP) and why do they matter

Web security has evolved to protect against cross-origin data leaks and to enable powerful features like `SharedArrayBuffer`. Three complementary HTTP headers—**Cross-Origin Resource Policy (CORP)**, **Cross-Origin Embedder Policy (COEP)** and **Cross-Origin Opener Policy (COOP)**—allow sites to opt in to stricter isolation. Together, COOP and COEP provide **cross-origin isolation**, which unlocks high-resolution timers and shared memory in browsers.

### ## Cross-Origin Resource Policy (CORP)

`Cross-Origin-Resource-Policy` instructs the browser whether a resource can be loaded by other origins. It has three possible values:

- `same-origin`: Only the same origin can load the resource. Requests from other origins will be blocked.
- `same-site`: Only the same site (including subdomains) can load the resource.
- `cross-origin`: Any origin may load the resource (default). Use this for public assets.

CORP helps prevent *side-channel attacks* where a malicious page attempts to fetch a resource and infer its content based on response timing or error messages. For example, an image served with `same-origin` cannot be embedded by another origin.

### ## Cross-Origin Embedder Policy (COEP)

`Cross-Origin-Embedder-Policy` controls which cross-origin resources (scripts, images, etc.) your page is allowed to load. It has two primary values:

- `unsafe-none`: The default; allows embedding cross-origin resources without restrictions.
- `require-corp`: Your page may only load cross-origin resources that either explicitly allow being embedded with `Cross-Origin-Resource-Policy: cross-origin` or come from the same origin. Resources without CORP or CORS headers will be blocked.

COEP effectively protects you from accidentally including third-party resources that might leak data, and is one half of achieving cross-origin isolation.

## ## Cross-Origin Opener Policy (COOP)

`Cross-Origin-Opener-Policy` defines the relationship between your page and any opened windows or tabs. It also determines whether your page shares the same browsing context group with pages from other origins. Important values:

- `unsafe-none`: Default; your page shares a browsing context group with any opener, meaning `window.opener` is available and resources like `document.cookie` can be leaked via side channels.
- `same-origin`: The opener of your window must be the same origin; otherwise the browsing contexts are isolated. `window.opener` becomes `null` for cross-origin openers.
- `same-origin-allow-popups`: Similar to `same-origin`, but allows popups to share a browsing context group with each other if they are same-origin.

By isolating browsing contexts, COOP prevents cross-origin pages from using `window.opener` to manipulate or peek into each other's state.

## ## Cross-origin isolation

To enable features like `SharedArrayBuffer` and `performance.now()` with nanosecond resolution, a document must be **cross-origin isolated**. This requires both COEP and COOP:

```
``http
Cross-Origin-Embedder-Policy: require-corp
Cross-Origin-Opener-Policy: same-origin
...
```

With these headers in place, the browser ensures that the page does not share its memory or performance timing information with any non-isolated cross-origin contexts. In return, you can safely use `SharedArrayBuffer`, `Atomics` and high-precision timers.

## ## Why they matter

- **Security:** They mitigate side-channel attacks and cross-site leaks by limiting who can load your resources and preventing untrusted pages from interacting with yours.
- **Performance features:** Cross-origin isolation is required for SharedArrayBuffer, which enables true multithreading via Web Workers with shared memory. It also restores `performance.now()` precision beyond the default reduced resolution.
- **Future web APIs:** Many upcoming APIs (like WebAssembly threads) rely on cross-origin isolation. Using CORP, COEP and COOP prepares your site for these features.

### ## Practice questions

1. **Theory:** Explain the difference between `Cross-Origin-Resource-Policy: same-site`` and `Cross-Origin-Embedder-Policy: require-corp``. How do they complement each other?
2. **Theory:** What must be set on a document to achieve cross-origin isolation and why is it required for SharedArrayBuffer?
3. **Coding:** Configure an Express.js server to serve images with `Cross-Origin-Resource-Policy: same-origin`` and a script with `Cross-Origin-Resource-Policy: cross-origin``. Test loading them from another origin and observe the results.
4. **Coding:** Use the `fetch()` API to load a resource from a server that does not send CORP or CORS headers. Then update the server to include `Cross-Origin-Resource-Policy: cross-origin`` and observe the difference when COEP is enabled on your page.

# What is a content-type sniffing attack and how can JS prevent it?

## # What is a content-type sniffing attack and how can JS prevent it

Browsers need to know how to interpret a resource—whether it is a script, an image, a PDF or plain text. They generally trust the `Content-Type` header sent by the server. However, to be user-friendly, browsers sometimes perform **content sniffing**: they attempt to infer the type of a resource if the `Content-Type` is ambiguous or missing. While helpful in some cases, this behavior opens the door to **content-type sniffing attacks**.

### ## The attack

A content-type sniffing attack occurs when a browser interprets a resource as executable (e.g. HTML or JavaScript) even though the server intended to serve it as something else (e.g. an image or text file). A malicious actor can exploit this by storing an HTML/JS payload on a server that expects only images. If the browser guesses incorrectly and executes the content, it may run attacker-supplied scripts in the context of your site.

Consider a file upload endpoint that stores user uploads in a public `/uploads` folder and serves them with a generic `Content-Type: application/octet-stream`. An attacker uploads a file containing `

For example:

```
``http
X-Content-Type-Options: nosniff
Content-Type: image/png

...binary PNG data...
``
```

If a script is served as `image/png` with `nosniff`, the browser will refuse to execute it.

### ### Validate and sanitize uploads

On the server, validate uploaded files: check their content matches the claimed MIME type, restrict allowed types, and sanitize filenames. Store uploads outside of publicly accessible paths or serve them from a different domain to isolate them from your main site.

### ### Use JavaScript safely

When fetching data with `fetch()` or XHR, examine the `Content-Type` header and avoid using potentially executable responses in dangerous ways. For example, do not insert untrusted text into `innerHTML`. Use `textContent` for plain text and parse JSON with `response.json()`.

Example of safe image loading in JavaScript:

```
``js
async function loadImage(url) {
 const res = await fetch(url, { mode: "cors" });
 const type = res.headers.get("Content-Type") || "";
 if (!type.startsWith("image/")) {
 throw new Error("Not an image");
 }
}
```



```

}

const blob = await res.blob();

const imgUrl = URL.createObjectURL(blob);

const img = new Image();

img.src = imgUrl;

document.body.appendChild(img);

}

...

```

This function ensures the server returns an image before creating an `` element. It avoids content sniffing by verifying the header.

## ## Practice questions

1. **Theory:** Why do browsers perform content sniffing? What risks does it introduce?
2. **Theory:** How does the `X-Content-Type-Options: nosniff` header help mitigate content-type sniffing attacks? What happens if the header is missing?
3. **Coding:** Write server code (in Express or another framework) that serves files with proper `Content-Type` headers and includes `X-Content-Type-Options: nosniff`.
4. **Coding:** Implement a fetch wrapper that verifies the MIME type before creating DOM elements. How would you extend it to handle JSON, images and text safely?

# What is the Observer pattern and how is it implemented in JavaScript?

## # What is the observer pattern and how is it implemented in JavaScript

The **observer pattern** is a design pattern in which an object (called the `_subject_` or `_observable_`) maintains a list of dependent objects (called `_observers_`) and notifies them automatically when its state changes. This allows the subject to publish state changes without knowing anything about the observers' internal details, promoting a loose coupling.

### ## Key components

- **Subject (observable):** Holds the state being observed and a collection of observers. Provides methods to subscribe or unsubscribe observers and a method to notify them of changes.
- **Observers:** Objects that want to be notified when the subject changes. They implement an `update()` or similar method called by the subject.

### ## Why use the observer pattern

- **Decoupling:** The subject does not need to know how many observers there are or what they do; it simply notifies them. Observers can be added or removed at runtime.
- **Reactivity:** Changes propagate automatically. This pattern underpins many reactive frameworks (e.g. RxJS, Vue's reactivity system) and event-driven architectures.
- **Maintainability:** The pattern centralizes state changes and reduces the number of direct method calls between components.

### ## Implementing a simple observer pattern in JavaScript

Below is an implementation of a subject that manages a list of observers. Observers are objects that define an `update()` method.

```
```\n\nclass Observable {\n  constructor(value) {\n    this._value = value;\n  }\n}
```

```

    this._observers = new Set();
  }
  get value() {
    return this._value;
  }
  set value(newValue) {
    if (newValue !== this._value) {
      this._value = newValue;
      this.notify(newValue);
    }
  }
  subscribe(observer) {
    this._observers.add(observer);
    // Immediately send current value to new observer
    observer.update(this._value);
    return () => this._observers.delete(observer);
  }
  notify(val) {
    this._observers.forEach((obs) => obs.update(val));
  }
}

// Usage
const state = new Observable(0);

const logger = {
  update(value) {
    console.log("State changed to", value);
  },
};

```

```

const unsubscribe = state.subscribe(logger);
state.value = 42; // logs "State changed to 42"
unsubscribe();
state.value = 100; // no log, observer unsubscribed
...

```

The `Observable` class encapsulates a value and notifies subscribed observers whenever it changes. Observers can unsubscribe by calling the returned function.

Observer pattern vs event emitters

JavaScript environments like Node.js provide an `EventEmitter` which implements a similar pattern. You can subscribe to named events and emit them when appropriate:

```

```js
const EventEmitter = require("events");
const emitter = new EventEmitter();

emitter.on("data", (payload) => {
 console.log("Received:", payload);
});

emitter.emit("data", { id: 1, value: 10 });
...

```

Here, `emitter` acts as a subject and handlers act as observers. The principle remains the same: decoupled notification of state changes or events.

### ## When to use the observer pattern

- Building reactive UI components that should update when underlying data changes.
- Implementing pub-sub systems, event emitters or state management libraries.

- Coordinating asynchronous processes (e.g. network events, WebSocket messages).

## ## Practice questions

1. **Theory:** Describe the roles of the subject and observer in the observer pattern. How does the pattern promote loose coupling?
2. **Theory:** What are some real-world examples of the observer pattern in browser APIs or JavaScript libraries?
3. **Coding:** Extend the `Observable` class above to support multiple values (e.g. an observable object with multiple keys). Write a test observer that listens to changes on a specific key.
4. **Coding:** Use Node.js `EventEmitter` to implement a chat room where users can subscribe to receive messages. Explain how this illustrates the observer pattern.

# What is the Publish–Subscribe pattern and how does it differ from Observer?

## # What is the publish-subscribe pattern and how does it differ from observer

The **\*\*publish-subscribe (pub-sub) pattern\*\*** is a messaging pattern where senders (*\_publishers\_*) emit events to a central broker, and receivers (*\_subscribers\_*) express interest in certain event types. Publishers and subscribers do not know about each other; the broker routes messages based on topic or event name. This decoupling makes the pattern useful for systems where components must communicate without tight coupling.

## ## How pub-sub works

1. **\*\*Subscribers\*\*** register interest in one or more topics. They provide a callback to be executed when a message on that topic is published.
2. **\*\*Publishers\*\*** emit messages to the broker, tagging each message with a topic.
3. The **\*\*broker\*\*** looks up subscribers for that topic and invokes their callbacks with the message payload. Subscribers can subscribe or unsubscribe at runtime.

## ### Example: a simple message bus in JavaScript

```
``js
class PubSub {
 constructor() {
 this.topics = new Map();
 }
 subscribe(topic, handler) {
 if (!this.topics.has(topic)) {
 this.topics.set(topic, new Set());
 }
 this.topics.get(topic).add(handler);
 return () => this.topics.get(topic).delete(handler);
 }
 publish(topic, data) {
```

```

const handlers = this.topics.get(topic);

if (!handlers) return;

handlers.forEach((handler) => handler(data));
}
}

const bus = new PubSub();

const unsubscribe = bus.subscribe("news", (article) => {
 console.log("Breaking news:", article.headline);
});

bus.publish("news", { headline: "New JavaScript release" });

unsubscribe();

bus.publish("news", { headline: "Second article" });
...

```

In this implementation, subscribers listen for the `'news'` topic. When a publisher publishes a message with that topic, the message is delivered to all subscribed handlers.

When you call `bus.subscribe('news', handler)`, you are telling the message bus: "Please call my `handler` function whenever someone publishes a `'news'` message." Internally, the `subscribe()` method adds your handler to a list of listeners for that topic. To give you an easy way to stop listening later, `subscribe()` returns another function.

Think of it like signing up for a newsletter: you give the publisher your email address, and they start sending you updates. Along with your subscription, they include a special "unsubscribe" link. If you click that link, they remove your email address from their mailing list so you stop receiving messages.

In the code, we store that "unsubscribe link" in a variable:

```

```js
const unsubscribe = bus.subscribe("news", (article) => {
  console.log("Breaking news:", article.headline);

```

```
});
```

```
...
```

The `unsubscribe` variable now holds a function. Whenever you no longer want to receive `'news'` messages, you call it:

```
```js
```

```
unsubscribe(); // stop receiving news updates
```

```
...
```

After calling `unsubscribe()`, any future calls to `bus.publish('news', ...)` won't trigger your handler. Saving the returned function in a variable like `unsubscribe` is just a convenient way to keep a reference to it so you can clean up your listener later.

## ## Differences between pub-sub and observer

Although the pub-sub and observer patterns both involve one-to-many communication, there are key differences:

- **Direct vs. mediated:** In the observer pattern, observers subscribe directly to a specific subject and the subject notifies them. In pub-sub, publishers and subscribers are decoupled via a broker. The subject (topic) does not directly know its subscribers.
- **Topics/queues:** Pub-sub uses a named channel (topic) or queue. Observers typically register with a specific object, not a global topic.
- **Unknown publishers:** Observers know which subject they observe; subscribers to a pub-sub system may not know who publishes messages.
- **Scalability:** Pub-sub scales to distributed systems and message queues (e.g. RabbitMQ, Kafka) where publishers and subscribers may run on different machines. Observer is often used within a single process or object graph.

## ## When to use pub-sub

- Decoupling modules in a large application. For example, a logging service can subscribe to `'error'` events without the application knowing about it.
- Implementing event buses or global event systems in browser or Node.js apps.



- Distributing events across processes or servers using message brokers or WebSockets.

## ## Practice questions

1. **Theory:** Explain how the publish-subscribe pattern decouples publishers and subscribers. What are the benefits of this decoupling?
2. **Theory:** List two differences between the pub-sub and observer patterns.
3. **Coding:** Implement a basic pub-sub system that supports wildcard subscriptions (e.g. subscribing to `'user.*'` to receive both `'user.login'` and `'user.logout'`).
4. **Coding:** Use the pub-sub pattern to implement a notification system where different modules can publish success or error notifications. Demonstrate subscribing and unsubscribing to a topic.

# Explain functional composition in JavaScript

## # Explain functional composition in JavaScript

Functional composition is the process of combining multiple functions so that the output of one becomes the input of the next. Rather than calling functions one after the other manually, composition creates a new function that represents the pipeline of operations. This leads to code that is declarative, reusable and easy to test.

### ## What is composition

Given two functions `f`` and `g``, the composition `f ∘ g`` is a function such that `(f ∘ g)(x) = f(g(x))``. In JavaScript, you can compose functions manually:

```
```js
const toUpper = (str) => str.toUpperCase();
const exclaim = (str) => `${str}!`;

const shout = (str) => exclaim(toUpper(str));

console.log(shout("hello")); // "HELLO!"
```
```

The `shout`` function composes `toUpper`` and `exclaim``. Composition becomes more powerful when you compose many functions and reuse the resulting pipeline.

### ## Building a generic `compose`` function

We can write a higher-order function that takes an arbitrary number of functions and returns their composition. A common implementation composes functions from right to left:

```
```js
function compose(...fns) {
  return (initial) => {
```

```

    return fns.reduceRight((value, fn) => fn(value), initial);
  };
}

```

```

const trim = (s) => s.trim();
const lower = (s) => s.toLowerCase();
const addPeriod = (s) => s + ".";

const tidy = compose(addPeriod, lower, trim);
console.log(tidy(" Hello World ")); // "hello world."
...

```

Here, `compose(addPeriod, lower, trim)` returns a function that first applies `trim`, then `lower`, then `addPeriod` to the input. You can swap the order of functions or replace one function without affecting others.

Alternatively, you can implement `pipe()` which composes functions left to right:

```

```js
const pipe =
 (...fns) =>
 (initial) =>
 fns.reduce((value, fn) => fn(value), initial);

const shoutPipe = pipe(trim, toUpper, exclaim);
console.log(shoutPipe(" hello ")); // "HELLO!"
...

```

## ## Advantages of composition

- **Reusability:** You write small, single-purpose functions and compose them into more complex operations.

- **Testability:** Each function can be tested in isolation. The composed function has no hidden state.
- **Readability:** A composed pipeline reads like a data flow: the value passes through a sequence of transforms.
- **Immutability:** Composed functions are pure if the individual functions are pure; they don't mutate shared state.

## ## Composition in libraries

Libraries like Lodash and Ramda provide helpers (`_.flow`, `R.pipe`, `R.compose`) to make composition easy. React's hooks and higher-order components also encourage composing functionality.

## ## Practice questions

1. **Theory:** In your own words, explain what function composition is and why it is useful.
2. **Theory:** Contrast composition with chaining methods on an object (e.g. `array.map().filter().reduce()`). When might one be preferred?
3. **Coding:** Write a `composeAsync()` function that handles functions returning promises. Compose three asynchronous functions (e.g. fetch data, parse JSON, extract a field).
4. **Coding:** Use function composition to build a data processing pipeline that normalizes an array of names (trim, capitalize first letter, append an index). Show how to swap out one step without modifying others.

# What are Higher-Order Components (HOC) and render-props patterns conceptually?

## # What are higher-order components (HOC) and render props patterns conceptually

React encourages code reuse by allowing you to encapsulate behavior and share it between components. Two common patterns for sharing logic are **higher-order components** (HOCs) and **render props**. Both allow you to abstract stateful logic away from the components that use it.

### ## Higher-order components (HOC)

A higher-order component is a function that takes a component and returns a new component with enhanced functionality. HOCs wrap the original component, injecting additional props or behavior. They are similar to higher-order functions in functional programming.

Example: a HOC to inject the current window width into any component.

```
```jsx
function withWindowWidth(WrappedComponent) {
  return class extends React.Component {
    state = { width: window.innerWidth };
    handleResize = () => this.setState({ width: window.innerWidth });
    componentDidMount() {
      window.addEventListener('resize', this.handleResize);
    }
    componentWillUnmount() {
      window.removeEventListener('resize', this.handleResize);
    }
    render() {
      return <WrappedComponent width={this.state.width} {...this.props} />;
    }
  };
}
```

```
// Usage

const DisplayWidth = ({ width }) => <p>Window width: {width}</p>;

const ResponsiveDisplay = withWindowWidth(DisplayWidth);

...

```

`withWindowWidth` adds resize listeners and passes the width as a prop. Any component can become responsive by being wrapped with this HOC. HOCs are commonly used for cross-cutting concerns such as authentication, routing or theming.

Render props

The render props pattern involves a component that takes a function as its `children` or `render` prop. Instead of returning JSX directly, the component calls this function with relevant data or callbacks. The caller decides how to render the UI.

Example: a `` component that tracks mouse position and uses a render prop to display it.

```
```jsx

class MouseTracker extends React.Component {
 state = { x: 0, y: 0 };

 handleMouseMove = e => {
 this.setState({ x: e.clientX, y: e.clientY });
 };

 render() {
 return (
 <div style={{ height: '100vh' }} onMouseMove={this.handleMouseMove}>
 {this.props.render(this.state)}
 </div>
);
 }
}

```

```
const App = () => (
 <MouseTracker render={({ x, y }) => (
 <p>The mouse position is ({x}, {y})</p>
)} />
);
...

```

`MouseTracker` manages state internally but leaves presentation up to the caller. Render props are flexible and work well for composing multiple behaviors. They avoid component names that wrap the original component and are easier to type-check compared to HOCs.

## ## Comparing the two patterns

\* **Composition style:** HOCs wrap components and return new components, while render props involve a component that invokes a function passed as a prop.

\* **Name collisions:** HOCs can cause prop name collisions if not careful. Render props avoid this by explicitly passing arguments to the render function.

\* **Ease of testing:** Both are testable; HOCs can be more opaque if many wrappers are stacked. Render props sometimes result in deeply nested JSX.

\* **Hooks era:** Since React 16.8, custom hooks have become the primary way to share logic. Hooks can replace many HOC or render prop patterns with simpler syntax.

## ## Practice questions

1. **Theory:** Explain the main difference between a higher-order component and a render prop. When would you choose one over the other?
2. **Theory:** What problems might occur if you wrap a component with multiple HOCs? How can you avoid them?
3. **Coding:** Write a HOC that injects network status (`online`/`offline`) into a component using the `navigator.onLine` API and `online`/`offline` events.
4. **Coding:** Implement a render prop component `Toggle` that manages a boolean `on` state and provides a function to toggle it. Show how to use it to build a custom switch UI.





# What is dependency injection and can it be achieved in JavaScript?

## # What is dependency injection and can it be achieved in JavaScript

**\*\*Dependency injection (DI)\*\*** is a design pattern in which an object's dependencies are provided from the outside rather than being created internally. Instead of hard-coding the creation of services or collaborators, you inject them through constructors, functions or setters. This approach promotes loose coupling, easier testing and flexibility.

## ## Why dependency injection matters

- **\*\*Decoupling:\*\*** Components do not need to know how to instantiate their collaborators. They can work with any implementation of a given interface.
- **\*\*Testability:\*\*** During unit tests, you can inject mocks or stubs instead of real services.
- **\*\*Flexibility:\*\*** You can switch implementations (e.g. swap a local storage service for an in-memory cache) without modifying the consumer.

## ## DI in JavaScript

JavaScript does not have built-in dependency injection containers like some server-side frameworks, but the pattern can still be applied through techniques such as:

### ### Constructor injection

Pass dependencies to a class or function at construction time.

```
``js
class UserService {
 constructor(api) {
 this.api = api;
 }
 async getUser(id) {
 return await this.api.fetch(`/users/${id}`);
 }
}
```

```
}
}
```

```
class ApiClient {
 async fetch(url) {
 const res = await fetch(url);
 return res.json();
 }
}
```

```
// Dependency injection
const apiClient = new ApiClient();
const userService = new UserService(apiClient);

userService.getUser(1).then((user) => console.log(user));
...
```

In tests, you can inject a fake `ApiClient`:

```
```js  
class FakeApi {  
  async fetch(url) {  
    return { id: 1, name: "Test User" };  
  }  
}  
  
const testService = new UserService(new FakeApi());  
...
```

Factory functions

Instead of using `new`, create objects via factory functions that accept dependencies.

```

```js
function createLogger(prefix) {
 return (msg) => console.log(`[${prefix}]`, msg);
}

function createUserController(userService, logger) {
 return {
 async showUser(id) {
 const user = await userService.getUser(id);
 logger(`User loaded: ${user.name}`);
 return user;
 },
 };
}

const logger = createLogger("App");
const controller = createUserController(userService, logger);
```

```

Dependency injection containers

For larger applications, you can build or use a DI container. A container registers providers for different tokens and resolves dependencies recursively. Libraries like InversifyJS offer decorators and metadata to declare dependencies.

Example of a simple container:

```

```js
class Container {
 constructor() {
 this.registry = new Map();
 }
}
```

```

```

register(token, provider) {
  this.registry.set(token, provider);
}

resolve(token) {
  const provider = this.registry.get(token);
  if (typeof provider === "function") {
    return provider(this);
  }
  return provider;
}
}

const container = new Container();
container.register("api", () => new ApiClient());
container.register("userService", (c) => new UserService(c.resolve("api")));

const svc = container.resolve("userService");
svc.getUser(1);
...

```

Caveats and best practices

- ****Avoid hidden dependencies:**** Explicitly list all dependencies in constructors or factory functions. Hidden dependencies via module imports make testing harder.
- ****Don't over-engineer:**** Small scripts don't need a full DI container. Use simple functions or parameters.
- ****Use interfaces or types:**** In TypeScript, define interfaces for dependencies so that implementations can vary.

Practice questions

1. ****Theory:**** Why is dependency injection beneficial for testing? Give an example scenario.

2. **Theory:** What is the difference between constructor injection and setter injection? Which is preferred in JavaScript and why?
3. **Coding:** Refactor a class that directly calls `fetch()` to instead accept an HTTP client dependency. Show how to inject a mock client for testing.
4. **Coding:** Implement a simple DI container in JavaScript that supports singleton providers and demonstrates resolving nested dependencies.

What are singletons and their drawbacks in JavaScript?

What are singletons and their drawbacks in JavaScript

A **singleton** is a design pattern that restricts the instantiation of a class to a single instance. Whenever you request the singleton, the same object is returned. Singletons provide a global point of access to resources such as configuration, logging or database connections.

Implementing a singleton in JavaScript

JavaScript's module system naturally lends itself to singleton-like behavior: modules are cached after the first import, so subsequent imports return the same instance. However, you can also implement a singleton class manually:

```
``js
class Logger {
  constructor() {
    if (Logger.instance) {
      return Logger.instance;
    }
    this.logs = [];
    Logger.instance = this;
  }
  log(message) {
    this.logs.push(message);
    console.log("[LOG]", message);
  }
  get count() {
    return this.logs.length;
  }
}

const logger1 = new Logger();
const logger2 = new Logger();
console.log(logger1 === logger2); // true
logger1.log("Hello");
```

```
console.log(logger2.count); // 1
...

```

The constructor checks if an instance already exists and returns it if so. This ensures only one instance is created.

Alternatively, you can use an IIFE (Immediately Invoked Function Expression) to encapsulate the instance:

```
``js
const Counter = (() => {
  let instance;
  function create() {
    let value = 0;
    return {
      increment() {
        value++;
      },
      getValue() {
        return value;
      },
    };
  }
  return {
    getInstance() {
      if (!instance) instance = create();
      return instance;
    },
  };
})();

```

```
const counterA = Counter.getInstance();

```

```
const counterB = Counter.getInstance();  
counterA.increment();  
console.log(counterB.getValue()); // 1  
...
```

Drawbacks of singletons

While singletons can simplify access to shared resources, they come with significant downsides:

- **Global mutable state:** A singleton introduces global state. Any part of your application can mutate it, making behavior unpredictable and complicating debugging.
- **Hidden dependencies:** Components that rely on singletons implicitly depend on them. Testing such components becomes harder because you must reset the singleton's state between tests.
- **Tight coupling:** Consumers become tightly coupled to the singleton implementation. Swapping implementations or running multiple instances (e.g. multiple databases) becomes difficult.
- **Concurrency issues:** In asynchronous environments, singletons may create contention or race conditions if not managed carefully.

Alternatives

- **Dependency injection:** Inject dependencies instead of accessing singletons directly. This makes dependencies explicit and testable.
- **Factory functions:** Use functions to create instances as needed. Pass them down the call chain rather than storing them globally.
- **Module exports:** Use ES modules to encapsulate state and functions. If you need multiple instances, export a factory function instead of exporting a single shared object.

Practice questions

1. **Theory:** Why do singletons make unit testing more complicated? Provide an example of test pollution caused by a singleton.
2. **Theory:** Describe scenarios where a singleton might be appropriate and scenarios where it should be avoided.

3. ****Coding:**** Implement a singleton pattern using ES module caching. Then show how you would refactor the code to avoid the singleton by injecting dependencies.
4. ****Coding:**** Create a logging module that stores logs in memory but allows multiple independent loggers. How would you structure the module to avoid a singleton?

What is event-driven architecture and how can it be implemented in JavaScript?

What is event-driven architecture and how can it be implemented in JavaScript

****Event-driven architecture (EDA)**** is a style of building software systems in which components communicate by emitting and responding to events rather than invoking each other directly. The system reacts to events as they occur, leading to loosely coupled components, better scalability and improved responsiveness.

Characteristics of EDA

- ****Producers and consumers:**** Components produce events when something happens (e.g. user input, a message arrives). Other components consume these events and perform actions.
- ****Asynchronous communication:**** Events are handled asynchronously, allowing the system to process multiple events concurrently without blocking.
- ****Decoupling:**** Producers and consumers do not call each other directly. They interact through an event broker or message bus.
- ****Event flow:**** Events can trigger other events, forming a reactive chain.

Event-driven JavaScript

JavaScript is well suited for EDA because of its event loop and asynchronous APIs. You already use event-driven programming when handling DOM events (click, keydown) or Node.js events.

Browser example: custom event bus

You can implement an event bus using the `EventTarget` API:

```
``js
// Create a global event bus
const bus = new EventTarget();

// Listen for a custom event
```

```

bus.addEventListener("user:login", (e) => {
  console.log("User logged in:", e.detail);
});

// Dispatch the event somewhere else in your app
function loginUser(user) {
  // ... authentication logic ...
  bus.dispatchEvent(new CustomEvent("user:login", { detail: user }));
}

loginUser({ id: 1, name: "Alice" });
...

```

Here, components fire events on the bus and others listen for them. The bus decouples producers from consumers. You can also use libraries like mitt or build more sophisticated buses that support namespaces or wildcard topics.

Node.js example: EventEmitter

Node.js has a built-in `EventEmitter` class used throughout the standard library:

```

```js
const { EventEmitter } = require("events");
const emitter = new EventEmitter();

// Consumer
emitter.on("order:created", (order) => {
 console.log("Processing order", order.id);
});

// Producer
function createOrder(items) {

```

```
const order = { id: Date.now(), items };
emitter.emit("order:created", order);
}
```

```
createOrder(["apple", "banana"]);
...
```

`EventEmitter` queues listeners and executes them asynchronously after the current call stack, making it ideal for I/O completion events, timers, etc.

### ### Scaling beyond a single process

In distributed systems, EDA is implemented using message brokers (e.g. RabbitMQ, Kafka, Redis Pub/Sub). Components publish events to a broker; other services subscribe to event streams. This approach decouples services, improves reliability (thanks to queues and persistence) and allows independent scaling. Libraries like `amqplib` (RabbitMQ) or `kafkajs` can be used in Node.js to integrate with such brokers.

### ## Benefits and challenges

#### **\*\*Benefits:\*\***

- Improved scalability and fault tolerance—components can be scaled independently.
- Loose coupling—producers and consumers do not depend on each other's implementation.
- Flexibility—new consumers can be added without modifying existing producers.

#### **\*\*Challenges:\*\***

- Harder to trace flow—events may pass through many handlers, making debugging more complex.
- Ordering guarantees—events might arrive out of order unless sequencing is enforced.
- Error handling—in distributed EDA, ensure failed event processing is retried or logged appropriately.

## ## Practice questions

1. **Theory:** What are the main benefits of event-driven architecture compared to a traditional request-response model?
2. **Theory:** Explain how the EventEmitter API in Node.js enables EDA. How do you handle errors thrown inside event handlers?
3. **Coding:** Implement an event bus in the browser that supports namespaced events (e.g. `chat:message`). Show how producers and consumers interact through the bus.
4. **Coding:** Set up a simple Node.js service that publishes messages to Redis Pub/Sub and another service that subscribes and processes them. Explain how this demonstrates EDA in a distributed system.

# Explain memoization strategies and cache invalidation techniques

## # Explain memoization strategies and cache invalidation techniques

**\*\*Memoization\*\*** is an optimization technique that stores the results of expensive function calls and returns the cached result when the same inputs occur again. When used properly, it can dramatically improve performance for pure functions that are called repeatedly with the same arguments. However, effective memoization requires choosing appropriate caching strategies and knowing when to invalidate stale entries.

### ## Basic memoization

A simple memoization function wraps another function and caches its results based on the input arguments. For functions with a single primitive argument, a plain object or `Map` suffices:

```
```js
function memoize(fn) {
  const cache = new Map();
  return function(arg) {
    if (cache.has(arg)) {
      return cache.get(arg);
    }
    const result = fn(arg);
    cache.set(arg, result);
    return result;
  };
}

const slowSquare = n => {
  console.log('Computing', n);
  return n * n;
};
```

```
const fastSquare = memoize(slowSquare);  
fastSquare(4); // logs "Computing 4", returns 16  
fastSquare(4); // returns 16 instantly, no log  
...
```

This approach works for deterministic functions with primitive arguments. If the function takes multiple parameters or objects, you need a more robust key—e.g. serializing arguments with JSON or using a `WeakMap` keyed by object.

Advanced strategies

LRU (Least Recently Used) cache

An LRU cache discards the least recently accessed items when it reaches a maximum size. This prevents memory from growing unbounded. You can implement an LRU cache by combining a map with a doubly linked list to track usage order. Libraries such as `lru-cache` handle this for you.

TTL (Time to Live)

You might want to invalidate cache entries after a certain period. A TTL cache associates each entry with an expiration timestamp. When retrieving a value, the cache checks whether it has expired. This is useful when underlying data changes over time.

Parameter normalization

Functions that accept many arguments or objects can benefit from normalizing parameters to a unique key. For example, you can create a composite key by joining arguments or by using a `WeakMap` that maps argument objects to results.

Selective memoization

Not all functions should be memoized. Functions with side effects, non-deterministic results (e.g. random or time based) or huge argument spaces may not benefit. Memoization works best with pure functions that return the same output for the same input.

Cache invalidation techniques

Memoization caches can become stale. You need strategies to invalidate or refresh entries:

- * **Manual invalidation**: Expose a method to clear the cache or remove specific keys when underlying data changes.

- * **Max size eviction**: In an LRU cache, old items are removed automatically when new items push the cache over its size limit.

- * **Time-based expiration**: Entries expire after a timeout (TTL). When retrieving, check if the entry is still valid.

- * **Event-driven invalidation**: In more complex systems, listen for events (e.g. database updates) and invalidate related cache entries.

Example: memoizing a recursive function

The classic example is the Fibonacci sequence. A naive recursive implementation has exponential complexity. Memoization turns it linear:

```
```\njs\nfunction memoizeFib() {\n  const cache = {};\n  function fib(n) {\n    if (n < 2) return n;\n    if (cache[n] !== undefined) return cache[n];\n    const result = fib(n - 1) + fib(n - 2);\n    cache[n] = result;\n    return result;\n  }\n  return fib;\n}
```



```
}
```

```
const fib = memoizeFib();
console.log(fib(40)); // computes quickly
...
```

Here, the inner `fib` function uses a closure over the cache. Without memoization, computing `fib(40)` would take millions of recursive calls.

### ## Practice questions

1. **Theory:** Why is memoization effective only for pure functions? Give an example where memoization would not help.
2. **Theory:** Compare LRU and TTL caches. When would you choose one over the other?
3. **Coding:** Implement a memoized version of a function that sorts arrays of numbers. How would you ensure the cache key uniquely identifies the input array regardless of reference?
4. **Coding:** Write a memoization helper that supports a maximum cache size and automatically evicts the least recently used entry. Test it with a computationally expensive function.

# What is reactive programming and how does it differ from imperative programming?

## # What is reactive programming and how does it differ from imperative programming

**\*\*Reactive programming\*\*** is an approach to software development focused on working with asynchronous data streams and the propagation of change. Instead of writing step-by-step instructions (imperative code) that pull data, you declare relationships and let the system react automatically when data arrives or changes. Reactive programming is particularly useful for dealing with user interactions, I/O events and real-time data.

## ## Imperative vs reactive

### ### Imperative programming

Imperative code describes *how* to achieve a result by giving explicit commands. You manage state and control flow manually. For example, consider updating the UI when a user types:

```
``js
const input = document.getElementById('search');
const resultsDiv = document.getElementById('results');
input.addEventListener('input', async (e) => {
 const query = e.target.value;
 const res = await fetch(`/search?q=${encodeURIComponent(query)}`);
 resultsDiv.textContent = await res.text();
});
``
```

Here you explicitly attach a listener, fetch results and update the DOM in response.

### ### Reactive programming

Reactive programming treats values as streams that you can transform, filter, combine and observe. When a stream emits a new value, any dependent computations automatically update. Libraries like RxJS provide observables for this.

Example using RxJS (conceptual):

```
``js
import { fromEvent, of } from 'rxjs';
import { debounceTime, switchMap, map, catchError } from 'rxjs/operators';

const input = document.getElementById('search');
const resultsDiv = document.getElementById('results');

fromEvent(input, 'input').pipe(
 debounceTime(300),
 map(e => e.target.value),
 switchMap(query => fetch(`/search?q=${encodeURIComponent(query)}`)
 .then(res => res.text())
 .catch(() => 'Error')), // convert promise to observable
).subscribe(text => {
 resultsDiv.textContent = text;
});
``
```

This code creates an observable stream of input events, debounces them, maps to query strings, switches to a new network request whenever the query changes and updates the UI whenever new results come back. The code describes *what* should happen rather than *how* to manage control flow. The RxJS library handles the timing and cancellation of requests.

### ## Benefits of reactive programming

**\* \*\*Declarative:\*\*** You express transformations on data streams, leaving orchestration to the framework.

\* **Composability:** Operators like ``map``, ``filter``, ``debounceTime``, ``combineLatest`` allow you to build complex data pipelines from simple pieces.

\* **Error handling and cancellation:** Streams can propagate errors and support cancellation semantics (e.g. using ``switchMap`` to cancel previous requests).

\* **Consistency:** Reactive code often looks similar on the client and server (e.g. RxJS in the browser and RxJava on the backend).

## ## Differences from imperative programming

\* **Control flow:** In imperative code, you control when and how operations run. In reactive code, you declare relationships and let the runtime schedule operations.

\* **State management:** Reactive systems maintain state through streams and reactive variables. Imperative code stores state in variables you update manually.

\* **Concurrency:** Reactive systems are inherently asynchronous and often use non-blocking I/O. Imperative code may block or require manual callback handling.

## ## When to use reactive programming

\* Real-time applications (chat, live data feeds).

\* Complex user interfaces with many interdependent events and asynchronous operations.

\* Systems that need to react to multiple sources of events and combine them elegantly.

## ## Practice questions

1. **Theory:** In your own words, define reactive programming and contrast it with imperative programming.

2. **Theory:** What are the advantages of using a reactive approach when dealing with user input and network requests?

3. **Coding:** Using RxJS (or a similar library), create a reactive autocomplete input that fetches suggestions as the user types, debouncing requests and cancelling previous ones.

4. **Coding:** Implement a simple reactive data flow without external libraries by creating a ``Signal`` class that notifies observers when its value changes. Use it to link two input fields so that changing one updates the other.

# How does decorator syntax enhance class behavior?

## # How does decorator syntax enhance class behavior

Decorators are a proposed language feature (currently at Stage 3 of the TC39 process) that allow you to attach behaviors or metadata to classes, methods, fields and accessors using a concise `@decorator`` syntax. Decorators enable modular composition of cross-cutting concerns—such as logging, memoization, access control or dependency injection—without modifying the core business logic. Several frameworks (e.g. Angular, NestJS) and libraries use decorators extensively.

## ## What are decorators

A **decorator** is a function applied to a class, method, field or accessor declaration. It receives metadata about the target and can alter its definition or attach additional behavior. Decorators run during class definition, not at runtime when methods are invoked.

For example, a class decorator might modify the class constructor or static fields; a method decorator can wrap a method to add logging; a property decorator can enforce validation.

## ## Examples of decorators

### ### Method decorator: logging

Suppose you want to log every call to a method. You could write a decorator that wraps the original function and logs arguments and return values.

```
````js
function log(target, context) {
  const { kind, name } = context;
  if (kind === 'method') {
    return function (...args) {
      console.log(`Calling ${name} with`, args);
      const result = target.apply(this, args);
      console.log(`${name} returned`, result);
    };
  }
}
```

```

    return result;
  };
}
}

```

```

class Calculator {
  @log
  add(a, b) {
    return a + b;
  }
}

```

```

const calc = new Calculator();
calc.add(2, 3); // logs inputs and result
...

```

The ``log`` decorator receives the original method (``target``) and a ``context`` object describing the method. It returns a new function that wraps the original method with logging. When ``calc.add()`` is called, the wrapper logs the call and delegates to the original implementation.

Field decorator: memoization

Decorators can also wrap getters to cache expensive computations:

```

```js
function memoizeAccessor(target, context) {
 const { kind, name } = context;
 if (kind === 'getter') {
 const cacheKey = Symbol(name);
 return function () {
 if (!(cacheKey in this)) {
 this[cacheKey] = target.call(this);
 }
 };
 }
}

```

```

 }
 return this[cacheKey];
 };
}
}

class Expensive {
 @memoizeAccessor
 get largeArray() {
 console.log('Computing large array');
 return new Array(1000000).fill(0).map((_, i) => i);
 }
}

```

```

const e = new Expensive();
e.largeArray; // computes and caches
e.largeArray; // returns cached value, no log
...

```

Here, the `memoizeAccessor` decorator wraps the getter so that it only executes once. Subsequent accesses return the cached result.

### ### Class decorator: registering metadata

A class decorator can attach metadata for frameworks:

```

```js
const registry = new Map();

function controller(path) {
  return function (target, context) {

```

```

    registry.set(path, target);
  };
}

@Controller('/users')
class UserController {
  // ...methods...
}

console.log(registry.get('/users') === UserController); // true
...

```

The `@controller('/users')` decorator stores the class in a registry keyed by path. Later, a framework could read this registry to configure routes.

Comparison with higher-order functions and classes

Decorators are syntactic sugar over higher-order functions applied at declaration time. You could achieve similar results by manually wrapping methods or classes, but decorators make the intent clear and reduce boilerplate. Unlike higher-order components in React, which wrap components at runtime, class decorators run once when the class is defined.

Current status and limitations

The decorator proposal is still experimental and may evolve. Not all environments support decorators yet. Transpilers like Babel or TypeScript support older decorator syntax, but it differs from the current proposal shown here. When using decorators today, check your tooling and be aware that the API may change.

Practice questions

1. **Theory:** What kinds of elements (class, method, field) can be decorated, and what does a decorator function receive as arguments?

2. **Theory:** How does a decorator differ from a higher-order function? When would you prefer one over the other?
3. **Coding:** Implement a method decorator ``time`` that measures and logs the execution time of a method. Apply it to a function that performs a heavy computation.
4. **Coding:** Create a class decorator ``sealed`` that prevents further modification of the class prototype (e.g. using ``Object.seal()``). Demonstrate its effect.

What are import assertions and how do they ensure module type safety?

What is import assertions and how do they ensure module type safety

When using JavaScript modules, the default behavior is to treat imported files as JavaScript. However, modern applications often import other types of resources, such as JSON or CSS modules. ****Import assertions**** provide a standardized way to explicitly declare the expected type of a module when importing it. By asserting the type, you help the JavaScript engine validate that the imported content matches your expectation and prevent silent failures or security issues.

Why import assertions were introduced

Historically, browsers and bundlers allowed syntax like ``import data from './data.json';`` to import JSON as if it were a JavaScript module. This was convenient but ambiguous: what if a `.js`` file is accidentally served as JSON? To remove this ambiguity and improve security, the ECMAScript spec introduced ****import assertions****—an extra clause in the import statement that declares the module's type.

Syntax of import assertions

Import assertions use the ``assert`` keyword followed by an object literal after the module specifier:

```
``js
import userData from "./user.json" assert { type: "json" };
```

// or with dynamic import

```
const strings = await import(`/i18n/${lang}.json`, {
  assert: { type: "json" },
});
...`
```

In the static form, the ``assert { type: 'json' }`` tells the loader to treat the imported resource as JSON. If the resource is not of the asserted type, the import will fail with a syntax error. In the dynamic form, the second argument is an options object with an ``assert`` property.

Supported types

At the time of writing, the ``type`` assertion is defined for JSON (``json``) in browsers and Node.js. Future proposals may define assertions for other module formats (e.g. ``css``, ``wasm``). Tools and loaders can extend this mechanism to support custom file types.

Detailed example: importing JSON with assertions

Imagine you have a ``config.json`` file:

```
``json
{
  "apiUrl": "https://api.example.com",
  "timeout": 5000
}
...

```

You can import it safely in your module:

```
``js
// config.js
import config from "./config.json" assert { type: "json" };

export function getApiUrl() {
  return config.apiUrl;
}
...

```

****What happens under the hood?****

1. The loader sees the ``assert { type: 'json' }` clause and knows to parse the file as JSON, not JavaScript.
2. If the server returns a MIME type that is not JSON (``application/json``), or the file contains invalid JSON, the import fails at parsing time rather than silently returning an unexpected value.
3. The imported value is the parsed JSON object (``config``), which you can use immediately. You don't need to call ``fetch()`` and ``JSON.parse()`` yourself.

If you omit the assertion and try to import a JSON file, many environments will throw an error because unasserted JSON imports are not allowed. Assertions explicitly opt into this behavior.

Benefits of import assertions

- **Type safety:** Assertions ensure that a module is only loaded if it matches the expected format. They prevent accidentally interpreting a JSON file as JavaScript or vice versa.
- **Security:** They help mitigate attacks where a malicious server returns a different file type (e.g. serving JavaScript when JSON was expected). The import will fail instead of executing unexpected code.
- **Clarity:** The import statement documents the developer's intent. Other developers reading the code know that a non-JavaScript resource is being imported.
- **Extensibility:** In the future, assertions may allow fine-grained control over module loading, such as specifying integrity hashes or custom loaders.

Practice questions

1. **Theory:** What problem do import assertions solve, and how do they improve module type safety?
2. **Theory:** How do import assertions differ between static and dynamic imports? Provide the correct syntax for each.
3. **Coding:** Create a module that imports a JSON configuration file with an import assertion and exports a function that reads a property from it. Then write a test showing that the import fails if the file contains invalid JSON.
4. **Coding:** Suppose browsers support CSS modules with ``type: 'css'``. Demonstrate how you would import a stylesheet using an import assertion and apply it to a component.

What is module federation and how does it enable micro-frontend architectures?

What is module federation and how does it enable micro-frontend architectures

Module Federation is a feature introduced in webpack 5 that allows multiple separate builds to dynamically share and load modules at runtime. Traditionally, JavaScript bundles are isolated—code in one bundle cannot import code from another unless they're built together. Module Federation breaks this barrier by enabling applications to **expose** modules and **consume** modules from remote builds on demand, without needing to publish them to a package registry or rebuild the consuming application.

Micro-frontend architecture divides a frontend application into smaller, independently developed and deployed fragments. Each micro-frontend may be built by a different team and has its own build process. Module Federation provides the tooling needed for these fragments to interoperate seamlessly.

How module federation works

Module Federation uses two key roles:

- **Host (or shell) application:** The main application that will load remote modules at runtime.
- **Remote application:** A separate build that exposes some of its modules for consumption by others.

Each build configures the webpack `ModuleFederationPlugin` with information about what it exposes and what it consumes.

Remote configuration

In the remote application's webpack config:

```
```js
// webpack.config.js in remote app
const ModuleFederationPlugin = require("webpack/lib/container/ModuleFederationPlugin");
```

```

module.exports = {
 // ... other settings ...
 plugins: [
 new ModuleFederationPlugin({
 name: "remoteApp",
 filename: "remoteEntry.js",
 exposes: {
 "./Button": "./src/components/Button.js",
 "./utils": "./src/utils/index.js",
 },
 shared: {
 react: { singleton: true },
 "react-dom": { singleton: true },
 },
 }),
],
};
...

```

- `name`: Identifies this remote.
- `filename`: The compiled file that contains the exposed modules. It is served via HTTP.
- `exposes`: Maps internal modules to public names. These paths will be available to hosts.
- `shared`: Declares shared dependencies so that only one copy is loaded (singleton).

### ### Host configuration

In the host application's webpack config:

```

```js
const ModuleFederationPlugin = require("webpack/lib/container/ModuleFederationPlugin");

```

```

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      remotes: {
        remoteApp: "remoteApp@https://remote.example.com/remoteEntry.js",
      },
      shared: {
        react: { singleton: true },
        "react-dom": { singleton: true },
      },
    }),
  ],
};
...

```

Here, the host declares a `remotes` object mapping the remote's name to a URL where `remoteEntry.js` is served. At runtime, webpack fetches this file, resolves the exposed modules and makes them available via `import()`.

Loading remote modules

Once configured, the host can import remote modules just like local ones:

```

```js
// In host application code
import("remoteApp/Button").then(({ default: Button }) => {
 // Use the remote Button component as if it were local
 ReactDOM.render(<Button label="Click me" />, document.getElementById("app"));
});

import("remoteApp/utils").then(({ formatDate }) => {

```

```
console.log(formatDate(new Date()));
});
...
```

Webpack loads `remoteEntry.js` asynchronously, resolves the module, ensures shared dependencies (like React) are not duplicated, and returns the exported value. This happens at runtime, not build time.

### ## Enabling micro-frontends

In a micro-frontend architecture, each team can build its part of the UI as a separate project with its own repository and deployment pipeline. Module Federation enables these fragments to be integrated into a host at runtime without central coordination.

For example, a dashboard might consist of several panels built by different teams. Each panel is a remote exposing a React component. The shell application dynamically loads the panels based on configuration and composes them together. When a panel team releases a new version, the host automatically picks it up without a rebuild. This independence accelerates deployment and reduces coupling between teams.

### ### Sharing dependencies

Shared dependencies (such as React) should be marked as singletons so that only one instance is loaded. Otherwise, multiple versions of the library might be imported, leading to inconsistent state or duplicate React copies.

### ### Limitations and considerations

- **Version compatibility:** All consuming apps must use compatible versions of shared libraries.
- **Runtime failures:** If a remote is unavailable or misconfigured, the host must handle errors gracefully.
- **Complexity:** Module Federation introduces additional build and deployment considerations (e.g. serving `remoteEntry.js` over HTTP). Proper versioning and contract testing are essential.

### ## Practice questions



1. **Theory:** Explain how the `ModuleFederationPlugin` enables a host application to load modules from a remote at runtime.
2. **Theory:** What is the purpose of the `shared` section in the module federation configuration? Why are singletons important?
3. **Coding:** Set up a minimal host and remote application using webpack. Expose a component from the remote and consume it in the host. Describe how you would handle errors if the remote fails to load.
4. **Coding:** Suppose you have two remotes that both depend on the same version of a UI library. Show how to configure module federation so that only one copy of the library is loaded.

## What are WeakMap-based private fields and how do they differ from native private fields ()?

### # What are WeakMap-based private fields and how do they differ from native private fields

Before the introduction of native private fields (`#field`) in JavaScript classes, developers often used `WeakMap`'s to simulate private data. Understanding how these patterns work helps you appreciate the benefits and trade-offs of the new syntax.

#### ## WeakMap-based privacy pattern

A `WeakMap` is a collection where keys must be objects and values can be arbitrary data. The keys are weakly referenced: if no other references to the key object exist, it can be garbage-collected and the corresponding entry in the `WeakMap` is removed.

Using a `WeakMap`, you can associate private data with each instance of a class:

```
```js
const _privateData = new WeakMap();

class Person {
  constructor(name, age) {
    // Store private fields in the WeakMap
    _privateData.set(this, { name, age });
  }
  getName() {
    return _privateData.get(this).name;
  }
  celebrateBirthday() {
    const data = _privateData.get(this);
    data.age++;
    console.log(`${data.name} is now ${data.age}`);
  }
}
```

```
}
```

```
const alice = new Person("Alice", 30);  
console.log(alice.getName()); // Alice  
alice.celebrateBirthday(); // logs "Alice is now 31"  
// Direct access is not possible:  
console.log(alice.name); // undefined  
...
```

****How it works:****

1. `\_privateData` is a module-level `WeakMap` keyed by instances of `Person`.
2. In the constructor, the private values are stored in the map with `this` as the key.
3. Methods access the private data by calling `\_privateData.get(this)`.
4. There is no public way to access the private data because it's scoped outside the class.
5. When the instance (`alice`) is no longer referenced, the entry in the `WeakMap` disappears automatically.

Pros and cons of the WeakMap pattern

- ****Pros:**** Works in older environments; private data is not exposed on the object; memory cleans up automatically. You can have truly private properties with dynamic names.
- ****Cons:**** Slightly verbose and error-prone (you must remember to use the map in every method); still accessible to privileged code that closes over the WeakMap; cannot define private methods this way; slower access due to the map lookup.

Native private fields

ECMAScript now supports ****private class fields**** and methods with a `#` prefix. Private fields are defined inside the class body and are only accessible from within the class declaration.

```
```js
```

```
class BankAccount {
```

```

 #balance;

 constructor(initial) {
 this.#balance = initial;
 }

 deposit(amount) {
 this.#balance += amount;
 }

 getBalance() {
 return this.#balance;
 }
}

const acct = new BankAccount(100);
acct.deposit(50);
console.log(acct.getBalance()); // 150
// console.log(acct.#balance); // SyntaxError: Private field '#balance' must be declared in an
enclosing class
...

```

### **\*\*Key features:\*\***

- **\*\*Lexical privacy:\*\*** The `#balance`` field is only accessible from within the class body. Attempting to access it outside results in a syntax error. No property with that name exists on the instance; it is stored in an internal slot.
- **\*\*Performance:\*\*** Native private fields have optimized access and don't require a map lookup.
- **\*\*Private methods and accessors:\*\*** You can declare `#method()` and private getters/setters: `get #secret() { ... }`. These are not possible with the WeakMap pattern without additional complexity.
- **\*\*Static private fields:\*\*** You can declare `static #counter = 0`` to have per-class private state.

## ## Differences at a glance

| Feature            | WeakMap pattern                             | Native private fields                        |
|--------------------|---------------------------------------------|----------------------------------------------|
| Syntax             | External map, verbose                       | Concise <code>#field</code> syntax           |
| Access control     | Hidden by closure, not enforced by language | Enforced by syntax; illegal to access        |
| Private methods    | Not directly supported                      | Supported ( <code>#method() { ... }</code> ) |
| Garbage collection | Automatically cleans up via WeakMap         | Managed internally by the engine             |
| Enumeration        | Private values not enumerable               | Private fields not enumerable                |
| Performance        | Slight overhead due to map lookup           | Faster access, stored in internal slots      |

## ## When to use which

Native private fields are the preferred way to declare private data in modern JavaScript. They provide language-level enforcement, better ergonomics and performance. However, if you need to attach private data to objects that are not class instances (e.g. DOM elements), or you need dynamic private keys, the WeakMap pattern remains useful.

## ## Practice questions

- Theory:** Describe how private data is stored and accessed in the WeakMap pattern versus native private fields.
- Theory:** What happens if you try to access a native private field from outside the class? Contrast this with the WeakMap pattern.
- Coding:** Rewrite the `Person` class using native private fields instead of a WeakMap. Include a private method that returns a greeting.
- Coding:** Show how you might attach private data to DOM elements using a WeakMap. Explain why native private fields wouldn't work for this use case.

# How does lazy vs eager evaluation affect performance in iterables?

## # How does lazy vs eager evaluation affect performance in iterables

**\*\*Lazy evaluation\*\*** delays computing values until they are needed, while **\*\*eager evaluation\*\*** computes values immediately when an operation is invoked. In the context of iterables, lazy evaluation allows you to process sequences element by element on demand, often through generators or iterators. Eager evaluation typically produces a fully realized array or collection up front.

Understanding the performance implications of these strategies helps you choose the right approach for your use case.

### ## Eager iterables

Eager methods (such as `Array.prototype.map()`, `filter()` and `reduce()`) create intermediate arrays for each step. Consider this chain:

```
```js
const numbers = [1, 2, 3, 4, 5];
const result = numbers
  .map((n) => n * 2) // [2, 4, 6, 8, 10]
  .filter((n) => n > 5) // [6, 8, 10]
  .reduce((sum, n) => sum + n, 0); // 24
```
```

Each call to `map` and `filter` creates a new array. For small collections, the overhead is negligible. For large datasets, however, repeatedly allocating intermediate arrays consumes memory and time.

### ## Lazy iterables with generators

Lazy evaluation uses generators to produce values on the fly. You define a generator that yields values one at a time, and consumers iterate through them as needed. You can build your own lazy counterparts to `map` and `filter`:

```

```js
function* map(iterable, fn) {
  for (const item of iterable) {
    yield fn(item);
  }
}

function* filter(iterable, predicate) {
  for (const item of iterable) {
    if (predicate(item)) yield item;
  }
}

function* range(start, end) {
  for (let i = start; i < end; i++) yield i;
}

const pipeline = filter(
  map(range(1, 1000000), (x) => x * 2),
  (x) => x % 3 === 0
);

// The result is a generator; nothing has been computed yet.
let total = 0;
for (const val of pipeline) {
  total += val;
  if (total > 1000) break; // early exit
}
console.log(total);
```

```

### **\*\*What's happening?\*\***

1. ``range(1, 1000000)`` yields numbers from 1 to 999,999 lazily.
2. ``map(range..., x => x * 2)`` wraps that generator and yields each value multiplied by 2.
3. ``filter(..., x => x % 3 === 0)`` yields only the values divisible by 3.
4. The ``for...of`` loop requests values from this pipeline one at a time. As soon as ``total`` exceeds 1000, the loop breaks and the generator pipeline stops producing more values. Most of the million values are never computed at all.

This example illustrates two advantages:

- **\*\*Memory efficiency:\*\*** No intermediate arrays are created. Only one value at a time exists in memory.
- **\*\*Short-circuiting:\*\*** You can stop iteration early, saving time.

### **## Benchmarks and trade-offs**

- **\*\*Large collections:\*\*** Lazy iteration shines when processing large or potentially infinite sequences. Eager methods may allocate large arrays and block the event loop, whereas generators yield values lazily.
- **\*\*Single pass vs multiple passes:\*\*** If you need to iterate over the data multiple times, a generator may recompute values each time. A fully realized array may be faster on subsequent passes.
- **\*\*Complex transformations:\*\*** For small arrays or simple operations, the overhead of generator functions may outweigh the benefits.
- **\*\*Debugging:\*\*** Lazy pipelines can be harder to debug because computations happen at iteration time. Eager arrays are easier to inspect.

### **## Choosing between lazy and eager**

Use lazy evaluation when:

- The dataset is large, infinite or expensive to compute.
- You might stop early (e.g. search or sampling tasks).
- You want to minimize memory footprint.



Use eager evaluation when:

- The dataset is small or fits comfortably in memory.
- You need random access or multiple passes over the data.
- Simplicity and ease of debugging are more important than raw efficiency.

### ## Practice questions

1. **Theory:** Explain why lazy evaluation can reduce memory usage compared to eager evaluation. Give an example where this matters.
2. **Theory:** What are some potential downsides of using lazy evaluation for small arrays? Describe scenarios where eager evaluation might be better.
3. **Coding:** Implement a lazy version of `take(n, iterable)` that yields the first `n` values of an iterable. Then use it to sum the first 100 even numbers from an infinite generator.
4. **Coding:** Compare the performance of summing all numbers from 1 to 1 million using eager array methods vs. a lazy generator pipeline. Measure memory usage and execution time, and discuss the results.

# What are WeakKeys and WeakRefs — new memory-safe references?

## # What are WeakKeys and WeakRefs: new memory-safe references

JavaScript memory management is automatic: the garbage collector frees objects that are no longer reachable. However, certain data structures (like `Map` and `Set`) hold strong references to their keys and values. As long as a key is present in a `Map`, the object cannot be reclaimed even if there are no other references. This can lead to memory leaks in caches or listeners.

To address these cases, ECMAScript introduced **weak references**—data structures that reference objects without preventing them from being garbage-collected. The new proposals include **WeakRefs** and **WeakKey maps/collections** that offer more granular control over memory without sacrificing safety.

## ## WeakRef

`WeakRef` is a class that lets you hold a weak reference to an object. A weak reference does not prevent the object from being garbage-collected. You can attempt to access the object via `.deref()`, which returns the object if it is still alive or `undefined` if it has been collected.

```
```js
class Cache {
  constructor() {
    this.store = new Map();
  }
  set(key, value) {
    // store a weak reference to the value
    this.store.set(key, new WeakRef(value));
  }
  get(key) {
    const ref = this.store.get(key);
    if (!ref) return undefined;
    const value = ref.deref();
  }
}
```

```

    if (value === undefined) {
      // value was collected; remove from cache
      this.store.delete(key);
    }
    return value;
  }
}

```

```

let obj = { data: "important" };
const cache = new Cache();
cache.set("foo", obj);
obj = null; // only weak reference remains
// Later, obj may be garbage-collected. cache.get('foo') will return undefined once collected.
...

```

****When to use WeakRef:**** WeakRefs are useful for caches or look-aside maps where you can recreate a value if it's been collected. They should not be used to reference critical data because derefing a collected object returns `undefined`. You should always check for undefined.

FinalizationRegistry

`FinalizationRegistry` lets you register a finalization callback that the engine calls when an object is garbage-collected. This is helpful for cleaning up resources associated with an object, such as removing it from other data structures.

```

```js
const registry = new FinalizationRegistry((heldValue) => {
 console.log(`Cleaning up for`, heldValue);
});

function trackResource(resource) {
 const obj = { resource };

```

```

registry.register(obj, resource.id);

return obj;
}

```

```

let tracked = trackResource({ id: 1, file: "tmp.txt" });

tracked = null; // when obj is collected, the callback logs "Cleaning up for 1"
...

```

Finalization callbacks run at an arbitrary time after the object becomes unreachable. You should not rely on them for timely cleanup or critical logic.

### ## WeakKey maps and sets

The traditional `WeakMap` and `WeakSet` require that the keys are objects and hold weak references to those keys. However, the values of a `WeakMap` are strongly referenced. New proposals (often called **WeakKey** collections) aim to extend this concept by allowing keys to be weak and values to be strong or weak, providing more flexibility for caches. As of now, these proposals are still being developed.

### ### Why are WeakKeys and WeakRefs memory-safe

- **No memory leaks:** If the only references to an object are weak, it is eligible for garbage collection. This avoids leaks in caches or observer lists.
- **No access after collection:** If you dereference a `WeakRef` after the object has been collected, you get `undefined`. There is no chance of accessing stale memory.
- **Controlled cleanup:** `FinalizationRegistry` gives you a hook to remove entries or free associated resources.

### ### Caveats

- **Unpredictable timing:** Garbage collection is nondeterministic. `WeakRef` values may disappear at any time, and finalization callbacks may run much later.
- **No strong references:** Do not store critical data solely via `WeakRefs`; always keep at least one strong reference if the data must persist.

- **Potential misuse:** Overusing WeakRefs can make code hard to reason about. Use them for cache layers or to break cycles, not for general data storage.

## ## Practice questions

1. **Theory:** Explain how a weak reference differs from a strong reference. Why can weak references help prevent memory leaks?
2. **Theory:** What does `FinalizationRegistry` do, and why should you not rely on it for critical cleanup logic?
3. **Coding:** Implement a memoization cache that uses WeakRefs to store results keyed by objects. Demonstrate how results may disappear from the cache when the keys are no longer referenced elsewhere.
4. **Coding:** Write a simple wrapper around WeakRef that attempts to get the value and, if it has been collected, computes a new value and stores a new WeakRef. Explain how this pattern can be used in caches.

# How do `structuredClone`, `postMessage`, and transferable objects relate?

## # How do `structuredClone`, `postMessage` and transferable objects relate

JavaScript environments often need to copy or send data between contexts—such as between the main thread and Web Workers, or between different windows or iframes. Copying complex objects safely and efficiently is non-trivial. Three related mechanisms address this: the **structured clone algorithm**, the `postMessage()` API, and **transferable objects**.

### ## Structured clone algorithm

The **structured clone algorithm** is a specification that defines how to deeply copy a broad range of JavaScript values. It supports primitives, plain objects, arrays, dates, typed arrays, Map, Set, RegExp, ArrayBuffer, and more. Crucially, it can handle circular references and shared references, preserving object graphs.

Historically, browsers implemented the structured clone algorithm internally, but there was no way for developers to invoke it directly. This changed with `structuredClone()`, a global function that allows you to deep-clone structured clone-serializable values:

```
``js
const original = { name: "Alice", list: [1, 2, 3] };
const copy = structuredClone(original);
copy.list.push(4);
console.log(original.list); // [1, 2, 3]
``
```

`structuredClone()` creates a new object graph. It is more powerful than `JSON.parse(JSON.stringify(obj))` because it can clone functions' absence (functions are not cloneable), Dates, Maps, Sets and typed arrays, and it preserves reference identity.

### ## `postMessage()` and structured cloning

`postMessage()` is an API used to send data between different execution contexts:

- **Window messaging:** `window.postMessage()` sends data to another window or iframe.
- **Web workers:** `worker.postMessage()` sends data to a worker thread.
- **Message ports:** `port.postMessage()` communicates over a `MessageChannel`.

When you pass an object to `postMessage()`, the environment uses the structured clone algorithm to copy the data from one context to another. The receiving context gets a deep clone of the original object. This ensures that the sender and receiver do not share mutable objects, preventing accidental sharing of memory and data races.

Example:

```
``js
// main thread
const worker = new Worker("worker.js");
const obj = { values: [1, 2, 3], time: new Date() };
worker.postMessage(obj);

// worker.js
self.onmessage = (event) => {
 const data = event.data;
 console.log(data.values); // [1, 2, 3]
 console.log(data.time instanceof Date); // true
 data.values.push(4);
};
// Mutating data inside the worker does not affect the original in the main thread
...

```

Because the clone is deep, modifying `data.values` in the worker does not affect the original `obj`.

## ## Transferable objects

Some objects hold underlying binary data, such as `ArrayBuffer`, `MessagePort`, `OffscreenCanvas` and `ImageBitmap`. Copying these objects can be expensive. **Transferable objects** allow you to transfer ownership of the underlying resource from one context to another instead of copying it. After transfer, the sender's object becomes unusable (it's "detached") and the receiver gains sole control.

To transfer objects with `postMessage()`, pass them in an array as the second argument:

```
``js
const sab = new SharedArrayBuffer(1024);
const worker = new Worker("worker.js");
// Transfer the buffer
worker.postMessage({ buffer: sab }, [sab]);
// sab is now detached in the main thread; accessing it throws an error

// worker.js
self.onmessage = (e) => {
 const { buffer } = e.data;
 // buffer refers to the transferred ArrayBuffer
 console.log(buffer.byteLength); // 1024
};
...

```

In this example, the `ArrayBuffer` moves to the worker without copying its bytes. Attempting to use the buffer in the main thread after transfer will throw a `TypeError: DetachedBuffer`. Transferables are essential for high-performance applications such as streaming media, image processing and parallel computations.

### ## Relationship between the three

- **Structured clone algorithm** defines how data is copied in both `structuredClone()` and `postMessage()`. It ensures deep cloning of complex objects and prevents sharing references across threads.



- **`structuredClone()`** exposes the clone algorithm directly for your own deep copy needs within a single context.
- **`postMessage()`** uses the clone algorithm to send data across execution contexts. It can optionally transfer ownership of transferable objects to avoid copying.
- **Transferable objects** are a special case handled by both `structuredClone()` and `postMessage()`; they let you move a resource instead of copying it. Passing them in the transfer list ensures efficient transfer.

## ## Practice questions

1. **Theory:** Describe how the structured clone algorithm differs from using `JSON.stringify()`/`JSON.parse()` for deep copying. What kinds of values can it handle that JSON cannot?
2. **Theory:** Why do browsers detach a transferable object from the sender after it is transferred via `postMessage()`? What would happen if they didn't?
3. **Coding:** Use `structuredClone()` to create a deep copy of an object containing a `Map` and a `Date`. Demonstrate that modifications to the copy do not affect the original.
4. **Coding:** Send an `ArrayBuffer` to a worker using `postMessage()` with and without specifying it as transferable. Measure the time taken and observe the behavior of the original buffer. Discuss when you should use transferables.